

从 C++ 到 AI 计算：第三册

张量、算子、量化与本地大模型推理引擎

CPU Performance Study

本册接续第一册的程序执行证据链和第二册的现代 CPU 账本，围绕一个能推理开源 7B 级 decoder-only 大模型的本地引擎展开：模型加载、tokenizer、张量布局、GEMM、attention、KV cache、量化、CPU/GPU 后端、调度、验证和性能对标都必须落到可测量的机器事实。

前言

第一册把 C++ 程序追到二进制、ABI、汇编和可信测量；第二册把同一个程序继续放进现代 CPU、多核心、缓存、TLB、SIMD、线程、运行时、I/O 和分布式边界中。第三册从这里进入 AI 计算，但它不会把 AI 写成框架 API 清单。真正的问题不是“怎样调用一个模型”，而是：一个能推理开源 7B 级大模型的本地引擎，怎样把模型文件、tokenizer、权重张量、GEMM、attention、KV cache、量化参数、CPU/GPU kernel、调度器和测量报告组织成一条可解释的机器链路。

本册的贯穿项目是一台本地大模型推理引擎。目标不是玩具 MLP，而是能加载真实开源 7B 左右 decoder-only Transformer 权重，完成 prompt 到 token 流的推理，并且同一套 runtime 可以挂接 CPU 后端和 GPU 后端。书中仍会使用一个极小 MLP 作为第一块显微镜切片：它让矩阵乘、bias、activation、量化权重和 oracle 能被手工检查。但这个切片只服务主线，不取代主线。后续每一节都要回到真实引擎：模型格式怎样承载几十亿参数，prefill 和 decode 为什么有不同性能形态，KV cache 怎样决定显存/内存账本，量化怎样让 7B 模型装进本地机器，CPU 后端怎样一步步接近成熟框架，GPU 后端又怎样共享同一份算子合同。

核心思想

第三册的主线是把大模型推理拆成可以验证的机器事实：张量是带形状、步长和生命周期的内存视图；算子是有输入合同和输出合同的 kernel；量化是整数表示、误差和吞吐之间的合同；KV cache 是跨 token 持续增长的状态；推理引擎是模型、内存、后端、调度和测量证据的组合。

本册正文仍然以原理和工程证据为主。现有 [books/cpu-volume-3/labs/linux_cpu_inference/](https://books.cmu.edu/cpu-volume-3/labs/linux_cpu_inference/) 是第一阶段的最小切片；完整路线会继续扩展出模型加载、tokenizer、Transformer 层、KV cache、CPU kernel、GPU backend adapter、benchmark harness 和 correctness report。正文会使用伪代码、关键 C++ 片段、抽象汇编路径、数据布局表、流图和测量边界来解释机制；工程目录负责把这些机制落实到可构建、可测试、可 benchmark 的实现。

怎样阅读本册

阅读第三册时，不要先把注意力放在模型名和框架名上。模型名称会变，框架接口会变，硬件指令集也会扩展；但本地大模型推理的基本账本长期稳定：权重字节从哪里来，token 怎样变成 id，张量按什么步长读取，在哪个类型中累加，KV cache 写到哪里，CPU/GPU 后端各自承担哪些 kernel，怎样判断误差可接受，怎样证明时间数字对应正确工作。

如果你完全不懂编译原理，也可以从本册开始。先把“编译器”理解成一个严格的工程流水线：读懂输入，建立中间表示，检查不变量，在语义不变的前提下改写成更适合机器执行的形式。AI 编译器只是把这件事用在模型文件、张量、量化、推理图、后端 kernel 和运行时调度上。遇到 IR、verifier、lowering、pass、runtime 这些词，不要先背定义，而要问：它在本地推理引擎里检查了什么、生成了什么、节省了哪部分机器成本。

本册会频繁回扣第一册和第二册。第一册教你把 C++ 代码追到 ABI、汇编、对象布局 and 可信 benchmark；第二册教你把同一段机器工作放进现代 CPU、cache/TLB、SIMD、线程、NUMA、运行时和 I/O 边界里解释。第三册讲 AI 编译器和推理后端时，会把这两类能力当成工具，而不是重新孤立发明一套 AI 术语。

建议按四条线并行阅读。

1. 架构线：模型资产、编译前端、IR 优化、后端、运行时分别负责什么，哪些事实不能跨层乱猜。
2. 语义线：每个算子定义什么输入、输出和错误边界。
3. 表示线：token、张量、权重、scale、zero point、activation buffer 和 KV cache 怎样存在内存里。
4. 机器线：核心循环生成什么地址序列、使用哪些指令、触发哪些 cache、带宽、NUMA、PCIe 或显存问题。
5. 后端线：同一个算子合同怎样分别落到 CPU SIMD/多线程 kernel 和 GPU kernel/stream 上。
6. 证据线：reference、误差 oracle、反汇编、profiler、benchmark 和框架对标报告怎样组成闭环。

若某个优化看起来很诱人，先问五件事：它是否保持算子语义，是否改变了机器账本中真正等待的部分，是否适用于 prefill、decode 或两者，C++20 工程实现是否有清楚模块边界和资源所有权，是否有测量证据证明它相对当前基线和现有框架缩小了差距。只要这些问题没有说清，就不要把它当成可靠优化。

本册结构

第三册不按“AI 名词清单”组织，而按一台 C++20 本地大模型推理引擎的建设顺序组织。它明确承接第一册和第二册：第一册给出 C++、二进制、汇编、ABI、对象布局和可信测量；第二册给出现代 CPU、cache/TLB、SIMD、多线程、NUMA、运行时、I/O 和背压模型；第三册把这些能力集中用在模型、张量、AI 编译器、推理 runtime 和 CPU/GPU 后端上。

读者即使完全没有学过编译原理，也应该先拿到全书地图：输入是什么，语义在哪里被检查，中间表示怎样建立，优化改变了哪条机器账本，后端怎样落到 CPU/GPU，运行时怎样证明结果没有跑偏。

```
model files / tokenizer / weights
-> manifest and schema resolution
-> shape, dtype and quantization verifier
-> quantization calibration, int8/int4 and KV cache policy
-> graph IR / tensor IR / loop IR
-> fusion, memory planning and lowering
-> CPU backend / GPU backend executable plans
-> runtime session, KV cache, scheduler and sampler
-> oracle, profiler, benchmark and framework comparison
```

本册重新分为六个大纲。

第一章：总架构与业务闭环。 先解释编译器到底在做什么，再把传统编译流水线映射到 AI 推理引擎，并明确第三册怎样继承第一册的 ABI/汇编/测量能力和第二册的硬件/并发/运行时能力。这里会定义全书五层架构：模型资产层、编译前端层、IR 与优化层、后端层、运行时层，并用一条端到端链路把模型文件、manifest、typed graph、executable plan、prefill、KV cache、decode、sampler、oracle 和 profiler 串起来。本册还会明确贯穿项目：一个可解释的 C++20 本地推理服务原型，包含 CLI、服务接口、报告文件、测试夹具、reference、oracle、profiler、资源预算、业务状态机、队列、调度、取消和 backpressure。读者先理解“语义合同”和“机器账本”，后续内容才不会把 manifest、IR、kernel 和 runtime 混成一团。

第二章：张量、算子、量化与计算账本。 从真实本地推理引擎边界开始，逐步进入张量布局、地址公式、矩阵向量乘、GEMM、packing、SIMD、量化合同和 7B 计算账本。这里重点回答：一个算子在数学上定义什么，在内存里怎样存在，在机器上怎样读写，低比特表示怎样保持数值合同，模型级 FLOPs、权重字节和 KV cache 容量怎样算清。

第三章：Transformer 推理、KV cache 与采样。 把 decoder-only Transformer、reference decoder、prefill、decode、KV cache、logits、sampler 和长上下文状态接起来。这里重点回答：一次真实请求怎样从 prompt 变成 token 流，RoPE、attention、GQA、KV cache、采样策略和停止条件各自改变什么状态，哪些错误必须由 oracle 定位。

第四章：AI 编译器与推理运行时。 讲推理引擎里为什么需要编译器，怎样从模型文件导入 typed graph，怎样建立 manifest、schema resolution、shape/dtype/quant verifier，怎样做 memory planning，怎样分层使用 graph IR、tensor IR、loop IR，怎样设计 analysis pass、transform pass、backend lowering 和 executable plan。所有编译原理概念都落到 AI infra 对象上解释。

第五章：C++20 CPU/GPU 后端优化。 先建立后端优化总路线：reference、scalar baseline、layout、packing、SIMD、线程、NUMA/TLB、runtime integration、oracle/profiler report。CPU 后端要细到 RAII 所有权、std::span 视图、tensor view、aligned allocator、packed weight cache、SIMD micro-kernel、寄存器账本、tail path、线程池、NUMA/TLB、KV cache 读流、perf counter、benchmark harness 和成熟框架对标；GPU 后端要从硬件事实落到 device buffer、stream、event、workspace、kernel launch、host-device staging、量化格式支持、fallback 和端到端 latency。

第六章：工程验收、路线图与作品集证据。 把 reference、oracle、profiler schema、测试矩阵、覆盖率、性能回归门禁、诊断质量、实施路线图和框架对标收束成作品集证据。这里重点回答：一个学习项目怎样从“能跑”升级为能被强团队认可的 AI Infra 资产。

因此，本册每个大章内部不再堆小标题，而按同一条工程推导线展开：概念定义、最小实现、性能瓶颈、优化设计、实验数据、工业对比、局限复盘。每个优化都要满足同一条验收线：它保持了哪条语义合同，改变了哪条地址流或生命周期，目标后端为什么因此更快，oracle 怎样证明没算错，profiler 怎样证明瓶颈真的下降。只要这些问题没有讲清，就不算完成一个高质量 AI infra 优化。

目录

前言	ii
怎样阅读本册	iii
本册结构	iv
第 1 章 总架构与业务闭环	1
第 2 章 张量、算子、量化与计算账本	23
第 3 章 Transformer 推理、KV Cache 与采样	87
第 4 章 AI 编译器与推理运行时	129
第 5 章 C++20 CPU/GPU 后端优化	160
第 6 章 工程验收、路线图与作品集证据	198
术语表	217

第 1 章

总架构与业务闭环

1.1 全书总架构：从零理解 AI 编译器、推理引擎与 C++20 后端

先把“编译器”讲成人话

如果完全没有学过编译原理，可以先把编译器理解成一个严谨的翻译和审计系统。它不只是把一种文本换成另一种文本，而是要回答三个问题：

1. 这份输入到底表达了什么语义。
2. 这个语义能不能在目标机器上安全、正确地执行。
3. 在不改变语义的前提下，怎样让机器少做无用工作。

普通 C++ 编译器的输入是源代码。它会先把字符切成 token，再按语法组成 AST，然后检查类型、名字、生命周期、调用规则和错误边界，再把程序降成 IR，做优化，最后生成目标机器代码。AI 编译器的输入不一定是 C++ 源码，而是模型配置、tokenizer、权重文件、量化元数据和一张推理图。它要做的仍然是同一件事：先理解语义，再建立可检查的中间表示，最后把它降到 CPU/GPU 后端能执行的 kernel 计划。

核心思想

编译原理不是一套抽象名词，而是一种工程纪律：任何优化都必须先知道输入语义、表示边界、错误条件和目标机器成本。AI 编译器只是把这套纪律用在模型、张量、量化和推理运行时上。

本册不会假设读者已经懂词法分析、语义分析、IR、优化、lowering、runtime 这些词。每次使用它们时，都要落到本地推理引擎里的具体对象。例如 manifest 就像模型文件的符号表；shape verifier 就像类型检查器；graph IR 就像算子级中间表示；memory planner 就像给大张量做寄存器/栈槽分配；backend lowering 就像把 IR 变成 AVX、线程池、GPU stream 和 kernel launch。

第三册怎样承接第一册和第二册

第三册不是另开一条 AI 支线，而是把前两册的能力集中用在 AI infra 上。第一册解决“一个 C++ 程序怎样落到机器事实”：源码怎样编译、汇编和链接，进程怎样获得虚拟地址空间，寄存器、栈、对象布局、指针和函数调用怎样受 ABI 约束，怎样用反汇编、调试器和 benchmark 证明自己运行的是哪份二进制。没有这部分基础，后端优化里的 `std::span` 张量视图、汇编形态、调用边界、aligned buffer、C ABI kernel wrapper 和可信测量都会变成空话。

第二册解决“同一段机器工作进入现代计算系统以后怎样变慢或变快”：取指前端、乱序执行、分支预测、cache、TLB、page fault、SIMD、线程、锁、原子、false sharing、NUMA、任务运行时、I/O、背压和分布式边界怎样改变成本。没有这部分基础，第三册里讨论 GEMM/GEMV、packing、KV cache、prefill/decode、线程池、NUMA 放置、GPU staging 和 profiler 时，只能停在“经验上这样更快”。

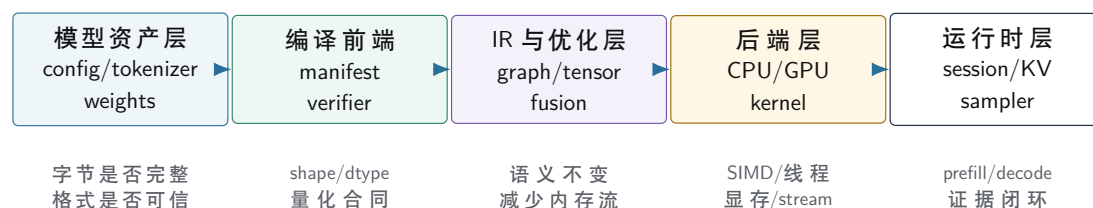
卷次	已建立能力	第三册中的用法
第一册	C++ 到二进制、汇编、ABI、对象布局、可信测量	读 kernel 汇编、设计 C++20 RAII/ABI 边界、验证 benchmark 身份
第二册	现代 CPU、cache/TLB、SIMD、多线程、NUMA、运行时、背压	解释算子瓶颈、设计 packing/threading、分析 KV cache 和 prefill/decode
第三册	模型、张量、AI 编译器、推理 runtime、CPU/GPU 后端	把前两册能力组合成可验证的本地大模型推理引擎

因此，第三册每个重要机制都要向前接两条线：语义和 ABI 线来自第一册，硬件和系统成本线来自第二册。比如一个 int8 GEMV kernel，第一册帮助我们看懂函数边界、指针、汇编和测量；第二册帮助我们解释 SIMD 宽度、cache line、TLB、线程划分和 NUMA；第三册再把它放进模型权重、量化 scale、typed graph、runtime segment 和 oracle/profiler 中。

AI infra 的五层：从文件到 token 流

一台真正可解释的本地大模型推理引擎，可以分成五层。每一层都有自己的输入、输出、错误边界和优化空间。

第三册的五层 AI infra 架构



图示：本图要证明的命题是：AI infra 不是一个大函数，而是模型资产、编译前端、IR 优化、后端和运行时五层合同。

模型资产层。 这里处理的是文件事实：模型配置写了多少层，tokenizer 有多少词，权重张量在哪个文件偏移，量化 scale 怎样存储。错误通常是文件缺失、shape 不一致、tokenizer 和 embedding 不匹配、量化 metadata 不完整。

编译前端层。 这里把文件事实变成编译器事实。manifest 记录张量和模型合同，schema resolution 把不同模型命名映射成统一角色，shape/dtype/quant verifier 拒绝不可信输入。前端做得越扎实，后端越不需要在热路径里猜。

IR 与优化层。 这里把推理图变成 graph IR、tensor IR 和 loop/kernel IR。Graph IR 负责算子依赖和状态边；Tensor IR 负责 shape、stride、layout 和 dtype；Loop IR 负责 tile、vector、thread 和 memory access。融合、消除冗余 reshape、layout 选择和内存生命周期分析都发生在这里。

后端层。 这里把计划落到机器。CPU 后端要处理 cache line、SIMD、packing、线程划分、NUMA、TLB 和分支；GPU 后端要处理显存布局、coalesced load、shared memory、kernel launch、stream 和同步。后端优化是本册后半部分的重头戏，不能停留在“调用一个库”。

运行时层。 编译器生成计划后，runtime 还要维护 session、token buffer、KV cache、activation arena、sampler、profiler 和 oracle。prefill 和 decode 是不同阶段，指标也不同：prefill 关心首 token 和 prompt 吞吐，decode 关心 tokens/s、单 token latency 和尾延迟。

整本书按六个大章组织

本册后续按六个大章推进。每一节都回答一个具体问题，而不是堆主题名。

第一章：总架构与业务闭环。 先把 AI infra、编译原理和本地推理引擎放到一张图里。读者需要先知道全书在建什么系统、每层交付什么、C++20 项目应该如何分模块，后面才不会把 manifest、IR、kernel、runtime 混在一起。

第二章：张量、算子、量化与计算账本。 从张量布局、地址公式、matvec、GEMM、packing、SIMD、量化合同和 7B 账本开始。这章的目标是让读者能看懂一个 kernel 为什么快或慢：不是看函数名，而是看地址流、累加器、对齐、尾块、scale 和字节账本。

第三章：Transformer 推理、KV cache 与采样。 把 decoder-only Transformer、reference decoder、prefill/decode、KV cache、logits 和 sampler 接成一条推理链路。这里开始把单算子性能放回整机系统。

第四章：AI 编译器与推理运行时。 讲模型导入、manifest、schema resolution、shape verifier、dtype verifier、memory planner、graph/tensor/loop IR、融合、backend lowering 和 executable plan。读者不需要先懂传统编译器，本册会用推理引擎对象逐步解释每个概念。

第五章：C++20 CPU/GPU 后端优化。 CPU 后端要细化到 C++20 模块划分、RAII 资源管理、`std::span` 张量视图、aligned allocator、packing cache、SIMD micro-kernel、线程池、NUMA 放置、perf 计数器和 benchmark harness；GPU 后端要细化到 device buffer、stream、kernel launch、host-device staging、workspace 和 fallback；所有优化都必须配 oracle、profiler 和可复现实验。

第六章：工程验收、路线图与作品集证据。 最后把测试、coverage、sanitizer、benchmark、perf 报告、框架对标、负例诊断和阶段路线图收束成作品集证据。没有这些证据，书稿再厚也只能证明学习过，不能证明能做生产级 AI Infra。

C++20 项目应该怎样分层

本册贯穿项目用 C++20 实现时，工程目录不能堆成一个巨大的 `engine.cpp`。一个保守、可维护的分层可以这样设计：

Listing 1.1 C++20 AI inference infra 的建议模块边界

```
include/ai/
  model/      manifest, tensor index, tokenizer contract
  graph/      graph IR, tensor descriptors, verifier facts
  compile/    passes, fusion, memory planner, lowering
  runtime/    session, KV cache, arena, scheduler, sampler
  backend/
    cpu/      packed weights, SIMD kernels, thread scheduler
    gpu/      device buffers, streams, kernel adapters
  test/      reference, oracle, profiler report schema

src/
  model/
  graph/
  compile/
  runtime/
  backend/cpu/
  backend/gpu/
  test/
```

这些目录不是形式主义。每个目录对应一组不变量：

- **model/** 只把外部文件变成可信 manifest，不选择 kernel。
- **graph/** 表达语义和 shape/dtype 合同，不知道 AVX 寄存器有多少个。
- **compile/** 做 pass、memory plan 和 lowering，不直接持有某次用户请求的 KV cache。
- **runtime/** 绑定 session 状态并调度 executable plan，不重新解释模型文件。
- **backend/cpu/** 和 **backend/gpu/** 接收已经验证过的 descriptor，输出可测的 kernel 结果和 profiler 事件。

C++20 代码应该优先使用明确所有权和轻量视图：权重文件用 RAII 对象管理映射或加载缓冲，张量视图用 `std::span` 和 descriptor 表达，错误用明确结果类型返回，pass 用小对象或函数组合，热路径避免字符串查表、重复分配和不必要拷贝。后续写到实现时，类成员访问、常量命名、无递归遍历、模块分解和测试覆盖都要按同一套工程纪律执行。

后端优化的总路线

后端优化不能从“写 SIMD”开始。正确路线是先建立 reference，再建立可测 baseline，然后每次只改变一个机器事实。

1. Reference：写最清晰的标量实现，定义正确输出和误差边界。
2. Baseline kernel：用普通连续循环实现，记录吞吐、延迟、内存峰值和 profiler 事件。
3. Layout 修正：确认 shape、stride、对齐、尾块和 cache line 行为。

4. Packing: 把权重从文件布局变成 kernel 友好布局, 并记录 packed descriptor。
5. SIMD micro-kernel: 明确向量宽度、累加器数量、unroll、tail path 和精度。
6. Thread scheduling: 决定按行、列、token、head 还是 layer 切分任务, 处理 barrier 和 false sharing。
7. NUMA/cache/TLB: 控制内存放置、预取、页行为、线程亲和和大页策略。
8. Runtime integration: 把 kernel 接进 prefill/decode plan, 不让单算子优化破坏整体延迟。
9. Oracle/profiler: 每一步都比较误差和性能, 不能只看最终 tokens/s。

这条路线会贯穿后续内容。只要某个优化不能说明它改变了哪条地址流、哪个生命周期、哪个同步点或哪个后端 layout, 它就还不是一个合格的工程优化。

本节验收线

读完本节, 应该先建立全书地图:

- 编译器的核心是理解语义、建立中间表示、检查不变量、在语义不变下优化机器执行。
- AI 编译器把模型资产、张量合同、量化元数据、推理图和后端能力编成 executable plan。
- 推理引擎不是单个 kernel, 而是模型资产层、编译前端层、IR 优化层、后端层和运行时层的组合。
- C++20 项目要按 model、graph、compile、runtime、backend、test 分层, 避免大文件和跨层猜测。
- 后端优化必须按 reference、baseline、layout、packing、SIMD、线程、NUMA、runtime、oracle/profiler 的顺序建立证据。

后面的章节都要回到这张地图。读者即使零基础, 也可以按“这一章属于哪一层、建立哪个合同、优化哪条机器账本、怎样验证”这四个问题往下读。量化相关章节还要多问一个问题: 低比特表示是否同时保持了数值质量、metadata 一致性和后端吞吐。

1.2 端到端链路: 从模型文件到下一个 token

先把一条请求链路钉住

第三册后续会讲很多对象: manifest、typed graph、量化、KV cache、CPU/GPU backend、oracle、profiler。若不先把它放进同一条请求链路, 读者很容易学到一堆孤立概念。这里先用一个最小对话请求把全链路钉住: 用户输入一段 prompt, 引擎加载模型, 编译计划, 执行 prefill, 进入 decode 循环, 最后不断产生 token。

Listing 1.2 本册贯穿链路的最小形态

```
model directory
-> manifest
-> verified typed graph
-> quantization and layout plan
-> CPU/GPU executable plan

request prompt
-> tokenizer
-> prefill plan
-> KV cache initialized
-> logits
-> sampler
-> decode plan loop
-> next token stream
```

这条链路有两个方向。第一条是“模型到计划”: 它尽量在请求前完成, 把能检查的语义、shape、dtype、量化、layout 和 kernel 选择提前固定。第二条是“请求到 token”: 它发生在 runtime 中, 绑

定 session、token buffer、KV cache、arena、workspace 和 sampler。把这两条线分开，是推理引擎不退化成图解释器的关键。

核心思想

本册的主线不是“写几个快 kernel”，而是把模型资产编成可执行计划，再把计划安全地绑定到一次会增长的请求状态上。优化只能发生在这条链路上的某个明确位置。

从一个坏的 `generate()` 看边界

最容易写出的推理程序，是一个巨大的 `generate()` 函数：读模型、解析 tokenizer、分配 activation、判断量化格式、选择 kernel、采样 token 全挤在一个循环里。

Listing 1.3 不可维护的推理主循环形态

```
generate(prompt):
    model = read_files()
    tokens = tokenize(prompt)
    for step in 0..max_new_tokens:
        for layer in model.layers:
            if layer.weight_format == "q4":
                unpack_and_run_q4()
            else:
                run_f32()
            maybe_allocate_more_memory()
            maybe_move_to_gpu()
        next = sample(logits)
        tokens.push_back(next)
```

问题不在于代码短，而在于它把四类生命周期混在一起：模型文件和权重属于模型生命周期；packed weight 和 executable plan 属于编译生命周期；token buffer、KV cache 和 sampler 属于请求生命周期；单个 kernel 的 workspace 和 profiler event 属于算子生命周期。边界混乱以后，错误定位、内存复用和性能测量都会失真。

生命周期	应该保存的对象	不应该做的事
模型生命周期	文件索引、manifest、原始权重视图	持有某次请求的 token 状态
编译生命周期	typed graph、packed weight、back-end plan	每个 token 重新解析模型结构
请求生命周期	session、KV cache、sampler、trace	修改模型 manifest 或后端 capability
算子生命周期	局部 workspace、临时 tile、计数器	泄漏到下一层或下一次请求

常见误区

推理引擎里很多难查的问题，本质上不是数学错，而是生命周期错：静态对象在热路径重复构造，请求对象被全局共享，临时 buffer 被跨 step 使用，或者 profiler 把 prepare 成本算进 decode。

模型到计划：哪些事情应该提前做

模型加载阶段不应该只读文件。它要把外部资产变成编译器可以相信的事实。

阶段	产物	拦住的问题
读取资产	raw tensor index、config、tokenizer	文件缺失、大小不一致、checksum 错
manifest	canonical tensor roles	模型方言、权重命名、tied weight
verifier	typed graph 与诊断	shape/dtype/quant/KV 合同错误
优化与 lowering	executable plan	融合、layout、memory、backend 选择
prepare	packed weight、device buffer	热路径重复 packing 或上传

这张表的核心是“早拒绝”。如果 tokenizer 词表大小和 embedding 行数不一致，应该在前端报错；如果 int4 scale 数量不匹配，应该在量化 verifier 报错；如果 GPU backend 不支持某种低比特格式，应该在 plan 阶段显式 fallback 或拒绝。不要等到 decode 第 128 个 token 时才用错误 logits 暴露问题。

请求到 token：哪些事情只能运行时知道

运行时不是编译器的替代品。它处理的是请求期才知道的状态：prompt 内容、当前 position、最大生成长度、sampler 参数、KV cache 已写入长度、当前选择的 backend、是否打开 oracle/profiler。

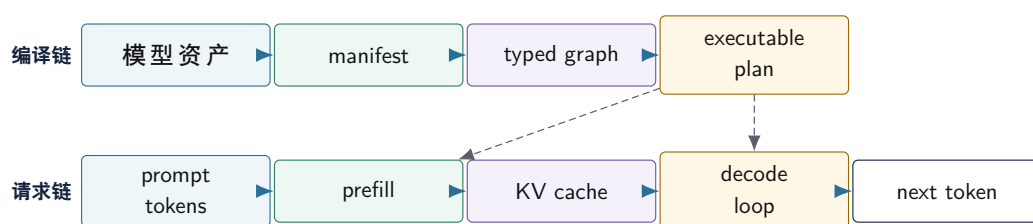
Listing 1.4 一次请求的运行期状态

```
Session:
  token_buffer
  current_position
  kv_cache
  activation_arena
  workspace
  sampler_state
  profiler_trace
  oracle_policy
```

prefill 和 decode 的职责不同。prefill 读取整个 prompt，批量写入 KV cache，并产出最后一个 prompt token 的 logits。decode 每次只处理一个新 token，读取历史 KV cache，追加当前位置的 K/V，再产出下一步 logits。很多后端优化之所以失败，是因为把这两种形态混成一个计划。

同一个算子名字在不同阶段不一定是同一种机器工作。QKV projection 在 prefill 里更像大 GEMM，在 decode 里更像小 batch GEMV；attention 在 decode 里还要读取从 0 到当前 position 的历史 K/V；sampler 若每步强制 GPU 同步并拷回完整 logits，也会进入端到端延迟。

模型编译链路与请求执行链路



图示：本图要证明的命题是：模型编译链和请求执行链共享计划，但生命周期和错误边界不同。

计划不是抽象名词，而是一份可执行合同

本册反复使用 **可执行计划**（executable plan）这个词。它不是架构口号，而是一份后端和运行时都能执行的合同：语义 segment、kernel 选择、layout、packed weight、workspace、KV cache 读写、profiler event 和 fallback 规则都要落在这里。

Listing 1.5 executable plan 的最小合同字段

```
ExecutablePlan:
  segments:
    id
    operator_or_fused_region
    input_values
    output_values
    state_effects: reads_kv | writes_kv | none
    backend_choice
    kernel_descriptor
    workspace_bytes
    profiler_label
    oracle_checkpoint_policy
```

这里的 **state_effects** 很重要。decode attention 会读历史 KV cache，同时写入当前位置的 K/V；sampler 会读 logits 并更新随机数状态。若 IR 和 plan 不表达这些状态边，memory planner 可能错误复用 buffer，profiler 可能漏掉 cache 写入，oracle 也无法定位漂移点。

一个真实业务接口长什么样

说“能做业务开发”，不能只说能生成文本。最小可用接口要把请求参数、资源预算、流式输出、停止原因和诊断 trace 都暴露出来。一个本地推理服务的请求可以先设计成：

Listing 1.6 GenerateRequest 的最小字段

```
GenerateRequest:
  request_id
  model_id
  prompt
  max_new_tokens
  max_context_tokens
  temperature
  top_k
  top_p
  repetition_penalty
  stream: true | false
  quant_profile_id
  backend_policy: cpu | gpu | auto
  trace_level: none | summary | debug
```

这些字段不是接口装饰。**max_context_tokens** 决定 KV cache 容量；**max_new_tokens** 决定最坏 decode 时长；采样参数决定结果可复现性；**quant_profile_id** 决定误差和性能合同；**backend_policy** 决定 CPU/GPU plan 选择；**trace_level** 决定是否允许 materialize 中间结果和更细 profiler。

响应也不能只是一个字符串。流式输出至少要区分 token 事件、错误事件和结束事件：

Listing 1.7 流式响应事件

```
TokenEvent:
```

```

request_id
token_id
token_text
index
model_position
finish_reason_optional
trace_event_id_optional

FinalResponse:
request_id
text
generated_tokens
finish_reason: stop_token | max_tokens | context_limit | cancelled | error
usage:
    prompt_tokens
    completion_tokens
    kv_cache_bytes
trace_summary_optional

```

这样设计后，业务层可以实现超时、取消、计费、审计、问题定位和重试。若接口只返回一个最终字符串，就无法判断卡在 prefill、decode、sampler、KV 容量还是后端 fallback。

流式生成不是 print 循环

流式生成的状态机要明确。prefill 完成之前，通常还没有第一个输出 token；prefill 之后每个 decode step 产生一个候选 token，sampler 决定是否结束，然后服务层把 token 事件推给客户端。

Listing 1.8 流式生成状态机

```

state = admitted
tokens = tokenizer.encode(prompt)

run prefill(tokens)
emit trace: first_token_ready

while not stopped:
    logits = decode_one_token(session)
    next = sampler.sample(logits)
    append next into session
    emit TokenEvent(next)
    if stop_condition(next):
        emit FinalResponse(stop_token)
        break

```

真实服务还要处理取消和背压。客户端断开时，runtime 必须停止后续 decode，并释放该 session 的 KV cache、arena 和 pending events。若下游消费 token 很慢，不能让输出队列无限增长；要设置队列上限，超过后暂停调度、丢弃请求或返回 backpressure 错误。否则模型算得再快，服务也会被 I/O 队列拖垮。

常见误区

流式输出是运行时协议，不是把每个 token 打印出来。它必须和 session 生命周期、KV cache 释放、取消、超时、trace 和 backpressure 绑定。

容量预算和 admission control

业务服务最先撞到的不是“能不能生成一句话”，而是能同时服务多少请求。admission control 的职责是在请求进入 decode 前判断资源是否足够。最小预算包括权重、packed weight、KV cache、activation arena、workspace、队列和系统余量。

Listing 1.9 请求进入前的容量判断

```
required_kv_bytes =
    layers * max_context_tokens * kv_heads * head_dim *
    2 /* K and V */ * kv_bytes_per_element

required_runtime_bytes =
    required_kv_bytes +
    activation_arena_bytes +
    workspace_bytes +
    trace_buffer_bytes
```

若使用前文 7B 级 GQA 参数, `L=32`、`kv_heads=8`、`head_dim=128`、fp16 KV、4096 context, 一个 session 的 KV cache 约 512MiB。若 int4 权重约 3.7GiB, 机器剩余可用内存 10GiB, 理论上不能简单做 10GiB/512MiB。还要扣掉 packed weight、arena、workspace、allocator 碎片、线程栈、页表和操作系统余量。

admission decision 应该结构化:

Listing 1.10 AdmissionDecision 字段

```
AdmissionDecision:
  accepted: true | false
  reason:
    ok | context_too_long | memory_budget_exceeded |
    queue_overloaded | backend_unavailable
  selected_plan_id
  reserved_kv_bytes
  reserved_workspace_bytes
  estimated_ttft_ms
  estimated_decode_tokens_per_s
```

这让业务层能给出明确错误: 上下文太长、内存不够、队列满、GPU 后端不可用, 而不是笼统返回“生成失败”。也让运维能知道是容量问题还是正确性问题。

SLO 要分首 token 和后续 token

推理服务的延迟目标不能只写一个平均响应时间。至少要拆成:

- **TTFT** (time to first token): 从请求进入到第一个 token 可输出。
- decode p50/p95: 每个后续 token 的延迟分布。
- prefill tokens/s: prompt 处理吞吐。
- decode tokens/s: 生成阶段吞吐。
- queue wait: 进入模型前等待了多久。
- memory peak: 请求期间峰值内存或显存。

prompt 很长但输出很短时, TTFT 主要由 tokenizer、prefill、KV 写入和首个 sampler 决定; prompt 较短但输出很长时, decode p95、KV 读流和 sampler 更重要。若服务端合批, 平均 tokens/s 可能上升, 但单请求 p95 可能变差, 因为请求要等待同批次其他 session。

Listing 1.11 业务 SLO 报告字段


```
ServiceSloReport:
  prompt_bucket
  max_context_bucket
  backend_plan_id
  quant_profile_id
  queue_wait_p50_p95
  ttft_p50_p95
  decode_step_p50_p95
  tokens_per_second
  rejection_rate
  memory_peak_bytes
  fallback_count
```

有了这些字段，才谈得上把第三册内容落到一个本地助手、RAG 服务、代码补全服务或批量摘要服务里。不同业务目标会选择不同 profile，但底层证据字段是同一套。

错误也要沿链路定位

链路完整的好处是错误能定位。若生成结果不对，不要直接怀疑 sampler 或模型质量，而要按链路缩小范围：

- tokenizer 输出是否和 reference 一致。
- manifest 中 embedding、lm head、Q/K/V、MLP 权重角色是否正确。
- shape/dtype/quant verifier 是否通过。
- packed weight 与 reference converter 是否一致。
- 单算子 oracle 是否通过。
- 层输出 oracle 从哪一层开始漂移。
- logits top-k 是否变化。
- sampler 固定 seed 后是否仍然分叉。

性能问题也一样。首 token 慢，不等于 decode 慢；decode 慢，不等于所有 kernel 都慢；GPU kernel 时间低，不等于端到端快。profiler 要沿链路拆：load、prepare、prefill、decode、sample、copy、sync、fallback。

用 trace 把链路变成证据

链路如果只存在于图里，仍然不能维护。每次运行都应该能输出一个轻量 trace，让 correctness 和 performance 共享同一套定位坐标。trace 不必一开始很复杂，但字段要稳定。

Listing 1.12 端到端推理 trace 的最小字段

```
RunTrace:
  run_id
  model_id
  manifest_hash
  backend_plan_id
  quant_profile_id
  prompt_tokens
  generated_tokens
  events:
    stage
    segment_id
    layer_id
    backend
    context_length
```

```

start_ns
end_ns
bytes_read_estimate
bytes_written_estimate
oracle_status

```

`manifest_hash` 追溯模型语义; `backend_plan_id` 追溯 kernel、layout 和 fallback; `quant_profile_id` 追溯误差阈值和量化格式; `context_length` 让 decode 指标可分档。trace 坐标稳定, 后续 CPU SIMD、GPU kernel 和 KV cache 量化报告才可比较。

本节验收线

读完本节, 应该能用一条链路解释全书:

- 模型资产先变成 manifest 和 typed graph, 再 lowering 成 executable plan。
- 请求运行时绑定 token、KV cache、arena、workspace、sampler、oracle 和 profiler。
- prefill 与 decode 是两种形态, 不能强行共用一个性能模型。
- 模型、编译、请求和算子生命周期必须分开, 否则内存、状态和测量都会串扰。
- executable plan 要显式记录 segment、backend、workspace、状态读写、oracle checkpoint 和 profiler label。
- 正确性和性能问题都应该沿链路定位, 而不是从最终文本倒猜。
- 端到端 trace 要让模型语义、后端计划、量化 profile、context length 和事件耗时可追溯。

1.3 贯穿项目: 做一个可解释的 C++20 本地推理服务

本册最终要让读者做成什么

如果读完第三册只知道一些名词, 例如 Transformer、KV cache、量化、GEMM、GPU、AI 编译器, 那还不够。本册的贯穿项目要更具体: 做一个受限但严肃的 C++20 本地推理服务原型。它不承诺一开始达到成熟框架的吞吐, 也不承诺支持所有模型格式; 它要做到的是把模型资产、编译检查、推理运行时、后端优化、正确性 oracle 和性能报告接成一条可解释链路。

核心思想

贯穿项目的目标不是“能生成几句话”, 而是“每个 token 为什么这样算、用了多少内存、读了多少权重、KV cache 怎样增长、量化误差从哪里来、性能瓶颈卡在哪一层”都能被追溯。

项目完成后的最小能力如下:

Listing 1.13 第三册贯穿项目的最终能力

```

load:
  read model manifest
  verify tensor shape, dtype, quantization and tokenizer contract
  build typed graph
  lower into CPU/GPU executable plan

run:
  tokenize prompt
  run prefill
  allocate and append KV cache
  decode token by token
  stream token events
  stop, cancel or reject by resource policy

```

```

prove:
  compare reference logits
  compare optimized operator outputs
  report quantization error
  report TTFT, decode latency, memory peak and fallback

```

这个项目能支撑真实业务原型，例如本地问答、RAG 检索增强回答、批量摘要、代码补全或内部文档助手。但业务能力必须建立在底层合同之上：上下文长度要能预算，取消要能释放 KV cache，trace 要能定位慢点，错误要能说明是模型资产、后端能力、量化 profile 还是请求参数的问题。

不是完整框架，但必须完整闭环

一个学习项目不需要一开始覆盖所有工业功能。可以暂缓的东西包括：全部模型方言、分布式推理、复杂动态 batching、PagedAttention 的生产级调度、所有 GPU kernel、所有低比特格式、服务多租户隔离。不能暂缓的是闭环本身。

层次	可以先做受限版本	不能缺失的合同
模型导入	只支持一种模型切片或一种 manifest	shape、dtype、量化、tokenizer 必须验证
推理数学	先跑小模型或一层切片	reference logits 必须可复现
CPU 后端	先标量，再 SIMD，再线程	operator oracle 和 profiler 必须存在
GPU 后端	先建立 device/stream/fallback 合同	copy、launch、sync 账本必须可见
服务接口	先单进程 HTTP 或 CLI	admission、取消、trace、finish reason 必须明确

闭环的含义是：输入资产先被验证，运行过程能被 trace，输出结果能被 oracle 比较，性能数字能回到具体阶段。若只写一个 `generate(prompt)`，即使它能输出文本，也无法回答“为什么慢”“为什么漂移”“为什么显存涨了”“为什么换量化格式后质量变差”。

可执行交付物

贯穿项目最终应该形成四类交付物。

命令行工具。 命令行最适合教学和回归。它不需要复杂服务依赖，容易固定输入和输出。

Listing 1.14 贯穿项目命令行工具

```

ai-inspect-model --model ./model
ai-compile-plan --model ./model --backend cpu --quant q4_group64
ai-generate --model ./model --prompt prompts/basic.txt --max-new-tokens 64
ai-bench --model ./model --prompt-set prompts/bench.jsonl
ai-oracle --fixture fixtures/tiny_decoder.json

```

`ai-inspect-model` 只读模型资产并输出 manifest 诊断；`ai-compile-plan` 只做 verifier、IR 和 lowering；`ai-generate` 跑端到端请求；`ai-bench` 输出性能报告；`ai-oracle` 对比 reference。把这些命令拆开，读者才能知道问题发生在哪一层。

本地服务。 服务形态可以先是单进程 HTTP 或本地 RPC。它不追求复杂微服务体系，但请求和响应字段必须严肃。

Listing 1.15 本地推理服务接口

```

POST /v1/generate
  model_id
  prompt

```

```

max_new_tokens
max_context_tokens
temperature
top_k
top_p
backend_policy
quant_profile_id
stream
trace_level

stream event:
  token | final | error | trace

```

业务层需要的不是一句“模型生成失败”。它要知道是上下文超限、队列满、内存预算不足、后端不可用、量化 profile 不支持，还是运行时被取消。服务接口因此必须和 admission control、session、KV cache、trace 和错误诊断绑定。

报告文件。 每次运行都应该能输出机器可读报告。报告让优化可以被比较，而不是靠主观体感。

Listing 1.16 贯穿项目报告文件

```

reports/
  manifest-report.json
  plan-report.json
  oracle-report.json
  trace.jsonl
  benchmark-report.json
  release-report.json

```

manifest-report 说明模型资产是否可信；**plan-report** 说明每个 segment 选择的 backend、layout 和 workspace；**oracle-report** 说明误差；**trace** 记录运行事件；**benchmark-report** 记录性能；**release-report** 记录当前版本支持范围和已知限制。

测试夹具。 没有固定 fixture，就无法形成教学和回归。夹具应从小到大：

Listing 1.17 测试夹具分层

```

fixtures/
  tokenizer_tiny/
  tensors_layout/
  quant_bytes/
  tiny_rmsnorm/
  tiny_linear/
  tiny_rope_attention/
  tiny_decoder_block/
  tiny_end_to_end/
  7b_shape_only/

```

tiny_end_to_end 可以只包含极小参数和固定 token，不需要真实 7B 权重；**7b_shape_only** 用来验证真实规模 shape、KV 容量和 plan，不必包含完整权重。这样项目既能快速测试数学正确性，也能训练读者面对真实 7B 规模的资源账本。

建议目录结构

C++20 项目不能把所有东西塞进一个大文件。下面是一套随章节增长的目录结构。

Listing 1.18 贯穿项目目录结构

```

include/ai/
  model/          manifest, tensor index, tokenizer contract
  graph/          tensor desc, graph node, verifier facts
  quant/          quant desc, converter, calibration record
  compile/        pass pipeline, memory plan, lowering
  runtime/        session, KV cache, arena, sampler, scheduler
  backend/
    cpu/          capability, packed weight, kernels, thread plan
    gpu/          device buffer, stream, event, workspace, fallback
  oracle/         reference, comparison, error report
  report/         trace schema, benchmark schema, release schema

src/
  model/
  graph/
  quant/
  compile/
  runtime/
  backend/cpu/
  backend/gpu/
  oracle/
  report/

tools/
  inspect_model/
  compile_plan/
  generate/
  bench/
  oracle/

tests/
  model/
  graph/
  quant/
  runtime/
  backend/cpu/
  backend/gpu/
  oracle/

```

这些目录对应的是责任边界。模型导入不选择 SIMD kernel；后端 kernel 不解析 tokenizer；runtime 不重新做 graph verifier；oracle 不复用被测后端的可疑路径；report 只描述事实，不偷偷改变执行。目录边界清楚，后面扩 GPU、量化、服务接口时才不会互相污染。

核心数据结构合同

项目的核心不是类名，而是这些对象之间的合同。下面的伪代码只表达字段，不要求读者照抄实现。

Listing 1.19 模型与图的核心合同

```

TensorDesc:
  name

```

```

    role
    shape
    dtype
    layout
    quant_desc_optional
    file_range

OperatorDesc:
    id
    op_kind
    inputs
    outputs
    attributes
    state_effects

ModelManifest:
    model_id
    tokenizer_contract
    tensor_table
    layer_count
    hidden_size
    head_count
    kv_head_count
    rope_config

```

TensorDesc 连接文件、shape、dtype、layout 和量化；**OperatorDesc** 表达算子语义与状态读写；**ModelManifest** 记录模型级合同。后端只应该接收已经验证过的 descriptor。

Listing 1.20 运行时与后端的核心合同

```

ExecutableSegment:
    id
    op_or_fused_region
    backend
    input_values
    output_values
    state_effects
    kernel_desc
    workspace_bytes
    profiler_label

SessionState:
    request_id
    token_buffer
    model_position
    kv_cache
    activation_arena
    sampler_state
    trace

BackendResult:
    status
    output_values

```



```
profiler_events
fallback_reason_optional
```

ExecutableSegment 是编译器和后端的接口；**SessionState** 是服务请求和 runtime 的接口；**BackendResult** 是后端和 profiler/oracle 的接口。若这些对象缺失，工程就会靠隐式全局状态和临时约定运行，后面很难定位错误。

一条请求的业务级状态机

业务开发必须把生成过程做成状态机，而不是阻塞函数。

Listing 1.21 推理请求状态机

```
received
-> validated
-> admitted
-> tokenized
-> prefill_running
-> decoding
-> finishing
-> completed

failure edges:
rejected
cancelled
timed_out
backend_failed
oracle_failed_debug_only
```

每条边都有资源语义。进入 **admitted** 时要预留 KV cache 和 workspace；进入 **prefill_running** 后要保证 plan 已准备好；进入 **decoding** 后每个 step 会追加 token 和 K/V；进入 **cancelled** 或 **timed_out** 必须释放 session 资源；进入 **completed** 要写 usage 和 trace summary。

状态	持有资源	必须记录的事实
admitted	KV 预算、队列 slot	接受原因、估计内存、计划 id
prefill	activation arena、workspace	prompt tokens、TTFT 分解
decoding	KV cache、sampler state	step latency、context length、finish reason
finishing	trace buffer、usage	token 数、峰值内存、fallback
cancelled	待释放资源	取消来源、已释放字节数

这张表直接回答“能不能做实例业务开发”。能，但前提是业务服务不是只调用一个黑箱函数，而是理解请求状态、资源预算、取消、错误和 trace。

第一个可运行版本的验收

第一个版本可以很小，但验收要严。

验收与评分标准

- 给定一个 tiny manifest，**ai-inspect-model** 能输出 tensor 表、shape 和 tokenizer 合同。
- 故意改坏 embedding 行数，verifier 能报出 tokenizer vocab 与 embedding shape 不匹配。
- 给定 tiny decoder fixture，reference 能输出固定 logits。

- `ai-generate` 能跑 prefill 和至少 8 个 decode step，并输出 token event。
- 超过 context limit 的请求被 admission control 拒绝，错误里包含实际 token 数和允许上限。
- 打开 trace 后，报告里能看到 tokenize、prefill、decode、sample 的耗时。
- 取消请求后，KV cache 预留字节数回到请求前。

如果这些验收都通过，项目已经不是玩具打印程序。它具备业务服务最小骨架，也具备继续扩 CPU SIMD、量化、GPU 和框架对标的证据基础。

贯穿项目和后续内容怎样对应

后续内容不是散讲。每一节都落到贯穿项目的某个对象。

主题	项目对象	验收证据
张量布局 GEMM/SIMD	<code>TensorView</code> 、 <code>TensorDesc</code> <code>PackedWeight</code> 、 <code>MicroKernelDesc</code>	地址公式、stride 测试 operator oracle、latency
量化	<code>QuantDesc</code> 、 <code>QuantConverter</code>	byte oracle、误差报告
KV cache AI 编译器	<code>KVCache</code> 、 <code>KVCacheDesc</code> <code>GraphNode</code> 、 <code>ExecutableSegment</code>	position 测试、容量报告 verifier、plan report
CPU/GPU 后端	<code>BackendResult</code> 、 <code>StepTrace</code>	profiler、fallback report
工程验收	<code>ReleaseReport</code> 、 <code>PerformanceGate</code>	CI、对标、回归门禁

读者读到任何小节，都应该能问三个问题：这个对象在项目里叫什么，谁生产它，谁消费它，怎样测试它。回答不了这三个问题的内容，不应该进入贯穿项目主线。

本节验收线

读完本节，应该明确第三册不是泛泛介绍 AI，而是围绕一个可解释本地推理服务推进：

- 最终交付物包括 CLI、本地服务、报告文件和测试夹具。
- 项目边界包括模型导入、typed graph、量化、runtime、CPU/GPU 后端、oracle 和 profiler。
- 业务接口必须包含 admission、取消、finish reason、usage 和 trace。
- 最小版本可以受限，但 manifest、reference、KV cache、trace、错误诊断和资源释放不能缺失。
- 后续内容所有概念都要落到具体对象、生产者、消费者和验收证据。

1.4 业务服务实战：请求、调度、队列、取消与资源预算

业务开发不是把 `generate()` 暴露成接口

很多本地大模型 demo 的接口只有一个输入 prompt 和一个输出 text。它能演示模型效果，却不能承载真实业务。业务开发要面对的问题更具体：多个用户同时请求怎么办，长 prompt 是否允许，用户断开后是否继续烧算力，队列满了怎样拒绝，CPU/GPU 后端不可用怎样降级，怎么知道慢在排队、prefill、decode 还是 sampler。

核心思想

推理服务的核心对象不是一个字符串函数，而是请求状态机、资源预留、调度策略、session 生命周期、流式事件和可追溯 trace。没有这些对象，模型能输出文本也不能算可交付业务服务。

本节把贯穿项目放到服务视角。目标是设计一个最小但完整的本地推理服务：它接收请求，验

证参数，估算资源，决定是否接纳，运行 prefill 和 decode，流式返回 token，处理取消和超时，最后输出 usage 与 trace。

请求字段要表达资源和语义

一个业务接口必须把会影响资源、质量和延迟的字段显式暴露出来。

Listing 1.22 GenerateRequest 的业务字段

```
GenerateRequest:
  request_id
  user_id_optional
  model_id
  prompt
  max_new_tokens
  max_context_tokens
  temperature
  top_k
  top_p
  repetition_penalty
  stop_token_ids
  stream
  timeout_ms
  backend_policy
  quant_profile_id
  trace_level
```

max_context_tokens 不是普通参数，它决定 KV cache 上限；**max_new_tokens** 决定最坏 decode 步数；**timeout_ms** 决定调度器能否中止；**backend_policy** 决定 CPU/GPU plan 选择；**quant_profile_id** 决定质量和容量；**trace_level** 决定是否允许记录更多中间证据。

请求验证要分两层。第一层是 API 参数验证，例如 **top_p** 是否在合法范围、**max_new_tokens** 是否为正。第二层是模型相关验证，例如请求的 context 是否超过模型最大上下文，选择的量化 profile 是否存在，后端是否支持该 profile。不要把这两类错误都混成 **badrequest**。

响应也要表达停止原因

响应不能只返回文本。最小响应应该能告诉业务层为什么结束。

Listing 1.23 GenerateResponse 的业务字段

```
TokenEvent:
  request_id
  index
  token_id
  text_fragment
  model_position
  trace_event_id_optional

FinalEvent:
  request_id
  finish_reason:
    stop_token | max_tokens | context_limit |
    timeout | cancelled | backend_error | internal_error
  prompt_tokens
  completion_tokens
  kv_cache_bytes
```

```

prefill_ms
decode_ms
trace_summary_optional

```

业务逻辑会依赖 `finish_reason`。若是 `max_tokens`，可以提示用户继续生成；若是 `context_limit`，应该让用户缩短输入；若是 `timeout`，可能需要重试或调小输出长度；若是 `backend_error`，要看 fallback 和后端日志。没有停止原因，服务层只能猜。

Admission control: 先预算，再接纳

服务端最重要的保护机制是 admission control。它在请求进入模型前决定是否接纳。最小预算包括 KV cache、activation arena、workspace、trace buffer 和队列 slot。

Listing 1.24 AdmissionRequest 字段

```

AdmissionRequest:
  model_id
  prompt_tokens
  max_context_tokens
  max_new_tokens
  backend_policy
  quant_profile_id
  trace_level

```

决策输出要结构化：

Listing 1.25 AdmissionDecision 字段

```

AdmissionDecision:
  accepted
  rejection_reason_optional:
    context_too_long
    memory_budget_exceeded
    queue_full
    backend_unavailable
    quant_profile_unsupported
  selected_plan_id
  reserved_kv_bytes
  reserved_workspace_bytes
  estimated_ttft_ms
  estimated_decode_tokens_per_s

```

一个常见错误是按当前 prompt 长度分配 KV cache，而不是按请求允许的最大上下文预算。若 prompt 是 500 token，`max_new_tokens` 是 3500，最终可能增长到 4000 token。admission 若只看 500，会在 decode 中途撞内存，业务体验比直接拒绝更差。

队列：排队时间也是用户延迟

推理服务中的队列至少有两类：等待进入模型的请求队列，以及流式输出事件队列。二者都必须有限。

队列	风险	控制方式
请求队列	排队时间吞掉 SLO	上限、优先级、超时、拒绝
decode ready 队列	某些 session 饿死	公平调度、step budget
流式事件队列	客户端慢导致内存膨胀	backpressure、断开检测
trace 队列	debug 模式产生日志洪峰	采样、等级、大小上限

排队时间要进入 trace。否则用户看到 TTFT 很慢时，工程师可能错误优化 prefill kernel，而真正的问题是请求在队列里等了 800ms。

调度粒度：一次跑完整请求还是一 token 一调度

最简单的调度方式是一个请求跑到结束再处理下一个请求。它容易实现，但对长输出请求不公平：一个用户生成 2048 token，会阻塞其他短请求。更合理的最小策略是 step-level 调度：每个活跃 session 每次执行一个或几个 decode step，然后让出。

Listing 1.26 step-level decode 调度

```
while active_sessions not empty:
    session = pick_next_session()
    if session.need_prefill:
        run_prefill(session)
    else:
        run_decode_step(session)
        emit_token_event(session)

    if session.finished:
        release_resources(session)
    else:
        requeue(session)
```

这不是完整动态 batching，但已经把“一个请求独占模型”的问题拆开。后续可以在 `pick_next_session` 和 `run_decode_step` 之间加入小批量合并：把多个 session 的当前 token 合成 batch，执行同一层 decode，再拆回各自 session。合批会提高吞吐，也会引入等待和复杂状态，因此必须由 trace 报告收益和 p95 代价。

prefill 调度和 decode 调度不同

prefill 通常更重，且 prompt 长度差异大。一个长 prompt prefill 可能占用较长时间，影响其他请求 TTFT。decode 每步较短，但会重复很多次。调度器至少要区分：

- 新请求的 prefill 是否允许插队。
- 长 prompt 是否拆块 prefill。
- decode session 是否按公平轮转。
- 是否允许为了 batch 吞吐牺牲单请求 p95。
- CPU 和 GPU 后端是否使用不同队列。

最保守的实现可以先不做复杂合批，但要把调度决策记录下来：

Listing 1.27 SchedulerEvent 字段

```
SchedulerEvent:
    request_id
    state
    queue_wait_ns
    selected_backend
    batch_size
```

```
prompt_tokens
context_length
reason
```

这样后续优化调度时，能判断吞吐提升是否以 p95 变差为代价。

取消和超时：资源释放是合同

用户关闭页面、网络断开、业务层超时，都应该触发取消。取消不是设置一个标志就完事。runtime 必须让正在运行的 session 到达安全点，然后释放资源。

Listing 1.28 取消处理的资源清单

```
on_cancel(request_id):
    mark session cancel_requested
    stop scheduling new decode steps
    drain or discard stream events
    release KV cache reservation
    release activation arena
    release workspace
    write final event with cancelled
    write trace summary
```

如果正在 GPU kernel 中，不能随意销毁底层 buffer。可以把取消点放在 step 边界：当前 kernel 完成后不再调度下一步。CPU 后端也类似，线程池任务要检查取消标志，但不能在持有共享结构时直接抛出异常导致资源计数不平衡。

常见误区

取消路径必须有测试。只测正常完成，会漏掉最真实的服务问题：客户端断开后继续生成、KV cache 不释放、事件队列持续增长、trace 文件缺 final event。

流式输出与 backpressure

流式 token 事件通常比模型计算慢得多，也可能被网络、客户端渲染或上游代理阻塞。服务端不能让事件队列无限增长。最小策略：

- 每个 session 设置事件队列上限。
- 队列满时暂停该 session 的 decode 调度。
- 超过最大阻塞时间后取消请求或返回 backpressure 错误。
- 客户端断开时立即进入取消路径。

backpressure 要进入调度器，而不是只在 I/O 层处理。否则模型会继续生成 token，I/O 层只是堆积事件，内存仍然失控。

错误分类：给业务可处理的错误

错误要分层返回。下面是一套最小分类：

错误类别	例子	业务处理
请求错误	参数范围错、prompt 为空	直接提示用户修改
容量错误	context 超限、队列满、内存不足	稍后重试或缩短输入
模型错误	manifest 不合法、tokenizer 不匹配	运维修模型资产
后端错误	GPU 不可用、dtype 不支持	fallback 或切 CPU
运行时错误	KV 越界、arena 冲突	记录 trace，阻断发布

不要把所有错误都变成 500。业务系统要根据错误类型决定重试、降级、提示、告警还是阻断部署。

服务级 SLO 报告

服务的 SLO 不只是一条平均延迟。最小报告：

Listing 1.29 ServiceSloReport 字段

```
ServiceSloReport:
  model_id
  backend_plan_id
  quant_profile_id
  prompt_bucket
  context_bucket
  request_count
  rejection_rate
  queue_wait_p50_p95
  ttft_p50_p95
  decode_step_p50_p95
  tokens_per_second
  cancel_count
  timeout_count
  memory_peak_bytes
  fallback_count
```

这份报告能指导业务选择。例如 RAG 问答更关注 TTFT 和长 prompt prefill；代码补全更关注短 prompt 低延迟 decode；批量摘要更关注吞吐；离线任务可以接受更高延迟换取更大 batch。不同业务可以使用同一引擎，但 SLO 和调度策略不同。

本节验收线

读完本节，应该能把第三册项目接成业务服务：

- 请求和响应字段能表达资源、采样、后端、量化、trace 和停止原因。
- admission control 在请求进入模型前预算 KV、workspace、arena 和队列。
- 调度器区分 prefill 与 decode，并能记录 queue wait、batch size 和 context。
- 取消、超时和客户端断开都能释放 KV cache 与事件队列。
- backpressure 会影响调度，而不是只在 I/O 层堆积。
- 错误分类能让业务决定重试、降级、提示或告警。
- SLO 报告拆开 queue、TTFT、decode、内存、fallback 和拒绝率。

第 2 章

张量、算子、量化与计算账本

2.1 从真实本地大模型推理引擎开始

第三册的主线不是玩具模型

AI 推理最容易被讲成一组名词: tensor、operator、layer、quantization、GEMM、SIMD、attention、KV cache、runtime、scheduler。名词本身没有错, 但如果没有机器账本, 它们很快会变成框架文档的索引。第三册的主线必须更具体: 写出一台本地推理引擎, 让它能加载开源 7B 左右 decoder-only Transformer 权重, 把 prompt 分词成 token id, 完成 prefill 和 decode, 在 CPU 后端先跑通, 再逐步优化, 并且让同一套 runtime 能挂接 GPU 后端。

一个真实请求不是“调用模型”四个字, 而是下面这条链路:

Listing 2.1 真实本地大模型推理请求的主链路

```
prompt text
-> tokenizer.encode()
-> token embedding
-> repeated transformer blocks
    RMSNorm
    Q/K/V projection
    RoPE
    attention with KV cache
    output projection
    MLP gate/up/down projection
-> final norm and lm_head
-> sampler
-> next token
-> append token and repeat decode
```

这条链路里有两种完全不同的性能形态。prefill 阶段一次处理 prompt 中的一串 token, 很多投影和 MLP 子层更像 GEMM; decode 阶段每次只追加一个 token, 许多地方退化成 GEMV 或小 batch GEMM, 同时还要读取越来越长的 KV cache。若不区分这两种阶段, 就无法解释为什么同一个模型在长 prompt、短回复、长上下文和 batch 服务下会表现不同。

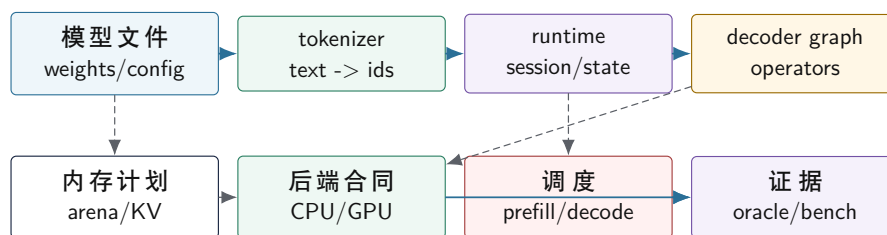
核心思想

第三册的贯穿项目是一台真实本地大模型推理引擎。最小 MLP 只是第一块显微镜切片, 用来手工检查张量、矩阵乘、量化和 oracle; 它不是本册目标, 也不是最终架构。

先画清楚引擎边界

一台本地大模型引擎至少有七个边界。第一, 模型文件边界: 权重、shape、dtype、量化参数、层数、hidden size、head 数、词表大小和 tokenizer 配置必须能被一致读取。第二, runtime 边界: session 保存当前 token 序列、position、KV cache、临时 activation buffer 和采样状态。第三, 算子边界: RMSNorm、matmul、RoPE、attention、softmax、MLP、sampler 都要有独立语义合同。第四, 后端边界: 同一个算子可以由 CPU kernel 或 GPU kernel 执行, 但输入输出合同不能变化。第五, 内存边界: 权重、packed weight、activation、KV cache 和后端工作区有不同生命周期。第六, 调度边界: prefill、decode、batch、线程池、GPU stream 和拷贝同步不能混成一团。第七, 证据边界: 正确性、误差、性能、峰值内存和对标对象必须写进报告。

真实推理引擎的七个边界



本图要先证明一件事：大模型推理引擎不是一个 kernel，而是模型表示、状态、内存、后端、调度和证据共同组成的系统。

图示：七个边界分别约束文件、文本、运行状态、算子语义、内存生命周期、执行后端和验收证据。

这张图也解释了为什么第三册不能只讲 CPU。CPU 是第一条完整落地路径，因为它最适合把地址流、cache、SIMD、多线程、NUMA 和反汇编讲清楚；GPU 是第二条后端路径，因为真实本地 7B 推理经常需要显存带宽和矩阵吞吐。两条路径共享同一份模型语义和算子合同，只是在 kernel 实现、内存驻留、调度同步和 profiler 证据上不同。

用最小切片建立显微镜

虽然主线是真实大模型引擎，第二章仍然从一个极小 MLP 切片开始。原因不是降低目标，而是建立显微镜。一个只有线性层、bias、activation、量化权重和 argmax 的模型，可以把推理引擎中最重要的几件事压缩到一页纸上：

Listing 2.2 最小切片的推理语义

```
hidden = relu(input * W0 + b0)
logits = hidden * W1 + b1
class_id = argmax(logits)
```

这段伪代码看起来只是在做矩阵和向量运算，但机器实际看到的是输入向量的连续字节、权重矩阵的布局、bias 的加载、乘加累加器、激活函数的分支或 `max` 指令、输出缓冲的写入，以及最终 `argmax` 的比较序列。若模型使用 int8 权重量化，核心问题又会变成：8 位权重怎样解码，输入 scale 和权重 scale 怎样组合，累加器为什么通常使用 32 位，重新量化时怎样舍入和饱和。

这些问题在 7B 模型里不会消失，只会被放大。MLP 切片中的 `input*W0` 对应 Transformer 里的 Q/K/V projection、output projection 和 MLP gate/up/down projection；`hidden` 对应 activation buffer；量化权重对应数十亿参数的压缩格式；oracle 对应每层输出、logits 和生成 token 的误差验证。后续内容会不断把显微镜下的局部机制接回真实大模型引擎。

张量不是数组别名

tensor（张量）可以先理解为“带形状的多维数据视图”，但这个说法还不够。对推理引擎来说，张量至少包含五类信息：

- 元素类型：例如 `float`、`std::int8_t`、`std::int32_t`、`bf16`、`fp16`。
- 形状：每个维度有多少元素，例如 `[tokens, hidden]`、`[heads, seq, head_dim]`。
- 步长：相邻维度在内存中跨多少元素或字节。
- 量化元数据：scale、zero point、group size、per-channel 或 per-group 参数。
- 所有权和驻留位置：CPU 内存、GPU 显存、mmap 权重、临时 arena 或 KV cache。

连续的一维数组只是最简单的张量。二维矩阵若按行主序存储，访问 `A[i,j]` 常接近：

Listing 2.3 行主序矩阵的地址计算

```
address = base + (i * stride_row + j) * sizeof(element)
```

若矩阵被转置视图、切片视图或 padding 后的视图引用，`stride_row` 和 `stride_col` 就不再等于直觉中的宽度。算子如果假设所有输入都连续，就可能在正确性或性能上出错；算子如果完全支持任意 stride，又可能让热循环多出复杂地址计算。工程实现必须明确边界：哪些 kernel 要求 contiguous，哪些路径负责转换布局，转换成本是否被计入测量。

在大模型推理里，张量还会变成状态。KV cache 不是普通临时张量：它跨 decode 步骤持续存在，每生成一个 token 都要向每层的 K/V 区域追加一行，并在后续 attention 中被重新读取。它的 shape、stride、容量、有效长度和后端驻留位置都属于模型正确性与性能合同。

算子合同：先定义结果，再谈 kernel

operator（算子）是有输入合同和输出合同的计算单元。以矩阵乘为例，语义可以写成：

Listing 2.4 矩阵乘 reference 语义

```
for m in 0..M:
  for n in 0..N:
    acc = 0
    for k in 0..K:
      acc += A[m, k] * B[k, n]
    C[m, n] = acc
```

这段 reference 不追求快，它定义正确结果。优化 kernel 可以换循环顺序、分块、展开、向量化、预打包权重，也可以使用 int8 乘加指令或 GPU tensor core；但只要算子合同没有变，输出就必须被 oracle 接受。浮点路径可能用误差界比较，整数量化路径可能用反量化后的误差界比较，生成路径还要检查 logits 排名、采样前概率和最终 token 序列是否在合同内。

常见误区

不要先写“高性能 kernel”再倒推语义。AI 推理里很多 bug 并不崩溃，而是悄悄改变 logits、概率、排名或边界 token。没有 reference 和误差 oracle，速度数字没有意义。

量化从哪里来

quantization（量化）的基本动机不是神秘技巧，而是机器账本约束。一个 7B 模型如果权重按 float32 保存，仅权重就接近 28GB；按 fp16/bf16 保存约 14GB；按 int8 保存约 7GB；若使用 int4 或分组量化，权重字节还能继续下降，但会引入更复杂的 scale、zero point、group metadata 和反量化开销。权重变小可以降低内存容量压力、cache/TLB 压力、内存带宽压力和 GPU 显存压力，但量化不是免费压缩，它引入表示误差和重新缩放成本。

最常见的线性量化可以写成：

Listing 2.5 线性量化和反量化

```
q = clamp(round(x / scale) + zero_point, qmin, qmax)
x_approx = (q - zero_point) * scale
```

这里 `scale` 决定实数间隔，`zero_point` 让某个整数值表示实数零。若张量范围很大，scale 变大，小差异会被吞掉；若范围估计太窄，极端值会被 clamp。量化校准要解决的正是这个问题：用什么数据估计范围，用 per-tensor、per-channel 还是 per-group scale，怎样在误差、内存和速度之间取舍。

整数矩阵乘常把 int8 输入和 int8 权重乘出中间结果，再累加到 int32：

Listing 2.6 int8 dot product 的累加器

```
int32 acc = 0
```

```

for k in 0..K:
    acc += int32(a_i8[k] - za) * int32(w_i8[k] - zw)
out = requantize(acc, scale_a, scale_w, scale_out, zero_out)

```

累加器不能仍然用 int8, 因为 K 稍大就会溢出。重新量化也不是简单右移: 真实 scale 往往不是 2 的幂, 需要乘法、舍入、饱和和边界处理。第三册后续会把这些步骤分别落到 C++ 类型、CPU/GPU 指令形态和测试 oracle。

从算子到推理引擎

单个算子正确还不等于推理引擎正确。decoder-only 推理会把多个算子按层重复连接起来: 上一层输出成为下一层输入, 中间 activation 可以被复用, 权重可能预打包, 某些算子可以融合, 某些布局转换必须插入, KV cache 要跨 token 保留。推理引擎至少要管理五类状态:

- 模型状态: 权重、shape、量化参数、层配置、tokenizer 配置、算子拓扑。
- 运行状态: token ids、position、logits、临时 activation、KV cache、采样器状态。
- 后端状态: CPU feature、线程池、packed weight、GPU buffer、stream、kernel registry。
- 证据状态: reference 输出、误差报告、benchmark 配置、二进制身份、对标框架版本。
- 资源状态: 内存峰值、cache 工作集、线程亲和性、NUMA 边界、显存占用和传输边界。

真实引擎的执行可以写成:

Listing 2.7 本地大模型推理引擎执行流程

```

load model metadata, tokenizer and weights
validate tensor shapes, dtypes and quantization parameters
prepare backend-specific packed weights or device buffers
session = initialize KV cache, activation arena and sampler

tokens = tokenizer.encode(prompt)
run prefill(tokens, session)

while not stopped:
    logits = run decode_one_token(session)
    next_id = sampler.sample(logits)
    session.append(next_id)
return tokenizer.decode(generated_ids)

```

这段流程故意没有隐藏工程边界。加载模型时要校验文件; 选择 kernel 时要检查 dtype、layout 和 backend capability; 复用 activation buffer 时要保证生命周期不重叠; 更新 KV cache 时要保证层号、head、position 和 batch 维度一致; 记录 timing 时要把 prefill tokens/s、decode tokens/s、首 token 延迟、内存峰值和对标对象分开。

学完这一册能做出的业务原型

把本节这些边界连起来, 读者能做出的不是完整商业平台, 而是一个严肃的本地推理原型。它应该具备:

- 加载一个受限 decoder-only 模型或模型切片。
- 验证 tokenizer、manifest、shape、dtype、量化参数。
- 支持 prefill/decode, 维护 session 和 KV cache。
- 输出流式 token、finish reason 和 usage。
- 支持至少一个 CPU reference path 和一个优化 path。
- 用 oracle 比较单算子、层输出或 logits。
- 用 profiler 报告 load、prepare、prefill、decode、sample。
- 在容量不足、context 超限、backend 不支持时给出明确诊断。

这已经足以支撑本地助手、RAG 问答、批量摘要或代码补全的原型。真正上线还需要安全策略、模型评测、权限、日志脱敏、监控、弹性调度和产品层交互，但底层 AI infra 能力已经闭合：输入可验证，状态可管理，后端可替换，误差可检测，性能可解释。

核心思想

第三册要教的不是“调用一个大模型 API”，而是实现本地推理系统的关键骨架：模型合同、张量地址、量化数值、KV 状态、后端计划、oracle 和 profiler。

本节收束

第三册从真实 7B 级本地推理引擎立题，再用小 MLP 切片建立显微镜。张量把内存、形状、量化和驻留位置连接起来；算子合同把语义和测试连接起来；量化把表示范围、内存容量和机器成本连接起来；KV cache 把单次算子变成跨 token 状态；后端合同把 CPU SIMD、多线程和 GPU kernel 放进同一个 runtime；性能报告把每一步优化放到当前基线和成熟框架旁边比较。

下一章不会急着跳到最快 GEMM 或某个 SIMD 指令，而是先把一层矩阵向量乘拆成 shape、stride、地址公式、float32 内层循环、基本块流图、抽象汇编账本和 int8 账本。只要这条最小路径能讲清，后面的 packing、向量化、量化校准、算子融合、attention、KV cache、CPU/GPU 后端和运行时调度才有稳定落点。

2.2 张量布局、地址流与第一个矩阵向量内核

为什么先讲布局再讲算子优化

第一章把第三册的主线钉在真实本地大模型推理引擎上：模型文件、tokenizer、decoder graph、KV cache、CPU/GPU 后端和性能证据都要被同一套 runtime 管住。但不能从这些大词直接跳到最快 kernel。接下来先拿最小切片中的一层矩阵向量乘做显微镜，因为每个 Transformer projection、MLP 子层和 decode 阶段的小 batch 路径，最终都会回到同一组问题：张量在内存里到底怎样排布，热循环每一步访问哪些地址，累加器和输出缓冲怎样被机器指令更新。

本节先放大最小切片中的第一层：

Listing 2.8 本节先追踪这一层

```
hidden = relu(input * W0 + b0)
```

为了便于手工检查，先把 batch 固定为 1，把它看成一个矩阵向量乘：

Listing 2.9 矩阵向量乘的 reference 语义

```
for out in 0..N:
    acc = bias[out]
    for k in 0..K:
        acc += input[k] * weight[out, k]
    hidden[out] = max(acc, 0)
```

这段 reference 很慢也没关系，它定义了结果。优化版本可以分块、展开、向量化、预打包权重、融合 ReLU，也可以换成 int8 路径；但每一次改造都必须回答同一组问题：`input[k]` 是否连续，`weight[out,k]` 是否连续，bias 何时加载，acc 用什么宽度保存，输出写到哪里，activation 是分支还是条件数据流。

在 decoder-only Transformer 的 decode 阶段，同样的形状会反复出现。当前 token 的 hidden state 是一条向量，`Wq`、`Wk`、`Wv`、`Wo`、`Wgate`、`Wup`、`Wdown` 都是巨大的权重矩阵。单 token decode 时，很多 projection 看起来就是“一个向量乘很多权重行”；只不过 `K` 可能是几千，`N` 也可能是几千到上万，权重字节和 cache/TLB 压力会远大于教学切片。

核心思想

AI 算子的第一层机器事实不是“矩阵乘很重要”，而是：循环次序决定地址流，地址流决定 cache/TLB 和 SIMD 能否工作，累加器宽度决定正确性边界，reference 和 oracle 决定优化是否可信。

张量元数据必须落到地址公式

一个张量视图至少包含 dtype、shape、stride 和 data pointer。以二维权重矩阵 $W0[N, K]$ 为例，若按行主序连续存储，地址公式是：

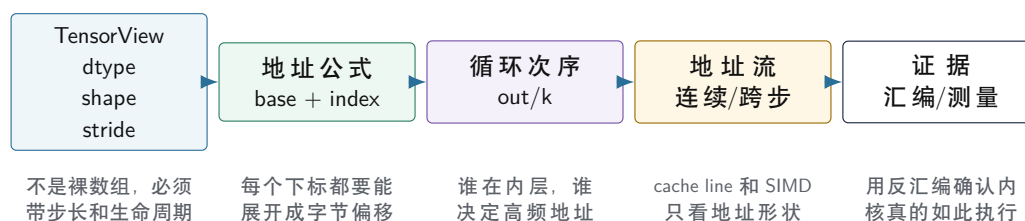
Listing 2.10 行主序权重矩阵的地址公式

```
address(W[out, k]) =
    base_W + (out * stride_out + k * stride_k) * sizeof(element)

row-major contiguous:
    stride_out = K
    stride_k   = 1
```

若 $\text{stride}_k=1$ ，内层循环扫描 k 时权重地址连续；若权重来自转置视图， stride_k 可能变成 N ，内层循环就变成跨步访问。源码里同样写 $W[\text{out}, k]$ ，机器看到的地址流却完全不同。第三册后面讲 packing，本质就是把“不适合内核的 stride”复制成“适合热循环的连续块”。

张量视图怎样变成地址流



图示：本图要证明的命题是：张量布局不是 API 信息，而是热循环地址流的来源。

第一个 float32 矩阵向量内核

先写一个朴素但清楚的 float32 内核：

Listing 2.11 float32 reference 风格矩阵向量乘

```
1 void matvec_relu_f32(float const* input,
2                     float const* weight,
3                     float const* bias,
4                     float* hidden,
5                     int n,
6                     int k)
7 {
8     for (int out = 0; out < n; ++out) {
9         float acc = bias[out];
10        float const* row = weight + out * k;
11        for (int i = 0; i < k; ++i) {
12            acc += input[i] * row[i];
13        }
14        hidden[out] = acc > 0.0f ? acc : 0.0f;
15    }
16 }
```

```

15     }
16 }

```

这段代码的内层地址流很明确：`input[i]` 连续读取，`row[i]` 连续读取，`acc` 通常留在寄存器里，`hidden[out]` 每个输出写一次。若 `K` 较大，权重流通常主导读取字节；若 `N` 较大，同一个 `input` 向量会被每个输出行复用。后续优化会围绕两件事展开：怎样让 `input` 和权重的复用更靠近核心，怎样让多个乘加并行进入 SIMD 或多个累加器。

一个抽象汇编账本可以写成：

Listing 2.12 float32 内层循环的典型机器形态

```

load  input[i]          ; stride-1 load
load  weight[row+i]     ; stride-1 load if row-major
mul   input, weight
add   product into acc
branch back to inner loop

```

这不是要求某个编译器必须生成标量 `mul/add`。在合适目标上，优化器可能生成向量 `load`、FMA、循环展开和水平归约。这里的重点是：无论标量还是向量，内核都必须消耗 `input` 地址流、`weight` 地址流和 `acc` 依赖链；SIMD 只是一次处理多个相邻元素，不能替代布局和地址分析。

把内层循环切成基本块

为了把第一册和第二册的方法接进 AI 算子，不能只停在 C++ 循环。下面这份伪反汇编把同一个 `matvec_relu_f32` 切成三个基本块：外层准备一行权重，内层做 dot product，尾部做 ReLU 和写回。

Listing 2.13 matvec 第一层的伪反汇编基本块

```

B0_row:
    acc = load bias[out]
    row = weight + out * K
    i = 0

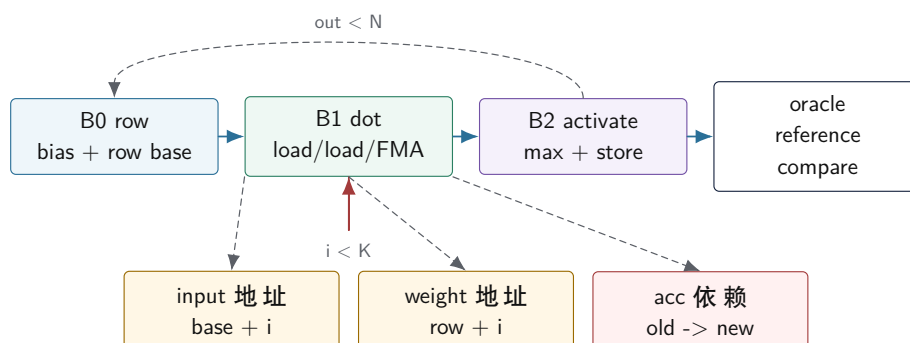
B1_dot:
    x = load input[i]
    w = load row[i]
    acc = fma_or_mul_add(acc, x, w)
    i = i + 1
    branch i < K back to B1_dot

B2_activate_store:
    y = max(acc, 0)
    store hidden[out] = y
    out = out + 1
    branch out < N back to B0_row

```

这份账本故意不写具体寄存器名，因为寄存器分配、展开和向量化会随编译器变化。但它保留了性能分析必须稳定存在的边：`B1_dot` 的两条 `load` 由 `i` 产生地址，`acc` 是跨内层迭代的归约依赖，`i < K` 是内层回边，`hidden[out]` 写回发生在归约完成之后。若后续优化把 `B1_dot` 展开成四个累加器，变化的是依赖图；若把权重预打包，变化的是 `row[i]` 的地址流；若融合下一层，变化的是 `B2_activate_store` 的写回边界。

matvec 热区的控制流、地址流和证据闭环



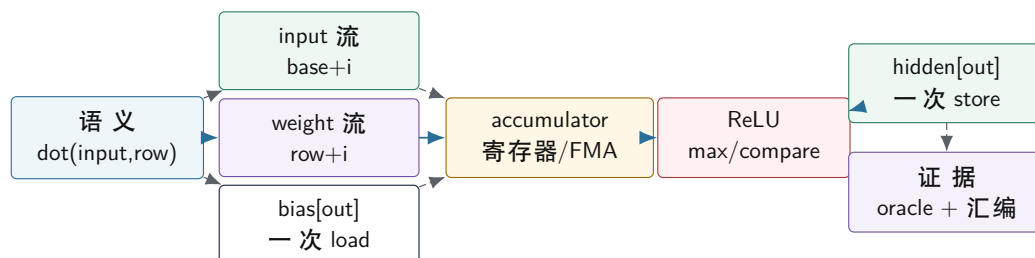
验证时先确认基本块和回边，再确认 load 地址是否连续，最后用 reference 输出和误差合同证明优化没有改变语义。

图示：本图要证明的命题是：AI 内核分析必须同时切出控制流、地址流、归约依赖和正确性证据。

按这张图读后续优化，很多术语会自然归位。循环展开不是“让代码更长”，而是让 **B1_dot** 中出现多条相互独立的 acc 链；SIMD 不是“自动快”，而是要求 **input** 和 **weight** 至少在热路径上形成可装载的相邻 lane；packing 不是“拷贝一份权重”，而是把 **weight** 的跨步或间接地址改造成内层连续流；ReLU 融合不是“少一个函数”，而是移动 **B2_activate_store** 的边界，减少中间张量的写回和读回。

用一张图同时看语义、地址和机器形态

矩阵向量内核的语义、地址流和硬件账本



本图先固定 float32 路径。int8 路径会把 input/weight load 改成窄整数，把 acc 改成 int32，并增加 requantize、round、clamp。

图示：本图要证明的命题是：一个 AI 算子要同时有语义账本、地址账本和机器账本，后续优化才能被验证。

按这张图读代码，bias 不是循环外的装饰，它是每个输出通道的一次初始 load；activation 不是数学符号，它要落成 **max**、条件移动、近似函数或查表路径；输出 store 不是性能主体，但它决定 hidden buffer 的生命周期和后续层的输入布局。若把 activation 与 matvec 融合，可以少一次写出再读回 hidden 的中间流；但融合后 oracle 必须仍能解释输出误差，调试路径也要能定位是哪一层出错。

布局错误怎样伪装成算子慢

常见误判是看到 **matvec** 慢，就直接想“需要 SIMD”。但如果权重不是行主序连续，或者每个输出行通过指针数组间接访问，SIMD 也只能在更差的地址流上工作。下面两种形状语义相同，机器成本不同：

Listing 2.14 两种语义相同但地址形状不同的权重访问

```

row-major packed:
    weight[out*K + 0], weight[out*K + 1], weight[out*K + 2] ...

pointer rows:
    rows[out] -> heap block
    rows[out+1] -> another heap block
    row pointers and weight bytes may live on many pages

```

第一种形状让内层循环沿着连续权重流前进；第二种形状先追一层指针，再进入某个堆块。若块很小、分配分散或页很多，TLB 和 cache line 利用率都会变差。对于 7B 级推理引擎，模型加载阶段把权重转换为内核友好的 packed layout 往往比在每次推理时容忍任意布局更合理。代价是加载时间、额外内存和格式版本；收益是热路径简单、可测、可向量化。CPU 后端可能选择 cache-friendly panel，GPU 后端可能选择显存连续、coalesced load 或 tensor-core 友好的 tile；两者都必须从同一份张量元数据出发。

TensorView 的 C++20 合同

工程里不能把张量参数写成一堆裸指针和整数。至少需要一个只观察内存、不拥有内存的 view；真正拥有内存的对象可以是 arena、mmap weight、DeviceBuffer 或 aligned buffer。view 的责任是描述地址公式。

Listing 2.15 TensorView 的最小字段

```

TensorView:
    data
    element_type
    rank
    shape[]
    stride[]
    byte_offset
    device
    layout_tag

```

它要满足几条不变量：

- `shape[d]>=0`，rank 与 shape/stride 长度一致。
- view 不拥有内存，不负责释放 `data`。
- 地址计算必须检查或由 verifier 证明不会越界。
- `layout_tag` 只是快速路径提示，真正地址仍由 stride 决定。
- kernel 若要求 contiguous，必须显式检查，而不是假设 stride 合适。

Listing 2.16 contiguous 检查的语义

```

expected = 1
for dim from rank-1 down to 0:
    if stride[dim] != expected:
        return false
    expected *= shape[dim]
return true

```

这个检查看起来简单，但能拦住大量错误。比如一个 `[T,H]` 视图经过转置后 shape 仍然可能是二维，但 stride 不再是 row-major contiguous；若后端把它送进只支持连续输入的 GEMM kernel，输出就会错或性能极差。正确路线是：verifier 标出非连续，compiler 插入 layout conversion 或选择

支持 stride 的 reference/slow path。

layout conversion 也要进入 profiler

很多推理代码把 layout conversion 当成“准备工作”，没有计入耗时。实际它可能非常贵。若每个 decode step 都把某个 activation 从非连续转换成连续，转换成本会进入热路径；若模型加载时一次性把权重 pack 好，成本属于 prepare 阶段，应该和 decode 分开报告。

Listing 2.17 layout conversion event 字段

```
LayoutConversionEvent:
    source_value
    source_layout
    target_layout
    bytes_read
    bytes_written
    stage: prepare | prefill | decode
    reason: backend_requirement | fusion_boundary | debug_oracle
```

这能避免一种常见误判：kernel 本身很快，但前后各有一次 layout conversion，端到端反而慢。profiler 若只记录 kernel 时间，就会把瓶颈藏起来。

layout 测试矩阵

张量布局相关测试不能只测连续 happy path。最小测试矩阵应该包含：

- row-major contiguous 二维矩阵。
- 带 padding 的矩阵，例如每行 stride 大于列数。
- 转置 view，shape 正确但 stride 交换。
- 子视图，byte offset 非零。
- tail shape，**K** 或 **N** 不能整除 tile。
- 量化 packed layout，逻辑坐标与物理位置不同。

Listing 2.18 layout oracle 的基本做法

```
for each logical coordinate:
    reference = dense_reference[coordinate]
    actual = read_through_tensor_view(view, coordinate)
    compare actual with reference
```

这个测试不是性能测试，而是证明地址公式正确。只要地址公式错，后面 SIMD、GPU、量化都只是更快地读错数据。

从 float32 路径过渡到 int8 路径

量化路径不是把 `float` 替换成 `int8_t` 那么简单。同一层 matvec 进入 int8 后，机器账本变成：

Listing 2.19 int8 matvec 的机器账本变化

```
load int8 input values
load int8 packed weights
convert or widen to int16/int32 lanes
dot product into int32 accumulators
add int32 bias or folded bias
requantize acc to output dtype
apply clamp or activation
```

store output

权重字节变小, cache 能容纳更多参数; 某些 CPU 有专门的 dot-product 或 madd 指令, 能提高吞吐。但累加器必须更宽, 重新量化必须处理 scale、舍入和饱和, oracle 也必须从“逐元素几乎相同”变成“误差在合同范围内”。这就是第三册的基本节奏: 每个算子先有 float reference, 再有地址账本, 再有量化语义, 最后才进入特定 ISA 和 kernel 优化。

把这条账本接回 Transformer

现在把本节的 `matvec` 账本放回 decoder-only Transformer 的一个 decode 步骤。假设当前层输入是一条 hidden 向量 `x[hidden]`, 这一层至少要做下面几组投影:

Listing 2.20 decode 单 token 中反复出现的投影账本

```
q = matvec(x, Wq)
k = matvec(x, Wk)
v = matvec(x, Wv)
append k, v into KV cache at current position

attn = attention(q, K_cache[0..pos], V_cache[0..pos])
x = x + matvec(attn, Wo)

gate = matvec(norm(x), Wgate)
up = matvec(norm(x), Wup)
x = x + matvec(silu(gate) * up, Wdown)
```

这里的每个 `matvec` 都可以用本节的方法拆开: 输入向量地址、权重行地址、accumulator 依赖、输出 store、量化 scale 和 oracle。不同的是, 真实模型把尺寸和状态放大了。Q/K/V 的输出还要 reshape 成 `[heads, head_dim]`; K/V 不是普通临时输出, 而要写入第 `layer` 层、第 `position` 个 token 的 KV cache; attention 会在后续 token 中反复读取这些 cache 行。

从 matvec 切片接回 decoder 层状态



图示: 本图要证明的命题是: 最小 `matvec` 账本不是玩具结论, 它会直接进入每一层 Transformer decode 的投影、KV cache 和 MLP 路径。

prefill 阶段则把 `x` 从一条向量扩成多条 token 行。此时 `matvec` 账本会变成下一章的 GEMM/GEMV 问题: prompt token 越多, 权重复用越明显, packing、tile、SIMD 和 GPU kernel 的价值越容易体现; decode 阶段每次只处理一个或少数 token, 权重流和 KV cache 读写又会成为另一种瓶颈。因此后续性能报告必须把 prefill tokens/s、decode tokens/s 和首 token 延迟分开写。

本节验收线

读完本节, 应该能对最小切片的一层 `matvec`, 以及 Transformer 中的 Q/K/V 或 MLP projection, 写出下面几件事:

- 用 shape 和 stride 推导 `input[k]`、`weight[out,k]`、`bias[out]`、`hidden[out]` 和 KV cache 写入位置的地址公式。
- 说明行主序连续权重为什么适合内层 `k` 循环, 转置或指针行为什么会改变地址流。

- 把 float32 matvec 写成 load、mul、add、branch、store 的机器账本。
- 解释 activation 融合减少了哪条中间内存流，也说明它怎样改变调试和 oracle 边界。
- 说明 int8 路径为什么需要 int32 accumulator、requantize 和误差合同。
- 区分 prefill 中的多 token GEMM 倾向和 decode 中的单 token GEMV 倾向。

这些输出看起来比“会调用矩阵乘 API”低级，但它们是后续 GEMM packing、SIMD、量化校准、算子融合、KV cache 规划、CPU/GPU 后端和推理图内存规划的共同地基。

2.3 从矩阵向量乘走向 GEMM、packing 与 SIMD

为什么不能直接从 matvec 跳到最快 GEMM

上一章把一层矩阵向量乘拆成了 shape、stride、地址公式、基本块、地址流和机器账本，并把它接回了 decoder-only Transformer 的 Q/K/V projection、KV cache 写入和 MLP 子层。那条路径故意很小，因为它让每个性能判断都有落点：input[i] 从哪里读，row[i] 是否连续，acc 为什么形成归约依赖，activation 和写回发生在什么边界。

本节继续沿同一条路径推进，但不直接背某个 BLAS 接口。真正的高性能矩阵乘不是一句“调用 GEMM”就结束。它要解决三个逐层出现的问题：

Listing 2.21 从 matvec 到 GEMM 的三个新问题

```
matvec:
  one input vector times many weight rows
  output is one hidden vector

prefill or batched inference:
  many token rows share the same weight matrix
  output becomes a token-by-channel matrix

GEMM kernel:
  compute a tile C[m,n] from panels A[m,k] and B[k,n]
  choose loop order, packing layout, SIMD lanes and accumulators
```

第一个问题是复用。matvec 中，同一个 input 会被每个输出行复用；prefill 中多个 token 行共享同一份权重，batch 服务时多个请求也共享权重。第二个问题是布局。数学上都是矩阵乘，机器上却要决定哪一块连续、哪一块预打包、哪一块留在 cache 或显存。第三个问题是寄存器和 SIMD。向量指令一次处理多个 lane，但只有当地址流、对齐、数据类型和累加器安排都配合时，它才会变成吞吐提升，而不是更复杂的写法。

核心思想

GEMM 优化的核心不是“矩阵乘很快”，而是把重复使用的数据切成能留在 cache 和寄存器里的小块，并让内层循环产生稳定、连续、可向量化的地址流。

把 token 维度加回模型

上一章把 token 行固定为 1。若 prefill 一次处理 T 个 prompt token，或者服务端把 B 个请求合批，线性层语义可以写成：

Listing 2.22 多 token 版线性层 reference 语义

```
for t in 0..T:
  for out in 0..N:
    acc = bias[out]
    for k in 0..K:
      acc += input[t, k] * weight[out, k]
    output[t, out] = activation(acc)
```

这仍然是 reference，不是最终内核。它暴露了两个复用方向。固定 **t**、遍历 **out** 时，一个 token hidden 行会被多次读取；固定 **out**、遍历 **t** 时，一个 weight 行会被多次读取。若 decode 每次只有一个 token，权重复用不明显，GEMV 或小 batch GEMM 可能是主形态；若 prefill token 多，权重矩阵成为多个 token 共享的热数据，GEMM 风格就有意义。

常见的矩阵表示可以写成：

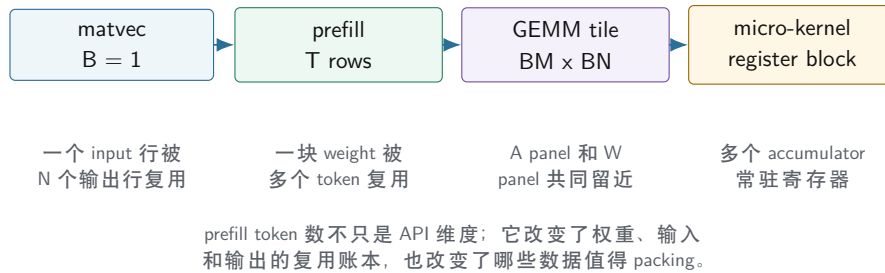
Listing 2.23 第一层的矩阵形状

```
A = input    shape [T, K]
W = weight   shape [N, K] row-major by output channel
C = output   shape [T, N]

semantic:
  C[t, out] = dot(A[t, :], W[out, :]) + bias[out]
```

如果把右侧矩阵看成 $W_T[K, N]$ ，语义就接近 $C=A*W_T$ 。这一步很容易在纸上完成，却不能在机器上忽略布局： $W[out, k]$ 行主序连续适合 matvec 的内层 **k**；但 GEMM 常希望在一个小 tile 里同时取多个 **out**，也就是读取 $W[out0, k]$ 、 $W[out1, k]$ 、 $W[out2, k]$ 。这些地址在原始行主序中相隔 **K** 个元素，不一定适合内层 SIMD。

prefill 把复用方向从一条行扩成一个 tile



图示：本图要证明的命题是：从 decode matvec 到 prefill GEMM 的关键变化是复用单位从一条向量扩大为一个可驻留的 tile。

三层循环不是任意交换

矩阵乘 reference 通常有三层循环。初学者容易把循环顺序理解成纯数学等价：只要结果相同，怎么换都可以。性能分析要更具体：谁在内层，谁就决定高频地址流；谁跨 tile 复用，谁就决定 cache 工作集；谁留在寄存器，谁就决定 accumulator 数量。

Listing 2.24 三种循环顺序的地址含义

```
t-out-k:
  for each token t:
    for each output out:
      scan A[t, k] and W[out, k]

t-k-out:
  for each token t:
    for each k:
      reuse A[t, k] across many out
      touch many W[out, k] and C[t, out]
```



```

tile-t-out-k:
  for blocks of tokens and out:
    keep C tile accumulators
    stream K panels of A and packed W

```

第一种像上一章的 matvec：每个输出行顺序扫描权重，简单、可验证，但每次只产生一个输出。第二种把 $A[b, k]$ 复用到多个 out ，但如果 $W[out, k]$ 在原始行主序中跨步很大，内层访问会变差。第三种把问题切成 tile：一次只处理 BM 个样本和 BN 个输出通道，用多个 accumulator 保存一个 C 小块，沿 K 扫描输入和权重面板。

这就是为什么 GEMM 教材经常出现 BM 、 BN 、 BK 。它们不是玄学参数，而是三个工作集边界：

- BM ：一次处理多少输入行，影响输入 panel 和输出 tile。
- BN ：一次处理多少输出列，影响 accumulator 数量和权重 panel。
- BK ：一次沿归约维度推进多少元素，影响 packing 块和 cache 复用。

Packing 的本质：把坏地址流变成后端友好的地址流

packing（预打包）指把原始张量中的一块数据复制成内核更喜欢的布局。它不是为了让模型文件好看，而是为了让热循环简单。以权重为例，原始行主序 $W[out, k]$ 对单个输出行很好；但一个微内核若想同时算 BN 个输出，内层可能需要在同一个 k 下读多条输出行的权重。原始布局给出的地址是：

Listing 2.25 原始权重在同一个 k 下访问多个 out

```

W[out0, k] = base + (out0 * K + k)
W[out1, k] = base + (out1 * K + k)
W[out2, k] = base + (out2 * K + k)

distance between neighboring out values: K elements

```

若把一个 $BN \times BK$ 的权重块打包成 k -major，内核看到的就可以是：

Listing 2.26 packed weight panel 的一种形状

```

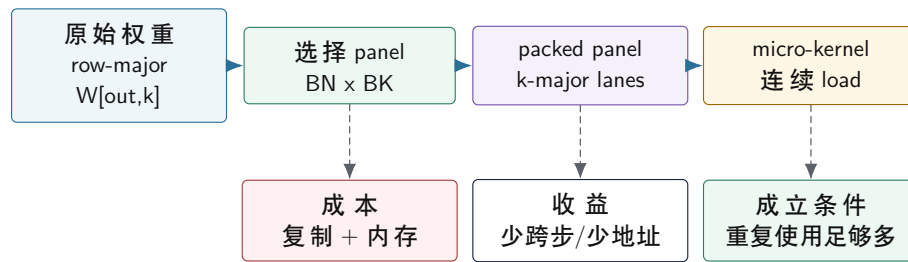
for k in 0..BK:
  packed[k, 0] = W[out0, k]
  packed[k, 1] = W[out1, k]
  packed[k, 2] = W[out2, k]
  ...

inner kernel reads packed[k, 0..BN)

```

这样做付出了一次复制、额外内存和格式管理成本，换来内层循环的连续读取、更少地址计算和更稳定的 SIMD lane。是否值得 packing，取决于这块权重会被复用多少次。7B 模型权重会在每个 token、每一层反复使用，所以在加载阶段或首次运行阶段打包常常合理；但不同后端喜欢的 packed layout 不一定相同。CPU 后端关心 cache panel、SIMD lane 和预取；GPU 后端关心 coalesced load、shared memory tile、tensor core 数据格式和显存对齐。runtime 因此不能把唯一一种 CPU packed layout 写死成模型格式。

packing 把原始布局改造成内核布局



packing 不是数学变换，而是把原始地址图复制成微内核更容易连续读取的地址图。

图示：本图要证明的命题是：packing 的价值来自热循环地址流改善，必须和复用次数一起判断。

SIMD 需要多条独立账本配合

SIMD（单指令多数据）可以理解为一指令同时处理多个 lane。例如一个 float32 向量寄存器可以装多个相邻浮点数，FMA 指令可以对多个 lane 同时做乘加。但 SIMD 对地址流有要求：连续 load 最容易，固定可预测步长次之，随机 gather 或指针追踪通常更难。

在 GEMM 微内核里，SIMD 常见两种方向。第一种是沿 **K** 维度装载多个相邻元素，做向量乘加，最后水平归约到一个输出。第二种是同时维护多个输出通道，让一个输入标量广播到多个 lane，与 packed 权重 lane 相乘，更新多个 accumulator。哪种方向更好，取决于 ISA、寄存器数量、数据类型、矩阵尺寸和权重布局。

Listing 2.27 一个抽象 micro-kernel 账本

```

for k in 0..BK:
    a0 = load A[b0, k]
    a1 = load A[b1, k]
    wv = load packed_W[k, out0..outN]
    acc0[0..N] += broadcast(a0) * wv
    acc1[0..N] += broadcast(a1) * wv

store C[b0, out0..outN]
store C[b1, out0..outN]
  
```

这段账本看起来已经像高性能内核，但仍然不能脱离前两册的方法。要问：**packed_W** 是否真的连续；**A[b, k]** 是否能留在 cache；accumulator 数量是否超过寄存器预算；输出 store 是否连续；尾部 **N** 不能整除 SIMD 宽度时怎样处理；反汇编是否真的生成了向量 load、广播和 FMA，而不是退回标量循环。

从标量 reference 到 micro-kernel 的切分路线

真正写后端时，不应该一上来就写最大化的 SIMD intrinsic。更稳的路线是把同一个 linear 语义切成四个阶段，每个阶段都能被 oracle 验证。

Listing 2.28 linear 内核的四阶段演进

```

stage 0: scalar reference
    one output channel at a time
    easiest to audit, slowest

stage 1: scalar tiled
    compute a small C[BM, BN] tile
  
```

```

    prove tile boundary and tail are correct

stage 2: packed weight
    keep scalar arithmetic
    prove packed descriptor maps to the same weights

stage 3: SIMD micro-kernel
    replace inner loop with vector loads/FMA/dot
    keep the same descriptor and oracle cases

```

这个顺序有一个好处：当 stage 3 出错时，问题范围很小。若 stage 2 已经证明 packed descriptor 的字节解释正确，stage 3 的错误就更可能来自 SIMD lane 顺序、accumulator、尾部 mask 或 dtype 转换，而不是模型导入或 shape verifier。

以 **BM=2**、**BN=4** 的小 tile 为例，标量 tiled 版本可以先写成：

Listing 2.29 标量 tiled GEMM 的参考形态

```

for b in 0..B step 2:
  for out in 0..0 step 4:
    acc[2][4] = 0
    for k in 0..K:
      for bi in 0..1:
        for oi in 0..3:
          acc[bi][oi] += A[b + bi, k] * W[out + oi, k]
        store acc into C[b..b+1, out..out+3]

```

这段代码不快，但它已经有了微内核的形状：`acc[2][4]` 是寄存器账本的影子，`b/out/k` 是 tile 循环骨架，尾块可以独立测。后续把 `W[out+oi,k]` 换成 packed 地址，把 `oi` 方向换成 SIMD lane，就不会改变上层语义。

一个 micro-kernel 的 descriptor 至少要把这些事实写清：

Listing 2.30 micro-kernel descriptor 的最小字段

```

MicroKernelDesc:
  bm
  bn
  bk
  input_dtype
  weight_dtype
  accumulator_dtype
  output_dtype
  weight_packed_layout
  required_alignment
  tail_policy
  supported_isa

```

这让内核选择变成编译器问题，而不是一堆 `ifcpuhasavx` 散落在调用点。typed graph 给出 shape 和 dtype，backend capability 给出 ISA 和支持格式，lowering 选择 descriptor，oracle 用同一 descriptor 解释 packed bytes。

验收与评分标准

一个 linear micro-kernel 进入主线前，至少要通过四组测试：整齐尺寸、**K** 维尾部、**O** 维尾部、非默认 stride。若是量化权重，还要额外覆盖 group 边界和 scale 尾组。

尾块、对齐和小尺寸陷阱

工业 GEMM 内核会花大量代码处理边界。教材如果只展示整齐尺寸，会让读者误以为优化就是把主循环写出来。实际推理中，隐藏层大小、输出类别数、batch size 和量化 block 不一定刚好整除 tile。

尾块不是异常路径。 若 **N** 不能整除 **BN**，最后一个输出 tile 是窄 tile；若 **B** 不能整除 **BM**，最后一个 batch tile 是矮 tile；若 **K** 不能整除向量宽度，归约尾部需要标量补齐、mask load 或专门小内核。尾块很常见，不能只靠“输入尺寸都规整”逃避。

对齐是性能条件，不是语义条件。 非对齐 load 通常仍然能得到正确结果，但可能跨 cache line 或页，成本更高。packing 阶段可以选择对齐缓冲，减少内核里处理复杂地址的机会。报告中要写清：对齐要求是硬性合同，还是只是 fast path 条件；不满足时是复制、降级还是报错。

小矩阵可能被 packing 成本吞掉。 一个只有几行几列的小层，打包和调度的固定成本可能大于内核收益。第三册后续讲推理图和运行时时会反复遇到这个问题：大算子适合重内核，小算子更需要融合、批处理或减少中间内存流。

prefill、decode 与后端合同

把 GEMM 放回 7B 推理引擎后，性能不能只写一个“矩阵乘耗时”。同一个权重矩阵在 prefill 和 decode 中的形状不同：

Listing 2.31 同一层在 prefill 和 decode 中的矩阵形态

```
prefill:
  A = tokens x hidden
  W = out x hidden
  C = tokens x out
  weight reuse across many token rows

decode:
  A = 1 x hidden
  W = out x hidden
  C = 1 x out
  weight stream dominates, KV cache also grows
```

CPU 后端通常先把 decode 的单 token latency 做稳，再逐步优化 prefill 吞吐：标量 reference、普通线程切分、SIMD micro-kernel、packed weight、NUMA-aware weight placement、算子融合、量化内核。GPU 后端则更依赖批量和 tile：prefill 往往容易吃到大 GEMM 吞吐，decode 若 batch 太小就可能被 kernel launch、内存访问和 KV cache 读写限制。真实 runtime 要把这些差异放到 backend contract，而不是让上层 graph 直接知道每个后端的细节。

Listing 2.32 GEMM 后端合同的最小字段

```
op: linear
input: dtype, shape, layout, device
weight: dtype, shape, packed_layout, scale_policy
output: dtype, shape, layout, device
mode: prefill or decode
backend: cpu_scalar, cpu_simd, cpu_threaded, gpu
evidence: latency, tokens_per_second, error_report
```

这份合同让第三册后续能一步一步把性能提高，而不是一次跳到“对标成熟框架”。每次优化都必须说明它改变了哪一栏：是 dtype 从 fp32 变成 int8，还是 packed layout 变了；是 mode 只覆盖 prefill，还是 decode 也受益；是 CPU 后端新增 SIMD path，还是 GPU 后端新增 kernel；是降低了 latency，还是提高了 batch 吞吐。

把现有 lab 放进这条路线

当前 `books/cpu-volume-3/labs/linux_cpu_inference/` 里的 `linear_i8` 是一个故意朴素的内核。它按输出遍历 int8 权重，把每个权重乘以 scale 后与 float 输入相乘，并把结果累加到 float：

Listing 2.33 当前 lab 的线性层机器账本

```
for out in output_size:
    acc = bias[out]
    row_base = out * input_size
    for in in input_size:
        weight_i8 = weights[row_base + in]
        acc += float(weight_i8) * layer.scale * input[in]
    output[out] = acc
```

这段代码适合作为 reference 风格的本地推理入口，因为它的语义清楚，测试能手工验证。但它只是第一阶段切片，不是第三册最终引擎。它没有 token tile，没有 packed weight panel，没有 int32 accumulator，没有 requantize，也没有 CPU/GPU 后端选择。下一章会专门讲 int8 的真实量化账本：为什么当前 lab 选择了“int8 权重 + float 输入 + float 累加”的简化路径，以及 7B 推理引擎需要怎样扩展到 per-channel/per-group quantization、packed weights、KV cache 量化和后端共享的误差报告。

本节验收线

读完本节，应该能把“GEMM 很快”拆成下面几条可检查事实：

- 说明 batch 怎样改变 input、weight 和 output 的复用账本。
- 写出 BM、BN、BK 分别约束哪一块工作集。
- 解释 packing 改善的是哪条地址流，以及它为什么需要足够复用才能抵消复制成本。
- 区分 SIMD lane、accumulator 寄存器、连续 load、广播和尾块处理。
- 说明同一个 projection 在 prefill 中为什么更像 GEMM，在 decode 中为什么更像 GEMV 或小 GEMM。
- 能把当前 lab 的 `linear_i8` 定位成清晰 reference/教学内核，而不是最终 7B 推理引擎的 GEMM 微内核。

下一章沿着同一个 `linear_i8` 继续推进，但落点会放大到 7B 模型：权重为什么能用 int8 或 int4 保存，scale 怎样进入计算，accumulator 为什么要更宽，重新量化到底在做什么，KV cache 不能量化，oracle 又应该怎样判定误差。

2.4 量化系统总览：从实数模型到低比特推理合同

量化为什么必须单独成章

量化很容易被误解成“把 float 改成 int8”。如果只是换类型名，确实几行代码就能写完；但真正的推理引擎量化是一套跨越模型文件、编译前端、IR、后端 kernel、runtime、oracle 和 profiler 的系统合同。它决定权重能不能装进内存，决定 decode 阶段要读多少 KV cache，决定 CPU/GPU kernel 怎样读取 scale，决定 logits 是否还能被 sampler 正确使用。

一个最小反例可以说明它为什么不能混在普通算子章节里。假设某个 linear 权重被保存成 int4，每 64 个权重共享一个 scale。若后端 packing 时只重排了 4-bit 权重，却没有按同样分组重排 scale，kernel 仍然会跑，也可能输出一堆“看起来像正常浮点数”的 logits。但这些 logits 已经被错误 scale 污染。这个错误不是普通矩阵乘错误，而是量化格式、metadata、packing 和 kernel 合同同时破裂。

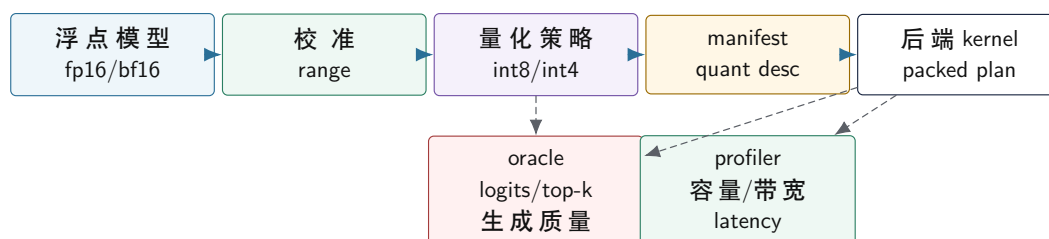
核心思想

量化是一条系统级合同：实数范围怎样选择，整数码点怎样解释，scale 和 zero point 怎样存储，accumulator 在哪里扩大，requantize 怎样舍入，packed layout 怎样保持 metadata 对齐，oracle 怎样判断误差是否可接受。

第三册里的量化流水线

本册把量化看成一条完整流水线，而不是一个局部函数。

量化从校准到后端执行的系统流水线



图示：本图要证明的命题是：量化不是单个 kernel，而是校准、策略、manifest、后端和验证共同组成的流水线。

校准阶段。 校准决定实数范围。权重可以离线扫描；activation 需要代表性输入；KV cache 是动态状态，通常要在真实 prompt/context 分布上观察。校准数据不代表真实分布时，scale 会偏，饱和会增多，生成质量会漂移。

策略阶段。 策略决定使用 int8、int4、per-tensor、per-channel、per-group、对称或非对称量化。策略不是越激进越好。int4 权重能省内存，但 kernel 要处理解包、scale 读取和更复杂误差；KV cache 量化能省长上下文内存，但 decode attention 的读路径会变复杂。

manifest 阶段。 量化信息必须进入 manifest。后端不能靠张量名字猜格式，必须读 quant descriptor：bit width、signedness、scale dtype、zero point policy、group size、axis、packed byte order、tail policy、checksum 和支持的 backend。

后端阶段。 后端负责把量化格式变成机器执行。CPU 可能选择 dequant-on-the-fly、int8 dot-product 或 int4 解包加 FMA；GPU 可能使用 vendor kernel、tensor core 支持格式或自定义 kernel。不同后端可以使用不同 packed layout，但必须共享同一逻辑量化合同。

验证阶段。 量化必须同时通过 oracle 和 profiler。oracle 证明误差在合同内；profiler 证明容量、带宽或 latency 的收益真实存在。只省文件大小但 decode 变慢，或 tokens/s 变快但 logits 漂移不可接受，都不是合格优化。

校准：scale 不是随便选出来的

线性量化的核心参数是 scale 和 zero point。scale 由范围决定，范围由校准决定。最简单的做法是扫描张量最小值和最大值：

Listing 2.34 最小最大校准的基本形状

```

min_value = min(x[i])
max_value = max(x[i])

scale = (max_value - min_value) / (qmax - qmin)
zero_point = round(qmin - min_value / scale)
  
```

这个算法容易理解，但不总是最好。若只有少数极端值，min/max 会把 scale 拉大，使大部分普通值的量化间隔变粗；若为了普通值缩小范围，极端值会被 clamp。两者都是误差，只是形态不同。量化校准的本质就是在舍入误差和饱和误差之间取舍。

校准策略	优点	风险
min/max	简单，可复现，适合第一版	极端值会拉大 scale
percentile	减少极端值影响	被截断值可能集中在关键通道
MSE/KL 搜索	更贴近分布误差	复杂，依赖校准集质量
per-channel 校准	减少通道间范围差异	元数据和 requantize 更复杂
动态校准	适合 activation 或 KV 状态	运行时成本和稳定性要验证

校准集必须写进报告。对图像模型，校准集可能是若干图片；对语言模型，校准集应该包含短 prompt、长 prompt、代码、中文、英文、对话、多轮上下文等场景。若只用很短英文 prompt 校准 KV cache，长中文对话的 activation 分布可能完全不同。校准不是离线脚本的附属品，而是模型合同的一部分。

线性量化的精确定义

先把最常见的线性量化写清楚。给定实数 x 、scale s 和 zero point z ，量化与反量化通常可以写成：

$$q = \text{clamp}(\text{round}(x/s) + z, q_{\min}, q_{\max})$$

$$\tilde{x} = s(q - z)$$

这里 q 是整数码点， $\tilde{x}$ 是反量化后的近似实数。若使用对称量化，通常 $z=0$ ，码点范围围绕 0；若使用非对称量化， z 让非零实数范围能映射到无符号整数区间。对称量化简单，适合很多权重；非对称量化能更好表达偏移分布，但后端计算和 zero point 修正更复杂。

形态	数学合同	工程影响
对称	$z=0$ ，正负范围近似对称	kernel 简洁，适合权重
非对称	z 可非零	activation 适配性强，但修正项更多
per-tensor	整个张量共用 scale	metadata 少，误差可能大
per-channel	每个输出通道一组 scale	精度好，scale 读流增加
per-group	每组权重一组 scale	int4 常见折中，packing 更复杂

per-group int4 的具体含义必须避免含糊。若 `group_size=64`，通常表示连续 64 个权重共享一个 scale。这个“连续”必须绑定到逻辑轴：是按输入通道连续，还是按 packed 后的物理顺序连续？正确做法是 descriptor 同时记录逻辑分组轴和 packed layout，并由 verifier 检查 scale 数量。

Listing 2.35 group scale 数量检查

```
groups_per_output =
    ceil(input_channels / group_size)

expected_scale_count =
    output_channels * groups_per_output
```

若 scale 少一个，kernel 可能越界；若 scale 多一个，说明 manifest、converter 或 packing 至少有一个环节对分组理解不同。不能让 kernel 用“能读多少读多少”的方式容错，因为这会把资产错误变成数值漂移。

校准算法怎样落到 C++20

校准不是只能依赖外部脚本。项目里至少应有可复现的校准结果读取和验证逻辑。对权重量化，离线 converter 可以扫描权重并写出 scale；对 activation 或 KV cache，运行时或校准工具需要收集统计量。

Listing 2.36 校准统计的最小结构

```
CalibrationStats:
  tensor_name
  axis
  granularity
  sample_count
  min_values
  max_values
  histogram_optional
  saturation_rate
  source_prompt_set_id
```

min/max 校准只需要最小值和最大值；percentile、MSE 或 KL 搜索通常需要直方图。直方图不是为了好看，而是为了回答“截断哪些值会让总体误差更小”。一个保守 MSE 搜索可以这样理解：枚举候选范围 $[-a, a]$ ，把范围外值 clamp，把范围内值量化再反量化，选择平均平方误差最小的 a 。

Listing 2.37 MSE calibration 的朴素形态

```
best_error = infinity
for candidate_absmax in candidates:
    scale = candidate_absmax / qmax
    error = 0
    for x in calibration_values:
        q = clamp(round(x / scale), qmin, qmax)
        x2 = q * scale
        error += (x - x2) * (x - x2)
    keep candidate with minimal error
```

这个朴素算法不一定是最高效实现，但它定义了语义。工程实现可以用直方图加速，而 oracle 应该能用小样本直接复现它。不要让校准工具和 runtime verifier 使用两套互相不透明的规则。

权重量化、activation 量化和 KV 量化不是一回事

三类量化对象生命周期不同，因此工程合同也不同。

对象	生命周期	关键风险
权重	离线静态，加载后常驻	packed layout、scale 对齐、后端格式支持
activation	每次推理临时产生	动态范围随输入变化，requantize 成本
KV cache	跨 token 状态，随上下文增长	误差会被未来 token 反复读取，meta-data 进入 attention 热路径

权重量化通常最先做，因为它直接降低模型容量和权重读流。activation 量化更难，因为每层中间值分布随 prompt 改变，且很多算子之间需要反量化/重量化边界。KV cache 量化介于两者之间：它不是静态权重，但每个 token 写入后会长期保存；它的容量收益很大，但误差会在长上下文里持续参与 attention。

因此本册建议的推进顺序是：先做权重-only 量化，让 activation 仍用 fp16/fp32 近似计算；然后做局部 activation 量化实验；最后再做 KV cache 量化。这样每次只改变一个主要误差来源，oracle 更容易定位。

量化收益要同时算字节和指令

低比特并不自动更快。以 int4 weight-only GEMV 为例，热路径少读了权重字节，但增加了三个成本：解包 nibble、读取 scale、把整数码点乘回实数或进入整数 dot-product 路径。若当前机器瓶颈是内存带宽，少读字节可能收益明显；若瓶颈是解包指令或 scale 读流，int4 可能不如 int8。

Listing 2.38 weight-only 量化的粗略收益账

```
fp16 weight bytes:
  params * 2

int8 weight bytes:
  params * 1 + scale_bytes

int4 weight bytes:
  params / 2 + scale_bytes + packing_padding

extra work:
  unpack integer code
  load scale metadata
  dequantize or use integer dot path
```

对于 7B 参数量级，fp16 权重大约 14GB；int8 约 7GB 加 metadata；int4 约 3.5GB 加 metadata。这个容量收益很直观。但 decode tokens/s 是否按比例提升，要看 effective bandwidth、cache、TLB、SIMD 指令、group size 和线程调度。量化报告必须把容量收益和 latency 收益分开写。

业务开发中的量化边界

用户真正关心的是“学完能不能做业务开发”。量化在业务系统里不能只说模型更小，还要说明什么时候可以用、什么时候不该用。

- 搜索补全、草稿生成、离线摘要等任务，可能更能接受轻微 logits 漂移。
- 法律、医疗、财务、代码自动修改等高风险任务，不能只看生成文本是否通顺，要固定评测集和人工验收。
- RAG 场景要特别看引用和数字是否稳定，因为量化误差可能改变排序靠前的候选 token。
- 多轮对话要看长上下文质量，KV cache 量化的误差不能只在短 prompt 上验收。
- 如果使用工具调用或 JSON 输出，要验证格式稳定性，不能只看自然语言回答。

工程上可以把量化 profile 做成可选择配置：**quality** 使用更保守的权重格式和 fp16 KV；**balanced** 使用 int4 权重和 fp16/int8 KV 实验；**memory_saver** 使用更激进 KV 量化和较短窗口。每个 profile 都要有 oracle、benchmark 和适用说明。这样业务方选择的是有证据的配置，而不是一句“开 int4”。

误差来源：不是所有误差都一样

量化误差至少有五类。

舍入误差。 实数值落在两个整数码点之间，需要按某种规则舍入。舍入规则必须固定，否则 CPU reference、SIMD kernel 和 GPU kernel 可能出现系统性差异。

饱和误差。 值超出可表示范围时被 clamp 到边界。少量饱和可能可接受；某层、某通道或某类 prompt 上大量饱和，通常意味着 scale 或校准策略有问题。

累加误差。 低精度乘法后通常在 int32、fp32 或 fp16/bf16 中累加。不同累加器宽度、不同 reduction 顺序、不同 FMA 形态都会改变误差传播。

元数据误差。 scale、zero point、group scale、tail padding 或 packed byte order 被错误解释，会产生结构性错误。这类错误比随机舍入更危险，因为输出可能稳定但错误。

状态误差。 KV cache 量化会把误差写入跨 token 状态。一次写入后，未来很多 decode step 会读取它。它不是一次性中间 activation 误差，而是会随上下文参与后续注意力。

因此，量化 oracle 不能只比较某个算子一次输出。它要能按层、按通道、按 token position、按 prompt 类别和按生成步数定位误差。

量化在 AI 编译器中的位置

在 AI 编译器里，量化至少影响四个阶段。

Listing 2.39 量化事实在编译流水线中的流动

```
model import:
  parse quant metadata
  build QuantDescriptor

verifier:
  check shape, bit width, group size, scale count
  reject unsupported quant formats

IR passes:
  insert dequant, requant, quantized matmul
  decide fusion and materialization boundaries

backend lowering:
  choose packed layout and kernel
  bind scale/zero point/group metadata
```

如果导入阶段没有把量化格式读清楚，后端会被迫猜；如果 verifier 不检查 scale 数量，kernel 可能越界读 metadata；如果 IR pass 不知道某条边是量化张量，就可能错误融合或错误复用 arena；如果 lowering 不知道 group size，就无法生成正确 packed descriptor。

常见误区

量化不是后端私有小技巧。只在 CPU kernel 里临时解释 int4 字节，而不让 manifest、verifier、IR 和 oracle 知道量化合同，会让错误绕过编译器所有安全边界。

量化 lowering 的常见形态

从 IR 到后端，量化通常有三种 lowering 形态。

dequantize then compute。 先把低比特权重或 activation 还原成 fp16/fp32 临时块，再调用普通 GEMM/GEMV。这条路径容易实现和验证，适合作 reference 或第一版 GPU fallback；缺点是会产生额外中间写回，可能抵消低比特节省。

dequantize on the fly。 kernel 读取低比特码点和 scale，在寄存器里反量化后立即参与 FMA。CPU int4 weight-only GEMV 常用这种思路。它减少中间写回，但 kernel 要处理解包、scale group、tail 和 SIMD lane 对齐。

integer dot-product。 若硬件支持 int8 dot-product 或低比特矩阵指令，可以让整数码点直接进入 dot，再用 scale 和 requantize 还原输出。它可能更快，但对 zero point、accumulator、溢出和 scale 合并要求更严格。

lowering	优点	风险
dequant + compute	简单，容易对 oracle	额外内存流量大
on-the-fly dequant	少写中间值，适合 weight-only	kernel 复杂，scale 热路径
integer dot	可利用专门指令	zero point、溢出、requantize 复杂

同一量化格式可以有多个 lowering plan。executable plan 必须记录实际选择的是哪一种，否则 profiler 和误差报告无法解释差异。

量化在 C++20 项目中的边界

C++20 实现中，量化不应该散落成若干裸字段。建议至少有一个稳定的 descriptor：

Listing 2.40 QuantDescriptor 的字段边界

```
QuantDescriptor:
  storage_dtype: int8 | uint8 | int4 | nf4 | fp8
  logical_dtype: approximate_f16 | approximate_f32
  scale_dtype: fp16 | fp32
  zero_point_policy: none | symmetric | asymmetric
  granularity: tensor | channel | group
  axis
  group_size
  scale_count
  packed_order
  tail_policy
  calibration_id
```

这个 descriptor 应该跟 tensor descriptor 一起流动，而不是藏在后端对象里。model loader 创建它，verifier 检查它，compiler pass 查询它，backend packing 消费它，oracle 报告它。这样一旦出错，日志能写清楚“某个张量使用 int4、group size 64、scale dtype fp16、tail padding zero、CPU AVX2 packed layout”，而不是只报 `invalidquantizedtensor`。

本节验收线

读完本节，应该能说明：

- 量化是跨模型文件、编译器、后端、runtime 和验证的系统合同。
- scale 来自校准，校准策略会在舍入误差和饱和误差之间取舍。
- 量化误差至少包含舍入、饱和、累加、元数据和状态误差。
- QuantDescriptor 必须进入 manifest、verifier、IR pass、backend lowering 和 oracle。
- 量化优化必须同时有正确性报告和性能报告。

下一章会把这套系统合同落到低比特权重和 KV cache 上：int4/group quant 怎样存储，scale 怎样跟 packed weight 对齐，KV cache 量化为什么是运行时状态问题，C++20 后端怎样把这些信息变成可测 plan。

2.5 int8 量化、累加器与重新量化

从当前 lab 的简化路径开始，但落点是 7B 权重

第三册的 lab 里已经有一个最小切片。模型文件 `tiny_mlp_i8.txt` 保存 int8 权重、float bias 和每层一个 scale。当前 `linear_i8` 的核心路径可以概括为：

Listing 2.41 当前 lab 的简化量化路径

```
weight_i8 = load int8 weight
weight_f32 = float(weight_i8) * layer.scale
acc_f32 += weight_f32 * input_f32
output_f32 = acc_f32 + bias
```

这条路径只量化了权重，输入和输出仍然是 float32，累加器也是 float32。它非常适合教学的第一步：模型文件已经变小，权重地址流已经是 int8，结果又能用普通浮点误差比较。但它不是完整 int8 推理，更不是 7B 大模型的最终量化方案。真实本地推理要同时面对权重容量、内存带宽、CPU/GPU kernel、activation 精度、KV cache 容量和生成质量。完整路径通常还会量化 activation，用 int32 做 dot-product 累加，再把 int32 累加结果重新映射到目标输出类型；更激进的权重量化还

会进入 int4、group-wise scale 和 dequant-on-the-fly。

核心理想

量化不是把类型名从 float 改成 int8。它是一份数值合同和后端合同：实数怎样映射到整数，乘加在哪个宽度里累加，bias 属于哪个尺度，输出怎样重新量化，KV cache 是否压缩，CPU/GPU kernel 怎样读 scale，误差怎样被 oracle 接受。

为什么 7B 推理绕不开量化

先只看权重容量，不看临时 activation、KV cache、allocator overhead 和 packed weight。一个 7B 参数模型若按不同 dtype 保存，大致账本是：

Listing 2.42 7B 权重容量的粗略账本

```
fp32: 7e9 * 4 bytes  ~= 28 GB
fp16: 7e9 * 2 bytes  ~= 14 GB
int8: 7e9 * 1 byte   ~= 7 GB, plus scales
int4: 7e9 * 0.5 byte ~= 3.5 GB, plus group scales
```

这个账本解释了为什么本地推理不会只讨论 fp32。许多普通机器装不下 fp32 权重；即使装得下，权重流也会压垮内存带宽。fp16/bf16 是常见精度基线，int8 和 int4 是本地部署的重要路径。量化的目标不是让文件更小这么简单，而是让模型能被加载、能在 cache/内存/显存带宽下持续供数，并且让 CPU/GPU kernel 能把压缩格式转化成可计算的乘加流。

线性量化的合同

最常见的线性量化把实数 x 映射到整数 q ：

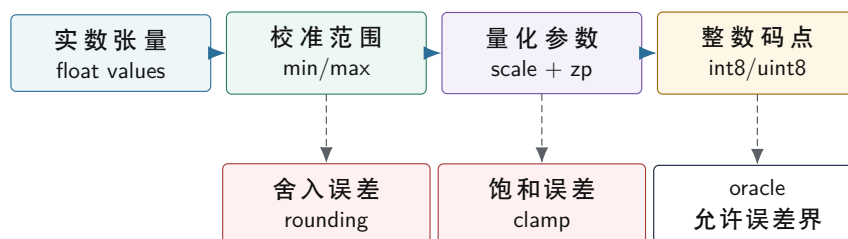
Listing 2.43 线性量化合同

```
q = clamp(round(x / scale) + zero_point, qmin, qmax)
x_approx = (q - zero_point) * scale
```

$scale$ 是整数相邻刻度之间代表的实数间隔， $zero_point$ 是实数零对应的整数位置。对称量化常让 $zero_point=0$ ，权重可以落在 $-128..127$ 或 $-127..127$ 附近；非对称量化允许零点偏移，适合 activation 范围不以零为中心的情况。

这个公式有三个必须写清的边界。第一， $round$ 采用哪种舍入规则；不同实现可能选择 ties-to-even、away-from-zero 或固定点近似。第二， $clamp$ 会吞掉超出范围的值；校准阶段若低估范围，极端值就会饱和。第三，反量化不是恢复原值，只是得到近似值；oracle 必须允许合同内误差。

实数、整数码点和误差合同



量化参数是数值合同的一部分；没有舍入、饱和和误差边界，int8 输出不能被可靠判定。

图示：本图要证明的命题是：量化误差来自范围选择、舍入和饱和，必须进入正确性合同。

为什么 accumulator 不能用 int8

两个 int8 数相乘，中间乘积已经不适合继续装在 int8。若 a 和 w 都接近 127，单次乘积约为 16129；再沿 K 累加，范围会继续扩大。因此 int8 dot-product 通常使用 int32 accumulator：

Listing 2.44 int8 dot-product 的整数账本

```
int32 acc = 0
for k in 0..K:
    int32 a = int32(input_i8[k]) - zero_input
    int32 w = int32(weight_i8[k]) - zero_weight
    acc += a * w
```

accumulator 的上界可以粗略估计：

Listing 2.45 accumulator 范围估计

```
worst_product ~= 127 * 127
worst_acc      ~= K * worst_product

if K = 4096:
    worst_acc is about 66 million
    int8 and int16 are not enough
```

真实模型不一定达到最坏情况，但内核不能依赖“通常不会”。溢出一旦发生，结果可能仍然像正常数字，不会崩溃，却会悄悄改变分类边界。使用 int32 accumulator 是把正确性边界先守住，再考虑指令吞吐和 requantize 成本。

Scale 怎样穿过乘加

若输入和权重都被量化，反量化语义是：

Listing 2.46 从量化值恢复 dot-product 语义

```
real_a[k] = (qa[k] - za) * scale_a
real_w[k] = (qw[k] - zw) * scale_w

real_acc = sum_k real_a[k] * real_w[k]
          = scale_a * scale_w *
            sum_k (qa[k] - za) * (qw[k] - zw)
```

这说明 int32 accumulator 保存的是无量纲整数和，真正的实数尺度在 $scale_a * scale_w$ 上。bias 必须进入同一个尺度。常见做法是把 float bias 预先量化成 int32 bias：

Listing 2.47 bias 折叠到 int32 accumulator

```
bias_i32[out] = round(bias_f32[out] / (scale_a * scale_w[out]))
acc_i32 += bias_i32[out]
```

若使用 per-tensor weight scale，所有输出通道共用一个 $scale_w$ ；若使用 per-channel weight scale，每个 out 有自己的 scale，精度通常更好，但 requantize 和参数管理更复杂。第三册在这一章先讲清账本，后面再讨论不同 scale 粒度对模型精度和内核复杂度的影响。

重新量化不是简单右移

如果下一层还希望接收 int8 activation，就要把 int32 accumulator 映射回 int8 或 uint8：

Listing 2.48 重新量化的数学形状

```

real_out = acc_i32 * scale_a * scale_w
q_out = clamp(round(real_out / scale_out) + zero_out, qmin, qmax)

combined_multiplier = scale_a * scale_w / scale_out
q_out = clamp(round(acc_i32 * combined_multiplier) + zero_out, qmin, qmax)

```

这里的 `combined_multiplier` 很少刚好是 2 的幂。高性能实现通常把它近似成固定点乘法加 shift，再处理舍入和饱和。于是重新量化至少包含四步：乘以缩放因子、舍入、加输出零点、clamp 到目标整数范围。若还有 ReLU 或 ReLU6，activation clamp 也可以融合进最后的范围限制。

int32 accumulator 到 int8 输出的重新量化路径



图示：本图要证明的命题是：requantize 是一条数值和整数实现共同参与的路径，不是把 int32 截断成 int8。

固定点 requantize 的边界

实际 int8 kernel 里不希望每个输出都做慢速浮点乘法。常见做法是把 `combined_multiplier` 近似为整数乘法和右移：

Listing 2.49 固定点 requantize 的形态

```

scaled = round(acc_i32 * multiplier / 2^shift)
q = clamp(scaled + zero_out, qmin, qmax)

```

这里要固定三件事。第一，`multiplier` 和 `shift` 怎样从实数 `multiplier` 生成。第二，右移前怎样舍入，负数是否对称处理。第三，乘法中间结果需要多宽，通常要用 64 位临时值避免溢出。

Listing 2.50 requantize 测试必须覆盖的值

```

acc_i32:
    0
    1
    -1
    near positive saturation
    near negative saturation
    values around rounding half
    large values requiring int64 intermediate

```

若这些边界没测，量化输出可能在普通样本上看起来正常，却在某些通道长期偏一。对分类模型，偏一可能不改变 top-1；对语言模型 logits，长期偏差可能改变 top-k 排名。

zero point 修正项为什么会拖慢内核

非对称量化中 dot-product 是：

$$\sum_k (q_a[k] - z_a)(q_w[k] - z_w)$$

展开后：

$$\sum_k q_a[k]q_w[k] - z_w \sum_k q_a[k] - z_a \sum_k q_w[k] + Kz_az_w$$

这条式子说明，非对称 zero point 不是免费字段。kernel 要么每次乘法前减 zero point，要么预计算某些和，例如每个权重行的 $\sum q_w[k]$ 。权重 zero point 若为 0，可以少掉一部分修正；activation zero point 若固定，也可以把某些项合并进 bias 或预处理。

选择	内核影响	常见用途
权重对称	少一个权重零点修正	weight-only 或 per-channel weight
activation 非对称 二者非对称	更好利用 activation 范围 精度可能好	int8 activation pipeline 修正项和测试更复杂

这也是很多推理后端先做对称权重量化的原因：收益明显，内核合同较清楚。等 reference、oracle 和 profiler 稳定后，再引入 activation 非对称量化更稳。

bias 的尺度错误会稳定污染输出

bias 常被忽略，但量化路径里它非常敏感。若 int32 accumulator 表示的是：

$$acc_{i32} = \sum_k q_a[k]q_w[k]$$

则它对应的实数尺度是 `scale_a*scale_w`。bias 若仍是 float，需要先除以这个尺度再加入 accumulator，或在 accumulator 变回实数后加入。两种做法语义都可以，但不能混用。

Listing 2.51 两种 bias 路径不能混用

```

path A:
acc_i32 += round(bias_f32 / (scale_a * scale_w))
output = requantize(acc_i32)

path B:
real = acc_i32 * scale_a * scale_w
real += bias_f32
output = quantize(real)

```

若 path A 的 bias 被当成 path B 使用，或者 per-channel `scale_w[out]` 被错用成 per-tensor scale，输出会出现稳定偏移。它不像随机舍入误差，而是每次都朝同一方向偏。oracle 应该单独测 bias-only、zero-input、one-hot-input 这些样本，便于定位。

对称、非对称、per-tensor、per-channel 与 per-group

量化参数的粒度会影响误差和内核复杂度。

对称权重量化。 权重通常正负都有，使用 `zero_point=0` 可以减少内核里的减零点操作，也让 dot-product 更简单。当前 lab 的模型就是这种教学形状：int8 权重乘以一个 float scale。

非对称 activation 量化。 activation 可能范围偏正，例如 ReLU 后没有负数。使用非对称 zero point 可以更充分利用整数码点，但内核要处理 `qa-za`，有时还要展开零点修正项。

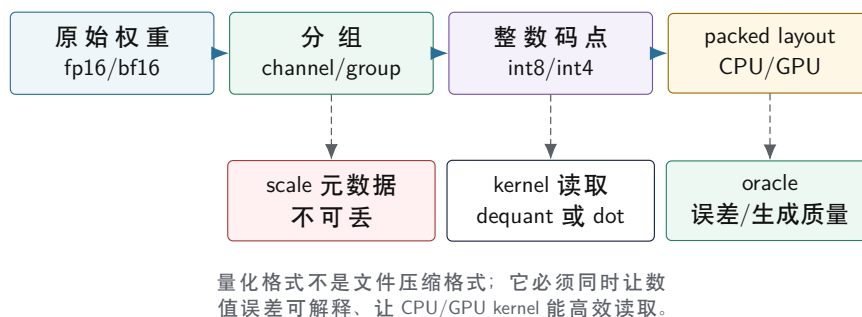
per-tensor scale。 整个张量共用一个 scale，实现简单，元数据少，但某些输出通道如果范围差异很大，小范围通道会损失精度。

per-channel scale。 每个输出通道拥有自己的权重 scale，常用于权重量化。它改善精度，但 requantize 阶段每个通道的 multiplier 不同，packing 和参数读取也要配合。

per-group scale。 大模型权重量化常按一小组连续权重共享 scale，例如每 32、64 或 128 个元素一组。它比 per-tensor 更能贴近局部范围，又比 per-element 元数据少。代价是 kernel 必须在读取权重块时同步读取 group scale，并且 packing 时不能把 scale 和数据关系打散。

工程上没有一个永远正确的选择。小型教学切片可以先使用 per-tensor scale，让模型文件、内核和 oracle 都清楚；7B 推理引擎要逐步引入 per-channel 或 per-group scale、校准数据集、后训练量化报告，以及 CPU/GPU 后端都能理解的 packed quantized layout。

7B 权重量化格式必须同时服务数值和后端



图示：本图要证明的命题是：7B 权重量化的 scale 粒度、整数码点和 packed layout 必须一起设计。

量化 oracle 要比较什么

浮点 reference 和 int8 实现不应该逐 bit 相等。对于分类切片，oracle 至少要比比较三类结果：

- 数值误差：每个输出元素的绝对误差、相对误差或反量化误差是否在合同内。
- 排名稳定性：分类任务的 top-1 或 top-k 是否保持，边界样本是否被标记。
- 饱和统计：有多少值被 clamp 到最小或最大码点，是否集中在某一层或某一通道。

如果只看 top-1，很多数值错误会被掩盖；如果只看逐元素误差，分类边界变化又可能被低估。对 7B 生成模型，还要增加层级 logits 检查和生成质量检查：固定 prompt、固定 sampler、固定随机种子时，logits 的 top-k 排名是否稳定；量化前后的困惑度或小验证集损失是否在阈值内；长上下文时 KV cache 量化是否导致后半段输出明显漂移。成熟报告应该同时写出 reference 版本、量化参数、输入样本集合、误差阈值、top-k 结果、饱和计数、logits 对比和生成样本摘要。

Listing 2.52 量化误差报告的最小字段

```

model_id
input_set_id
layer_name
scale_policy: per_tensor or per_channel
max_abs_error
mean_abs_error
top1_changed_count
clamped_min_count
clamped_max_count
logits_topk_changed_count
decode_token_mismatch_count
accepted: true or false
  
```

KV cache 能不能量化

KV cache 和权重不同。权重在加载后基本不变，可以离线量化、打包、校准；KV cache 是推理过程中不断写入的状态。每生成一个 token，每一层都会追加新的 K 和 V。若上下文很长，KV

cache 会成为显著内存开销，也会成为 decode attention 的读取流。

Listing 2.53 KV cache 的状态账本

```
K_cache[layer, batch, head, position, head_dim]
V_cache[layer, batch, head, position, head_dim]

on each decoded token:
    compute k, v for current position
    write k, v into cache
    attention reads positions 0..current_position
```

是否量化 KV cache，要分开判断容量、带宽和误差。量化后 cache 更小，长上下文可能明显受益；但每次写入要量化，每次 attention 读取可能要反量化或使用支持低精度的 kernel。若 scale 按 token、head 或 block 变化，元数据读写也会进入热路径。KV cache 量化因此不能只看“省了多少字节”，还要看 decode tokens/s、logits 漂移和长上下文质量。

常见误区

权重量化成功不代表 KV cache 也能用同一套策略。权重是静态参数，KV cache 是动态状态；权重误差在每层固定存在，KV cache 误差会随着上下文读取方式进入后续 token。

当前 lab 到 7B 量化路径的升级点

现在的 `linear_i8` 可以作为第一阶段验收：模型加载、shape 检查、int8 权重读取、float scale 和 bias、activation、argmax 都已经清楚边界。要把它升级成 7B 推理引擎里的量化路径，应分阶段推进，而不是一次重写：

Listing 2.54 从教学线性层到完整 int8 kernel 的阶段

```
stage 1:
    int8 weights, float input, float accumulator
    easy oracle and model parsing

stage 2:
    int8 weights, int8 activation, int32 accumulator
    explicit input scale and zero point

stage 3:
    int32 bias folded into accumulator scale
    requantize output to int8 activation

stage 4:
    packed weights, SIMD dot-product, per-channel scales
    benchmark and error report tied to one model id

stage 5:
    group-wise int4 or int8 weights for transformer projections
    dequant-on-the-fly and backend-specific packed layout

stage 6:
    optional KV cache quantization
    long-context error and decode throughput report
```

每个阶段都必须保持同一件事：reference 语义不变。若引入 int8 activation 后 top-1 变化，或

7B 模型的 logits/top-k/token 序列变化，要能判断是量化误差超出合同，还是实现 bug，例如 scale 用错、zero point 漏减、bias 尺度不一致、requantize 舍入规则不匹配、group scale 读错、packed layout 和 kernel 解释不一致。

本节验收线

读完本节，应该能完成下面几件事：

- 写出线性量化和反量化公式，并说明 scale、zero point、round 和 clamp 的作用。
- 解释 int8 dot-product 为什么需要 int32 accumulator。
- 把 `scale_a*scale_w/scale_out` 推导成 requantize 的 combined multiplier。
- 区分当前 lab 的简化路径和完整 int8 activation 路径。
- 解释 7B 权重在 fp16、int8、int4 下的容量压力，并说明 per-channel/per-group scale 的取舍。
- 说明 KV cache 量化为什么不同于权重量化。
- 设计一个量化 oracle，至少包含数值误差、top-k 稳定性、饱和统计、logits 对比和生成 token 变化。

后续内容会把这些量化账本放回算子实现：当权重被 packing、SIMD dot-product 指令被使用、GPU kernel 接入、多个层融合、activation buffer 被复用、KV cache 被压缩时，scale、zero point、accumulator、group metadata 和 oracle 仍然必须能被追踪。

2.6 低比特权重与 KV cache 量化：int4、group scale 与运行时状态

从 int8 走到 int4，问题变在哪里

int8 量化已经把 fp16 权重容量减半；int4 继续把 int8 再减半。对 7B 模型，容量差异非常直接：

Listing 2.55 7B 权重容量和元数据压力

```
fp16 weight: 7e9 * 2 bytes  ~= 14 GB
int8 weight: 7e9 * 1 byte   ~=  7 GB + scales
int4 weight: 7e9 * 0.5 byte ~=  3.5 GB + group scales
```

但 int4 不是“更小的 int8”。一个 int8 元素天然占一个字节；两个 int4 值通常打包在一个字节里。kernel 读取后要解包 nibble，处理符号扩展或 zero point，再乘 scale。为了精度，int4 权重很少整层共用一个 scale，通常按 group 保存 scale，例如每 32、64 或 128 个连续权重共享一个 scale。于是后端要同时处理两条流：压缩权重流和 scale metadata 流。

核心思想

int4 的主要难点不是 4-bit 算术本身，而是 packed byte order、group scale、尾部 padding、解包成本、scale 读取局部性和 oracle 合同必须同时正确。

先看一个会悄悄算错的实现

下面的伪代码看起来很自然，但它省略了 packed order、signedness、zero point、scale 粒度、tail policy 和后端重排合同。

Listing 2.56 容易算错的 int4 解包形态

```
for in in 0..input_channels:
    byte = packed[in / 2]
    q = (in % 2 == 0) ? (byte & 0xF) : (byte >> 4)
    w = (q - zero_point) * scale[in / group_size]
    acc += x[in] * w
```

常见误区

低比特实现最危险的错误，是 reference、converter、packing 和 kernel 各自持有一份隐含解释。正确做法是先定义一个可序列化的量化合同，再让 converter、verifier、packing、kernel 和 oracle 全部读取同一份合同。

group quant 的逻辑格式

以 group size 64 为例，一个输出通道的一段权重可以分成若干组。每组有 64 个 4-bit code 和一个 scale：

Listing 2.57 group-wise int4 权重的逻辑形状

```
weight[out, in]

group_size = 64
group_id = in / 64
offset_in_group = in % 64

q4_code = packed_weight[out, group_id, offset_in_group]
scale = group_scale[out, group_id]
real_weight ~= decode_int4(q4_code) * scale
```

若 `in` 维度不能整除 group size，就需要 tail policy。常见选择是补零、补中性码点或单独记录尾组长度。tail policy 必须进入 descriptor。否则不同 kernel 会对最后一组有不同解释。

字段	含义	错误后果
bit width	每个码点占几位	解包错位，全部权重错误
signedness	有符号还是无符号码点	零点和符号扩展错误
group size	多少权重共享 scale	scale 索引错误
scale dtype	scale 存 fp16/fp32 等	metadata 读取宽度错误
packed order	低 nibble 先还是高 nibble 先	每两个权重交换或错解
tail policy	尾组如何补齐	最后一块输出异常

量化合同要进入 manifest

如果 manifest 只写“这个权重是 int4”，后端仍然无法安全执行。manifest 需要把量化格式拆成可检查字段。

Listing 2.58 manifest 中的量化字段草图

```
QuantTensorManifest:
  tensor_name
  logical_shape
  role: q_proj | k_proj | v_proj | mlp_up | ...
  storage_bits
  code_signedness
  zero_point_policy
  scale_granularity
  group_size
  scale_tensor_name
  packed_order
  tail_policy
  calibration_record_id
```

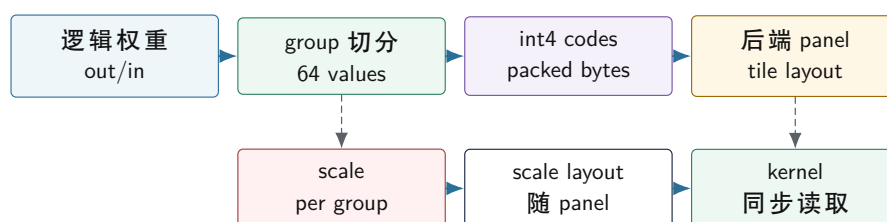

verifier 的任务不是重新做量化，而是拒绝不自洽的资产：packed byte 数、scale 数量、group size、tail policy、后端支持能力和角色一致性都要提前检查。

检查项	计算方式	失败时说明
packed bytes group count	$\text{ceil}(\text{elements} \times \text{bits} / 8)$ $\text{ceil}(\text{input_channels} / \text{group_size}) \times \text{output_channels}$	权重文件截断或格式解释错 scale 数量不匹配
backend support	capability 中是否支持该 profile	需要 fallback 或拒绝
tail policy	尾组是否有明确规则	最后一块输出不可验证
role consistency	同一逻辑角色是否 dtype/shape 一致	manifest 映射错误

packing 必须带着 scale 一起走

后端为了提高 cache locality 和 SIMD 利用率，通常会把权重从模型文件布局重排成 packed layout。对 int4/group quant，packing 不能只移动 4-bit 码点，还必须同步移动或重新索引 group scale。

int4 packing 同时重排权重码点和 scale



图示：本图要证明的命题是：int4 packing 不是只重排权重字节，scale metadata 必须跟着后端 panel 同步组织。

一个好的 packed descriptor 应该让 kernel 不需要查字符串、不需要理解原始文件格式，只需要按 descriptor 读取 panel：

Listing 2.59 int4 packed descriptor 草图

```

PackedInt4Weight:
  data: byte span
  scales: byte span
  output_channels
  input_channels
  group_size
  panel_rows
  panel_cols
  bytes_per_panel
  scales_per_panel
  nibble_order
  tail_policy
  
```

这个 descriptor 是第一册 ABI/对象布局知识和第二册 cache/SIMD 知识在第三册中的结合点。它既要在 C++20 中有明确对象生命周期和无拷贝视图，又要让后端按 cache line、SIMD 宽度和 tile 形状高效读取。

packing 要有字节级 oracle

只比较最终 logits, packing 错误定位会很晚。更可靠的办法是在 packed layout 形成后, 抽取逻辑坐标, 沿 descriptor 反查 packed byte、nibble、scale, 再和 reference converter 比较。

Listing 2.60 packed int4 字节级 oracle

```
for sample in coordinate_samples:
    logical = {out, in}
    reference = convert_from_original_file(logical)

    packed_location = descriptor.locate(out, in)
    code = read_nibble(descriptor.data, packed_location)
    scale = read_scale(descriptor.scales, packed_location.group)
    actual = decode(code, scale, descriptor.zero_point_policy)

    compare(actual, reference, tolerance)
```

验收与评分标准

int4 packed weight 的最低验收线:

- verifier 能计算 packed bytes、group count 和 scale count。
- byte oracle 覆盖高/低 nibble、group 边界、tail 和 panel 边界。
- 单算子 oracle 至少覆盖 input channel 非 group size 整数倍的形状。
- profiler 分开报告 packing time、packed bytes、scale bytes 和 kernel time。

dequant-on-the-fly 还是预反量化

int4 权重进入计算时, 有两种基本路线。第一种是 dequant-on-the-fly: kernel 每次读取 int4, 解包、乘 scale, 再参与 FMA 或 dot。第二种是预反量化: 加载或 prepare 阶段把 int4 转回 fp16/bf16/fp32 的 packed buffer, 热路径按浮点权重计算。

路线	优点	代价
dequant-on-the-fly 预反量化	内存带宽低, 容量小 kernel 简单, 可复用浮点路径	解包和 scale 乘法进入热路径 内存容量回升, prepare 成本高
混合路线	热层或热 tile 单独缓存	缓存策略和一致性更复杂

CPU decode 常常受权重读流和小 batch 形态影响, dequant-on-the-fly 可能有价值; 但如果解包成本和 scale 读取打断 SIMD pipeline, 收益会消失。prefill 阶段 GEMM 更大, 预反量化或 vendor kernel 可能更合适。没有 profiler, 不能靠直觉判断。

评估 dequant-on-the-fly 时, 不要只看“每个权重少读多少字节”。还要看 scale metadata、unpack/extend/convert、shuffle、寄存器压力和 tail path。低比特节省的是带宽, 也可能增加指令压力。

KV cache 量化是运行时问题

权重量化是静态问题: 模型加载前后, 权重不变。KV cache 量化是运行时问题: 每个 token 生成后, 所有层都会写入新的 K/V; 后续 attention 会反复读取历史 K/V。

Listing 2.61 KV cache 量化写入和读取路径

```
on kv_append(layer, position, k_f16, v_f16):
    scale_k = choose_scale(k_f16)
    scale_v = choose_scale(v_f16)
```

```

k_q = quantize(k_f16, scale_k)
v_q = quantize(v_f16, scale_v)
store k_q, v_q, scale_k, scale_v

on attention_decode(layer, q, context_length):
    for pos in 0..context_length:
        k = dequantize(load_k_q(pos), load_scale_k(pos))
        v = dequantize(load_v_q(pos), load_scale_v(pos))
        use k, v in attention

```

这段路径暴露三个关键问题。第一，scale 选择发生在 runtime，不能完全离线。第二，metadata 会随 position 增长，不能只统计 K/V 码点大小。第三，decode 每步会读取从 0 到当前 position 的历史；若每个 position 都有独立 scale，metadata 读取会变成热路径的一部分。

常见误区

KV cache 量化不能只报告“内存减半”。必须同时报告写入成本、metadata 成本、decode attention 耗时、logits 漂移和长上下文生成质量。

KV 量化误差先进入哪里

很多人会把 KV cache 量化想成“把历史 K/V 反量化回来，后面照常算 attention”。这句话只描述了数据流，没有描述误差流。真正先被扰动的不是最终输出文本，而是每个 head 的 attention score：

$$score_{i,j,h} = \frac{Q_{i,h} \cdot K_{j,g(h)}}{\sqrt{d}}$$

若量化后的 key 变成

$$\tilde{K}_{j,g(h)} = K_{j,g(h)} + \Delta K_{j,g(h)}$$

则 score 误差是：

$$\Delta score_{i,j,h} = \frac{Q_{i,h} \cdot \Delta K_{j,g(h)}}{\sqrt{d}}$$

这条式子比“量化会有误差”更重要，因为它说明了三件事。第一，误差会被当前 query 的方向放大或缩小，不是所有 token 位置都同样敏感。第二，同一个量化 profile，在不同 head、不同层、不同 query 上误差大小可能完全不同。第三，score 误差进入 softmax 后，会重新分配整行概率，而不是只影响一个位置。

如果 value 也量化，

$$\tilde{V} = V + \Delta V$$

则最终 attention 输出误差不只来自 ΔV ，还来自概率本身被改写。把概率写成

$$\tilde{P} = P + \Delta P$$

则：

$$\tilde{O} = \sum_j \tilde{P}_j \tilde{V}_j = \sum_j (P_j + \Delta P_j)(V_j + \Delta V_j)$$

展开后至少有三类误差项：

$$\Delta O \approx \sum_j P_j \Delta V_j + \sum_j \Delta P_j V_j + \sum_j \Delta P_j \Delta V_j$$

第一项是“value 被量化”的直接误差；第二项通常更危险，因为它表示 softmax 权重已经被改写，模型开始从历史中“看错位置”；第三项通常更小，但在长上下文和高压缩率下也不能忽略。

核心思想

KV cache 量化最敏感的地方往往不是 V 本身，而是 K 进入点积后对 softmax 排序和归一化的扰动。只看反量化后的 L2 误差，不足以判断生成质量。

softmax 为什么会放大量化误差

softmax 不是线性函数。若某一行 score 中本来有两个位置非常接近，比如：

$$s_a = 12.10, \quad s_b = 12.04$$

那么一个很小的量化误差就可能改变它们的排名。假设量化后变成：

$$\tilde{s}_a = 12.00, \quad \tilde{s}_b = 12.08$$

表面上每个位置只改了不到 **0.1**，但 attention 概率最大的那个位置已经从 **a** 变成 **b**。如果这两个位置对应的是两个不同语义片段，后续层读到的内容就会完全不同。长链条 decode 中，这种细小排序翻转会沿层数和 token 数累积。

再看稳定 softmax 的分母：

$$p_j = \frac{e^{s_j - m}}{\sum_t e^{s_t - m}}$$

当一行中存在几个接近最大值的位置时， $\Delta score$ 会通过指数放大为概率差异；当某个位置明显比其余位置大很多时，小误差反而可能不敏感。也就是说，量化误差的危害和“当前这行 attention 是尖锐分布还是接近平局”直接相关。

行分布形态	对 score 小误差的敏感性	典型现象
单峰明显领先	较低	top-1 不变，概率小幅扰动
多个位置接近	很高	排名翻转，读错历史片段
长尾很平	中等	概率整体重分配，输出更发散

因此 KV cache 量化评估不能只做 token 级平均误差。至少还要观察：

- attention score 的最大误差和分位数；
- top-1 attention 位置是否翻转；
- 重要 head/层是否在长上下文上更敏感；
- logits top-k 是否在长 context 档位下明显变化。

一个具体的 KV 量化误差数值例子

上面讲的是误差路径，现在把它压到可以手算的数字。设某个 head 的 query 和某一历史 key 为：

$$q = [0.60, -0.20, 0.50, 0.10]$$

$$K = [0.80, -0.35, 1.10, -0.50]$$

head 维度 **d=4**，所以原始 attention score 是：

$$score = \frac{q \cdot K}{\sqrt{4}} = \frac{0.60 \times 0.80 + (-0.20) \times (-0.35) + 0.50 \times 1.10 + 0.10 \times (-0.50)}{2} = 0.525$$

现在先看一种“局部范围较准”的 int4 对称量化。若这一个量化块的最大绝对值就是 **1.10**，则 scale 可取：

$$\alpha_{local} = \frac{1.10}{7} \approx 0.157$$

量化码点是：

$$q_K = \text{round}(K/\alpha_{local}) = [5, -2, 7, -3]$$

反量化后：

$$\tilde{K}_{local} = q_K \alpha_{local} \approx [0.786, -0.314, 1.100, -0.471]$$

新的 score 变成：

$$score_{local} = \frac{q \cdot \tilde{K}_{local}}{2} \approx 0.519$$

所以：

$$\Delta score_{local} \approx -0.006$$

再看一种更粗的情况。若这个 key 被塞进一个更大 block，block 内别的位置把最大绝对值拉到 **1.50**，则 scale 变成：

$$\alpha_{block} = \frac{1.50}{7} \approx 0.214$$

这时码点是：

$$q'_K = \text{round}(K/\alpha_{block}) = [4, -2, 5, -2]$$

反量化结果变成：

$$\tilde{K}_{block} \approx [0.857, -0.429, 1.071, -0.429]$$

新的 score 是：

$$score_{block} = \frac{q \cdot \tilde{K}_{block}}{2} \approx 0.546$$

于是：

$$\Delta score_{block} \approx +0.021$$

这组数字说明两个关键事实。第一，误差大小不只由 bit width 决定，还强烈依赖 scale 粒度和同块中的动态范围共享。第二，同一个原始 key，在两个都“合法”的 int4 配置下，score 偏移方向都可能相反：一个向下漂，一个向上漂。

现在把它放回 attention 排名。假设同一行里另一个候选位置的原始 score 是：

$$score_{rival} = 0.520$$

则原始情况下当前 key 领先：

$$0.525 > 0.520$$

局部量化后变成：

$$0.519 < 0.520$$

排名翻转；而粗 block 量化后又变成：

$$0.546 > 0.520$$

不但没翻转，领先还更大。

常见误区

这就是为什么很多人只看反量化后的均方误差会误判 KV 量化质量。真正决定 attention 行为的不是“整体误差小不小”，而是“关键候选位置之间的 score 排名会不会被改写”。

若继续往下一步看 softmax，排名翻转的影响会直接进入概率分配。尤其当一行里几个位置本来就接近时，**0.01** 到 **0.03** 级别的 score 扰动已经足以改写 top-1 attention 位置。对长上下文生成，这种翻转不是一次性的，它会把错误读出的上下文写回后续层 hidden，再影响下一步 query。

KV cache 的容量账本

先不考虑量化，一个 decoder-only 模型的 KV cache 容量可以用一条式子估算：

Listing 2.62 KV cache 基本容量估算

```
bytes =
  layers *
  batch *
  context_tokens *
  kv_heads *
  head_dim *
  2 /* K and V */ *
  bytes_per_element
```

这里的 **kv_heads** 不一定等于 attention heads。多查询注意力或分组查询注意力会减少 K/V head 数量，但不会消除 context length 线性增长。量化后还要把 scale metadata、page table、padding、对齐浪费和临时 dequant buffer 算进去。

项目	进入容量账本吗	说明
K/V 码点	是	最大主体，随 context 增长
scale metadata	是	粒度越细越大，也影响热路径
page table	是	分页 cache 或 sliding window 需要
padding/alignment	是	block、head 和设备对齐会产生浪费
临时 dequant buffer	视策略而定	若 attention 前反量化，峰值内存上升

把 metadata 字节也算出来

继续使用 7B 级 GQA 参数：layers=32, kv_heads=8, head_dim=128, context=4096, batch 为 1。fp16 KV 码点容量前面算过是 512MiB。如果把 K/V 码点改成 int8，码点主体约为：

Listing 2.63 int8 KV 码点主体容量

```
code_bytes =
  32 * 4096 * 8 * 128 * 2 * 1
  = 268435456 bytes
  = 256 MiB
```

但是 scale metadata 要看粒度。若每个 layer, position, kv_head 为 K 和 V 各存一个 fp16 scale，则：

Listing 2.64 per-token-per-head fp16 scale 容量

```
scale_bytes =
  layers * context * kv_heads *
  2 /* K and V */ *
  2 /* fp16 scale */

scale_bytes =
  32 * 4096 * 8 * 2 * 2
  = 4194304 bytes
  = 4 MiB
```


这个 metadata 相对 256 MiB 码点不大，但它在 attention 热路径中被反复读取。若改成 per-channel scale，每个 `head_dim` 都有 scale，则 metadata 变成：

Listing 2.65 per-token-per-channel fp16 scale 容量

```
scale_bytes =
    layers * context * kv_heads * head_dim *
    2 /* K and V */ *
    2 /* fp16 scale */

scale_bytes =
    32 * 4096 * 8 * 128 * 2 * 2
    = 536870912 bytes
    = 512 MiB
```

这就完全改变了账本：KV 码点降到 256 MiB，但 scale metadata 自己达到 512 MiB，容量反而比 fp16 KV 更差，还让 kernel 读取更多元数据。per-channel 可能精度好，但不能不算 metadata。

若使用 int4 KV，码点主体再减半到约 **128MiB**。但如果 scale 粒度太细，metadata 会吞掉大部分收益。因此 KV 量化报告必须同时列出 `code_bytes`、`scale_bytes`、`page_table_bytes` 和 `alignment_waste_bytes`。

核心思想

KV cache 量化的核心不是 bit width 越低越好，而是码点容量、scale metadata、读取热路径和误差共同闭合。metadata 不进入账本的量化报告没有工程意义。

KV dequant buffer 要不要物化

attention 读取量化 KV 时有两种路线。第一种是 on-the-fly dequant：读一个 block 的量化 K/V 和 scale，在寄存器或 shared memory 中还原并立即参与 score 和加权求和。第二种是先把一段历史 K/V 反量化到临时 buffer，再按普通 fp16 attention 处理。

路线	优点	风险
on-the-fly	不增加大临时 buffer，容量优势保留	kernel 复杂，scale 读流和解包进入热路径
物化 dequant buffer	可复用现有 attention kernel	峰值内存上升，额外写回和读取
分块物化	折中，适合 tiled attention	block 边界和 softmax 状态更复杂

如果物化整个 context 的 K/V，临时 buffer 大小接近 fp16 KV 的读取窗口。对 **4096** context、GQA 参数，单个 session 全层物化会非常大，通常不可接受。实际更可能按层、按 head 或按 block 物化，处理完立即释放。descriptor 和 memory planner 必须知道这个 workspace，否则峰值内存报告会少算。

KV cache scale 粒度

KV cache 的 scale 可以按不同粒度设计：

粒度	含义	风险
per-tensor	整个 cache 或整层共享 scale	范围过粗，误差可能大
per-head	每个 head 共享 scale	metadata 适中，但 position 差异被忽略
per-token	每个 position 一组 scale	精度好，metadata 热路径更重
per-block	每个 position block 一组 scale	精度与开销折中，需要 page/block 管理
per-channel	head_dim 内不同维度独立 scale	精度更细，metadata 和 kernel 复杂

对长上下文 decode，per-token 或 per-block 常更有解释力，因为不同 token 的 K/V 范围可能变化。但 scale 粒度越细，metadata 越多，attention kernel 的读取也越复杂。GPU 上还要考虑 coalesced load；CPU 上要考虑 cache line 和 TLB。这个选择必须由模型质量和 profiler 共同决定。

为什么 scale 粒度会改变误差分布

scale 粒度不是单纯的 metadata 体积选择，它直接决定误差怎样分布到 **head_dim** 和 **position** 上。设某个量化块的真实值向量为 $\mathbf{x}$ ，scale 为 α ，码点为 $\mathbf{q}$ ，则量化过程可以抽象成：

$$q = \text{clip}(\text{round}(x/\alpha)), \quad \tilde{x} = \alpha q$$

误差是：

$$\Delta x = \tilde{x} - x$$

若一个 scale 覆盖的数值范围太大，较小分量会被粗糙舍入；若 scale 覆盖范围太小，极值分量容易饱和。于是不同粒度的本质区别是：它们是在什么维度上共享动态范围。

per-head。一个 head 的所有位置或某个大块共用 scale。优点是 metadata 轻；缺点是如果这个 head 某几个 token 的 K/V 振幅特别大，其余位置的量化分辨率会被拖粗。

per-token。每个 position 各自选 scale。它能较好适应不同 token 的振幅变化，尤其适合长上下文中不同段落、代码块、表格或多语言文本混合时的局部范围变化；但每个 position 都要带 metadata，decode 热路径会反复读取这些 scale。

per-channel。**head_dim** 内不同维度各自选 scale。它能照顾某些维度长期幅值大、某些维度长期幅值小的情况，但 metadata 非常重，而且 kernel 需要更细粒度反量化。

per-block。在 position 和 dim 之间取一个折中块。它经常是工程上最实用的选择，因为既能比 per-head 更贴近局部范围，又不会像 per-token/per-channel 那么重。

可以把不同粒度理解成“谁共享同一个动态范围预算”。共享得越广，metadata 越省，但误差耦合越强；共享得越细，精度越好，但热路径越重。KV cache 和静态权重不同，它还要考虑历史长度增长后的累计读取成本，所以最优粒度往往不是权重量化里最优的那一个。

粒度	动态范围共享对象	metadata 成本	误差特征
per-head	整个 head 的较大区域	低	局部极值拖粗整体分辨率
per-token	每个 position	中高	适应 token 间范围变化
per-channel	dim 维度	很高	适应维度间长期振幅差异
per-block	局部块	中等	在局部适应性和成本间折中

长上下文为什么更容易暴露 KV 量化问题

短上下文中，decode 每步只看很少历史位置，attention 的候选集合小，很多误差还没有充分积累。长上下文中，问题会同时从三个方向放大。

第一，候选位置数变多。只要某些历史位置的 score 因量化误差发生排序翻转，softmax 就可能把注意力分到不同 token 上。第二，历史分布更复杂。长上下文里往往混有多段主题、代码、表格、指令头、自然语言解释，K/V 的局部动态范围比短 prompt 更不均匀。第三，误差会跨 token 累积。一次 decode 读错历史，不一定当步就输出错误 token，但它会把错误信息写入后续层 hidden，再生成下一步的 query。

这也是为什么 KV cache 量化报告必须按 context length 分桶，而不是只报一个全局平均值。一个合理的最小报告形态至少应包含：

Listing 2.66 长上下文量化报告的最小分桶

```
context buckets:
  0-512
  512-2048
  2048-8192
  8192+

for each bucket report:
  logits_topk_change
  attention_top1_flip_rate
  decode_tokens_per_second
  kv_metadata_bytes
```

常见误区

如果一个 KV 量化方案只在 256 或 512 token 上验证“几乎无损”，这个结论对 8K、32K 长上下文没有证明力。KV cache 是随上下文长度增长的运行时状态，不是一次性离线张量。

分页 KV cache 和 sliding window

真实请求不会总是从长度 0 开始生成固定短文本。服务端可能同时维护多个 session，每个 session 的 context length 不同，还可能触发上下文截断、prefix cache 复用或 sliding window。此时 KV cache 不再是一个简单的大连续数组，而更像一组 page 或 block。

Listing 2.67 分页 KV cache 的逻辑映射

```
logical position:
  session_id
  layer
  kv_head
  token_position
  head_dim_range

physical location:
  page_id = page_table[session_id, block_index]
  offset = position_in_block * stride_token + head * stride_head
```

分页设计便于增长、回收和 prefix 复用，但 attention kernel 要处理 page table，scale metadata 也要跟随 page 管理。sliding window 还需要逻辑 position 到物理 slot 的环形映射；descriptor 若不记录 epoch 或 base position，很容易读到旧 token。

工程案例

一个常见长上下文 bug 是：KV cache 物理 slot 被复用后，scale metadata 没有同步更新，或者 profiler 仍按旧 position 汇总。短 prompt 下这个 bug 不出现；只有 context 超过一个 page 或 sliding window 回绕后，logits 才开始漂移。

C++20 runtime 中的 KV cache descriptor

KV cache 量化需要 runtime descriptor，而不是只存在模型 manifest 中。

Listing 2.68 量化 KV cache 的运行时 descriptor

```
QuantizedKVCacheDesc:
  layers
  batch
  kv_heads
  head_dim
  capacity_tokens
  storage_dtype
  scale_dtype
  scale_granularity
  block_size
  layout
  page_table_policy
  dequant_policy
```

如果 KV cache 使用 page/block 增长，descriptor 还要指向 page table。若使用 sliding window，descriptor 必须记录逻辑 position 到物理 slot 的映射。若 CPU/GPU 各自维护 cache，runtime 必须知道数据在哪个设备上，什么时候需要同步或迁移。

Listing 2.69 KV cache append 的边界检查

```
append(layer, session, position, k, v):
  if position >= capacity_tokens:
    return context_limit_error
  slot = map_logical_to_physical(session, position)
  if slot.epoch != expected_epoch:
    return stale_slot_error
  quantize_and_store(k, v, slot)
  mark_written(layer, session, position)
```

KV cache 写入不是普通数组赋值。它至少要处理 capacity、逻辑到物理映射、epoch、量化数据、metadata 和 written 状态。

低比特量化的 oracle

低比特量化 oracle 至少要分四层：

- 权重还原层：随机抽查 packed int4 权重和 scale，确认反量化值匹配 reference converter。
- 单算子层：linear、attention、MLP 输出与 fp16/fp32 reference 比较。
- logits 层：比较 top-k、最大误差、KL 或分布差异。
- 生成层：固定 prompt、sampler 和 seed，比较 token 序列、困惑度或小验证集指标。

对 KV cache，oracle 还要增加 position 维度。短上下文正确不代表长上下文正确；前 32 个 token 正常不代表 2048 之后不漂移。报告必须包含 context length 分档。

Listing 2.70 低比特量化验收报告字段

```

quant_format:
  weight_bits
  group_size
  scale_dtype
  kv_cache_bits
  kv_scale_granularity

correctness:
  packed_weight_sample_errors
  layer_error_by_name
  logits_topk_change
  decode_mismatch_by_context_length

performance:
  model_load_time
  packing_time
  prefill_latency
  decode_tokens_per_second
  kv_cache_bytes
  metadata_bytes

```

量化决策要接回后端计划

量化 profile 最终必须影响 executable plan。后端可能支持 int8 GEMM，但不支持 int4 group size 128；可能支持 q4 权重，但要求 scale 连续对齐；可能支持 fp16 KV cache，但不支持 int8 KV cache attention。这些能力必须进入 plan。

Listing 2.71 量化 profile 与后端选择

```

QuantProfile:
  weight_format
  activation_format
  kv_cache_format
  error_tolerance_profile

BackendCapability:
  supported_weight_formats
  supported_kv_formats
  preferred_group_sizes
  requires_packed_scales

Plan decision:
  if backend supports profile:
    choose optimized kernel
  else if fallback allowed:
    choose reference or dequantized kernel
  else:
    reject with diagnostic

```

这让低比特量化成为编译器问题，而不是 kernel 小技巧：manifest 描述资产，verifier 检查自治，compiler 根据 backend capability 选择计划，runtime 执行计划，oracle 和 profiler 证明收益。

本节验收线

读完本节，应该能说明：

- int4 的难点在 packed byte order、group scale、tail policy、解包和 scale 读取。
- 量化合同必须进入 manifest 和 verifier，而不是由 kernel 隐式猜测。
- packing 必须同时组织权重码点和 scale metadata。
- packed int4 要有字节级 oracle，覆盖 nibble、group、tail 和 panel 边界。
- dequant-on-the-fly 与预反量化各有容量、带宽和计算成本取舍。
- KV cache 量化是运行时状态问题，不等同于静态权重量化。
- KV cache 容量账本必须包含码点、scale metadata、page table、padding 和临时 buffer。
- KV cache scale 粒度会同时影响误差、metadata 大小和 attention kernel 热路径。
- 分页 KV cache 和 sliding window 需要显式记录逻辑到物理映射和 stale slot 防护。
- 低比特量化 oracle 必须覆盖 packed weight、单算子、logits、生成序列和 context length。
- 量化 profile 必须参与 backend capability 检查和 executable plan 选择。

后续进入具体 C++20 后端实现时，本节的 descriptor 和报告字段会直接变成代码结构、测试 fixture 和 benchmark 输出，而不是停留在文字说明。

2.7 量化工程实现：C++20 descriptor、校准记录、oracle 与回归门禁

为什么量化实现不能只写一个转换脚本

前两章已经说明，量化不是把 float 改成 int8，也不是把两个 int4 塞进一个字节就结束。真正的工程问题是：模型文件、编译前端、IR pass、后端 packing、runtime、oracle 和 profiler 必须对同一份量化语义达成一致。只要其中一层用自己的口径解释 scale、zero point、group size 或 tail policy，输出就可能稳定地错。

先看一个实际会发生的错误。转换脚本把某层 MLP 的 Wup 权重量化成 int4，每 64 个输入通道共享一个 scale。后端为了 SIMD 方便，把权重重排成 16x64 的 panel。若 packing 只移动 int4 code，没有把每个 group 的 scale 按同样 panel 顺序重排，kernel 会读取错误 scale。这个错误不一定崩溃，也不一定产生 NaN；它更可能让某些 token 的 logits 轻微漂移，直到长生成时才表现为质量变差。

核心理想

量化工程实现的核心不是某个低位宽格式，而是结构化元数据和验收链路。C++20 项目必须让 descriptor、verifier、packing、kernel、oracle 和 profiler 使用同一套量化合同。

本节把量化落到 C++20 项目结构。目标不是给出某个库的完整代码，而是讲清楚每个模块应该负责什么、哪些字段必须显式保存、哪些错误应该在编译前端拦住、哪些误差应该在 oracle 中报告、哪些性能收益必须由 profiler 证明。

QuantDescriptor：量化合同的最小载体

一个量化张量至少有三层身份。第一层是逻辑身份：它近似表达的是浮点权重、activation 还是 KV cache。第二层是存储身份：字节里到底是 int8、uint8、int4、nf4、fp8，scale 是 fp16 还是 fp32。第三层是计算身份：kernel 怎样反量化、累加和输出。把这三层都压进一个 dtype=q4 字符串，是后续混乱的源头。

Listing 2.72 QuantDescriptor 的工程字段

```
QuantDescriptor:
  logical_value: weight | activation | kv_cache
  storage_format: int8 | uint8 | int4 | nf4 | fp8
  bit_width
  signedness
  scale_dtype: f16 | f32
  zero_point_policy: none | symmetric | asymmetric
```



```
granularity: tensor | channel | group | token | block
axis
group_size
block_size
scale_shape
zero_point_shape
packed_order
tail_policy
calibration_id
error_contract_id
```

这些字段看起来多，但每一个都能对应一个真实错误。没有 **signedness**，int4 code 的符号扩展可能错；没有 **axis**，per-channel scale 会沿错维度读取；没有 **scale_shape**，verifier 无法判断 scale 数量是否匹配；没有 **packed_order**，高 nibble 和低 nibble 的解释会不一致；没有 **tail_policy**，最后一组不整除 group size 时会出现后端分歧。

常见误区

descriptor 的目标不是让代码显得正规，而是禁止后端猜测。任何需要 kernel 根据 tensor name、文件路径或隐含模型家族推断的量化事实，都应该提升为 descriptor 字段。

在 C++20 中，descriptor 应该是一个轻量、可拷贝或可引用的值对象。它不拥有大内存，不打开文件，不选择 kernel。它只描述合同，并能被 manifest、verifier、IR、packing、runtime 和报告共同引用。

Listing 2.73 C++20 中 descriptor 的责任边界

```
QuantDescriptor
  owns: small enum fields, shape metadata, ids
  does not own: tensor bytes, scale buffer, backend packed bytes

TensorDesc
  shape, stride, dtype, layout, device
  optional QuantDescriptor id

TensorStorage
  byte span or owned buffer
  file mapping or allocated memory
```

这样拆分以后，模型导入可以先建立 **TensorDesc** 与 **QuantDescriptor**；后端 prepare 阶段再用它们生成 **PackedWeightDesc**；runtime 执行时只绑定已经验证过的 descriptor 和 buffer。

CalibrationRecord：校准不是临时日志

校准选择 scale。scale 选择错了，后端再快也只是高效地产生错误近似。工程上不能只在转换脚本里打印一行 **calibrated**，而要保存可追溯的校准记录。

Listing 2.74 校准记录需要保存的事实

```
CalibrationRecord:
  calibration_id
  tensor_name_or_role
  dataset_id
  sample_count
  token_length_buckets
```

```

algorithm: minmax | percentile | mse_search | kl | runtime
clipping_policy
observed_min
observed_max
saturation_rate
scale_summary
created_by_tool_version

```

对权重，校准记录主要说明扫描范围和算法。对 activation，校准记录必须说明代表性输入。对 KV cache，校准更复杂，因为 K/V 是运行时状态；报告应该按 prompt 类型、context length 和层/头维度统计范围。如果一个量化模型只给出 scale 文件，却没有校准记录，后续质量问题很难定位：是模型本身不适合低比特，还是校准集太偏，还是后端解释错了 metadata？

对象	校准重点	必须警惕的偏差
权重	每层/每通道/每组范围	极端值拉大 scale, 尾组处理不一致
activation	代表性 prompt 和 batch 形态	校准集短、语言单一、没有代码或长上下文
KV cache	position、head、context length 分布	短上下文通过，长上下文漂移
logits	采样前分布稳定性	top-k 变化被均值误差掩盖

校准记录要进入 manifest 或伴随 manifest 存放。verifier 不一定重新校准，但它应该能检查记录是否存在、是否与当前张量匹配、scale 数量是否一致、后端是否支持这种量化策略。

verifier：把格式错误停在热路径之前

量化 verifier 至少分三层。第一层检查元数据完整性；第二层检查 shape 与 scale 数量；第三层检查后端支持。不要把这些检查推给 kernel，因为 kernel 位于最难给出好诊断的地方。

Listing 2.75 量化 verifier 的分层检查

```

metadata checks:
  bit_width is supported
  storage format and signedness are explicit
  scale dtype exists when format requires scale
  zero point policy matches signedness

shape checks:
  quantized axis is valid
  group count matches tensor shape and group size
  scale tensor shape equals expected scale_shape
  packed byte length matches bit_width and element count
  tail policy exists when dimension is not divisible

backend checks:
  CPU/GPU backend supports this format
  chosen kernel supports accumulator and output dtype
  alignment and packed order can be represented

```

诊断要像编译错误一样具体：

Listing 2.76 量化 verifier 诊断示例

```

error: tensor layers.21.mlp.wup.weight has invalid q4 scale shape
weight shape: [11008, 4096]
quant axis: input channel
group size: 64
expected scales per output row: 64
expected scale shape: [11008, 64]
actual scale shape: [11008, 63]
note: input dimension 4096 requires 4096 / 64 groups

```

这样的诊断能把问题定位到转换脚本或模型资产，而不是让用户面对 `segmentationfault`、`invalidargument` 或一堆错误 token。第一册强调过二进制和 ABI 的可信边界；这里的 verifier 就是在 AI 模型资产进入二进制热路径之前建立可信边界。

IR 中的量化节点：显式边界胜过隐式魔法

量化进入 IR 后，有两种常见表达。第一种是显式节点：`quantize`、`dequantize`、`requantize`、`quantized_linear`。第二种是在 tensor edge 上附加 quant descriptor，让某些算子直接消费量化边。两者都可以，但不能让量化完全隐藏在后端。

Listing 2.77 显式量化 IR 的形状

```

%wq = const_weight "layers.0.attn.wq.weight" : tensor<[4096,4096], q4_group>
%x  = rmsnorm(%hidden, %gamma)           : tensor<[T,4096], f16>
%y  = quantized_linear(%x, %wq)
      {accumulator=f32, output=f16, qdesc=#q4_g64}
      : tensor<[T,4096], f16>

```

显式表达带来三个好处。第一，pass 能看到量化边界，不会错误地把 dequant 移到不安全位置。第二，memory planner 能知道 scale 和 packed weight 的生命周期。第三，oracle 能在量化算子边界挂比较点。缺点是 IR 更复杂，需要 pass 明确处理量化节点。

对于初学者，可以先采用保守策略：权重量化用 `quantized_linear` 直接表达；activation 量化用显式 `quantize/dequantize/requantize` 表达；KV cache 量化用 runtime descriptor 表达。这样语义边界最清楚，后续再根据性能需求做融合。

常见误区

不要让优化 pass 看不见量化边界。一个融合 pass 如果不知道 linear 权重是 q4 group，就可能把需要 materialize 的 scale 读流藏进大 kernel，导致 oracle 难以定位误差。

packing: descriptor 到后端物理布局的转换

packing 是 lowering 的一部分，不是加载阶段的随手优化。它把逻辑量化张量转换成后端 kernel 喜欢的物理布局。一个 C++20 后端应该把 packing 输出描述清楚：

Listing 2.78 PackedQuantizedWeightDesc 的最小内容

```

PackedQuantizedWeightDesc:
  source_tensor_id
  backend_id
  isa_or_device
  panel_rows
  panel_cols
  tile_k
  data_offset
  data_bytes
  scale_offset

```

```
scale_bytes
bytes_per_panel
scales_per_panel
alignment
tail_policy
checksum
```

这个 descriptor 让 kernel 可以按 panel 读取，不需要知道原始模型格式；也让诊断可以从 packed bytes 追溯到原始 tensor。若 oracle 发现某层误差突然变大，报告能指出它使用了哪个 packed descriptor、哪个 backend、哪个 ISA、哪个 group size，而不是只写 “linear failed”。

packing 要单独计时。若 int4 packing 花了 30 秒，但用户只生成 20 个 token，热路径的加速可能无法抵消加载时间；若服务端长期复用同一个模型，packing 成本就可以摊销。profiler 必须把 `pack_time_ms` 和 `decode_tokens_per_s` 分开，避免把 prepare 成本藏起来。

reference converter：先证明字节解释正确

低比特量化的第一层 oracle 不是比较最终生成文本，而是证明 “字节到实数” 的解释正确。reference converter 应该用最清楚的标量实现，不追求速度。它读取原始 int4/int8 bytes、scale、zero point 和 tail policy，输出一段 fp32 或 fp16 参考值。

Listing 2.79 reference converter 的测试输入

```
test cases:
  one full group
  dimension not divisible by group size
  positive and negative int4 codes
  high-nibble-first and low-nibble-first packed order
  symmetric and asymmetric zero point
  f16 and f32 scale storage
```

只有 converter 通过，才有资格测试 quantized linear。否则单算子误差可能来自字节解释、scale 数量、tail policy、SIMD 解包、累加顺序中的任何一个点。把验证分层，是编译器工程的基本方法：先验证低层事实，再验证组合行为。

reference decoder layer：把整层链条跑通

前一章和第 5 章已经把单个量化张量、attention 和 KV cache 的局部原理讲清楚。真正让工程变稳的，是有一条慢但可信的 reference decoder layer 路径，能把 embedding 之后的一层完整计算串起来。它不追求快，只追求语义清楚、边界明确、可逐点对比。

Listing 2.80 reference decoder layer 的最小输入

```
ReferenceLayerInput:
  x_in           // [T,H] or [1,H]
  rms_attn_gamma // [H]
  wq, wk, wv, wo
  rms_ffn_gamma  // [H]
  wgate, wup, wdown
  rope_cos, rope_sin
  causal_mask or visible_range
  kv_cache_in
  layer_config   // heads, kv_heads, head_dim, eps
```

一层 reference path 至少分成下面几步：

Listing 2.81 reference decoder layer 的语义步骤

```

1. h_attn = rmsnorm(x_in, rms_attn_gamma)
2. q = linear(h_attn, wq)
3. k = linear(h_attn, wk)
4. v = linear(h_attn, wv)
5. q, k = apply_rope(q, k, position)
6. kv_cache_out = append(kv_cache_in, k, v)
7. attn_out = attention(q, kv_cache_out, visible_range)
8. x_mid = x_in + linear(attn_out, wo)
9. h_ffn = rmsnorm(x_mid, rms_ffn_gamma)
10. g = linear(h_ffn, wgate)
11. u = linear(h_ffn, wup)
12. m = silu(g) * u
13. x_out = x_mid + linear(m, wdown)

```

这条路径的价值在于：它天然给出很多中间比较点。量化 kernel、fused kernel、paged KV cache、GQA kernel、packed int4 linear，都不需要一上来和最终文本比较；它们可以先和 reference path 的某一步比较。只要中间某一步开始漂，定位范围就会大幅缩小。

中间检查点应该怎么定

一个实用的 oracle 链，不会只在最后比较 **x_out**。至少应该暴露这些检查点：

检查点	张量形状	能定位的问题
h_attn	[N,H]	RMSNorm、输入 layout、dtype 转换
q/k/v	[N,heads,d]	linear、量化权重、head reshape
q_rope/k_rope	[N,heads,d]	RoPE、位置编号、左右 padding
scorerow	[visible]	GQA 映射、mask、量化 K 误差
probrow	[visible]	softmax、online normalization
attn_out	[N,H]	V 读取、KV 量化、head merge
x_mid	[N,H]	residual、Wo、accumulator dtype
g/u/m	[N,I]	MLP 权重、SiLU、逐元素乘
x_out	[N,H]	单层总输出合同

这样设计后，kernel 出现误差时就不会只得到一句“最终 logits 不一致”。例如：若 **q/k/v** 正确但 **q_rope/k_rope** 开始偏移，问题就在位置合同；若 **scorerow** 正确但 **probrow** 错，问题就在 softmax 或 mask；若 **attn_out** 正确但 **x_mid** 错，问题就在 **Wo** 或 residual。

单 token decode 的 reference 流程

对 decoder-only 推理，最有工程价值的不是整段 prefill 一次性 reference，而是单 token decode reference。因为大多数性能优化、KV cache 问题、长上下文问题，都发生在这个循环里。最小流程可以写成：

Listing 2.82 单 token decode 的 reference 流程

```

input:
  x_cur      // [1,H]
  kv_cache   // all previous positions
  pos        // absolute model position

for each layer:
  run reference decoder layer

```

```

update kv_cache

last_hidden = x_cur
logits = lm_head(last_hidden)

```

对这个流程，测试不能只断言“生成 token 对”。更好的做法是固定 prompt、固定 seed，把第 **t** 步 decode 的以下结果都落盘或比较：

Listing 2.83 decode oracle 的逐步证据

```

step t:
  layer 0 q checksum
  layer 0 score top entries
  layer 0 attn_out checksum
  layer L-1 x_out checksum
  logits top-k
  chosen next token

```

这样做虽然笨，但对工程极其有效。paged KV cache、GQA、int8/int4 KV、fused attention、speculative decode 验证失败时，都能回到第一个偏离的 step/layer/checkpoint。

把 reference decoder layer 落成 C++20 骨架

前面的步骤表已经说明“要算什么”，但工程上还需要回答“代码骨架怎么摆”。reference 路径最怕一开始就写成巨函数，把 tensor 视图、scratch buffer、checkpoint 和比较逻辑搅在一起。更稳的做法是先把数据边界钉死。

Listing 2.84 reference path 的最小数据结构骨架

```

enum class CheckpointKind {
    HAttn,
    Q,
    K,
    V,
    QRope,
    KRope,
    ScoreRow,
    ProbRow,
    AttnOut,
    XMid,
    G,
    U,
    M,
    XOut
};

struct TensorView final {
    const float* data = nullptr;
    std::array<int, 4> shape{};
    std::array<int, 4> stride{};
    int rank = 0;
};

struct MutableTensorView final {

```



```

float* data = nullptr;
std::array<int, 4> shape{};
std::array<int, 4> stride{};
int rank = 0;
};

struct LayerConfig final {
    int hidden_size = 0;
    int num_query_heads = 0;
    int num_kv_heads = 0;
    int head_dim = 0;
    int ffn_intermediate = 0;
    float rms_eps = 1.0e-5f;
};

struct CheckpointRecord final {
    CheckpointKind kind;
    std::vector<float> values;
    std::vector<int> shape;
};

```

TensorView 的目的不是炫技，而是强行把 reference path 和某个具体 runtime tensor 类解耦。reference 只需要“从哪里读、形状是什么、怎样走 stride”。这样 production path 用 arena、mmap、NUMA page、device buffer 都无所谓；reference 仍然能吃统一视图。

单层 reference 的输入输出可以进一步压成：

Listing 2.85 ReferenceLayer::run 的责任边界

```

struct ReferenceLayerInput final {
    TensorView x_in;
    TensorView rms_attn_gamma;
    TensorView wq, wk, wv, wo;
    TensorView rms_ffn_gamma;
    TensorView wgate, wup, wdown;
    TensorView rope_cos;
    TensorView rope_sin;
    int position = 0;
    int visible_begin = 0;
    int visible_end = 0;
};

struct ReferenceLayerOutput final {
    std::vector<float> x_out;
    std::vector<float> k_append;
    std::vector<float> v_append;
    std::vector<CheckpointRecord> checkpoints;
};

class ReferenceLayer final {
public:
    explicit ReferenceLayer(LayerConfig cfg) : cfg_(cfg) {}
    ReferenceLayerOutput run(const ReferenceLayerInput& in,
                           const KvCacheView& kv_in) const;
};

```

```
private:
    LayerConfig cfg_;
};
```

这里故意让 `run()` 返回 `k_append/v_append`, 而不是在内部偷偷写全局 cache。原因很直接: reference 的第一职责是把每一步算清楚, 第二职责才是和 runtime 状态衔接。若一开始就把 cache 写入隐藏在内部, `prefill/decode` 对齐和 `checkpoint` 定位都会变难。

真正的 `run()` 内部流程, 可以保持和数学合同一一对应:

Listing 2.86 ReferenceLayer::run 的最小伪代码

```
ReferenceLayerOutput ReferenceLayer::run(...) const {
    auto h_attn = rmsnorm(in.x_in, in.rms_attn_gamma, cfg_.rms_eps);
    save(HAttn, h_attn);

    auto q = linear(h_attn, in.wq);
    auto k = linear(h_attn, in.wk);
    auto v = linear(h_attn, in.wv);
    save(Q, q); save(K, k); save(V, v);

    auto q_rope = apply_rope(q, in.rope_cos, in.rope_sin, in.position);
    auto k_rope = apply_rope(k, in.rope_cos, in.rope_sin, in.position);
    save(QRope, q_rope); save(KRope, k_rope);

    auto score_row = attention_scores(q_rope, kv_in, k_rope,
                                      in.visible_begin, in.visible_end);
    save(ScoreRow, score_row);

    auto prob_row = softmax_stable(score_row);
    save(ProbRow, prob_row);

    auto attn_out = attention_reduce(prob_row, kv_in, v);
    save(AttnOut, attn_out);

    auto x_mid = residual_after_wo(in.x_in, attn_out, in.wo);
    save(XMid, x_mid);

    auto h_ffn = rmsnorm(x_mid, in.rms_ffn_gamma, cfg_.rms_eps);
    auto g = linear(h_ffn, in.wgate);
    auto u = linear(h_ffn, in.wup);
    auto m = silu_mul(g, u);
    save(G, g); save(U, u); save(M, m);

    auto x_out = residual_after_wdown(x_mid, m, in.wdown);
    save(XOut, x_out);
    return {...};
}
```

这段骨架要点不在“写了多少 helper 函数”, 而在于每一步都能独立落 checkpoint。生产后端可以融合 `QKV`、融合 `Wo+residual`、做 online softmax、做 paged KV、做低比特量化; 但 reference 层故意不融合, 因为它的职责是给每一条优化提供一个可对照的语义基线。

有了 layer 级骨架, 还需要一层专门做比较和报告:

Listing 2.87 decode oracle 的比较与报告骨架

```

struct CheckpointDiff final {
    CheckpointKind kind;
    float max_abs = 0.0f;
    float mean_abs = 0.0f;
    bool topk_changed = false;
};

struct DecodeStepReport final {
    int step = 0;
    int layer = 0;
    std::vector<CheckpointDiff> diffs;
    bool accepted = false;
    std::string first_failure;
};

class DecodeOracle final {
public:
    DecodeStepReport compare(const ReferenceLayerOutput& ref,
                           const ProductionLayerTrace& got,
                           const ToleranceProfile& tol) const;
};

```

这时 reference 和 oracle 的分工就非常明确：

- **ReferenceLayer** 只负责“算出可信结果并导出检查点”。
- **DecodeOracle** 只负责“按检查点规则比较并生成诊断”。
- production backend 只负责“按性能目标运行并暴露必要 trace”。

这个拆法在后面接 paged KV、GQA、int4 weight、int8 KV、FlashAttention 风格 softmax 时都不会塌。因为无论优化路径怎样变，最小闭环都还是：

同一输入 → reference checkpoints ↔ production checkpoints → diff report

没有这条骨架，所谓“oracle”往往只剩一句最终 logits 不一致，调试价值很低。

paged KV、GQA 和量化 KV 的 oracle 闭环

reference layer 只保存连续逻辑 KV 还不够。真实后端会把 KV cache 放进 page/block；GQA 会让多个 query head 共享一个 K/V head；KV 量化又会让 K/V 码点和 scale metadata 分开存。三者叠加后，错误不一定出现在 attention 算术本身，而可能出现在逻辑坐标到物理地址的映射。

先把生产路径必须暴露的 trace 字段列出来：

Listing 2.88 paged quantized KV 的最小 trace 字段

```

KvAccessTrace:
    step
    layer
    q_head
    kv_head
    model_position
    visible_begin
    visible_end
    page_id
    page_offset

```

```

slot_epoch
k_code_bytes
v_code_bytes
k_scale_id
v_scale_id
rope_position

```

这些字段看起来多，但每一个都对应真实故障。若 `q_head->kv_head` 映射错，是 GQA 错；若 `model_position` 和 `rope_position` 不一致，是位置合同错；若 `page_id/page_offset` 指向旧 slot，是 paged cache 或 sliding window 错；若 `k_scale_id` 指错，是 KV 量化 metadata 错。

reference 侧可以继续使用连续逻辑坐标：

Listing 2.89 reference KV 逻辑读

```

for q_head in query_heads:
    kv_head = q_head / gqa_group_size
    for pos in visible_begin..visible_end:
        k_ref = K_ref[layer, kv_head, pos]
        v_ref = V_ref[layer, kv_head, pos]
        score_ref[pos] = dot(q_ref[q_head], k_ref) / sqrt(head_dim)

```

production 侧可以使用 page table、量化码点和 scale：

Listing 2.90 production KV 物理读

```

for q_head in query_heads:
    kv_head = q_head / gqa_group_size
    for pos in visible_begin..visible_end:
        slot = page_table.lookup(session, pos)
        k_q = read_k_codes(slot, layer, kv_head)
        k_scale = read_k_scale(slot, layer, kv_head)
        k = dequantize(k_q, k_scale)
        score_got[pos] = dot(q_got[q_head], k) / sqrt(head_dim)

```

oracle 的闭环不是要求这两段代码长得一样，而是要求它们在同一逻辑坐标上给出可解释的差异。比较顺序应当从“坐标和字节解释”开始，而不是直接比较 logits：

Listing 2.91 paged KV oracle 的比较顺序

1. check visible set:


```
reference visible positions == production visible positions
```
2. check GQA mapping:


```
kv_head == q_head / group_size
```
3. check page mapping:


```
logical position -> page_id/page_offset is valid
slot epoch matches expected sequence
```
4. check quantized bytes:


```
code bytes and scale ids are present
dequantized K/V match reference tolerance
```

```

5. check attention:
    score row, prob row, attn_out

6. check layer/logits/token:
    x_out, logits top-k, chosen token

```

这套顺序能把错误压到第一个可解释边界。比如：

首个失败点	典型原因	不该先怀疑什么
visible set	mask、sliding window、prefix 保留策略	量化精度
GQA mapping	query head 到 KV head 映射错	softmax
page mapping	page table、slot epoch、回绕复用	linear 权重
quantized bytes	scale id、nibble order、tail policy	RoPE
score row	K 量化、RoPE、head dim stride	sampler
prob row	softmax、mask、online merge	Wq/Wk
attn_out	V 读取、head merge、Wo 前布局	logits head

对 KV 量化，oracle 还要同时保存两类误差：反量化误差和 attention 行为误差。反量化误差回答“bytes 是否按合同解释”；attention 行为误差回答“这些误差是否已经改变 score 排名和概率分布”。一个报告可以这样组织：

Listing 2.92 KV quant oracle report 的核心字段

```

KvQuantOracleReport:
  context_length
  layer
  q_head
  kv_head
  visible_count
  max_k_dequant_error
  max_v_dequant_error
  score_max_abs_error
  score_top1_flipped
  prob_kl_divergence
  attn_out_max_abs_error
  first_bad_position
  first_bad_page
  first_bad_scale_id

```

这份报告比“logits mismatch”更能指导修复。若 `max_k_dequant_error` 很小但 `score_top1_flipped` 很多，说明问题不是字节解释，而是量化 profile 对该层/head 太激进。若 `first_bad_page` 总在 page 边界，优先查 page table、block size 和 scale metadata 布局。若只在 sliding window 回绕后失败，优先查 `slot_epoch`、base position 和 RoPE position。

核心思想

paged KV、GQA、KV 量化同时存在时，oracle 的第一任务不是证明最终文本像不像，而是证明“逻辑位置、K/V head、物理 page、量化 bytes、scale metadata、RoPE position”指向的是同一个历史 token。

一个失败定位例子：不是 softmax，是 scale page 错

假设某个优化后端在 4096 token 以内全部通过，到了 4097 token 后 logits 开始漂。只看最终输出，很容易误判为长上下文量化精度不够。按上面的 oracle 链排查，会得到更具体的证据：

Listing 2.93 失败报告示例

```
step: 4097
layer: 18
q_head: 12
kv_head: 3
first_failure: quantized bytes
logical_position: 4096
page_id: 64
page_offset: 0
slot_epoch: expected 1, got 0
k_scale_id: expected 524288, got 516096
score_max_abs_error: 0.184
score_top1_flipped: true
```

这个报告已经把方向压得很窄：第 4096 位置刚好落到新 page，`page_offset=0`，`slot_epoch` 和 `k_scale_id` 都错。此时不应该去改 softmax，也不应该调大 logits 容忍度；应该检查 page 分配、scale metadata page stride、以及 sliding window 或 prefix cache 是否复用了旧 slot。

如果修复后同一 case 变成：

Listing 2.94 修复后报告应该收敛到数值误差

```
visible set: pass
GQA mapping: pass
page mapping: pass
quantized bytes: pass
score_max_abs_error: 0.012
score_top1_flipped: false
attn_out_max_abs_error: 0.006
logits_topk_changed: false
```

这才说明错误从“合同错误”退化成“可接受的数值近似”。量化优化必须先通过合同层，再讨论误差阈值。

prefill reference 不能只看最后一列

prefill 常被简化成“只看最后一个 token 的 logits”。这不够。prefill 一次性处理整个 prompt，任何一个中间位置的 causal mask、RoPE 位置或 attention 分块若出错，都可能在最后一列被部分掩盖。reference prefill 至少应该能检查：

- 第一个位置是否只看自己；
- 中间位置是否只看左侧历史；
- 最后位置的 logits 是否匹配；
- prefill 写入的 KV cache 是否与逐 token reference append 一致。

最后这一点很关键。一个正确的 prefill 实现，写出的 `K/V[0..T-1]` 应该和“把同一段 prompt 逐 token 跑 decode-style reference append”得到的结果一致。若两者不一致，后面的 decode 再快也建立在错误状态上。

常见误区

prefill 和 decode 不能各自有一套“差不多对”的 reference。它们必须在共享的 KV cache 逻辑合同上闭合，否则很容易出现 prefill 通过、decode 通过、但 prefill 后接 decode 的真实路径失败。

容忍度应该按检查点分层

reference oracle 不是要求所有路径 bitwise 相等。不同后端、不同累加顺序、fused kernel、online softmax 都可能带来合法的微小差异。关键是容忍度要按检查点分层，而不是用一个统一阈值糊过去。

检查点	更合理的比较方式	原因
字节解释	严格相等	合同层，不应漂移
q/k/v	max abs / relative error	累加顺序和量化会有微差
score row	top entries + max error	小误差可能改变 softmax 排名
prob row	KL 或 top-1/top-k 位置	概率分布比逐元素更有意义
x_out	max abs / cosine similarity	层输出是连续值
logits	top-k 变化 + max error	直接关系采样
token 序列	固定 seed 下完全一致或记录首个偏离步	离散行为最终验收

工程上最糟糕的情况是：前面所有连续值比较都在阈值内，但 logits 的 top-k 排名在某些长上下文点上开始翻转。这个时候不能简单说“误差很小所以没问题”，而应该把它报告成采样敏感点。

C++20 项目里 oracle 链怎么落地

reference path 不应该塞进生产 kernel 文件里，更不应该被宏条件塞得到处都是。更好的结构是：

Listing 2.95 reference/oracle 的项目结构草图

```
reference/
  reference_linear.cpp
  reference_rmsnorm.cpp
  reference_rope.cpp
  reference_attention.cpp
  reference_decoder_layer.cpp

oracle/
  quant_byte_oracle.cpp
  operator_oracle.cpp
  layer_oracle.cpp
  decode_oracle.cpp
```

这样做的好处是边界清楚。reference 模块只负责算出可信结果；oracle 模块只负责比较和报告；生产后端只负责跑快。需要时，CI 或本地 debug 可以把同一 prompt 同时送进 production path 和 reference path，在某个 checkpoint 失败后立即停止。

验收与评分标准

reference decoder layer 的最低验收线：

- 单层 decode 能输出 q/k/v、score、prob、attn_out、x_mid、x_out。
- prefill 写出的 KV cache 能与逐 token reference append 对齐。
- 容忍度按字节解释、连续值、概率分布、logits 和 token 序列分层。

- 报告能指出首个偏离的 step、layer 和 checkpoint。

oracle: 量化误差要按层次报告

量化 oracle 不能只给一个 **pass/fail**。它应该按层次报告误差，因为不同层次定位的问题不同。

oracle 层次	比较对象	能定位的问题
字节解释	packed bytes 反量化值	nibble order、scale、zero point、tail
单算子 层输出	linear/attention/MLP 输出 decoder block hidden state	kernel、累加器、requantize 融合、arena 覆盖、layout 转换
logits 生成序列	top-k、分布、最大误差 固定 seed 的输出 token	质量风险和采样敏感点 端到端可见行为变化

初学者最容易困惑的是：为什么 logits 有小误差，生成序列却完全不同？原因是采样是离散决策。若两个 token 的概率非常接近，轻微 logits 变化就可能改变 top-k 排序或采样结果。因此量化报告不能只看平均误差，还要看 top-k 变化、KL 或分布差异，以及固定 sampler 下的序列差异。

Listing 2.96 量化 oracle 报告字段

```
QuantOracleReport:
  model_id
  quant_profile_id
  prompt_set_id
  context_length_buckets
  tensor_decode_failures
  operator_error_by_layer
  layer_output_error_by_position
  logits_max_abs_error
  logits_topk_changed
  generated_token_mismatch
  tolerance_profile
```

这份报告应该和 profiler 报告一起出现。一个 int4 优化如果速度提升 35%，但 top-k 在某些中文长上下文 prompt 中大量变化，就不能直接算合格；反过来，如果 logits 几乎不变但 decode 变慢，也不能叫优化。

profiler: 量化收益要拆成容量、带宽和计算

量化常被宣传为“模型变小，所以更快”。这句话只说对了一部分。量化可能减少权重带宽，也可能增加解包、scale 读取和 requantize 成本。KV cache 量化可能减少 cache 容量和 attention 读带宽，也可能增加 metadata 读取和反量化成本。profiler 必须拆开这些项。

Listing 2.97 量化 profiler 的关键字段

```
QuantProfilerReport:
  original_weight_bytes
  quantized_weight_bytes
  scale_metadata_bytes
  packed_weight_bytes
  kv_cache_bytes_by_context
  kv_scale_metadata_bytes
  pack_time_ms
```

```

prefill_latency_ms
decode_latency_ms_by_context
dequant_time_ms
requant_time_ms
kernel_time_ms
memory_bandwidth_estimate

```

对 CPU，量化收益经常体现在减少内存读流，但如果解包逻辑让 SIMD pipeline 停顿，收益会下降。对 GPU，低比特格式若不能进入高效 tensor core 或 vendor kernel，可能会因为自定义解包 kernel 和同步开销而变慢。第二册讲过 cache、TLB、SIMD 和线程；本节的 profiler 字段就是把那些硬件事实拉回量化优化。

回归门禁：哪些测试必须常跑

量化改动风险高，不能靠人工看几条生成文本。C++20 项目至少应该有四类回归门禁：

- descriptor/verifier 单元测试：非法 group size、scale shape、tail policy、packed byte length 必须被拒绝。
- converter 测试：固定小张量的 int4/int8 bytes 反量化结果必须匹配 reference。
- kernel oracle 测试：常见 shape、tail shape、对齐/非对齐、不同 group size 都要覆盖。
- 端到端小模型测试：固定 prompt、固定 seed、固定 sampler，比较 logits 和生成序列合同。

性能也要有门禁，但不能用单次运行的绝对数字硬卡所有环境。更实用的是保存硬件信息、线程数、模型片段、prompt set 和相对基线，在同一机器上做回归判断。若一次改动让 decode latency 明显上升，CI 或本地 benchmark 应该提醒，而不是等到整本引擎集成后才发现。

验收与评分标准

量化功能进入主线前，至少要满足四条线：descriptor/verifier 能拒绝坏格式；reference converter 能解释 bytes；kernel oracle 能覆盖误差；profiler 能证明容量或速度收益。缺一条，就只能算实验分支。

本节验收线

读完本节，应该能把量化从“脚本转换”提升为工程系统：

- QuantDescriptor 要显式记录 bit width、signedness、scale、zero point、granularity、axis、group、packed order 和 tail policy。
- CalibrationRecord 要保存校准数据、算法、范围、饱和和工具版本，不能只留下 scale 文件。
- verifier 要在热路径之前检查 metadata、shape、scale 数量、packed byte length 和后端支持。
- IR 要显式表达量化边界，让 pass、memory planner、lowering 和 oracle 都看得见。
- packing 是后端 lowering 的物理产物，必须有 packed descriptor 和诊断映射。
- oracle 要按字节解释、单算子、层输出、logits 和生成序列分层报告。
- profiler 要拆开容量、metadata、packing、dequant、requant、prefill 和 decode 成本。

到这里，量化大章不再只是数值公式，而是已经能进入 C++20 项目的模块设计、测试设计和验收设计。后续讲后端 micro-kernel 时，本节的 descriptor、oracle 和 profiler 会直接成为 kernel 合同的一部分。

2.8 7B 推理计算账本：FLOPs、字节、KV cache 与延迟上限

为什么必须算账

讨论本地 7B 推理时，最常见的空话是“优化矩阵乘”“减少显存”“提高 tokens/s”。这些话如果不落到 FLOPs、字节、cache、带宽和延迟，就无法指导实现。本节用一个典型 7B decoder-only GQA 模型建立账本。具体模型可能略有差异，但方法通用。

Listing 2.98 典型 7B 账本参数

```

layers L = 32
hidden H = 4096
query_heads n_q = 32
kv_heads n_kv = 8
head_dim d = 128
intermediate I = 11008
vocab V = 32000
context S = 4096

```

这些数字不是装饰。它们决定每个 token 要读多少权重、KV cache 占多少内存、attention 随上下文怎样增长、CPU/GPU 后端为什么在 prefill 和 decode 阶段表现不同。

参数量账本

先算一层主要权重。Q projection 输出 $n_q * d = 4096$, K/V projection 各输出 $n_{kv} * d = 1024$ 。

权重	形状	参数量
Wq	4096x4096	16.78M
Wk	1024x4096	4.19M
Wv	1024x4096	4.19M
Wo	4096x4096	16.78M
Wgate	11008x4096	45.09M
Wup	11008x4096	45.09M
Wdown	4096x11008	45.09M

一层线性权重大约：

$$16.78 + 4.19 + 4.19 + 16.78 + 45.09 + 45.09 + 45.09 \approx 177.2M$$

32 层约 **5.67B** 参数。加上 embedding、lm head、norm 和少量元数据，就接近 7B 级别。这个账本直接解释了 decode 为什么常常被权重读流支配：每生成一个 token，主要线性权重基本都要参与一次。

权重字节账本

参数量乘以每个权重的字节数，就是权重读流的粗略下界。

格式	7B 权重字节	备注
fp16	约 14GB	精度高，带宽压力大
int8	约 7GB	还要加 scale/zero point
int4	约 3.5GB	metadata 和 packing 会增加一些

若每个 decode token 都要流过大部分权重，那么 CPU 上限首先受内存带宽约束。假设有效内存带宽是 **100GB/s**，int4 权重加 metadata 后每 token 读 **3.7GB**，只看权重的理论上限也只有：

$$\frac{100}{3.7} \approx 27 \text{ tokens/s}$$

这还没有算 KV cache 读取、activation、线程同步、解包、scale、sampler、allocator 和调度开销。真实速度低于这个上限并不奇怪。这个公式比单纯说“CPU 慢”更有用，因为它告诉我们应该优先减少权重字节、改善 packing、提高带宽利用率，而不是盲目堆线程。

decode 每层 FLOPs 账本

一次 decode 只处理一个新 token。线性层 MAC 数大约等于权重参数量。每层主要线性 MAC：

$$QKV = H(H + 2n_{kv}d) = 4096(4096 + 2048) \approx 25.17M$$

$$O = H^2 \approx 16.78M$$

$$MLP = 3HI = 3 \times 4096 \times 11008 \approx 135.27M$$

总计每层约：

$$25.17 + 16.78 + 135.27 \approx 177.22M \text{ MACs}$$

若把一次乘加按 2 FLOPs 计，每层约 **354MFLOPs**，32 层约 **11.36FLOPs/token**。attention 的点积和加权求和还要另算，随上下文长度 **S** 增长：

$$attention_macs \approx 2 \times n_q \times S \times d$$

S=4096 时，一层 attention 约：

$$2 \times 32 \times 4096 \times 128 \approx 33.55M \text{ MACs}$$

32 层约 **1.07BMACs**。所以在长上下文下，attention 计算也很可观；但在很多 CPU decode 场景，低比特权重的解包和权重/KV 字节流仍然是更早撞到的墙。

KV cache 容量账本

KV cache 每个 token、每层要保存 K 和 V。GQA 下保存的是 **n_kv** 个 head，不是 **n_q** 个 head。

$$kv_bytes_per_token_per_layer = 2 \times n_{kv} \times d \times bytes$$

fp16 下每个元素 2 字节：

$$2 \times 8 \times 128 \times 2 = 4096 \text{ bytes}$$

32 层就是每个 token **128KiB**。上下文 **4096** 时，一个 session 的 KV cache 是：

$$4096 \times 128KiB = 512MiB$$

这个数字对业务开发非常关键。一个本地服务如果同时接 8 个 **4096** context 的会话，仅 KV cache 就可能接近 **4GiB**，还不包括权重、packed weight、activation arena、workspace、线程栈和系统余量。

context	单 session fp16 KV	8 session fp16 KV
1024	128MiB	1GiB
2048	256MiB	2GiB
4096	512MiB	4GiB
8192	1GiB	8GiB

这就是为什么 admission control 必须在请求进入前估算 KV cache，而不是等到 decode 过程中分配失败。

KV cache 读取账本

容量只是第一半。decode 每步还要读取历史 K/V。当前 token 在每层 attention 中读取长度为 **S** 的 K 和 V：

$$kv_read_bytes_per_layer = S \times 2 \times n_{kv} \times d \times bytes$$

S=4096、fp16 时每层约 **16MiB**，32 层约 **512MiB/token**。也就是说，在长上下文下，即使用 int4 权重，每生成一个 token 还要额外读约半 GB 的 KV cache。这解释了两个现象：

- 上下文越长，decode token 越慢，不只是 attention FLOPs 增加，KV 读字节也线性增加。
- KV cache 量化可能有效，因为它减少的是每步反复读取的历史状态，而不是只减少静态权重。

但 KV 量化比权重量化更敏感。权重是固定的，可以离线校准；KV 是动态 activation，每个请求、每个位置都不同。若 scale 选择不好，attention score 会被放大后进入 softmax，少量误差可能改变 top-k。后面的低比特 KV 章节会专门展开。

prefill 和 decode 不是同一账本

prefill 处理整个 prompt。若 prompt 长度 $T=512$ ，一层 linear 变成 $[512, H] \times [H, 0]$ 的矩阵乘。权重被同一批 token 复用，GEMM 能更好利用计算单元。decode 每步 $T=1$ ，权重复用少，容易被权重读流限制。

阶段	主要形态	常见瓶颈
prefill	大 GEMM、批量 attention	算力、显存带宽、attention tile
decode	GEMV/小 GEMM、KV 读取	权重带宽、KV 带宽、launch/同步

这就是为什么一个后端可能 prefill 很快、decode 一般；也可能 CPU decode 经过 int4 和线程优化后可用，但长 prompt prefill 明显慢。报告必须分开写 TTFT、prefill tokens/s 和 decode tokens/s。

roofline：用带宽和算力约束 tokens/s

粗略上限可以用 roofline 思路判断。对每个 decode token，假设需要读：

Listing 2.99 decode token 的粗略字节项

```
weight_bytes_per_token
kv_read_bytes_per_token(context)
activation_bytes
metadata_bytes
temporary_workspace_bytes
```

带宽上限：

$$\text{tokens/s} \leq \frac{\text{effective_bandwidth}}{\text{bytes_per_token}}$$

算力上限：

$$\text{tokens/s} \leq \frac{\text{effective_flops}}{\text{flops_per_token}}$$

实际上限取二者较小者，再扣除线程调度、解包、sampler、同步和 cache miss。举例：int4 权重 **3.7GB/token**，长上下文 KV **0.5GB/token**，总计约 **4.2GB/token**。若有效带宽 **100GB/s**：

$$\frac{100}{4.2} \approx 23.8 \text{ tokens/s}$$

若同一机器由于解包、NUMA、TLB、线程调度只能达到 **55GB/s** 有效带宽，上限就是 **13tokens/s**。这比“开更多线程”更能解释问题：线程只能提高并行利用率，不能凭空减少每 token 字节数。

量化为什么有效，为什么也危险

权重量化主要减少静态权重字节。int8 把 fp16 权重从 2 字节变成 1 字节；int4 变成 0.5 字节，再加 scale metadata。若 decode 被权重带宽限制，低比特能直接提高上限。

但量化不是免费午餐。计算中会出现：

- 解包成本：int4 需要从 byte 中取 nibble。
- scale 成本：per-group 或 per-channel scale 要参与反量化。

- accumulator 精度：低比特乘法需要更宽累加或合理 requantize。
- 误差传播：Q/K/V、MLP、lm head 对最终 logits 的敏感度不同。
- metadata 读流：scale、zero point 和 group table 也占带宽。

所以量化报告不能只写“模型从 14GB 变成 3.7GB”。它至少要写：

Listing 2.100 量化收益报告字段

```
original_weight_bytes
packed_weight_bytes
scale_metadata_bytes
decode_tokens_per_s
prefill_tokens_per_s
operator_error
logits_topk_overlap
generation_diff_under_fixed_seed
```

低比特格式能不能进入业务服务，必须由容量、速度和质量三者共同决定。

并发容量账本

业务服务要算的是总内存，不是单请求。一个粗略公式：

Listing 2.101 服务内存预算

```
total =
  model_weight_bytes
+ packed_weight_bytes
+ backend_static_workspace
+ concurrent_sessions * kv_cache_bytes_per_session
+ concurrent_sessions * activation_arena_bytes
+ trace_buffers
+ safety_margin
```

假设 int4 packed 权重和 metadata 约 **4.0GB**，静态 workspace **0.5GB**，每个 **4096** context session 的 KV 为 **512MiB**，每个 session arena 和 trace 估计 **128MiB**，机器可给进程使用 **12GB**。最多并发不能简单算 $12/0.5=24$ ，而应先扣静态项：

$$12 - 4.0 - 0.5 - safety \approx 6.5GB$$

每 session 约 **640MiB**，最多约 10 个，还要考虑 allocator 碎片和峰值。因此 admission control 应该保守，并按 **max_context_tokens** 预留，而不是按当前 prompt 长度乐观估计。

把账本写进 trace

计算账本如果只在书里，工程上仍然不可用。trace event 应该包含估计字节和实际耗时：

Listing 2.102 账本相关 trace 字段

```
TraceEvent:
  stage
  layer_id
  segment_id
  context_length
  estimated_flops
  estimated_weight_bytes
  estimated_kv_read_bytes
```

```
estimated_kv_write_bytes
elapsed_ns
backend
quant_profile_id
```

有了这些字段，报告就能回答：

- decode 变慢是上下文变长导致 KV 读增加，还是后端 fallback。
- int4 变快是否符合权重字节下降的预期。
- prefill 慢是 GEMM 吞吐问题，还是 tokenizer/prepare 被算进了 TTFT。
- 某层异常慢是否因为 layout 转换或 workspace 分配。

本节验收线

读完本节，应该能自己给一个 7B 模型算粗账：

- 根据 $L, H, n_q, n_{kv}, d, I, S$ 估算每层参数量和 decode FLOPs。
- 估算 fp16、int8、int4 权重字节，并用带宽上限推 tokens/s。
- 计算单 session KV cache 容量和多 session 并发内存。
- 区分 prefill 的 GEMM 账本和 decode 的 GEMV/KV 读账本。
- 解释为什么长上下文会增加 KV 读字节和 attention 计算。
- 把 FLOPs、权重字节、KV 字节和耗时写进 trace，而不是只报最终速度。

第 3 章

Transformer 推理、KV Cache 与采样

3.1 AI 推理原理、KV cache 与计算账本：从一句 prompt 到本地 7B 引擎

先把问题讲清楚：模型不是在“理解一句话”

本节重新从最小问题开始：用户输入一句话，本地大模型怎样生成下一个 token？如果这件事讲不清，后面的张量、量化、KV cache、CPU/GPU 后端和 benchmark 都会变成孤立名词。

假设用户输入：

Listing 3.1 一个最小生成请求

```
prompt: "C++ 为什么需要内存模型？"
```

推理引擎不会直接处理这串字符。第一步是 tokenizer，把文本切成 token id。token id 是整数，不一定对应一个汉字、一个英文单词或一个完整词。第二步是 embedding，把每个 token id 映射成一个 hidden 向量。第三步是多层 decoder block，不断改写每个位置的 hidden。第四步是 `lm_head`，把最后一个位置的 hidden 投影成词表大小的 logits。第五步是 sampler，从 logits 里选出下一个 token id。然后把这个新 token 追加到序列尾部，继续下一轮。

Listing 3.2 从 prompt 到 next token 的主链路

```
text
-> tokenizer.encode(text)
-> token ids
-> embedding vectors
-> repeated decoder blocks
-> final hidden at last position
-> lm_head logits over vocabulary
-> sampler chooses next token id
-> append token and repeat
```

核心思想

decoder-only 大模型推理的核心任务不是一次输出整篇文章，而是不断重复同一个步骤：给定已有 token 序列，预测下一个 token 的分数分布，再由 sampler 选出一个 token。

这句话决定了本节后面的所有机器账本。因为一次只追加一个 token，decode 阶段天然有强状态性；因为每一步都要看历史 token，attention 会读取越来越长的上下文；因为历史 K/V 可以复用，KV cache 变成推理引擎的核心运行时状态；因为每步都要过所有层，权重读流和矩阵向量乘会支配 decode 性能。

训练目标和推理目标：为什么模型会预测下一个 token

decoder-only 语言模型在训练时常用下一个 token 预测任务。给定一段 token 序列：

Listing 3.3 训练样本的移位关系

```
input:  t0, t1, t2, t3, ...
target: t1, t2, t3, t4, ...
```

模型在位置 i 只能看见 $0..i$ 的历史, 然后预测 $i+1$ 。这叫 **因果语言建模** (causal language modeling)。因果的意思是当前位置不能偷看未来 token。推理时也沿用同一规则: 已有 token 是上下文, 模型预测下一个 token。

训练和推理的区别在于, 训练知道整段 target, 可以一次计算很多位置的损失; 推理不知道未来, 只能生成一个 token 后再把它加入上下文。因此推理有两个阶段:

阶段	输入形态	目标
prefill	用户 prompt 的多个 token	批量处理 prompt, 填好 KV cache, 得到首个 logits
decode	每次一个新 token	追加 K/V, 读取历史 cache, 生成下一个 logits

这不是 API 名称差异, 而是计算形态差异。prefill 能把很多 token 一起送进矩阵乘, 更像 GEMM; decode 每步只有一个 token, 更像 GEMV 或小 GEMM, 并且要读历史 KV cache。很多人以为“模型推理就是跑一遍网络”, 这句话只对 prefill 的直觉接近, 对 decode 不够准确。decode 是一个带状状态的循环。

token、embedding、hidden、logits 和 sampler

先把常见对象定义清楚。

token id。tokenizer 把文本映射成整数序列。一个中文词、英文词、标点、空格、子词片段都可能对应一个或多个 token。业务开发中经常要关心 token 数, 因为计费、上下文长度、KV cache 容量和 latency 都按 token 变化。

embedding。embedding 表可以理解成一个矩阵 $E[\text{vocab}, H]$ 。token id 查表后得到长度为 H 的向量。若 prompt 有 T 个 token, embedding 输出形状就是 $[T, H]$ 。

hidden。hidden 是模型内部的向量表示。它不是概率, 也不是文本。每个 decoder block 接收 hidden, 输出新的 hidden。hidden size H 是计算量和内存量的核心参数之一。

logits。**lm_head** 把最后位置的 hidden 投影成 $[\text{vocab}]$ 分数。logits 没有归一化, softmax 后才变成概率。做贪心解码时选最大 logits; 做温度、top-k、top-p 采样时会改变采样分布。

sampler。sampler 不是模型主体。它是从 logits 选择 token 的策略。工程上必须把“模型算错”和“采样策略不同”分开定位。固定 seed、temperature、top-k、top-p 后, 同一 logits 应该产生可复现的 token 序列。

对象	形状示例	归属
token ids	$[T]$	tokenizer/runtime
embedding	$[T, H]$	模型权重 + activation
hidden	$[T, H]$ 或 $[1, H]$	decoder block 中间状态
logits	$[\text{vocab}]$	lm_head 输出
next id	标量整数	sampler 输出

decoder block 的骨架

多数 7B 级本地模型可以先按 decoder-only Transformer 解释。不同模型会在 RMSNorm/LayerNorm、RoPE、SwiGLU、GQA/MQA、量化格式和词表上有差异, 但一层的骨架大致相同:

Listing 3.4 一个 decoder block 的高层账本

```
x = input hidden

h = norm(x)
q = h * Wq
k = h * Wk
```

```

v = h * Wv
q, k = apply_position_encoding(q, k, position)
append k, v to KV cache
a = attention(q, K_cache[0..position], V_cache[0..position])
x = x + a * Wo

h = norm(x)
g = h * Wgate
u = h * Wup
m = activation(g) * u
x = x + m * Wdown

```

这段伪代码里，最重的计算通常是线性投影：**Wq/Wk/Wv/Wo/Wgate/Wup/Wdown**。它们本质上是矩阵乘或矩阵向量乘。RMSNorm、RoPE、激活函数和残差加法算术量小一些，但会产生 activation 读写；如果这些读写没有融合，也会消耗内存带宽。

这里第一次出现了 KV cache。当前 token 产生的 **k** 和 **v** 不能像普通临时 activation 一样丢掉，因为未来 token 的 attention 会读取它们。KV cache 把推理引擎从“无状态算子流水线”变成“跨 token 状态机”。

单层 decoder 的数学合同

把上面的伪代码写成数学合同，才能判断一个推理引擎有没有算对。设第 **l** 层输入为 **X_l**，形状是 **[N, H]**。prefill 时 **N=T**，decode 时 **N=1**。RMSNorm 先对每个 token 的 hidden 做归一化：

$$\hat{x} = \frac{x}{\sqrt{\frac{1}{H} \sum_{i=1}^H x_i^2 + \epsilon}} \odot \gamma$$

然后做 Q/K/V 投影。若 query head 数是 **n_q**，K/V head 数是 **n_{kv}**，每个 head 的维度是 **d**，通常有 **H=n_q*d**：

权重	形状	输出
Wq	[H, n_q*d]	Q: [N, n_q, d]
Wk	[H, n_{kv}*d]	K: [N, n_{kv}, d]
Wv	[H, n_{kv}*d]	V: [N, n_{kv}, d]
Wo	[n_q*d, H]	attention 输出回到 [N, H]

GQA 的关键是 **n_q** 可以大于 **n_{kv}**。若 **n_q=32**、**n_{kv}=8**，每 4 个 query head 共享同一个 K/V head。可以写成：

$$g(h) = \left\lfloor \frac{h}{n_q/n_{kv}} \right\rfloor$$

其中 **h** 是 query head 编号，**g(h)** 是它读取的 K/V head 编号。这个映射必须进入 KV cache 的逻辑合同，否则 head 读错不会立刻崩溃，只会让 logits 慢慢偏掉。

对第 **h** 个 query head，attention 的核心公式是：

$$score_{i,j,h} = \frac{Q_{i,h} \cdot K_{j,g(h)}}{\sqrt{d}} + mask_{i,j}$$

$$P_{i,:,h} = \text{softmax}(score_{i,:,h}), \quad O_{i,h} = \sum_j P_{i,j,h} V_{j,g(h)}$$

prefill 时 **i** 和 **j** 都在 prompt 内，**mask_{i,j}** 会把未来位置置为负无穷；decode 时 **i** 只有当前 token，**j** 扫描历史 cache。RoPE 通常在算 score 前作用到 **Q** 和 **K**，它不改变张量形状，但会把位置编号注入到每个 head 的二维旋转对里。位置编号错一位，shape 仍然正确，logits 会错。

attention 输出拼回 $[N, n_q \times d]$ 后乘 W_o ，再和残差相加。随后进入 MLP。许多 7B 级模型使用 SwiGLU：

$$MLP(x) = (\text{silu}(xW_{gate}) \odot xW_{up}) W_{down}$$

其中 W_{gate} 和 W_{up} 的形状通常是 $[H, I]$ ， W_{down} 是 $[I, H]$ 。 $\odot$ 是逐元素乘法。SwiGLU 不是“一个激活函数小算子”这么简单，它让 MLP 有三块大矩阵，也解释了为什么前面 7B 单层账本里 MLP FLOPs 是大头。

核心理想

单层 decoder 的正确性合同可以压成一句话：norm 改尺度，Q/K/V 建立检索空间，RoPE 写入位置，mask 保证因果性，softmax 把相似度变成权重，V 汇聚内容，MLP 做逐 token 非线性变换，residual 保留主干信息。

RoPE 到底把位置写到哪里

RoPE 的全名是 rotary positional embedding。它不是把一个位置向量加到 hidden 上，而是在每个 attention head 内，把 Q 和 K 的相邻两个维度当成一个二维平面，对它们按位置编号做旋转。设某个二维对为 $[x_0, x_1]$ ，位置为 p ，这个维度对的角频率为 θ ，旋转后是：

$$\begin{bmatrix} x'_0 \\ x'_1 \end{bmatrix} = \begin{bmatrix} \cos(p\theta) & -\sin(p\theta) \\ \sin(p\theta) & \cos(p\theta) \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$$

一个 head 的 $\text{head_dim}=128$ ，就有 64 个这样的二维旋转对。低频维度转得慢，高频维度转得快。推理实现通常会预先准备 $\cos/\sin$ 表，或者在 kernel 中即时计算/递推。无论实现方式怎样，位置 p 必须和 token 的真实绝对位置一致。

RoPE 的关键效果体现在点积里。若 query 在位置 i ，key 在位置 j ，二者分别旋转后再点积，分数会携带相对位置 $i-j$ 的信息。也就是说，模型不只知道“这两个 token 内容像不像”，还知道“它们相隔多远”。这正是 KV cache 必须保存已经加过位置的 K 的原因之一：未来 query 来时，会用自己的位置旋转 Q，再和历史位置旋转过的 K 做点积。

Listing 3.5 RoPE 在 prefill 和 decode 中的位置合同

```
prefill:
    for position p in 0..prompt_tokens-1:
        q[p], k[p] = rope(q[p], k[p], p)
        store k[p] into KV cache

decode:
    p = prompt_tokens + generated_tokens
    q_new, k_new = rope(q_new, k_new, p)
    append k_new at KV cache position p
    attend q_new to cached K[0..p]
```

这里最容易错的是 position 计数。左 padding、prefix cache、分页 cache、被截断的滑动窗口、批量合并请求，都会让“数组下标”和“模型位置编号”不再天然相等。实现可以把 KV 放在任意物理 page，但 RoPE 用的位置编号必须仍然是模型语义位置。

常见误区

RoPE 错误通常不是 shape 错误。程序会正常运行，KV cache 大小也对，甚至前几个 token 看起来还行；但长上下文、prefix 复用、续写请求会出现 logits 漂移。调试时要记录每个 token 的 model position，而不是只打印 cache slot。

场景	容易犯的错	后果
左 padding batch	把 batch 内数组列号当位置	短句和长句位置错开
prefix cache	复用 K/V 但新 token 从 0 计数	续写 logits 偏移
滑动窗口	cache slot 回绕后位置也回绕	RoPE 相对距离语义损坏
分页 KV	page offset 当全局位置	跨页 attention 错

Q、K、V 到底是什么意思

自注意力要解决的问题是：当前位置应该从历史哪些 token 中取信息。每个位置经过三个投影得到 **Q**、**K**、**V**：

- **Q**：query，可以理解成当前位置提出的问题。
- **K**：key，可以理解成历史位置用于匹配的问题索引。
- **V**：value，可以理解成历史位置真正提供的内容。

对当前 token 的一个 attention head，计算可以写成：

Listing 3.6 单 head decode attention

```
for pos in 0..current_position:
    score[pos] = dot(q_current, k_cache[pos]) / sqrt(head_dim)

prob = softmax(score)

out = 0
for pos in 0..current_position:
    out += prob[pos] * v_cache[pos]
```

第一轮循环计算“当前 token 和每个历史 token 有多相关”。第二步 softmax 把分数变成权重。第三轮循环把历史 **V** 按权重加起来，得到当前位置从历史上下文中取回的信息。

如果是 prefill，模型会同时处理 prompt 中多个位置。为了保持因果约束，位置 **i** 只能看 **0...i**，不能看后面的 token。这就是 causal mask。decode 阶段每次只处理最后一个新 token，它天然只能看历史 cache，因此 mask 形态更简单，但读历史 K/V 的长度会随着上下文增长。

一个能手算的 attention 例子

用一个二维 head 看清 attention。设当前 query 和两个历史 key/value 为：

$$q = [1, 0], \quad k_0 = [1, 0], \quad k_1 = [0, 1]$$

$$v_0 = [2, 0], \quad v_1 = [0, 4], \quad d = 2$$

两个 score 是：

$$s_0 = \frac{q \cdot k_0}{\sqrt{2}} = 0.707, \quad s_1 = \frac{q \cdot k_1}{\sqrt{2}} = 0$$

softmax 后：

$$p_0 = \frac{e^{0.707}}{e^{0.707} + e^0} \approx 0.67, \quad p_1 \approx 0.33$$

输出就是：

$$out = p_0 v_0 + p_1 v_1 \approx 0.67[2, 0] + 0.33[0, 4] = [1.34, 1.32]$$

这个小例子说明三件事。第一，**Q/K** 决定从历史位置拿多少权重；第二，**V** 决定真正被汇聚的内容；第三，softmax 不会只选一个 token，它通常产生一个加权混合。实际模型只是把二维 head 换成 **128** 维 head，把两个历史位置换成长上下文，把一个 head 换成几十个 head。

常见误区

不要把 attention 解释成“模型查找最相关的一个词”。更准确地说，它是在每个 head 内对可见历史位置做加权汇聚。某些 head 可能很尖锐，某些 head 可能很分散；最终还要经过 **Wo** 和 MLP 混合。

一个能手算的最小 decoder block

只看 attention 还不够，因为读者真正要实现的是整层 decode。下面给一个极小、可以手算完的单层单 head 例子。设 hidden size **H=4**，query head 数和 KV head 数都为 **1**，**head_dim=2**，当前是 decode 第 **1** 步，历史里已有 1 个 token。

先给输入和权重。为了把账本压到最小，这里故意选近似单位选择矩阵：

Listing 3.7 最小 decoder block 的输入与权重

```
x_in = [1, 0, 1, 0]
```

```
Wq =
[[1,0],
 [0,1],
 [0,0],
 [0,0]]
```

```
Wk = Wq
```

```
Wv =
[[2,0],
 [0,4],
 [0,0],
 [0,0]]
```

```
Wo =
[[1,0,1,0],
 [0,1,0,1]]
```

```
Wgate =
[[1,0],
 [0,1],
 [0,0],
 [0,0]]
```

```
Wup =
[[1,0],
 [0,1],
 [1,0],
 [0,1]]
```

```
Wdown =
[[1,0,1,0],
 [0,1,0,1]]
```

假设 RMSNorm 的 $\gamma = [1, 1, 1, 1]$, ϵ 足够小可忽略; 历史 cache 中已经有

$$k_0 = [1, 0], \quad v_0 = [2, 0]$$

当前 token 经过 $Wq/Wk/Wv$ 后得到:

$$q_1 = [1, 0], \quad k_1 = [1, 0], \quad v_1 = [2, 0]$$

为了和前一节的二维 attention 例子保持一致, 再假设历史里还有一个位置

$$k_2 = [0, 1], \quad v_2 = [0, 4]$$

于是当前 query 实际可见的是三个位置 **0, 1, 2**, 其中 **0** 和 **1** 都与 **q_1** 同向, **2** 与 **q_1** 正交。
score 是:

$$s_0 = s_1 = \frac{1}{\sqrt{2}} \approx 0.707, \quad s_2 = 0$$

稳定 softmax 后:

$$p_0 = p_1 = \frac{e^{0.707}}{2e^{0.707} + 1} \approx 0.401, \quad p_2 \approx 0.198$$

因此 attention 输出是:

$$attn_out = 0.401[2, 0] + 0.401[2, 0] + 0.198[0, 4] \approx [1.604, 0.792]$$

Wo 把二维 head 拼回 **[4]**:

$$attn_proj = [1.604, 0.792, 1.604, 0.792]$$

做第一次残差:

$$x_{mid} = x_{in} + attn_proj = [2.604, 0.792, 2.604, 0.792]$$

现在进入 MLP。先做第二次 RMSNorm:

$$r = \sqrt{\frac{2.604^2 + 0.792^2 + 2.604^2 + 0.792^2}{4}} \approx 1.939$$

$$h_{ffn} = \frac{x_{mid}}{r} \approx [1.343, 0.408, 1.343, 0.408]$$

然后按上面的 $Wgate/Wup$ 投影:

$$g = [1.343, 0.408]$$

$$u = [1.343 + 1.343, 0.408 + 0.408] = [2.686, 0.816]$$

SwiGLU 的逐元素结果是:

$$\text{silu}(1.343) \approx 1.064, \quad \text{silu}(0.408) \approx 0.245$$

$$m = \text{silu}(g) \odot u \approx [1.064 \times 2.686, 0.245 \times 0.816] \approx [2.858, 0.200]$$

最后经 **Wdown** 回到 **[4]**:

$$ffn_out = [2.858, 0.200, 2.858, 0.200]$$

第二次残差后的层输出是:

$$x_{out} = x_{mid} + ffn_out \approx [5.462, 0.992, 5.462, 0.992]$$

这个例子虽然极小, 但已经包含了单层 decode 的完整合同:

Listing 3.8 最小 decoder block 的逐步账本

```

1. x_in -> RMSNorm -> h_attn
2. h_attn -> Wq/Wk/Wv -> q/k/v
3. append current k/v into KV cache
4. q dot historical K -> score row
5. softmax(score) -> prob row
6. prob weighted sum historical V -> attn_out
7. attn_out -> Wo -> x_mid
8. x_mid -> RMSNorm -> h_ffn
9. h_ffn -> Wgate/Wup -> g/u
10. silu(g) * u -> m
11. m -> Wdown -> ffn_out
12. x_mid + ffn_out -> x_out

```

核心思想

只要你能把这个最小例子逐标量复现，就已经具备了写 reference decoder layer 的能力。大模型真实工程只是把维度、head 数、层数、GQA 映射、RoPE 和量化合同扩展出去，而不是换了一套完全不同的数学。

causal mask 不是一个随手写的 if

attention 能保持“当前位置不能偷看未来”，靠的不是口头约束，而是 mask 的数学合同。对 prefill 中第 i 个位置看第 j 个位置，最基本的 causal mask 是：

$$mask_{i,j} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

它加到 score 上之后，未来位置经过 softmax 的概率就会变成 0。于是：

$$P_{i,j} = \frac{\exp(score_{i,j} + mask_{i,j})}{\sum_t \exp(score_{i,t} + mask_{i,t})}$$

这条式子看起来简单，但实现时有三个常见坑。第一，mask 是语义合同，不是某个 kernel 私有约定；prefill、decode、reference 和 optimized kernel 的可见位置集合必须一致。第二，很多高性能实现不会真的存一个完整 $[T, T]$ mask 矩阵，而是在 kernel 中按 i, j 规则即时判断。第三，若 softmax 在 fp32 accumulator 中做，可以直接使用 `-\infty`；若某条路径在低精度上直接做指数，常会改用足够小的有限负数，让它在指数后安全下溢到 0。

一旦从 full attention 改到局部窗口，mask 公式也会变化。若窗口宽度是 W ，则第 i 个位置只看最近 W 个可见历史：

$$mask_{i,j}^{(W)} = \begin{cases} 0, & \max(0, i - W + 1) \leq j \leq i \\ -\infty, & \text{otherwise} \end{cases}$$

这已经不是“同一个模型只是省点内存”，而是改变了可见上下文集合。任何 sliding window、sink token、prefix LM 或者 block sparse attention，都必须先把“谁能看见谁”写成明确合同，再去实现 kernel。

形态	第 i 个位置的可见集合	复杂度直觉
full causal	$0..i$	长度随 i 线性增长
window W	$\max(0, i-W+1)..i$	每行最多 W 个位置
prefix + window	固定前缀并加最近窗口	语义更复杂，缓存复用更敏感

为什么没有 KV cache 会慢得离谱

先想一个没有 KV cache 的 decode。已经生成了 S 个 token，现在要生成第 $S+1$ 个。如果没有 cache，模型为了让当前 token attend 到历史，必须重新把 $0..S$ 这些 token 跑过每一层，重新计算每一层每个历史位置的 K/V。下一步 $S+2$ 又要再算一遍 $0..S+1$ 。

这会造成重复计算。历史 token 的 K/V 一旦算出来，在后续 decode 中不会因为新 token 到来而改变。它们应该保存下来。KV cache 做的正是这件事：

Listing 3.9 有无 KV cache 的 decode 差异

```
without KV cache:
  step S:
    recompute K/V for tokens  $0..S$  in every layer
    compute attention for current token

with KV cache:
  prefill:
    compute and store K/V for prompt tokens
  step S:
    compute only current token K/V
    append K/V at position S
    read cached K/V[ $0..S$ ] for attention
```

核心思想

KV cache 省掉的是历史 token 的重复 K/V 投影和中间层重算；它没有省掉读取历史 K/V 的成本。decode 仍然要随上下文长度读取越来越多的 K/V，所以 KV cache 同时带来性能收益和内存压力。

这句话非常重要。KV cache 不是让 attention 变成常数时间；它让“重算历史网络”变成“读取历史 K/V”。前者会让每步重复大量矩阵乘，后者会让每步产生随上下文增长的内存读流。后续优化 KV cache 的核心，就是围绕容量、布局、读带宽、量化和分页管理做取舍。

KV cache 的逻辑形状和物理布局

KV cache 的逻辑形状通常可以写成：

Listing 3.10 KV cache 的逻辑形状

```
K_cache[layer, batch, kv_head, position, head_dim]
V_cache[layer, batch, kv_head, position, head_dim]
```

每一层都有自己的 cache；每个 batch/session 有自己的 cache；每个 K/V head 有自己的历史；每个 position 保存一个 `head_dim` 向量。这里用 `kv_head` 而不是 `head`，是因为一些模型使用 multi-query attention 或 grouped-query attention。它们让多个 query head 共享较少的 K/V head，以降低 KV cache 容量和读流。

逻辑形状不等于物理布局。CPU 后端可能希望 `position, head_dim` 连续，便于按历史顺序扫描；GPU 后端可能希望按 page、block 或 head 分组，让 warp 读取得到更好的 coalescing；分页 KV cache 还会引入 page table。无论物理布局怎样变，逻辑合同不能变：给定 `layer, batch, kv_head, position, head_dim`，读到的 K/V 必须一致。

布局选择	适合的访问	风险
position 连续	CPU 长上下文顺序扫描	多 session/page 管理较麻烦
head 连续	按 head 分工的 kernel	position 读流可能跨步
page/block	动态增长、释放、prefix 复用	page table 和边界处理复杂
量化 block	降低容量和带宽	scale metadata 进入热路径

KV cache 的容量账本

KV cache 容量可以用一条公式估算：

Listing 3.11 KV cache 容量估算

```
bytes =
  layers *
  batch *
  context_tokens *
  kv_heads *
  head_dim *
  2 /* K and V */ *
  bytes_per_element
```

举一个接近 7B 模型的账本。假设 `layers=32`, `kv_heads=8`, `head_dim=128`, 上下文长度 `S=4096`, batch 为 1, KV 用 fp16, 也就是每个元素 2 字节：

Listing 3.12 GQA 模型的 KV cache 例子

```
bytes = 32 * 1 * 4096 * 8 * 128 * 2 * 2
       = 536870912 bytes
       = 512 MiB
```

如果 `kv_heads` 不是 8, 而是 32, 容量会变成 4 倍, 也就是约 2 GiB。若 batch 从 1 变成 4, 再乘 4。若上下文从 4096 变成 32768, 再乘 8。KV cache 不是小 buffer, 它经常决定本地推理能不能跑长上下文、能跑多少并发 session、能不能放进 GPU 显存。

常见误区

权重量化解决的是静态模型容量和权重读带宽；KV cache 量解决的是运行时状态容量和历史 K/V 读带宽。两者对象不同、生命周期不同、误差来源不同，不能混为一谈。

KV cache 量化也不是免费。比如 int8 KV cache 会减少 K/V 码点字节, 但需要 scale metadata; per-token scale 精度较好, 但 metadata 和读取更重; per-block scale 开销小一些, 但误差和 block 边界要验证。报告不能只写“KV cache 减半”, 必须同时写 decode latency、metadata bytes、logits 误差和长上下文质量。

MHA、GQA、MQA 为什么影响 KV cache

attention 的 query head 数决定输出表达能力的一部分, 但 KV cache 的容量由 K/V head 数决定。三种常见形态可以这样区分：

形态	K/V head 数	含义
MHA	<code>n_kv=n_q</code>	每个 query head 有自己的 K/V head
GQA	<code>1<n_kv<n_q</code>	多个 query head 共享一组 K/V head
MQA	<code>n_kv=1</code>	所有 query head 共享一个 K/V head

继续用 `n_q=32`, `head_dim=128`, `L=32`, `S=4096`, fp16 KV。不同 K/V head 数的 cache 容量是：

形态	n_kv	KV 容量	每 token KV 读流
MHA	32	约 2 GiB	约 2 GiB
GQA	8	约 512 MiB	约 512 MiB
MQA	1	约 64 MiB	约 64 MiB

这张表解释了为什么很多本地模型选择 GQA。它保留多个 K/V head，比 MQA 更有表达能力；同时把 KV cache 容量和读流从 MHA 的四分之一甚至更低。注意，GQA 降低的是 Wk/Wv 输出、KV cache 容量和历史 K/V 读流；它不降低 Wq/Wo 和 MLP 的主计算量。

实现上，GQA 不能靠简单复制 K/V 到 32 个 query head 来“方便计算”。复制会把 KV cache 容量和读流重新放大，抵消 GQA 的主要收益。正确做法是在 attention kernel 中让多个 query head 映射到同一个 K/V head：

Listing 3.13 GQA head 映射不能靠物理复制糊弄

```
group_size = num_query_heads / num_kv_heads

for q_head in 0..num_query_heads-1:
    kv_head = q_head / group_size
    k = K_cache[layer, batch, kv_head, position, :]
    v = V_cache[layer, batch, kv_head, position, :]
    compute_attention(q[q_head], k, v)
```

核心想

GQA 是模型结构和运行时账本同时成立的设计：模型训练时学会共享较少的 K/V head，推理时才能少存、少读历史 K/V。没有这个前提，运行时强行合并 K/V head 会改变模型语义。

KV cache 缓存的不是 hidden

一个常见误解是：既然历史 token 已经算过，保存 hidden 就够了。实际不是这样。每一层 attention 需要的是这一层自己的 K 和 V ，它们来自当前层输入 hidden 经过 Wk/Wv 投影，再叠加 RoPE 等位置信息。第 3 层的 K/V 不能替代第 18 层的 K/V ；token embedding 也不能替代任意层的 K/V 。

对象	是否长期保存	原因
token ids	是	tokenizer 结果和上下文合同，需要用于位置、日志和重放
hidden	通常否	每层临时状态，下一层用完即可释放或复用 buffer
K/V	是	后续 token 的每一层 attention 都要读取历史 K/V
logits	通常否	sampler 用完即可丢弃，除非调试或审计需要保存

因此 KV cache 的正确性不能只看最终文本是否“差不多”。更严格的 oracle 应该能固定 prompt、权重、采样参数和位置编号，比较某层某位置的 K/V 或最终 logits。KV cache 布局、分页、量化、batch 合并都可以改，但逻辑坐标和数值合同不能漂移。

每生成一个 token 要读多少 KV

KV cache 的容量公式只回答“占多少内存”，还没有回答“每步读多少”。full attention decode 下，当前 token 在每一层都要扫描历史 K 算 score，再扫描历史 V 做加权求和。粗略读流就是：

Listing 3.14 decode 单 token 的 KV 读写粗账

```

kv_read_bytes_per_token =
    layers *
    context_tokens *
    kv_heads *
    head_dim *
    2 /* K and V */ *
    bytes_per_kv_element

kv_write_bytes_per_token =
    layers *
    kv_heads *
    head_dim *
    2 /* new K and V */ *
    bytes_per_kv_element

```

继续使用前面的 7B 级参数， $L=32$ ， $kv_heads=8$ ， $head_dim=128$ ，fp16 KV：

上下文长度	每 token 读 KV	每 token 写 KV
$S=4096$	约 512 MiB	约 128 KiB
$S=32768$	约 4 GiB	约 128 KiB

这张表比“KV cache 很大”更有工程意义：长上下文 decode 的热路径不是写入新 K/V，而是反复读取历史 K/V。KV int8 大致把码点读流减半；KV int4 大致降到四分之一，但 scale metadata、反量化和误差验证会进入热路径。滑动窗口、稀疏 attention、prefix cache、page attention 的价值，也要放在这条读流上判断。

若 prompt 长度为 P ，继续生成 G 个 token，KV 读取不是简单的 $G \times P$ 。因为上下文每步增长 1，累计读流近似为：

Listing 3.15 生成 G 个 token 的累计 KV 读流

```

total_kv_read =
    layers * kv_heads * head_dim * 2 * bytes_per_kv *
    (G * P + G * (G - 1) / 2)

```

这解释了为什么同一个模型在短回答和长回答中的平均 tokens/s 会不同。不是 sampler 变慢了，而是 decode 后半段每步要扫描更长的历史。

prefix cache 和 sliding window 改变的是语义

很多工程讨论会把 prefix cache、window attention、KV eviction 都叫成“cache 优化”。这不够准确。prefix cache 在语义上仍然是 full attention，它只是复用已经算过、并且逻辑位置完全一致的前缀 K/V；sliding window 则真的改变了每一步的可见历史集合；KV eviction 若没有模型层面的对应设计，往往直接改变 logits。

设用户 prompt 长度为 P ，已经生成了 t 个 token。full attention decode 的可见集合是：

$$Visible(t) = \{0, 1, \dots, P + t\}$$

如果某次请求复用了前缀缓存，只要前缀文本、tokenization 结果、RoPE 位置编号和模型权重都一致，那么前 $P-1$ 个位置的 K/V 可以直接重用；新 token 仍然会看见完整前缀。这时变的是“算不算前缀”，不是“能不能看前缀”。

若改成窗口大小为 W 的 sliding window，并保留完整前缀 $0..P-1$ ，后续第 t 步可见集合会变成：

$$Visible_{prefix+window}(t) = \{0, \dots, P-1\} \cup \{\max(P, P+t-W+1), \dots, P+t\}$$

若没有保留完整前缀，而是纯窗口 attention，则更激进：

$$Visible_{window}(t) = \{\max(0, P+t-W+1), \dots, P+t\}$$

这两者的差异非常大。前者还能长期保留系统提示词、指令头和固定 schema；后者在上下文够长时会把最早的系统指令也裁掉。实现上如果只是“把最旧 page 释放掉”，却没有同步修改 mask、position 合同和 benchmark 口径，那么报告里的模型已经不是原来的 full attention 模型。

常见误区

prefix cache 的复用条件比“前缀文本一样”更严格。tokenization 版本、special token 规则、chat template、RoPE 位置起点、模型权重版本，只要有一项不同，前缀 K/V 就不能安全复用。

计算量账本：线性层、attention 和 sampler

现在把计算量讲清。设 hidden size 为 H ，层数为 L ，prompt token 数为 T ，decode 当前上下文长度为 S 。一个线性层从 H 投影到 O ，处理 N 个 token，乘加量大约是：

Listing 3.16 线性层 FLOPs 粗账

```
linear_flops ~= 2 * N * H * O
```

系数 2 来自一次乘法和一次加法。这个公式足够解释 prefill 和 decode 的差别：

- prefill: $N=T$ ，一次处理很多 token，线性层更像 GEMM。
- decode: $N=1$ ，一次处理一个 token，线性层更像 GEMV 或小 GEMM。

attention 的账本分两部分。第一，当前 query 和历史 K 做点积，大约是 $2*S*head_dim$ 每个 head。第二，用 softmax 权重加权历史 V，也大约是 $2*S*head_dim$ 每个 head。所有层和所有 head 加起来，decode attention 的读流和计算量都随 S 线性增长。

sampler 也不能忽略。若词表大小是 V ，贪心采样至少要扫描 V 个 logits；top-k/top-p 还会有排序、选择或前缀概率累加。对于很大的 vocab 或 GPU 后端，如果每步都把完整 logits 从 GPU 拷回 CPU，sampler 和同步也会进入端到端瓶颈。

阶段	主要工作	常见瓶颈
prefill	多 token 线性层、prompt attention	GEMM 吞吐、activation 峰值、workspace
decode	单 token 投影、读取历史 KV	权重读带宽、KV cache 读流、小 kernel 开销
sampler	logits 后处理	词表扫描、top-k/top-p、设备同步

同一组 7B 参数下的 prefill/decode 对账

前面给了公式，现在把它们放进同一张账本里。仍取 $H=4096$ 、 $L=32$ 、 $I=11008$ 、 $n_q=32$ 、 $n_{kv}=8$ 、 $head_dim=128$ 。对 decode 单 token，单层主要线性部分大约是：

$$Q : 2H^2, \quad K/V : 2H(n_{kv}d) \times 2, \quad O : 2H^2, \quad MLP : 2HI \times 3$$

代入后：

项目	单层 FLOPs	32 层 FLOPs
Q projection	约 33.6M	约 1.07G
K/V projection	约 16.8M	约 0.54G
O projection	约 33.6M	约 1.07G
MLP 三矩阵	约 270.5M	约 8.66G
合计	约 354.4M	约 11.34G

这只是线性层，不含 attention 读历史 K/V、softmax、sampler 和 runtime 调度。若 prompt 长度 $T=2048$ ，prefill 线性层会把这 **11.34G** 乘上 **2048**，得到约 **23.2TFLOPs**；若 $T=4096$ ，就是约 **46.5TFLOPs**。但是 prefill 的权重 tile 能被多个 token 行复用，所以它不能按“每 token 读完整模型权重”粗暴相乘。

prefill attention 的二次项也要算。单 head 的 QK^T 和 PV 合起来约 $4T^2d$ ；所有 query head 和所有层合起来约：

$$F_{prefill_attn} \approx L \times n_q \times 4T^2d$$

代入后得到：

prompt 长度	线性层 FLOPs	attention 二次项 FLOPs
T=2048	约 23.2T	约 2.2T，因果半矩阵约 1.1T
T=4096	约 46.5T	约 8.8T，因果半矩阵约 4.4T

这张表说明：长 prompt 的 TTFT 不是只由线性层决定。prompt 从 **2048** 到 **4096**，线性层约翻倍，attention 二次项约翻四倍。若一个 benchmark 只测 **128** token prompt，它不能证明 **8K** prompt 的首 token 延迟。

再看 decode 长生成。假设 prompt 长度 $P=2048$ ，继续生成 $G=256$ 个 token。线性层 FLOPs 近似是：

$$F_{decode_linear} \approx 11.34G \times 256 \approx 2.90TFLOPs$$

decode attention 的累计历史长度不是 **256\times2048**，而是：

$$\sum_{t=0}^{G-1} (P+t) = GP + \frac{G(G-1)}{2} = 556928$$

attention 计算量约：

$$F_{decode_attn} \approx L \times n_q \times 4d \times 556928 \approx 0.292TFLOPs$$

注意这个 **0.292TFLOPs** 看起来不如线性层大，但它伴随大量 KV 读流。fp16 GQA KV 的累计读字节是：

$$32 \times 8 \times 128 \times 2 \times 2 \times 556928 \approx 68GiB$$

所以长生成的 decode 后半段常常表现为两条压力叠加：每个 token 都要读权重并做线性层，同时还要随上下文增长扫描历史 KV。权重量化主要压低第一条；KV 量化、paged KV、sliding window、GQA/MQA 主要压低第二条。但任一项只要改变 mask、position 或 K/V 数值解释，就必须回到 oracle 验证。

核心理想

prefill 的核心问题是“多 token 大矩阵 + 长 prompt 二次 attention”；decode 的核心问题是“每 token 权重流 + 随上下文增长的 KV 读流”。优化方向不同，验收指标也不能混在一个平均 tokens/s 里。

batch decode 不是免费吞吐

服务端常把多个 session 合并成 batch decode，希望让单 token GEMV 变成小 GEMM，提高权重复用。这个方向是对的，但它不是免费吞吐。设 batch 中有 B 个 session，线性层从 $[1, H]$ 变成 $[B, H]$ ，权重 tile 可以被 B 行复用；但是每个 session 的上下文长度可能不同，KV 读流仍然要分别扫描：

$$KVRead_{batch} \approx L \times n_{kv} \times d \times 2 \times bytes \times \sum_{b=0}^{B-1} S_b$$

若 batch 内有一个 32K 长上下文 session 和多个短上下文 session，attention kernel、page table 和调度可能被长请求拖慢。若为了凑 batch 等待太久，吞吐提高但 p95 延迟变差。真正的服务端账本至少要同时报告：

- batch size 分布，而不是只报最大 batch；
- batch 内 context length 分布；
- 每步实际扫描的 KV token 总数 $\sum S_b$ ；
- 合批等待时间和 decode kernel 时间；
- 是否出现 padding 计算、page table 间接访问和长短请求互相阻塞。

这也是 paged KV cache 的价值之一：它让不同 session 的 KV 可以分散增长和回收，调度器可以按 page 管理上下文。但 page table 引入间接寻址，kernel 必须把逻辑位置、物理 page、RoPE position、scale metadata 一起处理对。

logits 到 token 的算法细节

模型主体输出的是 logits，不是最终文字。设词表大小为 V ，最后位置的 hidden 是 h ，`lm_head` 权重是 W_{vocab} ，则：

$$z = hW_{vocab}, \quad z \in \mathbb{R}^V$$

z_i 是第 i 个 token 的未归一化分数。softmax 概率是：

$$p_i = \frac{\exp((z_i - m)/\tau)}{\sum_j \exp((z_j - m)/\tau)}, \quad m = \max_j z_j$$

这里减去 m 是为了避免指数溢出；`\tau` 是 temperature。temperature 越小，分布越尖，越接近贪心；temperature 越大，尾部 token 概率会被抬高。若实现没有先减最大值，长尾 logits 在 fp16/fp32 下都可能出现数值问题。

Listing 3.17 sampler 的常见执行顺序

```
logits = lm_head(last_hidden)
logits = apply_repetition_penalty(logits, generated_tokens)
logits = logits / temperature
logits = apply_top_k_filter(logits, k)
logits = apply_top_p_filter(logits, p)
prob = softmax_stable(logits)
next_id = sample(prob, rng_state)
```

top-k 是保留分数最高的 k 个 token，其他 token 置为负无穷。top-p 是先按概率从高到低排序，保留累计概率达到阈值 p 的最小集合。两者可以叠加，但顺序会影响结果。工程测试时必须固定顺序、随机数算法和 seed；否则同一个 logits 也会生成不同文本。

策略	算法含义	典型风险
greedy	取最大 logits	稳定但容易重复和死板
temperature	缩放 logits 分布	过高会放大低质量尾部 token
top-k	只在前 k 个候选中采样	k 太小会截断合理候选
top-p	保留累计概率核	排序和前缀和成本进入热路径
repeat penalty	惩罚已生成 token	过强会破坏术语和代码一致性

sampler 的 oracle 要分两层。第一层比较 logits：同一 prompt、同一权重、同一 KV cache 状态下，优化前后的 logits 误差应在阈值内。第二层比较 token 序列：固定 sampler 参数和随机数状态后，next token 应一致。只看最终自然语言文本是不够的，因为采样随机性会掩盖模型计算错误。

为什么 7B decode 常常像“读权重”

对 dense decoder-only 模型，decode 每生成一个 token 都要经过所有层。粗略说，每步会触碰模型的大部分权重。若权重是 fp16，7B 参数约 14GB；若是 int8，约 7GB 加 scale；若是 int4，约 3.5GB 加 group scale。实际 kernel 会有 cache、packing 和分块，但 decode batch 小，权重复用有限，内存带宽往往很显眼。

这解释了为什么量化能显著影响本地 CPU 推理：它不仅降低模型容量，也降低每步权重读流。但低比特会引入解包、scale 读取和反量化成本。收益来自“少读字节”，代价来自“多做转换”。是否更快，要看当前瓶颈是带宽、指令、cache、TLB 还是线程调度。

prefill 不同。prefill 有多个 token 行，权重能被多个 token 复用，GEMM kernel 更容易吃到算术吞吐。于是同一个模型可能表现为：prefill 很快，decode 慢；或者 GPU prefill 很强，但单用户 decode tokens/s 提升不如峰值算力那样夸张。

核心理想

不要用一个“模型速度”概括推理性能。至少要分开报告：加载时间、packing 时间、time to first token、prefill tokens/s、decode tokens/s、p50/p95 单 token 延迟、KV cache bytes 和峰值内存。

7B 级模型的一层粗账

为了让账本落地，取一组常见 7B 级参数做估算： $H=4096$ ，层数 $L=32$ ，MLP 中间维度近似 $I=11008$ ，query head 数 32，GQA 下 $kv_heads=8$ ， $head_dim=128$ 。不同模型数值会变，但量级判断相同。

decode 单 token 时，一层的主要线性层 FLOPs 近似如下：

子层	FLOPs 公式	量级
Q projection	$2 \cdot H \cdot H$	约 33.6M
K/V projection	$2 \cdot H \cdot kv_heads \cdot head_dim \cdot 2$	约 16.8M
O projection	$2 \cdot H \cdot H$	约 33.6M
MLP gate/up/down	$2 \cdot H \cdot I \cdot 3$	约 270M

这张表说明两个事实。第一，MLP 往往是单层 FLOPs 大头；第二，GQA 会显著降低 K/V projection 和 KV cache 容量，但不会降低 Q projection 和 MLP。粗略相加，一层 decode 线性部分约 350M FLOPs，32 层就是 11G FLOPs 量级。这个数字不是精确 benchmark，但足以解释为什么每 token latency 对本地机器很敏感。

再看权重字节。如果一层权重按 fp16 读，Q、O 各约 $H \cdot H \cdot 2$ 字节，MLP 三个矩阵约 $H \cdot I \cdot 3 \cdot 2$ 字节。仅这些线性层，一层就接近数百 MB 级权重流；32 层接近完整模型权重规模。int8、int4 的意义首先是减少这条读流，但它们会引入 scale、解包和反量化成本。

Listing 3.18 单 token decode 的粗略机器判断

```
if batch == 1:
    projection shape is mostly GEMV
```



```

weight reuse is low
weight bandwidth and cache/TLB behavior are critical
else:
    projection becomes small/batched GEMM
    weight reuse improves
    arithmetic throughput becomes more visible

```

把权重字节和带宽下限算出来

单 token decode 的线性层 FLOPs 很大，但在 batch 为 1 时，另一个更硬的约束是权重字节。可以先用最朴素的下限估算：

Listing 3.19 单 token decode 的字节下限

```

weight_bytes_per_token ~= parameter_count * bytes_per_weight
lower_bound_ms =
    1000 * (weight_bytes_per_token + kv_read_bytes) /
    effective_memory_bandwidth_bytes_per_second

```

对 7B 参数模型，忽略少量非矩阵参数和格式头，权重读流大致如下：

权重格式	码点字节	加上 scale 后的直觉
fp16	约 14 GB	几乎就是完整权重流
int8	约 7 GB	还要读 group scale
int4	约 3.5 GB	scale 和解包成本更显眼

把它和 KV 读流合在一起看。若 int4 权重实际要读约 **3.7GB**，上下文 **S=4096** 的 fp16 KV 每 token 约 **512MiB**，那么单 token 至少要搬运约 **4.2GB** 热数据。若有效内存带宽是 **100GB/s**，光搬数据的下限就是约 **42ms/token**，也就是最多约 **24token/s**。若有效带宽是 **300GB/s**，下限约 **14ms/token**，最多约 **71token/s**。

常见误区

这里的数字不是 benchmark 结果，而是反证工具。如果实测声称 batch 1、7B、长上下文、低带宽机器能远超这个下限，就必须解释权重是否常驻更高层 cache、是否跳过了部分层、是否用了 speculative decoding、是否测的是 prefill，或者统计口径是否把等待时间排除了。

从账本推到可服务容量

推理引擎能不能落地，不取决于一句“7B 可以本地跑”，而取决于容量预算是否闭合。最小预算可以先拆成三项：静态权重、运行时 KV cache、临时 workspace。

Listing 3.20 单机推理服务的内存预算

```

usable_memory >=
    model_weight_bytes +
    max_sessions * max_context_tokens * kv_bytes_per_token +
    workspace_bytes +
    runtime_overhead_bytes

```

前面例子里，fp16 KV 的 **kv_bytes_per_token** 是：

Listing 3.21 每个 session 每个 token 的 KV 容量

```

kv_bytes_per_token =
    layers * kv_heads * head_dim * 2 * bytes_per_kv
    = 32 * 8 * 128 * 2 * 2
    = 131072 bytes
    = 128 KiB

```

因此一个 4096 token 上下文的 session 需要约 512MiB KV；16 个并发 session 就是约 8GiB KV。若模型权重 int4 后约 3.7GiB，机器可用内存 16GiB，再扣掉 workspace、页表、线程栈、allocator 碎片和操作系统余量，能稳定承载的并发 session 不会等于 16GiB/512MiB=32。这个除法忽略了权重和运行时开销，是错误容量评估。

吞吐预算也要分 prefill 和 decode。一个请求的首 token 延迟可以粗略拆为：

Listing 3.22 请求延迟拆分

```

TTFT =
    queue_wait +
    tokenize_time +
    prefill_time(prompt_tokens) +
    first_sampler_time

per_output_token_latency =
    decode_compute_time +
    kv_read_time(current_context) +
    sampler_time +
    scheduler_overhead

```

这组公式能直接指导工程取舍。prompt 很长而输出很短，优先看 prefill；prompt 不长但输出很长，优先看 decode 后半段和 KV 读流；并发 session 多，优先看 KV 容量、分页和调度；batch 合并能提升权重复用，但会增加排队等待，p95 延迟可能变差。没有这些数字，只说“部署一个问答服务”没有工程意义。

prefill 的账本为什么不同

prefill 假设 prompt 长度是 T 。同一个 Q projection 从 $[1, H] \times [H, H]$ 变成 $[T, H] \times [H, H]$ 。FLOPs 乘以 T ，但权重不是读 T 份独立副本；优秀 GEMM 会让一块权重被多个 token 行复用。于是 prefill 的算术强度比 decode 高，GPU tensor core 或 CPU 高质量 GEMM 更容易发挥作用。

阶段	数据复用	典型优化方向
prefill	多个 token 复用同一权重 tile	GEMM、tile、batch、tensor core、activation planner
decode	单 token 复用弱，逐步读历史 KV	packed weight、低比特、GEMV、小 kernel、KV layout

attention 也不同。prefill 的 attention 处理 T 个位置，因果 mask 下每个位置看左侧历史，总体有近似 T^2 的交互；decode 每步只处理当前 token，看长度为 S 的历史，单步是 S 级别。若 prompt 很长，prefill attention 可能显著；若生成很长，decode 的累计 KV 读取会显著。

prefill attention 的矩阵账本

decode attention 是当前 query 扫描历史 K/V；prefill attention 是 prompt 内所有 query 同时看左侧历史。对单个 head，prefill 可以写成三个矩阵步骤：

$$S = \frac{QK^T}{\sqrt{d}} + M, \quad P = \text{softmax}(S), \quad O = PV$$

其中 $Q/K/V$ 的形状都是 $[T, d]$, S/P 的形状是 $[T, T]$, M 是 causal mask。第 i 行只能保留 $0..i$ 列, 未来列被设为负无穷。这个 $[T, T]$ 分数矩阵解释了为什么长 prompt 的 prefill attention 会突然变重。

步骤	形状	粗略 FLOPs
QK^T	$[T, d] * [d, T]$	$2 * T * T * d$
softmax	每行长度 T	指数、求和、归一化
PV	$[T, T] * [T, d]$	$2 * T * T * d$

若 $T=4096$, $d=128$, 单个 head 的 QK^T 就约 4.3GFLOPs, PV 也是同量级。32 个 query head 合起来是数百 GFLOPs 级别。实际高性能实现不会天真地把完整 $[T, T]$ 矩阵长期写到内存里, 而会分块计算、在线 softmax、尽量减少 HBM/DRAM 流量。这就是 FlashAttention 这类算法的核心方向: 数学结果不变, 改变中间矩阵的物理存取方式。

CPU 本地推理也要理解这一点。短 prompt 时, prefill 主要看线性层 GEMM; 长 prompt 时, attention 的二次项会变明显。若 benchmark 只用 32 token prompt, 就不能推断 8K prompt 的 TTFT。反过来, decode 单步没有 $[T, T]$ 分数矩阵, 但它会随着上下文增长持续读 KV cache。

常见误区

不要把 prefill tokens/s 和 decode tokens/s 混在一起平均。prefill 的主项包含批量线性层和 T^2 attention; decode 的主项包含单 token 权重流和 S 级 KV 读取。两个数代表不同问题。

softmax 为什么能在线分块算

前面写了 prefill attention 的主式:

$$S = \frac{QK^T}{\sqrt{d}} + M$$

若天真实现, 先把完整 $[T, T]$ score 矩阵写到内存, 再逐行做 softmax, 再把概率矩阵乘 V 。这样数学上没错, 但内存流量很大。高性能 attention 的关键观察是: softmax 可以在不物化整行的情况下分块计算。

先看一行 score $s_1, \dots, s_n$ 的稳定 softmax:

$$p_i = \frac{e^{s_i - m}}{\sum_j e^{s_j - m}}, \quad m = \max_j s_j$$

若把一整行拆成多个 block, 可以对每个 block 先算局部最大值和局部指数和, 再合并。设 block A 的局部最大值和局部和是 m_A, l_A , block B 的是 m_B, l_B , 则合并后的全局值是:

$$m = \max(m_A, m_B)$$

$$l = e^{m_A - m} l_A + e^{m_B - m} l_B$$

输出向量也能按同样的缩放关系合并。若 block A 的加权值和 block B 的加权值分别是 o_A, o_B , 则:

$$o = \frac{e^{m_A - m} l_A}{l} o_A + \frac{e^{m_B - m} l_B}{l} o_B$$

这说明 attention 不需要把完整概率矩阵 P 永久落到内存里。只要每处理一个 block 时维护“当前行的最大值、归一化分母和累计输出”, 就能得到与稳定 softmax 相同的数学结果。FlashAttention 这一类算法的核心就是这个思想: 不改变结果, 改变中间矩阵的物理存取路径。

Listing 3.23 一行 attention 的 online softmax 直觉

```

row_max = -inf
row_sum = 0
row_out = 0

for each score/value block:
    block_max = max(scores)
    new_max = max(row_max, block_max)

    row_sum = exp(row_max - new_max) * row_sum +
              sum(exp(scores - new_max))

    row_out = exp(row_max - new_max) * row_out +
              sum(exp(scores - new_max) * values)

    row_max = new_max

output = row_out / row_sum

```

核心思想

online softmax 的价值不是“少做了 softmax”，而是避免把巨大中间矩阵写回再读回。attention 的数学没有变，变的是内存交通组织方式。

roofline 判断：算力瓶颈还是带宽瓶颈

第二册讲过 roofline 思路：性能取决于算术强度，也就是每读写一个字节做多少计算。把它放到推理里：

Listing 3.24 推理阶段的算术强度直觉

```

prefill GEMM:
    many FLOPs per reused weight byte
    more likely compute-bound on strong backend

decode GEMV:
    fewer FLOPs per weight byte
    often bandwidth/cache/TLB sensitive

KV attention:
    reads K/V proportional to context length
    bandwidth grows with S

```

所以“加 GPU”不是万能解释。GPU 对 prefill 大 GEMM 很强；但单用户 decode 若 batch 小，可能被 launch、KV 读流、logits 拷贝和 sampler 同步限制。CPU 优化也一样：SIMD 指令很重要，但若主要时间在权重读流和 KV cache 读流上，单纯增加 FMA 吞吐不会线性提升 tokens/s。

验收与评分标准

一个高质量推理报告至少要回答：

- 当前测的是 prefill、decode，还是混合端到端。
- 模型参数、量化格式、上下文长度、batch、线程/GPU 配置是什么。
- 每 token 权重读流和 KV cache 读流大致是多少。
- 主要瓶颈更像算力、带宽、cache/TLB、同步，还是 sampler。

- 优化后 logits/oracle 是否仍然通过。

CPU 后端、GPU 后端和同一份算子合同

理解原理后，再谈后端。CPU 和 GPU 不应该各自定义一套模型语义。它们应该共享同一份算子合同：输入 shape、dtype、layout、量化 profile、误差界、KV cache 逻辑坐标一致。不同的是物理实现。

Listing 3.25 共享算子合同，分离后端实现

```
OperatorContract:
  op_name
  input_shape
  output_shape
  dtype
  quant_profile
  logical_layout
  accuracy_tolerance

CpuKernel:
  packed_weight_layout
  simd_width
  thread_plan
  numa_policy

GpuKernel:
  device_layout
  block_shape
  stream
  workspace
  copy_policy
```

CPU 后端的优势是可观察：可以看汇编、cache miss、TLB miss、线程扩展曲线和 NUMA 行为。GPU 后端的优势是吞吐：prefill 大 GEMM、attention kernel 和显存带宽更强，但 kernel launch、host-device 同步和小 batch decode 会带来固定成本。第三册的工程路线应该先让 CPU reference 和 CPU optimized path 把语义、oracle、profiler 建稳，再把同一合同挂到 GPU 后端。

本节验收线

读完本节，应该能回答下面问题：

- 文本怎样变成 token id，token id 怎样变成 hidden，hidden 怎样变成 logits。
- logits 和 sampler 的关系是什么，为什么 sampler 不应该掩盖模型计算错误。
- decoder block 中 Q/K/V、attention、MLP、residual、norm 分别做什么。
- KV cache 省掉了哪些重复计算，又引入了哪些内存和读带宽压力。
- 写出 KV cache 的逻辑形状和容量公式，并能做 7B 级粗略估算。
- 解释 prefill 为什么更像 GEMM，decode 为什么更像 GEMV 加历史 KV 读取。
- 说明为什么 decode 常常受权重带宽、KV cache 读流、小 kernel 和 sampler 同步影响。
- 用一组 7B 级参数粗算单层投影、MLP、KV cache 容量和 decode 权重读流。
- 用 roofline 思路判断当前问题更像算力瓶颈、带宽瓶颈还是同步/采样瓶颈。

本节之后再看量化、memory planner、AI 编译器和后端优化，才有稳定落点：不是为了堆名词，而是为了让这条从 prompt 到 next token 的链路更正确、更快、更可测、更能落地成服务。

3.2 Reference decoder 实现：先把一层算对

为什么必须先写 reference decoder

很多推理项目失败，不是因为缺少 SIMD 或 GPU，而是因为一开始没有可信 reference。后端越快，越难调；量化越低，越难判断误差来源；KV cache 越复杂，越容易把位置、head 映射和缓存槽混在一起。reference decoder 的作用是把数学合同写成最朴素、最容易审计的 C++20 代码，让后续所有优化都有比较对象。

核心思想

reference decoder 不是低质量版本，而是正确性基准。它可以慢，但必须清楚、可复现、少隐藏状态、少复用复杂优化代码。

本节目标不是实现完整 7B 框架，而是实现一个小型 decoder slice: token embedding、RMSNorm、Q/K/V linear、RoPE、causal attention、MLP、lm head 和 sampler 前 logits。这个 slice 可以用极小维度，例如 **H=16**、**heads=2**、**head_dim=8**、**layers=1**，也可以用真实 shape 的小权重 fixture。重点是每一步都能输出中间张量并被 oracle 比较。

为了避免每一节换例子，本册使用同一条 TinyTrace-16 作为手算主线：

Listing 3.26 TinyTrace-16 decoder 参数

```
vocab = 16
hidden H = 8
layers L = 2
query_heads = 2
kv_heads = 1
head_dim = 4
intermediate I = 16
prompt token ids = [3, 1, 4, 2]
```

真实 7B 模型只是把这些维度放大：**H** 从 8 变 4096，**L** 从 2 变 32，**I** 从 16 变 11008，**vocab** 从 16 变几万。reference decoder 的价值在于：TinyTrace 上每一步能手算，真实 shape 上同一套 shape 合同仍然成立。

reference 的边界

reference decoder 应该故意避开复杂优化：

- 不做 packed weight，直接读普通 row-major 权重。
- 不做 SIMD intrinsic，使用普通循环。
- 不做线程并行，避免浮点归约顺序变化。
- 不复用优化后端的 int4 解包或 GPU kernel。
- 不把多个算子融合成一个不可观察的大函数。

它应该保留的能力是检查和观察：

- 每个张量都有 shape、dtype 和 layout。
- 每个阶段能选择性 dump 中间结果。
- 每个算子能独立运行 fixture。
- 每个错误能指出哪一层、哪一个 head、哪一个 position。
- 固定 seed 的 sampler 能复现。

最小类型：先让 shape 说话

不要让裸指针和裸长度在 reference 中到处流动。最小实现也需要一个张量视图。

Listing 3.27 reference TensorView 的语义字段


```
TensorViewF32:
  data: span<float>
  shape: vector<int64>
  stride: vector<int64>
  name: string_view

requirements:
  shape.size == stride.size
  every index is checked in debug mode
  view does not own memory
  owner outlives view
```

reference 里可以接受 debug 检查稍慢，因为它换来定位能力。优化后端可以在 verifier 通过后走 unchecked fast path，但 reference 不应该这样做。每次 oracle 报错时，张量名、shape 和索引比一个段错误有用得多。

模型参数也要结构化，不要用字符串 map 在热路径查。

Listing 3.28 一层 decoder 的 reference 权重集合

```
DecoderLayerWeights:
  rms_attn_weight: [H]
  wq: [Q, H]
  wk: [KV, H]
  wv: [KV, H]
  wo: [H, Q]
  rms_mlp_weight: [H]
  w_gate: [I, H]
  w_up: [I, H]
  w_down: [H, I]

ModelWeights:
  token_embedding: [V, H]
  layers: array<DecoderLayerWeights>
  final_norm_weight: [H]
  lm_head: [V, H]
```

这里使用 `[out, in]` 形式表达线性层权重，和前面 `matvec` 章节一致：输出通道是行，输入通道是列。若模型文件来自别的布局，导入阶段应该转换或记录 layout，不要让 reference 每次猜。

RMSNorm：第一个数值合同

RMSNorm 的公式是：

$$y_i = x_i \cdot \frac{1}{\sqrt{\frac{1}{H} \sum_j x_j^2 + \epsilon}} \cdot w_i$$

reference 实现要固定三件事：累加精度、epsilon、输出 dtype。建议 reference 用 fp32 累加，即使真实后端用 fp16/bf16。原因是 reference 要更接近数学定义，而不是模仿某个硬件的舍入。

Listing 3.29 RMSNorm reference 伪代码

```
sum_sq = 0
for i in 0..H:
  sum_sq += x[i] * x[i]
```

```
scale = rsqrt(sum_sq / H + epsilon)

for i in 0..H:
    y[i] = x[i] * scale * weight[i]
```

RMSNorm 的测试不要只测随机数。至少包含全零输入、极小值、大值、负值和 **H** 不能被 SIMD 宽度整除的形状。虽然 reference 不用 SIMD，但后续 optimized path 会用这些 fixture。

Linear: 所有大算子的共同骨架

decoder 中多数重计算都是 linear。reference 版本应尽量朴素：

Listing 3.30 Linear reference 语义

```
for out in 0..OUT:
    acc = bias_or_zero(out)
    for in in 0..IN:
        acc += weight[out, in] * x[in]
    y[out] = acc
```

这段代码要配 shape 检查：

Listing 3.31 Linear shape 合同

```
x.shape == [IN]
weight.shape == [OUT, IN]
y.shape == [OUT]
if bias exists:
    bias.shape == [OUT]
```

后续 CPU 后端的 packed weight、int8、int4、SIMD micro-kernel 都要和这个 reference 比较。若 reference 自己也绕过 shape 或复用 packed descriptor，oracle 就失去意义。

RoPE: 位置编号必须作为参数传入

RoPE 不能从数组下标偷偷推导位置。reference 函数应该显式接收 `model_position`：

Listing 3.32 RoPE reference 合同

```
apply_rope(q_or_k, head_count, head_dim, model_position, rope_config):
    for head in 0..head_count:
        for pair in 0..head_dim/2:
            theta = frequency(pair, rope_config)
            angle = model_position * theta
            rotate two adjacent dimensions by angle
```

测试要覆盖下面几类错误：

- position 为 0 时旋转应保持不变。
- 同一向量在不同 position 输出不同。
- prefix 长度为 **P** 时，新 token 的 position 是 **P**，不是 0。
- 滑动窗口或分页 cache 改变物理槽时，position 不随槽回绕。

RoPE 的 oracle 不只比较最终 logits。最好在 tiny fixture 里直接比较旋转后的 Q/K。这样一旦长上下文漂移，可以更早定位到位置合同。

KV cache: reference 也要严格建模状态

即使 reference 很慢，也不能每步重新计算所有历史 K/V 后假装有 KV cache。那样无法测试真实运行时状态。reference 的 KV cache 可以用普通连续数组：

Listing 3.33 KV cache reference 字段

```
KVCache:
  layers
  max_context
  kv_heads
  head_dim
  key: [layers, max_context, kv_heads, head_dim]
  value: [layers, max_context, kv_heads, head_dim]
  filled_tokens
```

append 合同必须检查：

- 写入 position 小于 `max_context`。
- K/V 的 head 数是 `kv_heads`，不是 query head 数。
- 写入后 `filled_tokens` 和 session position 一致。
- debug 模式下不能覆盖已经写入但未被窗口策略淘汰的槽。

GQA 读取时从 query head 映射到 KV head：

Listing 3.34 GQA head 映射

```
group_size = query_heads / kv_heads
kv_head = query_head / group_size
```

这个公式必须由 verifier 保证整除。若 `query_heads` 不能整除 `kv_heads`，模型资产应在加载阶段被拒绝，而不是让 attention 函数自己补救。

Causal attention: 先写能看懂的版本

decode 阶段的单 token attention 最适合作为 reference 起点：

Listing 3.35 单 token decode attention reference

```
for q_head in 0..query_heads:
  kv_head = map_query_to_kv(q_head)

  for pos in 0..current_position:
    score[pos] = dot(q[q_head], key_cache[pos, kv_head]) / sqrt(head_dim)

  prob = softmax(score[0..current_position])

  out[q_head] = 0
  for pos in 0..current_position:
    out[q_head] += prob[pos] * value_cache[pos, kv_head]
```

softmax 要做数值稳定处理：

Listing 3.36 稳定 softmax

```
max_score = max(score)
sum = 0
```

```

for i:
    e[i] = exp(score[i] - max_score)
    sum += e[i]
for i:
    p[i] = e[i] / sum

```

prefill reference 可以先用同一函数逐位置执行，再逐步加批量版本。这样容易证明 causal mask：第 i 个 prompt token 只能看 $0..i$ 。后续优化可以把 prefill attention 做成矩阵形式或 fused kernel，但 oracle 仍然回到这个语义。

MLP: SwiGLU 的中间值不能丢

SwiGLU 的 reference 要保留 gate、up 和乘积中间值：

Listing 3.37 SwiGLU MLP reference

```

gate = linear(w_gate, x)
up = linear(w_up, x)

for i in 0..I:
    hidden[i] = silu(gate[i]) * up[i]

down = linear(w_down, hidden)

```

$\text{silu}(x)=x/(1+\exp(-x))$ 。这里的中间值很重要，因为 MLP 往往是 FLOPs 大头。若最终 logits 漂移，保留 gate/up/down 的 oracle checkpoint 能迅速判断是第一段线性、激活、逐元素乘，还是 down projection 出错。

单层 forward：把状态边写出来

一层 decoder forward 可以写成下面的形态：

Listing 3.38 单层 decoder reference forward

```

forward_layer(layer_id, x, model_position, kv_cache):
    h = rms_norm(x, rms_attn_weight)

    q = linear(wq, h)
    k = linear(wk, h)
    v = linear(wv, h)

    apply_rope(q, query_heads, head_dim, model_position)
    apply_rope(k, kv_heads, head_dim, model_position)

    kv_cache.append(layer_id, model_position, k, v)

    attn = attention_decode(q, kv_cache, layer_id, model_position)
    x = x + linear(wo, attn)

    h = rms_norm(x, rms_mlp_weight)
    mlp = swiglu_mlp(h)
    x = x + mlp

    return x

```

注意 `kv_cache.append` 是状态写入，`attention_decode` 是状态读取。reference 也要把这两步分开记录。若后续 memory planner 错误复用 buffer，或者 decode step 提前读取未写完的 K/V，trace 才能定位。

TinyTrace 的完整 shape trace

下面只跟踪一个 token 的一层 forward。设当前 token 是 prompt 中第 2 个 token，`model_position=2`，进入第 0 层前 hidden 形状是 `[H]=[8]`。

阶段	输出 shape	必须检查的合同
embedding	<code>[8]</code>	token id 在 <code>[0,16)</code> 内
RMSNorm(attn)	<code>[8]</code>	weight 是 <code>[8]</code> ，epsilon 固定
Q linear	<code>[8]</code>	<code>Wq=[8,8]</code>
K linear	<code>[4]</code>	<code>Wk=[4,8]</code> ，因为 <code>kv_heads*head_dim=4</code>
V linear	<code>[4]</code>	<code>Wv=[4,8]</code>
RoPE(Q)	<code>[2,4]</code>	使用 <code>position=2</code> ，不是 cache slot
RoPE(K)	<code>[1,4]</code>	KV head 数不同于 query head 数
KV append	<code>cache[0,2,0,:]</code>	写第 0 层、逻辑位置 2、KV head 0
attention	<code>[2,4]->[8]</code>	每个 query head 读 <code>pos0..2</code>
O linear	<code>[8]</code>	<code>Wo=[8,8]</code>
RMSNorm(MLP)	<code>[8]</code>	residual 后再 norm
gate/up	<code>[16], [16]</code>	<code>Wgate/Wup=[16,8]</code>
SwiGLU hidden	<code>[16]</code>	<code>silu(gate)*up</code> 逐元素
down	<code>[8]</code>	<code>Wdown=[8,16]</code>
layer output	<code>[8]</code>	第二次 residual

跑完整个 prompt 后，最后一个 token 的 hidden 进入 final norm 和 lm head：

Listing 3.39 从最后 hidden 到 logits 的 shape trace

```
last_hidden: [8]
final_norm_weight: [8]
normed: [8]
lm_head_weight: [16, 8]
logits: [16]
predicted_token = argmax(logits)
```

这条 trace 是 reference decoder 的最低验收线。若某个实现只告诉你“logits shape 是 `[16]`”，但不能说明第 0 层 position 2 的 K 写到了哪个 cache 坐标、query head 1 映射到哪个 KV head，它还不是可调试的 decoder。

把 shape trace 绑定到仓库代码

本仓库的 reference slice 落在 `books/cpu-volume-3/labs/linux_cpu_inference/src/reference_decoder.cpp`。它不是完整生产 runtime，但已经覆盖了这条链路的关键函数：

- `rms_norm`：用 fp32 累加定义 norm 合同。
- `linear_f32`：定义 row-major `[out,in]` 权重语义。
- `apply_rope`：显式接收 `model_position`。
- `ReferenceKVCache::append/key/value`：按 `layer,position,kv_head,dim` 寻址。
- `attention_decode`：实现 GQA head 映射和稳定 softmax。

- `run_reference_decode`: 从 token ids 跑到 logits 和 argmax。

测试入口在 `books/cpu-volume-3/labs/linux_cpu_inference/tests/lcqi_tests.cpp`。目前它覆盖 RMSNorm、RoPE、KV attention、reference decode end-to-end、packed int8 kernel 与 scalar 对齐。下一步要把 TinyTrace-16 的每个 checkpoint 固化成 golden 文件，而不是只检查最终 `predicted_token`。

端到端 reference decode loop

端到端 reference 可以先只输出 logits，不做复杂采样：

Listing 3.40 reference decode loop

```
tokens = tokenizer_fixture.encode(prompt)

for pos in 0..tokens.size:
    x = embedding[tokens[pos]]
    for layer in 0..layers:
        x = forward_layer(layer, x, pos, kv_cache)

last_hidden = x
normed = rms_norm(last_hidden, final_norm_weight)
logits = linear(lm_head, normed)
```

prefill 的最后一个 token 产生首个 logits。之后 decode 追加 sampler 选出的 token：

Listing 3.41 decode 生成循环 reference

```
for step in 0..max_new_tokens:
    next_id = sampler(logits, seed, params)
    tokens.push_back(next_id)

    pos = tokens.size - 1
    x = embedding[next_id]
    for layer in 0..layers:
        x = forward_layer(layer, x, pos, kv_cache)

    logits = linear(lm_head, rms_norm(x, final_norm_weight))
```

这里故意没有 batch、线程和 fusion。它的价值是每个 token、每层、每个中间值都能被检查。

Golden fixture 怎么设计

reference 必须配 golden fixture。一个高质量 fixture 至少包含：

Listing 3.42 tiny decoder fixture 内容

```
model_dims:
  vocab
  hidden
  layers
  query_heads
  kv_heads
  head_dim
  intermediate
  max_context
```



```

weights:
  token_embedding
  layer weights
  final_norm
  lm_head

input:
  prompt_token_ids
  sampler_params
  seed

expected:
  checkpoints
  logits
  generated_token_ids

```

权重不要全用随机数。建议混合使用小整数、正负数、接近零的值和非对称值。全随机 fixture 容易覆盖到计算路径，却不容易人工定位错误。比如 RoPE 可以用只有两个维度非零的向量，attention 可以用能手算 softmax 排序的 Q/K/V。

checkpoint 策略

不要只保存最终 logits。推荐 checkpoint 分层：

checkpoint	目的	常见错误
embedding	tokenizer 和查表	token id 错、词表错
norm output	RMSNorm	epsilon、累加精度
q/k/v	linear 与 head reshape	权重转置、shape 错
rope q/k	位置编码	position、维度配对
attention prob	mask 与 softmax	未来泄漏、溢出
attention out	KV cache 读取	head 映射、cache slot
mlp hidden	SwiGLU	激活、逐元素乘
layer output	residual	加法位置、覆盖 buffer
logits top-k	可见输出	lm head、采样差异

checkpoint 太多会增加文件体积，可以通过 `trace_level` 控制。日常测试比较关键 checkpoint；调试漂移时打开 debug 全量输出。

reference 和优化后端怎样比较

比较不能只看完全相等。不同 dtype 和不同归约顺序会造成微小差异。oracle 应按层级使用不同阈值：

Listing 3.43 oracle tolerance profile

```

f32_scalar:
  absolute: 1e-5
  relative: 1e-5

f16_or_bf16:
  absolute: 2e-2
  relative: 2e-2

int8_weight:
  per_operator_absolute: profile dependent

```

```
logits_topk_overlap: required
```

```
int4_weight:
```

```
compare logits, top-k and generation under fixed prompt set
```

阈值必须随 profile 记录到报告里。不要让测试代码里一个魔法数决定所有后端。特别是量化后端，operator 误差、logits top-k 和生成序列要一起看；单个最终文本“看起来不错”不能证明模型没有漂移。

本节验收线

读完本节，应该能动手写一个小型 reference decoder：

- 用 **TensorView** 和结构化权重表达 shape，而不是裸指针乱传。
- 分别实现 RMSNorm、Linear、RoPE、KV append/read、attention、SwiGLU 和 lm head。
- 显式传入 **model_position**，不把物理 cache slot 当 RoPE 位置。
- 用 tiny fixture 保存 checkpoint 和 logits golden。
- 用 oracle profile 比较 reference 与优化后端。
- 把 reference 当正确性基准，而不是被优化路径污染的慢版本。

3.3 推理算法：logits、softmax、采样、搜索与投机解码

模型输出的不是文字，而是分数分布

decoder 最后一层输出 hidden 后，**lm_head** 会得到词表大小的 logits。logits 是每个 token 的未归一化分数，不是概率，也不是最终文字。推理算法的任务是把 logits 变成下一个 token id。

核心想

“模型会不会回答得好”不只由权重决定，也由解码算法决定。贪心、温度、top-k、top-p、重复惩罚、beam search 和投机解码都会改变输出、延迟、可复现性和服务成本。

如果把模型主体看成 **hidden->logits**，采样器就是 **logits->nexttoken**。它在数学上很小，在业务上很重要：同一组 logits，贪心可能稳定但死板，高温采样可能多样但不稳定，top-p 可能减少低质量尾部 token，重复惩罚可能降低复读，但也可能破坏术语一致性。

softmax 和数值稳定

softmax 把 logits 转成概率：

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

直接计算容易溢出。稳定做法是减去最大值：

$$p_i = \frac{e^{z_i - z_{max}}}{\sum_j e^{z_j - z_{max}}}$$

这不会改变概率，因为分子分母同时乘了同一个常数。采样器必须使用稳定 softmax，尤其在低温或 logits 幅度较大时。

Listing 3.44 稳定 softmax 伪代码

```
max_logit = max(logits)
sum = 0
for i in vocab:
    prob[i] = exp(logits[i] - max_logit)
    sum += prob[i]
for i in vocab:
    prob[i] /= sum
```

在工程实现中，完整 softmax 会扫描整个词表。若词表有三万到十几万 token，sampler 的成本并非完全可以忽略。GPU 后端若每步把完整 logits 拷回 CPU 再采样，会引入同步和传输成本；CPU 后端则要注意 top-k 选择和概率归一化的复杂度。

贪心解码

贪心解码直接选择最大 logit：

$$next = \arg \max_i z_i$$

它的优点是确定、简单、可复现，适合测试、oracle 和某些结构化任务。缺点是缺少多样性，容易陷入重复，且不一定得到全局最优序列。贪心解码不需要 softmax，因为最大 logit 对应最大概率。

Listing 3.45 贪心解码

```
best_id = 0
best_value = logits[0]
for i in 1..vocab:
    if logits[i] > best_value:
        best_value = logits[i]
        best_id = i
return best_id
```

reference 和回归测试常用贪心，因为它去掉了随机性。若贪心下 logits top-1 都不一致，先不要讨论采样策略，应该回到模型计算、量化或后端 oracle。

温度：控制分布尖锐程度

温度采样先把 logits 除以温度 T ：

$$p_i = \text{softmax}(z_i/T)$$

T 越小，分布越尖锐，模型更接近贪心； T 越大，分布越平，低分 token 更容易被采到。温度不能简单理解成“创造力”。它改变的是概率分布熵，过高会增加胡言乱语，过低会增加模板化和重复。

温度	效果	常见用途
接近 0	接近贪心，稳定	测试、代码、结构化输出
中等	保留多样性	问答、摘要、普通对话
较高	分布更平，随机性更强	创意任务，但风险高

实现上要防止 $T=0$ 直接除零。工程中通常把 $T \leq 0$ 解释成贪心，或者拒绝参数。

top-k：只在前 k 个候选中采样

top-k 先保留 logit 最大的 k 个 token，把其他 token 概率置零，再归一化采样。它减少低质量尾部 token 的机会，也让采样成本从全词表 softmax 变成 top-k 选择加小集合 softmax。

Listing 3.46 top-k 采样流程

```
selected = find_top_k(logits, k)
for token not in selected:
    probability[token] = 0
softmax over selected logits
sample one token from selected distribution
```

$k=1$ 等价于贪心。较大的 k 给更多多样性。top-k 的工程关键是选择算法。完整排序是 $O(V \log V)$ ，只取前 k 可以用 heap、partial selection 或后端专用 kernel。对每 token decode 来说，词表扫描成本会累积。

top-p: 按累计概率截断

top-p 又叫 nucleus sampling。它先按概率从大到小排序，选择最小集合，使累计概率超过 p ：

$$\sum_{i \in S} p_i \geq p$$

然后只在集合 S 中重新归一化采样。top-p 的直觉是：当模型很确定时，候选集合小；当模型不确定时，候选集合大。相比固定 top-k，它能根据分布形态自适应。

代价是需要概率和排序或近似排序。若实现每步对全词表完整排序，sampler 可能成为短 decode 的显著成本。高性能实现通常会结合 top-k 上限、分块选择或 GPU sampler。

重复惩罚和频率惩罚

语言模型容易重复。重复惩罚会根据已生成 token 调整 logits。最简单的策略是：若 token 已出现，把正 logit 除以 penalty，把负 logit 乘以 penalty。这样已出现 token 的相对分数下降。

Listing 3.47 重复惩罚示意

```
for token in generated_tokens:
    if logits[token] > 0:
        logits[token] /= penalty
    else:
        logits[token] *= penalty
```

频率惩罚则按出现次数扣分。存在惩罚只看是否出现过。它们都是采样策略，不是模型本体。业务上要谨慎：代码、法律、医学、术语解释中，重复某些 token 可能是正确行为；过强惩罚会破坏格式、变量名和专有名词。

停止条件

生成循环不能只靠 `max_new_tokens` 停止。常见停止条件包括：

- 生成了模型的 EOS token。
- 生成了业务指定 stop token 或 stop sequence。
- 达到 `max_new_tokens`。
- 达到 `max_context_tokens`。
- 请求超时或被取消。
- sampler 发现非法分布，例如全部概率为零。

stop sequence 比 stop token 更难，因为它可能跨多个 token。服务端要在 token 流里维护一个小窗口，检查最近 token 是否匹配 stop sequence。若流式输出已经把 stop sequence 的一部分发给客户端，可能需要延迟输出或在业务协议中接受这种行为。不同产品会选择不同策略，但必须明确。

随机数和可复现性

随机采样必须有可控随机数源。固定 seed、固定 sampler 参数、固定 logits，应该得到同一 token 序列。多线程服务中不要使用全局共享随机数直接采样，否则请求之间会互相影响。

Listing 3.48 SamplerState 的最小字段

```
SamplerState:
    seed
    step
    temperature
    top_k
```

```
top_p
repetition_penalty
generated_token_counts
```

把 **step** 放进状态是为了让 trace 可重放。若第 37 步出现异常，可以用同一 logits、同一 sampler state 复现该步选择。业务上，完全可复现有助于审计和回归；创意任务可以使用随机 seed，但也应该把 seed 记录到 trace。

beam search：搜索多个候选序列

beam search 不是每步只保留一个 token，而是保留多个候选序列。每个候选有累计 log 概率。每一步扩展所有候选，保留得分最高的 **beam_width** 条。

Listing 3.49 beam search 伪代码

```
beams = [empty sequence with score 0]
for step in 0..max_new_tokens:
    candidates = []
    for beam in beams:
        logits = run_model(beam.sequence)
        for token in top_candidates(logits):
            candidates.add(beam + token, updated_score)
    beams = best beam_width candidates
```

beam search 常用于翻译、较短结构化任务或需要高概率序列的场景。但对聊天式大模型，它可能输出更保守、更模板化的文本，而且计算成本明显更高。若没有共享 KV cache 和批量扩展，beam width 为 **B** 就接近把 decode 成本放大 **B** 倍。实现 beam search 时，每个 beam 都有自己的 token 序列和 KV cache 分支，不能简单共享一个 session 状态。

投机解码：用小模型减少大模型步数

投机解码的目标是减少大模型 decode 次数。思路是用一个小模型先快速提出多个候选 token，再让大模型一次验证这些候选。若候选被接受，就相当于大模型一次前进多个 token；若被拒绝，则回退到大模型给出的分布。

简化流程：

Listing 3.50 投机解码简化流程

```
draft_tokens = small_model.generate_k_tokens(context)
big_model evaluates context + draft_tokens
for each draft token:
    accept if verification rule passes
    otherwise sample from corrected distribution and stop draft chain
```

投机解码不是简单“用小模型代替大模型”。它的关键是验证规则要保持目标大模型的分布不变或可控近似。工程上还要处理两套模型的 tokenizer 一致性、KV cache、显存/内存占用、调度和 trace。它适合在 decode 被大模型单步开销限制时使用，但会增加系统复杂度。

采样器放 CPU 还是 GPU

如果 logits 在 GPU 上，采样器有两种选择。把 logits 拷回 CPU 采样，实现简单，但每步有设备到主机拷贝和同步。把 sampler 放在 GPU，能减少同步，但实现 top-k/top-p、随机数和 stop 检查更复杂。

位置	优点	风险
CPU sampler	简单、易调试、业务逻辑方便	每步 copy/sync, GPU decode 受阻
GPU sampler	减少同步, 适合高吞吐	实现复杂, trace 和可复现性更难
混合策略	top-k 在 GPU, 最终选择在 CPU	仍需管理拷贝和候选集合

本册贯穿项目可以先用 CPU sampler, 但 trace 必须记录 logits copy 和 sampler 时间。否则未来接 GPU 后, 会误以为 kernel 很快就代表端到端快。

采样错误怎样定位

若生成结果异常, 不要先改温度。按下面顺序定位:

1. 固定 sampler 为贪心, 检查 logits top-1 是否和 reference 一致。
2. 固定 seed, 检查同一 logits 下 sampler 是否复现。
3. 检查 temperature、top-k、top-p 的应用顺序。
4. 检查重复惩罚是否只作用于已生成 token, 而不是 prompt 或全部词表。
5. 检查 stop token 和 stop sequence 是否在 token 边界处理。
6. 检查 GPU/CPU sampler 是否使用同一随机数和同一排序规则。

采样器看起来是“小组件”, 但它决定最终可见输出。工程报告应该把 `sampler_profile` 记录到 trace, 否则两次输出不同可能根本不是模型或后端问题。

本节验收线

读完本节, 应该能实现一个可控采样器:

- 知道 logits、softmax、概率和 token 选择的关系。
- 能实现稳定 softmax、贪心、温度、top-k、top-p 和重复惩罚。
- 能解释采样策略对确定性、多样性、质量和延迟的影响。
- 能设计 `SamplerState`, 固定 seed 后复现生成序列。
- 能区分 beam search 和普通采样的 KV cache 成本。
- 能说明投机解码为什么需要验证, 而不是直接相信小模型。
- 能把 sampler 时间、logits copy、seed 和参数写进 trace。

3.4 KV cache 实现细节: 布局、分页、位置、量化与故障模式

KV cache 是运行时数据结构, 不只是公式

前面已经讲过 KV cache 的数学作用: 历史 token 的 K/V 被保存下来, 未来 decode step 可以复用。真正写引擎时, KV cache 不是一句“缓存 K/V”, 而是一个运行时数据结构。它要回答: 内存按什么布局放, 逻辑 position 如何映射到物理槽, GQA head 怎样映射, 多个 session 如何分配, 超过上下文怎么办, 量化后 scale 放哪里, 取消时怎样释放。

核心思想

KV cache 的正确性由三个坐标共同决定: layer、model position、KV head。物理内存可以连续、分页或回绕, 但不能破坏这三个语义坐标。

本册后续用一个固定 tiny trace 贯穿 KV cache、reference decoder、memory planner 和 oracle:

Listing 3.51 TinyTrace-16 的 KV 参数

```
vocab = 16
hidden H = 8
layers L = 2
query_heads = 2
```



```
kv_heads = 1
head_dim = 4
sequence_tokens = [3, 1, 4, 2]
max_context = 4
```

这个例子故意小到可以手算，但它保留了真实 decoder 的关键结构：**query_heads!=kv_heads**，所以有 GQA；**seq=4**，所以每一步 decode 读到的历史长度不同；**layers=2**，所以地址公式必须带 layer 维度，不能只看一个数组。

最朴素的 fp16 KV cache 形状是：

Listing 3.52 连续 KV cache 形状

```
key: [layers, max_context, kv_heads, head_dim]
value: [layers, max_context, kv_heads, head_dim]
```

这个布局容易理解，适合 reference 和早期实现。但生产级 runtime 常会遇到变长 session、并发、内存碎片、长上下文和 prefix 复用，连续大块内存不一定够灵活。

从第 0 个 token 手算到第 3 个 token

在 TinyTrace-16 中，每层、每个 token 写入一条 K 和一条 V。因为 **kv_heads=1, head_dim=4**，一条 K 是 4 个元素，一条 V 也是 4 个元素。fp16 下每元素 2 字节，所以每层每 token 的 KV 字节是：

$$2 \times kv_heads \times head_dim \times 2 = 2 \times 1 \times 4 \times 2 = 16 \text{ bytes}$$

两层就是每 token 32 字节；4 个 token 的整个 session KV cache 是 128 字节。这个数字很小，但公式和 7B 完全一样。

decode step	写入	attention 可读逻辑位置	每层读 KV 字节 (fp16)
0	K0, V0→pos0	0	16
1	K1, V1→pos1	0, 1	32
2	K2, V2→pos2	0, 1, 2	48
3	K3, V3→pos3	0, 1, 2, 3	64

这里的关键不是 128 字节，而是增长规律：decode 第 **p** 步会读取 **p+1** 个历史位置的 K/V。长上下文变慢，不只是因为 softmax 更长，也是因为 KV 读字节随上下文线性增长。

连续布局地址公式

连续布局下，**key[layer, position, kv_head, dim]** 的元素偏移是：

$$offset = (((layer \times max_context + position) \times kv_heads + kv_head) \times head_dim + dim)$$

TinyTrace-16 中 **max_context=4, kv_heads=1, head_dim=4**，所以：

$$offset = ((layer \times 4 + position) \times 1 + 0) \times 4 + dim$$

坐标	元素偏移	fp16 byte offset	含义
key[0, 0, 0, 0]	0	0	第 0 层第 0 个 token 的 K 起点
key[0, 1, 0, 0]	4	8	第 0 层第 1 个 token 的 K 起点
key[1, 0, 0, 0]	16	32	第 1 层第 0 个 token 的 K 起点
key[1, 3, 0, 2]	30	60	第 1 层第 3 个 token 的第 2 维

value 数组用同一套偏移公式，但 base pointer 不同。真实 runtime 可以把 K 和 V 分开，也可以交错存；无论哪种布局，都必须能把 **layer, position, kv_head, dim** 映射回确定地址。

GQA head 映射的最小例子

TinyTrace-16 有 2 个 query head、1 个 KV head，因此：

$$group_size = query_heads / kv_heads = 2$$

$$kv_head = query_head / group_size$$

query head	kv head	读哪些 K/V
0	0	<code>key[layer, 0..pos, 0, :]</code>
1	0	<code>key[layer, 0..pos, 0, :]</code>

这说明 GQA 不是少算 attention head，而是多个 query head 共享同一组 K/V。若代码把 `kv_head=query_head`，在 TinyTrace-16 的 query head 1 上就会越界；在某些更大的模型上则可能读到错误 head，程序不一定崩，但 logits 会漂移。

常见误区

KV cache 最危险的 bug 往往不是 shape mismatch，而是语义坐标错位：RoPE 用了物理 slot、GQA head 映射错、prefix 后 position 从 0 重启、或 sliding window 读到了已经淘汰的槽。这些错误都可能输出“看起来像中文/英文”的文本，但长上下文质量会突然下降。

逻辑 position 和物理 slot 分开

模型语义使用 `model_position`。内存使用 `slot`。连续布局里二者常常相等，但实现不要依赖这个巧合。

Listing 3.53 KV 逻辑到物理映射

```
KvAddress:
    layer_id
    model_position
    kv_head
    head_offset

PhysicalAddress:
    block_id
    slot_in_block
    byte_offset
```

RoPE 使用的是 `model_position`，attention mask 使用的是历史逻辑范围，内存读写使用的是物理地址。若滑动窗口把物理 slot 回绕，`model_position` 不能跟着回到 0。若 prefix cache 复用前 1024 个 token，新 token 的 position 应该从 1024 开始，而不是从新 session 的局部数组下标开始。

连续布局的优点和限制

连续布局可以按 session 一次分配最大上下文：

Listing 3.54 连续 KV 分配

```
bytes =
    layers * max_context * kv_heads * head_dim *
    2 /* K,V */ * bytes_per_element
```

优点是地址计算简单，cache locality 可预测，debug 容易。缺点是浪费：很多请求实际不会用满 `max_context`，但 admission 为了安全会预留整块。并发多时，大量未使用 KV 空间会占住内存。

连续布局适合本册早期项目，因为它能先把正确性和账本讲清。等到并发和长上下文成为主要问题，再引入分页 KV。

分页 KV：把上下文拆成 block

分页 KV 把每个 session 的上下文拆成固定大小 block，例如每个 block 容纳 16 或 32 个 token。session 维护一个 block table：

Listing 3.55 分页 KV 元数据

```
KvBlock:
  block_id
  capacity_tokens
  used_tokens
  device_or_host
  ref_count

SessionKvTable:
  logical_block_index -> block_id
```

逻辑 position 到 block：

Listing 3.56 分页 KV 地址计算

```
logical_block = model_position / block_tokens
slot = model_position % block_tokens
block_id = session_table[logical_block]
address = block_base(block_id, layer, kv_head, slot, head_offset)
```

分页的优点是按需增长，便于 prefix block 共享，也便于释放。缺点是 attention 读历史 K/V 时不再是一个完全连续区间，kernel 要读取 block table。GPU 上还要考虑 block table 访问、coalescing 和 shared memory staging。

prefix cache 和引用计数

多个请求可能共享同一段 prompt，例如系统提示词或 RAG 模板前缀。prefix cache 可以复用这段 K/V。实现上，prefix block 应该只读共享，引用计数记录多少 session 使用它。

Listing 3.57 prefix cache 生命周期

```
build prefix:
  run prefill for shared prompt
  store KV blocks
  mark blocks read-only

new session:
  attach prefix blocks
  increase ref_count
  model_position starts after prefix length

session release:
  decrease ref_count
  free block when ref_count becomes zero
```

prefix cache 最容易错的位置是 RoPE 和 model position。复用 prefix K/V 时，新 token 的 position 不能从 0 开始；attention 也必须把 prefix token 算进历史长度。

滑动窗口和上下文截断

有些模型或服务会限制最近窗口，例如只保留最近 W 个 token 的 KV。此时物理内存可以回绕，但语义要明确：

- attention 只能读窗口内 token。
- RoPE position 是否继续增长，取决于模型的长上下文策略。
- 被淘汰 token 的 K/V 不能被读到。
- stop sequence 和业务 token buffer 可能仍需保留完整文本。

不要把滑动窗口实现成“数组下标取模”就结束。取模只是物理槽复用；语义上还要维护窗口起点、逻辑 position、可读范围和 RoPE 策略。

KV cache 量化

KV cache 量化的目标是减少容量和读带宽。fp16 KV 一个 4096 context session 约 512 MiB；int8 可接近减半；int4 可进一步降低。但 KV 是动态 activation，比权重量化更敏感。

Listing 3.58 KV 量化 descriptor

```
KvQuantDesc:
  bit_width
  scale_policy:
    per_token_head
    per_block_head
    per_channel
  group_size
  scale_dtype
  store_k_quantized
  store_v_quantized
  dequant_stage:
    before_score
    inside_attention_kernel
```

K 的误差会进入 dot product，再被 softmax 放大；V 的误差会在线性加权中传播。二者敏感度不同。有些实现可能只量化 V，或对 K 使用更高精度。不能简单把权重量化策略照搬到 KV。

KV 量化的 scale 放哪里

量化不只是压缩数据，还要保存 scale。若按 token/head 存 scale，metadata 随上下文增长；若按 block/head 存 scale，metadata 少，但误差可能增大。

scale 策略	优点	风险
per token/head	精度较好，适配动态范围	metadata 多，读流增加
per block/head	metadata 少，分页友好	block 内动态范围差异大
per channel	某些维度更稳定	实现和访问更复杂

报告不能只写 KV 从 512 MiB 变成 256 MiB，还要写 scale metadata 字节、attention 误差、logits top-k 和 decode latency。若 metadata 访问破坏 coalescing，实际速度可能不升反降。

KV cache 的 oracle

KV cache 需要专门 oracle，不要只看最终文本。

Listing 3.59 KV oracle 层次

```

append oracle:
    written K/V equals reference at model_position

address oracle:
    logical position maps to expected physical block and slot

read oracle:
    attention reads exactly positions in visible range

quant oracle:
    dequantized K/V within tolerance

score oracle:
    q dot k score drift within profile

```

score oracle 很重要。K 的小误差经过 dot product 和 softmax 后，可能改变 attention 分布。直接比较最终 logits 会太晚。

KV cache descriptor

为了让编译器、runtime 和后端使用同一套合同，KV cache 应该有 descriptor，而不是把参数散落在构造函数里。

Listing 3.60 KVCacheDesc 字段

```

KVCacheDesc:
    layers
    max_context
    query_heads
    kv_heads
    head_dim
    element_dtype
    layout:
        contiguous
        paged
        sliding_window
    block_tokens_optional
    quant_desc_optional
    rope_position_policy
    device_residency:
        host
        device
        unified

```

query_heads 和 **kv_heads** 同时出现，是为了让 verifier 检查 GQA 映射；**layout** 决定地址计算；**block_tokens** 只对分页布局有意义；**quant_desc** 决定 K/V 的存储 dtype 和 scale；**rope_position_policy** 说明 position 是否绝对递增、滑动窗口内重映射或使用模型特定 scaling；**device_residency** 决定后端是否需要 copy。

这份 descriptor 要进入 plan report。后端 kernel 不应该从 session 对象里临时读取一堆字段再猜 layout。若 CPU 和 GPU 使用不同 KV layout，也应该产生两个不同 plan，而不是一个 descriptor 被不同后端解释成不同含义。

KV append 和 read 的接口

KV cache 至少需要两个热路径接口：append 当前 token 的 K/V，读取历史可见范围。

Listing 3.61 KV append/read 合同

```
append(layer_id, model_position, key, value):
    require model_position < max_context or window policy allows it
    require key.shape == [kv_heads, head_dim]
    require value.shape == [kv_heads, head_dim]
    write into physical storage
    update logical length

read_range(layer_id, query_head, visible_begin, visible_end):
    kv_head = map_query_head(query_head)
    return K/V view for visible logical positions
```

append 是写状态，**read_range** 是读状态。它们要分别打 trace。比如一个 decode step 中，某层应该先 append 当前 token 的 K/V，再让当前 query 读 **0..position**。有些实现会先读历史再写当前 token，再单独把 self attention 加进去；这也可以，但 plan 必须明确。不要让调用顺序成为隐式约定。

多 session 分配器

业务服务里不止一个 session。KV cache manager 要负责分配、释放和统计。

Listing 3.62 KVCacheManager 职责

```
KVCacheManager:
    reserve(request_id, KVCacheDesc, max_tokens)
    attach_prefix(request_id, prefix_id)
    append(request_id, layer, position, k, v)
    release(request_id)
    stats()
```

reserve 要和 admission control 绑定；**release** 要在 completed、cancelled、timeout 和 backend error 路径都执行；**stats** 要输出已预留、已使用、空闲、碎片和共享 prefix block。若只在正常完成路径释放，长时间服务一定会出现内存只涨不降。

KV cache 与 memory planner 的边界

activation arena 可以按一层或一个 step 复用；KV cache 不能这样复用，因为它跨 token 存活。memory planner 必须知道哪些 value 是普通临时 activation，哪些是 session state。

内存对象	生命周期	planner 策略
临时 activation	一个 segment 或一层内	last-use 后复用
workspace	一个 step 或一个 plan	后端独占或按 stream 分配
packed weight	模型/plan 生命周期	prepare 后常驻
KV cache	session 生命周期	不参与普通 activation 复用
trace buffer	请求生命周期	有大小上限

如果 planner 把 K/V 当普通 activation，单步测试可能通过，第二个 decode token 就会读到被覆盖的数据。这类 bug 很隐蔽，必须通过 state effect 和 layer oracle 抓住。

容量压力下的策略

当内存不足时，有几种策略：

- 直接拒绝新请求，最简单也最可控。
- 降低 max context，但需要业务明确接受。
- 使用 KV 量化，减少每 session 容量。
- 使用分页 KV，减少未使用预留。
- 使用 prefix 共享，减少重复 prompt。
- 把部分 session 排队，而不是立即分配。

这些策略都要体现在 admission decision 中。不要在 runtime 中途偷偷把 KV 从 fp16 改成 int8，也不要静默截断上下文。容量策略改变的是语义或质量边界，必须对业务可见。

常见故障模式

故障	表现	定位证据
position 重置	prefix 后输出漂移	RoPE checkpoint
GQA head 映射错	某些 head 输出异常	head-level oracle
物理 slot 覆盖	长文本中途突然漂移	address oracle
context off-by-one	第一个新 token 或最后 token 错	visible range trace
scale metadata 错位	量化 KV 长上下文变差	quant oracle
取消未释放	并发后内存只涨不降	resource counter

这些故障都可能在 shape 正确、程序不崩溃的情况下发生。因此 KV cache 必须有自己的 trace 和 oracle。

KV trace 示例

一条 KV trace 不需要记录完整张量，但要记录坐标和容量。

Listing 3.63 KV trace event

```
KvTraceEvent:
  request_id
  layer_id
  operation: reserve | append | read | release
  model_position
  visible_begin
  visible_end
  kv_head_count
  head_dim
  layout
  physical_block_optional
  bytes_reserved
  bytes_used
  quant_profile_optional
```

当长上下文性能变差时，可以按 `visible_end-visible_begin` 分桶；当内存泄漏时，可以看 `reserve` 和 `release` 是否配对；当 prefix 复用错位时，可以看 `model_position` 是否从 prefix 长度后继续。

本节验收线

读完本节，应该能设计一个可靠 KV cache：

- 区分 layer、model position、KV head 和物理 slot。
- 能实现连续 KV，并知道它在并发下的浪费。
- 能解释分页 KV 的 block table、地址计算和 prefix 共享。
- 能处理 prefix cache、滑动窗口、取消释放和 context limit。

- 能设计 KV 量化 descriptor，并估算 metadata 成本。
- 能用 append、address、read、quant、score oracle 定位 KV 错误。

第 4 章

AI 编译器与推理运行时

4.1 推理运行时与内存规划：typed graph 如何服务一次真实请求

先从一个会抖动的请求开始

第 6 章把 AI 编译器讲成从推理图到 executable plan 的流水线，第 7 章把模型导入、manifest 和 verifier 固定下来。现在看计划真正落到一次请求时会遇到什么问题。假设用户输入 512 个 token 的 prompt，希望继续生成 128 个 token，本地引擎已经通过 verifier，权重也能被 kernel 读取。一个最朴素的 runtime 可能这样写：

Listing 4.1 朴素 runtime 的热路径

```
tokens = tokenizer.encode(prompt)

for op in graph:
    input_tensors = lookup_inputs(op)
    output_tensor = allocate(op.output_shape, op.output_dtype)
    run_kernel(op, input_tensors, output_tensor)

for step in 0..max_new_tokens:
    for op in graph:
        input_tensors = lookup_inputs(op)
        output_tensor = allocate(op.output_shape, op.output_dtype)
        run_kernel(op, input_tensors, output_tensor)
    next_token = sample(logits)
```

这段代码很容易在小模型上“跑通”，也很容易在 7B 级模型上暴露几个实际问题。第一，每个算子动态分配输出，会把 allocator、缺页、清零和碎片化放进热路径；decode 每生成一个 token 都重复这些成本，尾延迟会抖。第二，每次遍历 graph 时还在解释 op、查 dtype、查 layout、判断 shape、分支和间接调用压进最热循环。第三，所有 activation 都当作独立对象存活，内存峰值会被中间张量堆起来。第四，KV cache 是跨 token 状态，却被朴素代码混在普通临时张量里，容易出现覆盖、重复分配或错误增长。

真实推理运行时要做的不是“按图执行一遍”，而是把第 6、7 章的 typed graph、shape 合同、layout 合同和后端计划绑定到一个 session 状态上。编译期已经能决定的事情，不能留到每个 token 再判断；每个 token 才知道的事情，也不能假装编译期已经固定。

核心思想

推理运行时的核心任务，是把编译器产出的 executable plan 绑定到一次会增长、会采样、会持有 KV cache 的真实请求。内存规划不是运行时小优化，而是让计划稳定执行的前提。

runtime state：请求不是一串独立 kernel

一次本地对话请求至少包含下面这些运行期状态：

- session：模型实例、后端实例、编译计划、线程池、设备句柄和错误报告上下文。
- token buffer：prompt token、已生成 token、当前位置、最大上下文和 stop 条件。
- KV cache：每层已经写入的 K/V、当前可读长度、容量、布局和可能的 page table。
- activation arena：prefill 或 decode 当前阶段可复用的临时张量区域。

- workspace: 某些后端 kernel 需要的临时区, 例如 attention scratch、GEMM packing scratch 或 GPU workspace。
- sampler state: temperature、top-k、top-p、seed、重复惩罚和 logits 后处理状态。
- profiler/oracle: 记录事件、内存峰值、kernel 耗时和可选 reference 对比。

这些状态不属于同一种生命周期。权重从模型加载活到模型卸载; packed weight 从后端准备完成活到后端销毁; activation arena 通常只活在一个 plan invocation 内; KV cache 活在整个 session 内, 并且随 token 增长; sampler state 随生成循环推进; profiler 事件则是运行期证据。把它们全部塞进一个 “tensor map” 会让内存所有权模糊, 后续优化也没有安全边界。

Listing 4.2 推理 runtime 的核心对象关系

```
Engine
  ModelManifest
  TypedGraph
  ExecutablePlans
  BackendContext

Session
  TokenBuffer
  KVCache
  ActivationArena
  WorkspacePool
  SamplerState
  ProfilerTrace
```

注意这里的分层。Engine 持有模型级事实和编译计划, 多个 session 可以共享它; Session 持有请求级状态, 不能被另一个请求随意复用。CPU 后端的 packed weight、线程池和 NUMA 放置属于 Engine/BackendContext; 某个用户这次对话的 KV cache 和 token buffer 属于 Session。这个边界一旦错了, 多请求并发时会出现更隐蔽的问题: 一个 session 覆盖另一个 session 的 cache, 或者一个用户的 profiler/oracle 开关拖慢所有用户的热路径。

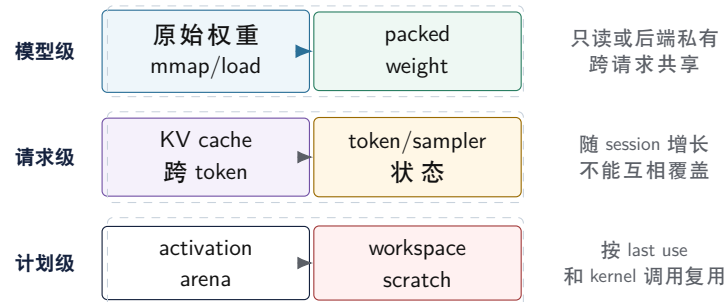
内存分类: 先分清谁能复用

内存 planner 的第一步不是写分配器, 而是给所有数据分类。下面这张表比 “都叫 tensor” 更接近真实运行时:

类别	生命周期	典型位置	复用规则
原始权重 packed weight	模型加载到卸载 后端准备到卸载	mmap 文件或主内存 CPU 对齐内存或 GPU 显存	只读, 不能当 scratch 后端私有布局, 按 kernel 合同读取
activation workspace	一段执行计划内部 单个 kernel 或 segment 内部	arena 或 device buffer workspace pool	last use 后可复用 按算法临时借用, 调用后归还
KV cache	session 生命周期	主内存、显存或分层缓存	追加写入, 历史可读, 不能被普通 arena 覆盖
logits	每步采样前后	小 buffer 或 device-to-host staging	可能需要保留 top-k 或完整分布

这张表背后是编译器理论里的 liveness analysis。一个 activation 的定义点是某个算子的输出, 最后一次使用点是最后一个消费者; 最后一次使用之后, 它占用的内存槽可以给别的 activation。原始权重和 KV cache 不满足这个条件: 权重会被每层、每步反复读取; KV cache 的旧 position 虽然不会再被写入, 却会在后续 attention 中反复读取。因此它们不能进入普通 activation arena。

推理内存的生命周期分类



图示：本图要证明的命题是：推理内存不能按“都是张量”统一处理，模型级、请求级和计划级生命周期必须分开。

底层机器后果也直接来自这张分类。packed weight 需要对齐到 cache line、页或 GPU backend 要求的边界；activation arena 希望少量大块内存重复使用，减少 allocator 和 TLB 抖动；KV cache 希望读流连续，避免 decode attention 每步跨太多页面；workspace pool 要能处理不同算法的最大临时需求，而不是每次 kernel 调用分配释放。

activation arena：把生命周期分析落到地址

activation arena 的目标很明确：预先申请一块或少数几块大内存，让所有短生命周期 activation 在里面复用。它类似传统编译器里的寄存器分配和栈槽复用，只是单位从标量变量变成了大张量。

planner 至少要知道每个中间值的这些字段：

Listing 4.3 arena planner 使用的 value descriptor

```
ValueDesc:
  id
  shape
  dtype
  layout
  byte_size
  alignment
  defined_at
  last_used_at
  materialization_required
  backend_location
```

defined_at 和 last_used_at 来自 typed graph 或 lowered plan 的拓扑顺序。若一个值被 residual 分支、oracle dump 或多个后续算子消费，它的 last_used_at 必须延长。若某个 debug 模式要求保存层输出用于 reference 比较，materialization_required 会阻止 planner 提前覆盖它。若某个值在 GPU device 上，不能被 CPU arena 的地址复用。

一个简化的 arena planner 可以用线性扫描实现：

Listing 4.4 线性扫描式 activation arena 规划

```
events = values sorted by defined_at
free_list = []
active = []

for v in events:
    expire all active values with last_used_at < v.defined_at
    if v is stateful or materialization escapes plan:
```

```

    allocate dedicated storage or mark external
    continue
    block = find_free_block(free_list, v.byte_size, v.alignment)
    if no block:
        block = grow_arena(v.byte_size, v.alignment)
    assign v.offset = block.offset
    add v to active

```

工业实现会更复杂：要处理 in-place 更新、别名、跨 stream 同步、不同 device、best-fit/first-fit 策略、碎片回收和峰值优化。但教材先抓住主线：只要 typed graph 告诉我们每个值何时产生、何时最后使用，就可以把多个不重叠生命周期的 activation 放在同一段物理内存上。

对齐不是语义细节。 CPU kernel 往往希望输入和输出至少按 32 或 64 字节对齐，便于向量 load/store 和 cache line 行为稳定。GPU 后端可能要求更大的 alignment 或特定 pitch。arena planner 若只按 byte size 紧密拼接，最后会把复杂的 unaligned 处理推给每个 kernel。

清零不是免费操作。 有些临时 buffer 不需要初始化，因为 kernel 会完整写入；有些 accumulator 或 mask buffer 必须清零。planner 应该把 zero-fill 作为 buffer 属性，而不是让每个算子默认 `memset`。否则一个看起来很小的 scratch 可能在长 prompt prefill 时变成可观带宽成本。

debug/oracle 会改变生命周期。 打开 oracle 对比时，某些中间结果需要物化并保留到 reference 比较之后。生产模式下能复用的 arena slot，在验证模式下可能不能复用。这不是“debug 模式慢一点”这么简单，而是 planner 必须显式支持不同 materialization policy。

常见误区

不要让 kernel 自己偷偷缓存输入指针。arena planner 会复用地址，如果某个异步 kernel 或 debug hook 在生命周期结束后仍持有旧指针，就会读到另一个张量的数据。异步后端必须把 stream/event 同步也纳入生命周期边界。

一个小图的 liveness 手算例子

内存规划不是抽象名词。看一个最小 residual block：

Listing 4.5 最小 residual block 的值流

```

v0 = input_hidden
v1 = rmsnorm(v0)
v2 = linear_qkv(v1)
v3 = attention(v2, kv_cache)
v4 = linear_wo(v3)
v5 = add(v0, v4)
v6 = rmsnorm(v5)
v7 = mlp(v6)
v8 = add(v5, v7)

```

若只看最后输出，`v1/v2/v3/v4/v6/v7` 都是短生命周期 activation；`v0` 必须活到第一次 residual；`v5` 必须活到第二次 residual；`kv_cache` 是外部状态，不进入 arena。可以把生命周期写成：

值	定义点	最后使用	规划含义
<code>v0</code>	输入	<code>add(v0, v4)</code>	不能在 attention 前覆盖
<code>v1</code>	RMSNorm	<code>linear_qkv</code>	QKV 后可复用
<code>v2</code>	QKV	<code>attention</code>	attention 后可复用
<code>v3</code>	attention	<code>linear_wo</code>	Wo 后可复用
<code>v5</code>	residual	<code>add(v5, v7)</code>	MLP 期间必须保留
<code>v7</code>	MLP	<code>add(v5, v7)</code>	输出后可复用

于是 planner 可以让 **v1** 和 **v3** 复用同一块 arena，让 **v2** 和 **v6** 复用另一块，只要它们的 shape/alignment 兼容。这个手算例子也解释了为什么 oracle/debug 会改变峰值内存：若要求保存 **v2** 的 Q/K/V 检查点，它的 last use 就不再是 attention，而是 oracle compare。

Listing 4.6 debug policy 对生命周期的影响

```
production:
    v2 last_use = attention

oracle mode:
    v2 last_use = compare_checkpoint_after_layer
    materialization_required = true
```

所以运行时不能把“打开 oracle”当成单纯多打印日志。它会改变 materialization policy、arena 峰值和调度同步点。

packed weight: 把文件布局变成内核布局

权重是只读的，但这不代表它可以直接以文件布局进入热路径。模型文件常按通用格式存储：张量名、shape、dtype、连续字节。高性能 kernel 需要的却是物理布局：按 tile 重排、按 SIMD 宽度分组、把 scale 和 weight 放在合适距离、补齐尾部、甚至为不同后端生成不同 packed format。

Listing 4.7 从 manifest 到 packed weight 的准备过程

```
for each linear weight tensor:
    verify logical shape and quantization metadata
    choose backend packing format
    allocate aligned packed buffer
    reorder or decode metadata into kernel-friendly panels
    record mapping: logical tensor -> packed descriptor
    keep checksum and source offset for diagnostics
```

packed descriptor 不是简单指针。它至少要告诉 kernel：每个 panel 的行列范围、stride、group scale 位置、tail padding、原始 dtype、计算 dtype、对齐和是否允许并行读取。对 int4/group quant 权重来说，scale 与 packed nibble 的关系尤其关键：如果 packing 只重排了 4-bit 数据，却没有同步重排 scale，kernel 会用错量化比例，输出仍然可能看起来“数值正常”。

这一步和传统编译器的后端 lowering 很像。高层 IR 中的 `linear(h, Wq)` 是语义；packed weight 是目标机器上的物理表示。语义值相同，不代表物理布局相同。CPU AVX2、AVX-512、ARM NEON 和 GPU tensor core 可能需要完全不同的 packed descriptor；runtime 选择 plan 时必须同时选择对应的 packed weight。

KV cache planner: 跨 token 状态不能当 scratch

KV cache 是 decoder-only 推理的核心状态。prefill 会批量写入 prompt 中所有 position 的 K/V；decode 每步追加一个 position，并让 attention 读取从开头到当前位置的历史 K/V。它的生命周期跨越整个 session，不能像 activation 一样在 last use 后复用，因为“last use”会随着未来 token 增长而不断后移。

Listing 4.8 KV cache 的逻辑合同

```
append(layer, batch, kv_head, position, k_vector, v_vector)
read(layer, batch, kv_head, range[0..position])

invariants:
    position is monotonically increasing inside one session
    every readable position has been written exactly once
```

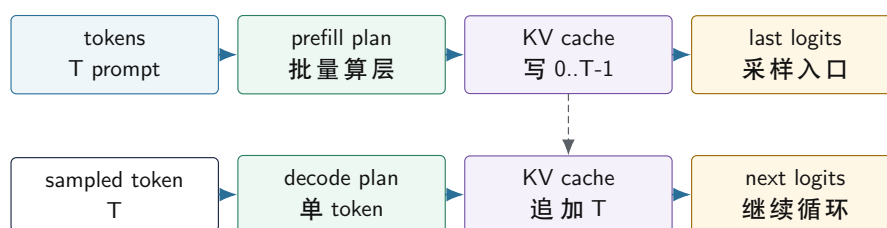
layout maps logical (layer, head, position, dim) deterministically
cache capacity is checked before write

planner 需要先回答容量问题。若直接按最大上下文一次性分配，地址简单，性能稳定，但峰值内存高。若按 page 或 block 增长，短对话更省内存，但 attention kernel 要通过 page table 读取，读流可能变碎。若使用 sliding window 或 KV eviction，必须把语义写进模型合同：并不是所有模型都允许丢弃远处 token；即使允许，position encoding 和 attention mask 也要同步处理。

策略	优点	风险
固定最大容量 按 block/page 增长	地址简单，kernel 容易优化 多请求场景更省内存	空间峰值高，短请求浪费大 page table、碎片和跨页读取增加复杂度
滑动窗口	长对话空间可控	改变可见上下文，必须匹配模型语义
KV 量化	降低 cache 带宽和容量	attention 精度、scale 存储和反量化成本要验证

KV cache 的物理 layout 要围绕后端访问方式设计。CPU decode 常常让一个线程或一组线程处理部分 head/output 行，连续读取某个 head 的历史 position；此时 **position** 和 **head_dim** 的局部连续性很重要。GPU kernel 可能按 block/warp 分摊 position 或 head，需要 coalesced load。两者可以使用不同物理 layout，但必须由同一个 logical descriptor 描述，不能让业务层知道后端私有排列。

prefill/decode 与 KV cache 的状态迁移



last logits 经 sampler 产生下一行的 decode 输入；prefill 写入的历史在 decode 阶段继续可读，decode 每步再追加一个位置。

图示：本图要证明的命题是：prefill 和 decode 不是同一形状下的两次执行，而是围绕 KV cache 读写合同形成的两个计划。

paged KV allocator: 不是 vector push back

固定最大 KV cache 简单，但并发请求一多就浪费。paged KV cache 把每个 session 的上下文切成固定 token 数的 block，例如每 page 存 16 或 32 个 token 的所有层/head K/V。allocator 管理空闲 page，session 的 page table 把逻辑 block 映射到物理 page。

Listing 4.9 paged KV allocator 的状态

```

KvPage:
  page_id
  capacity_tokens
  owner_session
  logical_block
  epoch
  written_mask
  k_bytes

```

```

v_bytes
scale_bytes_optional

SessionPageTable:
  logical_block -> page_id
  base_position
  window_size_optional
    
```

append 不能只是 **push_back**。它要先计算逻辑 block 和 block 内 offset，缺 page 时向 allocator 申请，写入 K/V 和 scale metadata，然后标记 written。读取时必须检查 page 属主、epoch 和 written bit，否则回收后的旧 page 会被当成当前上下文读取。

Listing 4.10 paged KV append/read 的核心检查

```

append(position, k, v):
  block = position / page_tokens
  offset = position % page_tokens
  page = page_table.get_or_allocate(block)
  require page.owner_session == session_id
  require page.epoch == expected_epoch(block)
  store k/v/scale at offset
  page.written_mask.set(offset)

read(position):
  block = position / page_tokens
  offset = position % page_tokens
  page = page_table.lookup(block)
  require page.written_mask.test(offset)
  return K/V view at offset
    
```

这套检查在 debug/oracle 模式必须严格执行；生产模式可以把部分检查移到 page 分配边界，但不能取消语义。否则一旦出现 sliding window 回绕、请求取消、page 复用或 prefix cache 共享，错误会表现成“长上下文偶发漂移”，极难定位。

prefill 和 decode：两套计划，两套指标

prefill 和 decode 的差异不是参数不同，而是计算形态不同。prefill 输入多个 prompt token，Q/K/V projection 和 MLP 更像 GEMM，attention 要处理较大的 token 块；decode 通常一次只处理一个新 token，线性层更像 GEMV 或 small-batch GEMM，attention 主要压力变成读取越来越长的 KV cache。一个 plan 若试图同时覆盖两者，通常会在其中一个阶段浪费机器资源。

维度	prefill	decode
输入形状	多 token prompt	通常 1 个新 token
主要矩阵形态	GEMM / batched GEMM	GEMV / small-batch GEMM
attention 压力	块内计算和写 cache	读历史 KV，随 context 增长
关键指标	time to first token、prompt 吞吐	tokens/s、单 token latency、尾延迟
内存行为	activation 大，arena 峰值高	activation 小，KV cache 带宽关键

runtime scheduler 需要把请求分成两个阶段，并在每个阶段选择对应 executable plan：

Listing 4.11 prefill/decode 调度主循环

```

plan_prefill = select_plan(stage=prefill, tokens=T, backend)
bind(plan_prefill, session.kv_cache, session.arena)
logits = run(plan_prefill, prompt_tokens)

while not stop:
    next_id = sample(logits, sampler_state)
    session.tokens.append(next_id)
    plan_decode = select_plan(stage=decode, context=session.position, backend)
    bind(plan_decode, session.kv_cache, session.arena)
    logits = run(plan_decode, next_id)

```

这里有两个容易忽略的细节。第一，`select_plan` 不应该在热路径里重新做 shape inference、fusion 和 packing；它只是在已经编译好的候选计划里选一个。第二，decode plan 可能按 context 长度分档，例如短上下文使用一个简单 kernel，长上下文使用分块 attention 或不同线程划分。分档选择要足够便宜，否则优化收益会被 dispatch 开销吃掉。

单请求调度和多请求调度不同。 教材前半部分可以先把一个 session 跑稳；服务端批量推理还要处理多个 session 的 prefill/decode 交错、batching、优先级、取消、超时和 backpressure。无论是否做连续批处理，底层不变量相同：每个 session 的 KV cache 和 token position 不能混，跨请求共享的只有模型级计划和权重。

线程池不是越满越好。 CPU decode 的单 token latency 常被小 kernel、同步和内存带宽限制。盲目让每个小算子都抢满线程池，可能增加 barrier 和 cache 竞争。scheduler 应该知道哪些 segment 值得并行，哪些 segment 用单线程或少量线程更稳。

GPU launch 也是成本。 GPU 对大 prefill 很有吸引力，但 decode 的单 token 阶段可能被 kernel launch、host-device 同步和 logits 取回成本影响。runtime 不能只看峰值 FLOPS，必须记录每个阶段的真实事件时间。

调度边界：segment 比单算子更适合热路径

如果 runtime 仍然按单算子逐个 dispatch，就算内存已经规划好，也会留下大量固定开销：查表、虚调用、线程池 barrier、设备同步和 profiler 事件过细。编译器可以把多个已经确定顺序和 buffer 绑定的节点合成 segment。segment 不是融合成一个 kernel，而是一个更粗的调度单元。

Listing 4.12 executable plan 中的 segment 草图

```

Segment layer_0_attention_prefill:
  inputs: hidden, position, kv_cache
  arena_bindings:
    norm_out  -> arena + 0x000000
    q_buffer  -> arena + 0x410000
    k_buffer  -> arena + 0x820000
    v_buffer  -> arena + 0xC30000
  kernels:
    rmsnorm_linear_qkv_fused
    rope_and_kv_append
    attention_prefill
    linear_wo
  sync:
    none on CPU
    stream event before host logits read on GPU

```

segment 的价值是把“计划已经决定”这件事落到数据结构里。runtime 执行 segment 时，不再解析 graph，也不重新分配 activation；它只绑定 session 中的外部状态，按预定 kernel 序列推进。

profiler 也可以按 segment 汇总，再在需要时打开 kernel 级事件。这样既保留调试能力，又不让热路径永远停留在解释器形态。

这 and 传统编译器里的基本块、trace 或机器指令调度有相似之处。单条 IR 指令适合分析，最终执行时却希望形成更稳定的机器代码或调度块。AI 推理计划也一样：Graph IR 适合表达语义，segment 适合表达运行期调度。

profiler 和 oracle：运行时要留下证据

没有 profiler，内存 planner 的改动很容易变成主观优化；没有 oracle，融合、量化和 layout 改动很容易悄悄改变 logits。运行时应该把证据作为一等输出，而不是出了问题才临时加日志。

一个最低可用的 profiler 事件可以这样设计：

Listing 4.13 推理 profiler 事件字段

```
event:
  session_id
  stage: load | prefill | decode | sample
  segment_id
  layer
  backend
  start_time_ns
  end_time_ns
  input_tokens
  context_length
  arena_peak_bytes
  workspace_bytes
  kv_cache_bytes
  kernel_name_optional
```

这些字段能回答几个关键问题：首 token 慢在 tokenizer、prefill 还是权重 packing；decode 慢在 linear、attention 还是 sampler；上下文变长后 attention 是否按预期变慢；arena 峰值是否符合 memory plan；某个后端切换是否真的改善了当前阶段。

oracle 则回答正确性问题。生产模式不可能每步都跑 reference，但开发和回归测试应该支持可控开关：

- 单算子 oracle：比较 RMSNorm、linear、RoPE、attention、MLP 的输出误差。
- 层级 oracle：比较某一层输入/输出 hidden state，定位误差扩散起点。
- logits oracle：比较 top-k、最大绝对误差、相对误差和采样前分布变化。
- 序列 oracle：固定 seed 和 sampler 后，比较生成 token 序列是否满足合同。
- 内存 oracle：检查 arena offset 是否重叠错误、KV position 是否越界、debug materialization 是否被提前覆盖。

oracle 不是简单打印浮点数。它要知道误差合同：fp32 reference 对 fp16/bf16/int8/int4 后端的容忍度不同；不同 reduction 顺序可能不 bitwise identical；量化 KV cache 会改变 attention 输出，需要单独设误差边界。把这些合同写进测试和报告，才能让优化有可追溯的安全线。

本节验收线

读完本节，应该能把 typed graph 到真实请求之间的运行时边界讲清楚：

- runtime state 至少包含 session、token buffer、KV cache、activation arena、workspace、sampler 和 profiler/oracle。
- 权重、packed weight、activation、workspace、KV cache 和 logits 的生命周期不同，不能放进同一种分配策略。
- activation arena 依赖 liveness analysis、last use、alignment、materialization policy 和后端位置。

- packed weight 是后端 lowering 的物理产物，必须保存 layout、scale、padding、对齐和诊断映射。
- KV cache 是跨 token 状态，容量、layout、page/block、sliding window 和量化策略都必须受模型语义约束。
- prefill 和 decode 应该使用不同 executable plan，并用不同指标评估：time to first token、prompt 吞吐、tokens/s 和尾延迟。
- segment 把图解释开销移出热路径，profiler 和 oracle 则把性能与正确性证据留在运行时。

下一章继续进入具体执行阶段时，就可以用本节的账本判断每个优化是否站得住：它缩短的是哪段生命周期，减少的是哪条地址流，改变的是哪个后端 layout，profiler 应该看到什么，oracle 又必须保证什么。

4.2 AI 编译器：从推理图到机器执行计划

推理引擎里为什么会出现编译器

前一章把本地 7B 级推理引擎拆成模型加载、tokenizer、decoder graph、KV cache、CPU/GPU 后端和证据报告。这里要再向下压一层：真实引擎不能每次遇到一个算子就临时解释执行。它必须把模型结构、张量形状、量化格式、设备能力和运行时状态编成一个可执行计划。这个过程就是 AI 编译器在推理引擎中的位置。

不要把 AI 编译器理解成“把 Python 变成机器码”的窄概念。推理场景里，输入通常已经不是普通源代码，而是模型文件、配置、tokenizer 规则、权重量化元数据和一张 decoder-only 计算图。编译器要做的事情是把这张图降到后端能高效执行的 kernel 序列，同时保持 logits、KV cache 和采样状态的语义不变。

Listing 4.14 推理引擎中的 AI 编译流水线

```
model/config/tokenizer
-> graph import and shape checking
-> typed tensor IR
-> graph rewrite and operator fusion
-> memory planning and lifetime analysis
-> backend lowering
-> kernel selection and packed layout generation
-> executable plan
-> runtime dispatch for prefill/decode
```

这条流水线和传统编译器很像。模型文件相当于源程序，图导入相当于 parsing，shape/dtype/quantization 检查相当于 semantic analysis，typed tensor IR 相当于中间表示，fusion 和 layout 选择相当于优化，CPU/GPU lowering 相当于后端代码生成，executable plan 则相当于目标程序。区别在于它编译的对象不是标量控制流为主的 C++ 函数，而是张量、状态、量化参数和设备 kernel 组成的推理程序。

核心思想

AI 编译器在推理引擎中的核心职责，是把稳定的模型语义编成稳定的机器执行计划。它既不是单个 kernel，也不是普通 runtime 调度器，而是连接图语义、张量布局、内存生命周期、后端能力和正确性证据的编译流水线。

前端：导入图时先建立语义合同

普通编译器前端会把源代码变成 token、AST 和符号表。AI 编译器的前端面对的是模型结构。以 decoder-only Transformer 为例，前端至少要确认这些事实：

- 每层有哪些算子：RMSNorm、Q/K/V projection、RoPE、attention、Wo projection、MLP gate/up/down、残差。
- 每个张量的 dtype、shape、stride、量化参数和驻留位置。

- 每个权重是否已经按某种 group size、scale 类型和物理布局保存。
- prefill 和 decode 是否共享同一张图，还是需要两套 specialization。
- KV cache 的逻辑 shape、最大上下文、当前位置和写入规则。

这些检查不是文档整理，而是编译器不变量。若 **Wq** 的输入维度和 hidden size 对不上，后端 kernel 不应该继续猜；若某个 int4 权重缺少 group scale，量化 lowering 不能把它当 int8；若 KV cache 的 head 数和当前 layer 的 attention head 数不一致，decode 会在很晚的位置产生错误 logits。前端要么拒绝模型，要么产出满足合同的 typed tensor IR。

Listing 4.15 typed tensor IR 的最小信息

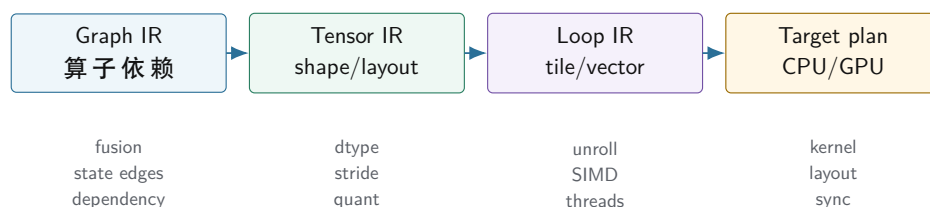
```
%0 = input_tokens      : tensor<[T], i32>
%1 = embedding(%0)      : tensor<[T,H], f16>
%2 = rmsnorm(%1, gamma) : tensor<[T,H], f16>
%3 = linear(%2, Wq_i4)   : tensor<[T,Q], f16>
%4 = linear(%2, Wk_i4)   : tensor<[T,K], f16>
%5 = linear(%2, Wv_i4)   : tensor<[T,V], f16>
%6q, %6k = rope(%3, %4, pos) : tensor<[T,Q], f16>, tensor<[T,K], f16>
%7 = kv_append(%6k, %5)  : state<KVCache>
%8 = attention(%6q, %7)  : tensor<[T,H], f16>
```

这段 IR 不是为了好看。它让每条边都带上类型、形状和状态含义。没有这些信息，后面的融合、内存复用、layout 转换和后端选择都只能靠脆弱约定。

IR：图 IR、张量 IR 和循环 IR 要分层

AI 编译器最容易犯的错误，是用一层表示承载所有事情。图 IR 擅长表达算子依赖：RMSNorm 后接 Q/K/V projection，RoPE 后写 KV cache，attention 后接 Wo projection。张量 IR 擅长表达 shape、stride、dtype、broadcast、reduction 和 layout。循环 IR 或 kernel IR 才适合表达 tile、向量宽度、展开、线程划分和寄存器累加器。

AI 编译器的三层 IR



图示：本图要证明的命题是：AI 编译器不能把算子图、张量布局 and 机器循环混成一层，否则优化会互相污染。

分层的好处是边界清楚。Graph IR 可以做无关后端的代数改写，例如把连续的 reshape 消掉，或把 RMSNorm 与后续线性层之间的临时 buffer 标成可融合候选。Tensor IR 可以决定某个 **[tokens, hidden]** 张量是否连续、某个 int4 权重是否要按 group 解码、某个 broadcast 是否会制造隐式 stride 0。Loop IR 再决定 CPU 上一个 tile 处理多少列、用几个累加器、尾部怎样 mask，或者 GPU 上一个 block 负责多少 token 和 head。

传统编译器也有类似分层。AST 不应该直接塞寄存器分配细节；SSA IR 不应该直接保存源代码排版；机器 IR 才关心调用约定和寄存器。AI 编译器同理：高层 graph 不应该知道 AVX2 的寄存器数，低层 kernel 也不应该重新推断模型层数和 KV cache 语义。

优化：融合不是越多越好

算子融合是 AI 编译器最常见的优化，但它经常被误讲成“把算子合在一起就快”。实际判断必须回到内存流和生命周期。若两个算子之间的中间张量很大、只被消费一次、且融合后不会增加过

多寄存器压力，那么融合可以减少一次写回和一次读取。若中间结果还被 residual、debug oracle、另一个分支或 KV cache 使用，盲目融合会破坏语义或让 buffer 生命周期变复杂。

Listing 4.16 融合决策要检查的事实

```
candidate: RMSNorm -> Linear

check:
  output of RMSNorm has exactly one consumer
  no oracle boundary requires materialized RMSNorm output
  fused kernel can stream input and gamma without spilling too much
  quantized Linear still has access to scale/group metadata
  backend supports the fused dtype/layout combination

if all true:
  lower to fused_rmsnorm_linear
else:
  keep two operators and expose the materialized buffer
```

attention 更能说明问题。prefill attention 可能是大块矩阵计算，decode attention 是一个 query 读取越来越长的 K/V cache。它们的形状不同，适合的 kernel 不同，融合边界也不同。prefill 可以倾向于高吞吐 tile；decode 更在意小 batch latency、KV cache 读流和 softmax 的数值稳定。一个“通用 attention 融合”若不区分这两种模式，很容易在其中一种场景下退化。

常见误区

融合优化必须由数据依赖、生命周期、数值误差和后端能力共同决定。只看算子名字相邻，就把它们写进一个大 kernel，是把编译器优化退化成手工拼接。

内存规划：生命周期分析是推理编译器的核心

本地推理最贵的不只是算术，还有内存容量和带宽。权重、packed weight、activation、KV cache、临时 workspace 和输出 logits 的生命周期完全不同。AI 编译器要像传统编译器做寄存器分配一样，为张量做内存分配。

Listing 4.17 张量生命周期的简化账本

```
weights:
  live from model load to session destroy
  read-only, often mmap or packed once

packed weights:
  live after backend preparation
  backend-specific physical layout

activation:
  live inside one layer or a small operator group
  can often reuse arena slots

KV cache:
  live across all decode steps
  grows by position and cannot be reused as scratch

workspace:
  live during one kernel or one scheduled segment
```

size depends on backend algorithm

这就是为什么内存规划不能靠 `new/delete` 随手分配。activation 可以进入 arena 并按生命周期复用；KV cache 必须按最大上下文或可增长策略预留；packed weight 可能比原始权重多占一份内存，但换来热路径布局稳定；GPU workspace 要考虑 stream 同步和设备内存峰值。若编译器没有显式生命周期，runtime 只能在执行时保守分配，结果是峰值内存上升、cache locality 变差、调试也更困难。

这一步和传统编译器的 liveness analysis 很接近。变量最后一次使用后可以释放寄存器；张量最后一次消费后可以释放 arena slot。区别在于张量尺寸大得多，KV cache 还跨 token 存活，后端 layout 可能要求额外对齐和 padding。

后端 lowering: CPU 和 GPU 的目标不同

同一个 Tensor IR 降到 CPU 和 GPU 时，目标函数不同。CPU 后端关心 cache line、TLB、SIMD 宽度、指令吞吐、分支、线程划分、NUMA 和 false sharing。GPU 后端关心 device memory、coalesced load、shared memory、warp/block 划分、kernel launch、stream 同步和 tensor core 格式。AI 编译器的后端 lowering 不能假装它们只是两个函数指针。

Listing 4.18 同一 linear 算子的两个 lowering 方向

```
linear(input[T,H], weight[0,H]):

CPU decode lowering:
  choose GEMV or small-batch GEMM
  use packed weight panels
  select SIMD width and tail handling
  partition output rows across worker threads

GPU prefill lowering:
  choose batched GEMM kernel
  ensure device-resident input and weight
  select vendor layout or internal tile layout
  schedule stream dependencies
```

这里的关键是 specialization。prefill 的 `T` 可能较大，decode 的 `T` 通常是 1；长上下文 attention 和短上下文 attention 也不该强行共享同一个计划。编译器可以为不同 shape、batch、上下文上限和后端生成多个 executable plan，再让 runtime 在请求进入时选择。这样做比每次动态判断更复杂，但能把热路径从“解释图”变成“执行计划”。

运行时：编译计划仍然需要调度

编译器生成计划后，runtime 仍然有工作。它要管理 session，接收 prompt，维护当前 position，选择 prefill 或 decode 计划，推进 KV cache，处理 sampler，记录 profiler 事件，并在必要时切换后端或回退到 reference。编译器负责把能提前决定的事情提前决定，runtime 负责处理请求期才知道的状态。

Listing 4.19 编译期和运行期的责任划分

```
compile time:
  validate model
  build typed graph
  choose layouts and fusion
  allocate static arenas
  prepare packed weights
  build backend executable plans
```

```

run time:
  tokenize prompt
  choose prefill/decode plan by shape and backend
  bind session buffers and KV position
  dispatch kernels
  sample next token
  collect correctness and performance evidence

```

这个划分也解释了为什么“推理引擎”和“AI 编译器”不是二选一。没有编译器，引擎会在每次请求中重复解释图、判断 layout、临时选择 kernel；没有 runtime，编译器生成的计划没有 session、KV cache 和 sampler 可以驱动。成熟系统通常是二者配合：前者把模型变成计划，后者把计划用在一个个真实请求上。

runtime 调度器：执行计划怎样变成 token 流

编译器产出 executable plan 后，runtime 调度器要把它用在真实请求上。调度器至少处理五件事：请求进入、prefill 排队、decode 循环、取消/超时、资源释放。它不是简单循环调用 `plan.run()`。

Listing 4.20 runtime 调度器的最小状态

```

Scheduler:
  loaded_models
  plan_cache
  ready_prefill_queue
  active_decode_sessions
  memory_reservations
  backend_contexts
  trace_sink

```

prefill 和 decode 队列要分开。prefill 可能是长 prompt 的大块计算，decode 是很多 session 的单 token 步进。若长 prefill 一直占住所有工作线程或 GPU stream，已经在交互中的请求会断流；若 decode 过度优先，长 prompt 首 token 会等太久。调度策略要根据业务目标取舍。

策略	优点	风险
prefill 优先	首 token 更快进入模型	活跃会话 decode 抖动
decode 优先	流式输出更平滑	新请求 TTFT 上升
分配时间片	两者较平衡	实现复杂，profile 更难读
按 deadline	符合 SLO	需要更准的耗时估计

取消、超时和资源释放

业务服务必须支持取消。用户关闭连接、上游超时、内容策略中止、队列过载，都可能要求停止某个 session。取消不是只设置一个 bool。runtime 要保证：

- 不再为该 session 提交新的 decode step。
- 已提交到 GPU stream 的工作要么等待完成再释放 buffer，要么走后端支持的安全取消路径。
- KV cache page、activation arena、workspace reservation 和 trace buffer 被释放。
- sampler state 和 token buffer 不再被其他线程访问。
- FinalResponse 标出 **cancelled** 或 **timeout**，方便业务层区分。

Listing 4.21 取消路径的资源顺序

```

cancel(session):
  mark session as cancelling

```

```

stop scheduling new decode steps
wait or fence backend work using this session buffers
release KV pages and arena reservations
close stream with final cancelled event
write trace summary

```

这个顺序和第二册讲过的异步生命周期一致：释放资源之前，必须证明没有异步执行还会读写它。CPU 后端也有类似问题，只是表现为线程池任务；GPU 后端表现为 stream/event。

plan cache 和 shape bucket

如果每个请求都重新编译 plan，热路径会很慢。runtime 通常需要 plan cache。cache key 至少包含模型、backend、量化 profile、stage、shape bucket 和 debug/oracle 配置。

Listing 4.22 PlanCacheKey 字段

```

PlanCacheKey:
  model_id
  manifest_hash
  backend
  quant_profile_id
  stage: prefill | decode
  prompt_length_bucket
  context_length_bucket
  oracle_policy
  trace_level

```

shape bucket 是折中。完全为每个 prompt length 编译一个计划，cache 会爆；所有长度共用一个计划，kernel 可能低效。可以把 prompt length 分成 1-32、33-128、129-512、513+ 等桶，把 decode context 也按 0-512、512-2048、2048+ 分桶。每个桶对应不同 attention plan、workspace 和 profiler baseline。

debug/oracle 配置也要进入 key。因为打开 layer oracle 可能强制 materialize 某些中间张量，导致 fusion 和 memory planning 不同。不能把 debug plan 和 production plan 混用。

正确性：优化后的计划必须能被验证

AI 编译器的优化会改变执行顺序、buffer 物化点、累加精度、量化缩放和后端 kernel。它不一定能做到 bitwise identical，但必须有可解释的误差合同。对一个本地推理引擎来说，至少要比这些层次：

- 单算子输出：RMSNorm、linear、RoPE、attention、MLP 与 reference 的误差。
- 层输出：一个 decoder block 前后 hidden state 的误差。
- logits：最大绝对误差、top-k 排名变化和采样前分布变化。
- 生成序列：固定 seed 和固定 sampler 下 token 序列是否在合同内。
- 性能证据：每个优化计划的 prefill/decode 分项耗时和峰值内存。

传统编译器有 verifier、IR dump、debug info 和测试套件；AI 编译器也需要对应工具。Graph IR dump 用来查融合是否正确，memory plan dump 用来查 buffer 生命周期，backend plan dump 用来查 kernel 选择，oracle report 用来查误差，profiler report 用来查优化是否真的改善瓶颈。

本节验收线

读完本节，应该能把“AI 编译器”和“推理引擎”的关系讲清楚：

- AI 编译器把模型、图、张量合同和后端能力编成 executable plan。
- 推理引擎在运行期维护 prompt、session、KV cache、sampler 和 profiler。
- Graph IR、Tensor IR 和 Loop IR 分别负责算子依赖、张量布局和机器循环。

- 融合优化必须检查消费者、生命周期、误差边界和后端支持。
- 内存规划本质上是张量级生命周期分析，KV cache 不能当普通 scratch buffer。
- CPU/GPU lowering 共享语义合同，但按不同硬件目标选择不同物理布局和 kernel。

后续进入真实模型加载、tokenizer 和具体算子时，这套编译视角会反复出现：每个优化都要说明它在哪一层 IR 生效，改变了哪条地址流或生命周期，后端计划如何验证，以及 runtime 怎样在 prefill 和 decode 中使用它。

4.3 AI 编译器前端：模型导入、manifest 与 shape verifier

先从一个会悄悄跑错的模型开始

推理引擎最危险的失败，不是程序立刻崩溃，而是模型能跑、速度也正常、生成结果却已经被污染。假设一个 7B 模型目录里有这些文件：

Listing 4.23 一个本地模型目录的最小文件组

```
config.json
tokenizer.model
tokenizer_config.json
weights-00001.bin
weights-00002.bin
quantization.json
```

如果 `config.json` 写着 `hidden_size=4096`，但某个 `Wq` 权重实际按 `[4096, 4096]` 以外的形状保存，后端 linear kernel 仍然可能读到连续字节并给出一个输出。若 tokenizer 的词表大小和 `lm_head` 的输出维度差一个特殊 token，采样器也可能只在极少数 prompt 上出错。若某个 int4 权重缺少 group scale，kernel 甚至能按照错误 scale 反量化出一串“看起来像数”的浮点值。

这就是 AI 编译器前端存在的理由。它不只是把文件读进内存，而是要把模型文件转换成一个已检查的 **manifest**（清单）和 typed graph。manifest 不是随手写的说明文档，而是“后端可以相信哪些事实”的合同：有哪些张量、每个张量在哪里、形状是什么、dtype 是什么、量化参数是什么、哪个算子消费它、输出应该是什么。

核心理想

模型导入阶段的目标不是“加载成功”，而是建立可验证的语义合同。没有 manifest、shape verifier 和 typed graph，后端 kernel 只能在热路径里猜测输入是否正确。

manifest：把文件事实变成编译器事实

传统编译器的源文件需要词法和语法分析；AI 编译器的模型文件需要 manifest。manifest 至少要收集四类事实：

- 模型结构：层数、hidden size、intermediate size、head 数、KV head 数、head dimension、最大上下文长度。
- tokenizer 合同：词表大小、特殊 token、BOS/EOS/PAD 规则、是否需要加入前缀空格、byte fallback 或 normalizer。
- 权重目录：每个权重张量的名字、文件偏移、字节长度、dtype、shape、逻辑 layout 和 checksum。
- 量化元数据：量化格式、group size、scale dtype、zero point 策略、per-tensor/per-channel/per-group 边界。

下面是一个压缩后的 manifest 草图。真实工程里可以来自 JSON、二进制索引或模型格式内置 metadata；教材先写成文本，是为了看清不变量。

Listing 4.24 模型 manifest 的核心字段

```
model:
  architecture: decoder_only_transformer
```



```

layers: 32
hidden_size: 4096
intermediate_size: 11008
attention_heads: 32
kv_heads: 8
head_dim: 128
max_context: 4096

tokenizer:
  vocab_size: 32000
  bos_id: 1
  eos_id: 2

tensor:
  name: layers.0.attn.wq.weight
  role: linear_weight
  shape: [4096, 4096]
  dtype: q4_group
  group_size: 64
  scale_dtype: f16
  file: weights-00001.bin
  offset: 1048576
  bytes: 9437184

```

manifest 的作用不是增加格式复杂度，而是把“文件里有什么”变成“编译器知道什么”。后续 graph import 不再直接扫文件名；它查询 manifest。后续 shape verifier 不再靠字符串约定；它读 tensor descriptor。后续 backend lowering 不再猜某个权重是不是 int4；它读 dtype 和 quantization descriptor。

导入不是字符串匹配

许多教学代码会用简单规则找权重名，例如看到 `wq` 就当作 Q projection，看到 `w1` 就当作 MLP gate。真实模型格式会让这种做法很快崩掉：不同模型使用不同命名，某些权重已经被合并为 QKV packed projection，某些模型有 grouped-query attention，某些模型把 `lm_head` 和 embedding tied 在一起，某些模型的 RoPE scaling 规则也不同。

前端应该把导入分成三步：

Listing 4.25 模型导入的三阶段

```

parse_files:
  read config, tokenizer config, tensor index, quant metadata
  build raw manifest entries

resolve_schema:
  map model-specific names to canonical roles
  detect tied weights and packed projections
  attach tokenizer and generation contracts

verify_and_import:
  run shape/dtype/quant checks
  produce typed graph nodes and tensor descriptors

```

第一步只读事实，不解释太多。第二步把模型方言映射到本引擎的 canonical roles，例如 `q_proj.weight`、`attention.wq.weight`、`layers.0.attention.wq` 都可能被解析为同一个角色。第

三步才产生 typed graph。这个顺序很重要：如果一边扫文件一边直接创建 graph node，错误通常会散落在导入代码、运行时代码和后端代码里。

schema resolution: 把模型方言翻译成统一角色

不同模型家族最烦人的地方不是数学完全不同，而是同一数学对象有不同名字、不同转置约定和不同融合方式。前端必须把它们翻译成本引擎内部的统一角色。否则每个后端 kernel 都会开始识别模型名字，系统很快失控。

外部命名示例	canonical role	verifier 要确认
<code>model.embed_tokens.weight</code>	token embedding	行数等于 vocab size, 列数等于 hidden
<code>self_attn.q_proj.weight</code>	attention Q	输出维等于 <code>n_q*head_dim</code>
<code>self_attn.k_proj.weight</code>	attention K	输出维等于 <code>n_kv*head_dim</code>
<code>mlp.gate_proj.weight</code>	MLP gate	输出维等于 intermediate
<code>lm_head.weight</code>	logits projection	是否 tied embedding, 词表大小一致

有些模型把 Q/K/V 合并成一个大张量，例如 `qkv_proj.weight`。这时 schema resolution 不能简单把它当成普通 linear，而要记录切片规则：

Listing 4.26 QKV packed projection 的导入合同

```
PackedQKV:
  source_tensor
  q_range: [0, n_q * head_dim)
  k_range: [q_end, q_end + n_kv * head_dim)
  v_range: [k_end, k_end + n_kv * head_dim)
  storage_layout: row_major | col_major | backend_packed
```

如果这个合同不显式保存，后端就可能把 Q/K/V 的切片顺序写死在 kernel 里。等另一个模型把顺序改成 Q/V/K，程序仍然能跑，但 attention 会稳定地错。

一层 decoder 的 shape 推导例子

把 shape verifier 写成代码之前，要能在纸上推导一层。设配置为：

Listing 4.27 一组 7B 级 decoder 配置

```
hidden_size = 4096
intermediate_size = 11008
attention_heads = 32
kv_heads = 8
head_dim = 128
```

则 verifier 应该推出：

张量角色	期望形状	推导来源
Q weight	[4096, 4096]	$n_q * d = 32 * 128$
K weight	[1024, 4096]	$n_{kv} * d = 8 * 128$
V weight	[1024, 4096]	同 K
O weight	[4096, 4096]	query heads 拼回 hidden
MLP gate/up	[11008, 4096]	intermediate by hidden
MLP down	[4096, 11008]	hidden by intermediate
RMSNorm gamma	[4096]	每个 hidden 维度一个权重

这个例子很重要，因为它能立即抓住 GQA 配置错误。若 K/V 权重被导入成 [4096, 4096]，但配置写着 `kv_heads=8`，前端必须拒绝或明确标记这是非 GQA 模型；不能让 attention kernel 到运行时才发现 `kv_head` 索引和权重输出维度对不上。

shape verifier: 让每条边都有可证明的形状

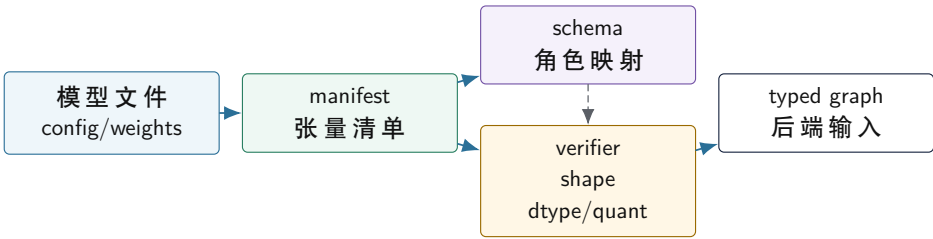
shape verifier 是 AI 编译器前端的语义分析器。它的输入是 manifest 和 graph schema，输出是带 shape 的 typed graph，或者一组明确诊断。以一层 decoder block 为例，若 hidden size 记为 H ，head 数为 A ，KV head 数为 KVA ，head dimension 为 D ，那么至少有：

Listing 4.28 decoder 层 shape 约束

```
H == A * D
K_shape == [KVA * D, H]
V_shape == [KVA * D, H]
Q_shape == [A * D, H]
Wo_shape == [H, A * D]
Wgate_shape == [I, H]
Wup_shape == [I, H]
Wdown_shape == [H, I]
RMSNorm_gamma_shape == [H]
KV_cache_shape == [layers, batch, KVA, max_context, D]
```

这些约束决定的不只是正确性，还决定性能路径。若 $KVA < A$ ，这是 grouped-query attention，decode attention kernel 要知道多个 query head 共享一组 K/V head；若 D 不是 SIMD-friendly 的倍数，CPU 后端要准备尾部处理；若 I 和 tile size 不对齐，MLP kernel 需要 remainder path。shape verifier 把这些事实提前暴露出来，而不是让后端在热循环里临时发现。

manifest 到 typed graph 的前端流水线



前端先把文件事实稳定成 manifest，再把模型方言解析成角色，最后由 verifier 产出可被后端相信的 typed graph。

图示：本图要证明的命题是：模型导入不是直接读权重跑 kernel，而是 manifest、schema 和 verifier 共同建立编译器事实。

dtype verifier: 不是每个字节数组都能进 kernel

shape 对了, dtype 仍然可能错。一个 linear 权重可能是 fp16、bf16、int8、int4 group-wise, 也可能是某个后端专用 packed format。dtype verifier 要检查三层事实:

- 逻辑 dtype: 算子语义中权重代表什么数值类型, 例如 f16 weight 或 q4 group weight。
- 存储 dtype: 文件中每个元素或压缩块怎样编码, 例如 4-bit pair、int8、f16 scale。
- 计算 dtype: kernel 中使用什么累加器和输出类型, 例如 fp32 accumulator、int32 dot accumulator、f16 output。

这三层不能混为一谈。int4 权重的逻辑值是近似浮点权重, 存储上可能两个 4-bit value 打包在一个字节, 计算时可能先反量化到寄存器再做 FMA, 也可能走整数 dot 再 requantize。若 manifest 只写 **dtype=int4**, 后端还不知道 group size、scale layout、zero point、padding 和尾部怎么处理。

Listing 4.29 量化权重 verifier 要检查的边界

```
for each quantized weight tensor:
    require group_size > 0
    require input_channels % group_size == 0 or tail_policy exists
    require scale tensor exists
    require scale length matches groups
    require packed byte length matches shape and bit width
    require backend supports this quant format
```

这些检查看似繁琐, 但比在 kernel 中读越界 scale 或错误解码权重便宜得多。前端拒绝模型是清晰错误; 后端悄悄算错 logits 是高成本错误。

verifier 负例矩阵

前端测试不能只加载一个正确模型。必须构造坏 manifest, 证明 verifier 会在正确层次拒绝它。最小负例矩阵如下:

负例	应报位置	典型风险
embedding 行数少于 vocab	tokenizer/embedding verifier	token id 越界
K/V head 数与权重维度不一致	shape verifier	GQA attention 读错 head
QKV packed 切片顺序缺失	schema verifier	Q/K/V 语义交换
int4 scale 数量不足	quant verifier	kernel 越界或误反量化
group size 为 0 或不支持	quant verifier	除零或 unsupported plan
RoPE head dim 为奇数	position verifier	二维旋转对不完整
max context 小于 prompt	runtime admission	KV cache 越界
GPU 不支持目标 dtype	backend capability	静默 fallback 或失败

每个负例都应该检查诊断内容, 而不只是“失败了”。比如 K/V head 维度错误的诊断要写出 **kv_heads*head_dim** 的推导; int4 scale 数量错误要写出 expected scale count; GPU dtype 不支持要写出 requested profile 和 backend capability。

RoPE 和位置参数也要验证

RoPE 参数如果错, shape 仍然可能全部正确。前端至少要检查:

- **head_dim** 是否能拆成二维旋转对, 通常要求偶数。
- RoPE base、scaling 类型、原始最大位置和扩展规则是否存在。

- prefill 与 decode 使用同一 position 语义。
- sliding window 或 prefix cache 是否记录 base position。

Listing 4.30 RoPE verifier 字段

```
RopeDesc:
  head_dim
  base_theta
  scaling_policy
  original_max_position
  effective_max_position
  position_base_policy
```

如果模型使用扩展上下文策略，而 manifest 没记录 scaling policy，引擎可以选择拒绝，而不是默认用基础 RoPE。默认继续跑会得到 shape 正确但长上下文错误的结果。

tokenizer 也是编译合同的一部分

很多人把 tokenizer 当作预处理，但本地推理引擎不能这样切开。tokenizer 决定 token id 序列，token id 决定 embedding 访问和位置编码，最终影响每一步 logits。若 tokenizer 的 BOS/EOS 规则、normalizer 或特殊 token 与模型训练时不一致，后端再快也只是高效地产生错误输入。

前端至少要验证：

- tokenizer 词表大小等于 embedding 行数，或有明确扩展规则。
- `bos_id`、`eos_id`、`unk_id` 等特殊 token 在词表范围内。
- generation config 中的 stop token 与 tokenizer 合同一致。
- prompt encode 后的 token 数不会超过当前 session 的 max context。
- tokenizer 输出 dtype 能被 embedding kernel 接受，通常是 32 位整数索引。

这部分和编译器的符号解析有相似性。源代码里的名字必须解析到某个声明；prompt 里的文本必须解析到模型词表中的 token id。若解析规则错了，后面所有 IR 和 kernel 都建立在错误输入上。

诊断：错误要停在前端，而不是后端崩溃

shape verifier 的诊断要像编译器错误，而不是像运行时崩溃。一个有用诊断应该包含：哪个 tensor、期望形状、实际形状、这个期望来自哪个 config 字段或上游算子，以及可能的修复方向。

Listing 4.31 好的模型导入诊断示例

```
error: tensor layers.12.attn.wk.weight has incompatible shape
expected: [1024, 4096]
  reason: kv_heads(8) * head_dim(128), hidden_size(4096)
actual:   [4096, 4096]
note: this looks like full multi-head K projection,
      but config declares grouped-query attention
```

这类诊断让使用者知道是模型文件、config、转换脚本还是引擎 schema 出了问题。反过来，如果只在后端报 `invalidargument` 或 `segmentationfault`，调试会被迫从机器码和二进制文件倒推语义，成本完全不对等。

前端产物：typed graph 必须足够给后端使用

前端通过后，输出不应该只是“一个 graph 指针”。它至少要包含：

- Graph nodes：算子类型、输入输出边、状态边和执行阶段标签。
- Tensor descriptors：shape、stride、dtype、layout、quantization、file location、ownership。
- Symbol table：canonical tensor name 到 descriptor 的映射。
- State descriptors：KV cache、position、session buffer 和 sampler state。

- Verification report: 通过了哪些约束, 哪些路径需要 fallback 或 tail handling。

这些产物会直接进入第 6 章讲过的后续流水线: Graph IR rewrite、Tensor IR layout 选择、memory planning、CPU/GPU lowering 和 runtime dispatch。前端如果偷懒, 后端就会被迫补课; 而后端补课通常发生在最难调试、最要求性能稳定的地方。

本节验收线

读完本节, 应该能说明 AI 编译器前端在本地推理引擎中的职责:

- manifest 把模型目录、权重文件、tokenizer 和量化 metadata 转换成编译器事实。
- schema resolution 把不同模型的命名方言映射到统一算子角色。
- shape verifier 用 config 和算子合同推导每条边的形状, 不让后端猜。
- dtype 和 quant verifier 区分逻辑 dtype、存储 dtype 和计算 dtype。
- tokenizer 合同必须和 embedding、generation config、max context 一起检查。
- 好诊断应该指出 tensor、期望、实际、推导来源和可能错因。

下一步进入 runtime 和 memory planner 时, 这些前端产物会成为所有优化的前提: 只有 typed graph 的边界稳定, 内存生命周期、kernel selection 和 CPU/GPU 后端计划才有可信输入。

4.4 AI 编译器细化: IR、pass、lowering 与 executable plan

先把 IR 讲清楚: 它是可检查的中间账本

如果没有学过编译原理, **IR** (中间表示) 这个词容易显得抽象。可以先把它理解成“机器可读、可检查、可改写的中间账本”。源代码、模型文件或 JSON 配置都不是优化的好对象, 因为它们包含太多外部格式细节。最终机器代码或 kernel 调用又太低层, 已经失去了很多语义。IR 站在中间: 保留足够语义让优化安全发生, 又足够结构化让程序可以分析和改写。

传统 C++ 编译器不会直接从字符生成机器指令。它先把字符切成 token, 组成 AST, 做语义检查, 降成 IR, 再进入优化和后端。AI 编译器也是同一个思路, 只是输入换成模型资产, 输出换成 executable plan。

核心理想

IR 的价值不是“多一层抽象”, 而是让每个优化都有检查对象。没有 IR, 融合、layout 选择、内存复用、量化 lowering 和后端选择都会退化成分散在运行时代码里的猜测。

本节把 AI 编译器从“概念流水线”细化到工程结构: Graph IR 表示算子依赖, Tensor IR 表示 shape/layout/dtype/quant, Loop IR 表示 tile/vector/thread, pass pipeline 表示改写顺序, lowering 表示逐步靠近 CPU/GPU 后端, executable plan 表示 runtime 真正执行的结果。

Graph IR: 先表达依赖和状态

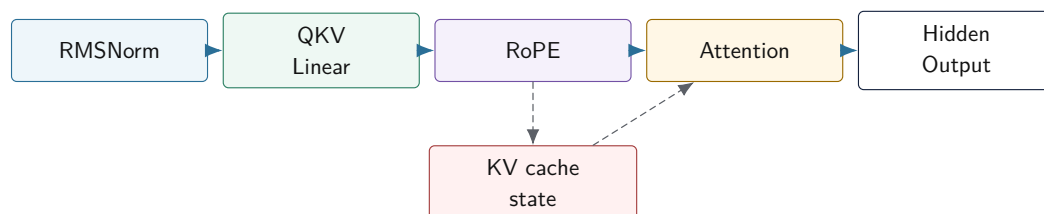
Graph IR 关心的是“谁依赖谁”。在 decoder-only Transformer 中, RMSNorm 的输出进入 Q/K/V projection, RoPE 的输出写入 KV cache, attention 读取 KV cache 并输出 hidden state, MLP 读取 residual 后的 hidden state。Graph IR 不应该关心 AVX2 有几个寄存器, 也不应该关心 GPU block 有多少线程。

Listing 4.32 Graph IR 节点的最小字段

```
GraphNode:
  id
  opcode: rmsnorm | linear | rope | attention | mlp | sampler
  inputs
  outputs
  state_reads
  state_writes
  stage: load | prefill | decode | both
  debug_name
```


`state_reads` 和 `state_writes` 很重要。KV cache 不是普通张量边；它跨 token 存活，会被 decode 反复读取。若 Graph IR 把 KV cache 当成普通 activation，memory planner 可能错误复用它的存储。传统编译器里，内存写、异常、volatile 和函数调用都需要特殊建模；AI 编译器里，KV cache、sampler state 和 profiler/oracle materialization 也需要特殊建模。

Graph IR 表达数据边和状态边



图示：本图要证明的命题是：Graph IR 不只记录张量数据流，还要显式记录 KV cache 这样的跨 token 状态边。

Tensor IR: shape、stride、layout 和 dtype 是语义的一部分

Graph IR 知道有一个 `linear` 节点，但还不够。后端需要知道输入是 `[T,H]` 还是 `[H]`，是否连续，stride 是多少，权重是 fp16 还是 q4 group，输出要放在 CPU 内存还是 GPU 显存。这些属于 Tensor IR。

Listing 4.33 Tensor IR value descriptor

```

Value:
  id
  element_type
  shape
  stride
  logical_layout
  physical_layout
  quant_descriptor_id
  device
  alias_info
  lifetime_info
  
```

初学者常把 shape 当作数组长度，把 stride 当作底层细节。实际不是。shape 决定算子数学语义；stride 决定地址公式；layout 决定 cache 和 SIMD 读流；dtype 决定数值误差和 kernel 选择；quant descriptor 决定 scale/zero point 和反量化规则。它们都是编译器需要理解的合同。

例如连续的 `[T,H]` hidden state 和转置视图可能拥有同样 shape，但地址流完全不同。一个 pass 如果只看 shape，不看 stride，就可能把一个本来连续的 kernel 应用到非连续视图上。第一册讲过对象布局和指针地址；这里就是把那些知识用于张量。

Loop IR: 机器优化从循环账本开始

当 Tensor IR 已经确定某个 `linear` 的输入、权重和输出，后端还要决定怎样循环。Loop IR 表示 tile、向量化、展开、线程划分和 memory access。它不再关心模型层名，而是关心循环变量和地址流。

Listing 4.34 一个 GEMV lowering 后的循环账本

```

for out_tile in output_channels step TILE_N:
    accum[TILE_N] = 0
    for k in input_channels step TILE_K:
  
```

```

x_vec = load input[k : k + TILE_K]
w_panel = load packed_weight[out_tile, k]
accum += dot_or_fma(x_vec, w_panel)
store output[out_tile]

```

这里的每个选择都对应第二册的硬件账本。TILE 的大小影响 cache line、SIMD 寄存器和预取；`out_tile` 的划分影响线程任务；`packed_weight` 的布局影响 TLB 和顺序读；tail path 影响分支和 mask；accumulator 数量影响 dependency chain 和寄存器压力。

Loop IR 不一定要做成复杂编译器基础设施。对于本册 C++20 项目，可以先用结构化 descriptor 和少量 lowering 规则表达它：

Listing 4.35 LoopPlanDesc 的保守工程形态

```

LoopPlanDesc:
  operator_id
  backend
  tile_m
  tile_n
  tile_k
  vector_width
  unroll_k
  tail_policy
  thread_partition
  packed_weight_desc
  accumulator_dtype

```

重点不是形式，而是让后端选择可见、可测试、可报告。不要把 tile 和 tail 都埋在某个巨大 `matmul.cpp` 里。

Pass 是带前置条件和后置条件的改写

Pass（编译遍）不是“随便遍历图改一改”。一个合格 pass 应该声明它读什么、要求什么、产出什么、会让哪些分析失效。传统编译器这样管理优化顺序；AI 编译器同样需要。

Listing 4.36 pass 合同的最小字段

```

PassContract:
  name
  input_ir
  required_analyses
  preconditions
  transformations
  produced_analyses
  invalidated_analyses
  verifier_after_pass

```

例如 `FuseRmsNormLinearPass` 的前置条件可能是：RMSNorm 输出只有一个消费者；consumer 是 linear；debug/oracle 不要求 materialize RMSNorm 输出；后端支持 fused dtype/layout；量化权重的 scale 可以在 fused kernel 中正确读取。后置条件是：原来的两个节点替换成 fused node；输出 shape 不变；lifetime 信息失效，需要重新计算。

常见误区

pass 如果不声明前置条件，就会把 bug 伪装成优化。特别是量化、KV cache 和 oracle materialization 相关 pass，必须明确哪些边界不能跨。

一个具体 pass pipeline

把 pass 讲成抽象概念还不够。一个能支撑本册推理引擎的保守 pipeline 可以先写成下面这样：

Listing 4.37 第三册项目的保守 pass pipeline

```
ImportCanonicalization
ShapeDtypeInference
QuantContractAttach
GraphVerification
SimpleCanonicalize
UseDefAndLivenessAnalysis
ConservativeFusion
MemoryPlanning
BackendLowering
PlanVerification
```

ImportCanonicalization。 模型导入后，先把外部格式里的名字、轴顺序、常量表示和算子别名统一成项目内部格式。比如某些模型文件里 Q/K/V 是三个权重，某些是一个 packed QKV 权重；某些图里 RMSNorm 写成多个基础算子，某些直接写成一个算子。canonicalization 的目标不是优化，而是让后续 pass 面对稳定输入。

ShapeDtypeInference。 这一遍推导每条 value 的 shape、dtype、layout 和 device。它必须能解释 prefill 的 `[T,H]` 和 decode 的 `[1,H]`，也必须能解释 GQA 中 `n_q` 与 `n_kv` 不同的情况。若 shape 推导失败，不能让后端自己猜。

QuantContractAttach。 量化不是后端小技巧。导入器读到的 int8、int4、group scale、zero point、packed order、tail policy，要在这一阶段附到 Tensor IR 的 descriptor 上。后续 pass 做 fusion、memory planning 和 lowering 时都要能查到。

GraphVerification。 在任何优化前先检查图：输入存在、输出唯一、状态读写合法、KV cache position 合法、sampler 只消费 logits、debug/oracle 要求的 materialization 点存在。这个阶段发现的问题通常是模型资产或导入逻辑问题。

SimpleCanonicalize。 做不会改变语义的小整理，比如删除恒等 reshape、合并连续 view、把等价 dtype 标记统一。它不应该跨过状态边，也不应该改量化解释。

UseDefAndLivenessAnalysis。 收集每个 value 的定义点、使用者和最后使用位置。memory planner 和 fusion 都要消费这些事实。不要在 fusion pass 里临时扫描一遍再自己做决定；这样会让测试粒度变差。

ConservativeFusion。 只做前置条件足够明确的融合。比如 RMSNorm 与 Linear 融合可以减少一次 activation 写回，但前提是 RMSNorm 输出没有被 oracle 要求物化、没有被多个消费者使用、后端支持对应 dtype/layout、量化 scale 读取顺序仍然正确。

MemoryPlanning。 根据 liveness、alignment、backend workspace 和 debug/oracle materialization 点分配 arena offset。KV cache、packed weight、workspace 和 activation arena 不属于同一类生命周期，不能混在一个简单线性 buffer 里。

BackendLowering。 把语义节点降到 CPU/GPU plan。CPU 计划包含 packed weight、tile、SIMD kernel、线程划分；GPU 计划包含 device buffer、stream、workspace、staging 和 sync。

PlanVerification。 最后一遍检查 executable plan：segment 顺序是否满足依赖，arena offset 是否冲突，kernel 是否支持实际 dtype/layout/quant，KV cache state binding 是否完整，profiler schema 是否能覆盖端到端指标。

这个 pipeline 故意保守。它没有一开始就做复杂图重写、自动调参或跨层大融合。原因是第三册的目标不是堆优化名词，而是让每个阶段都有可验证产物。等 reference、oracle、profiler 和回归门禁稳定后，再增加更激进的 pass 才有意义。

RMSNorm + Linear 融合的前置条件

用一个具体融合说明 pass 合同。RMSNorm 后面常接 QKV projection 或 MLP projection。若把 RMSNorm 输出写回 arena，再由 linear 读出，会多一次 $[N, H]$ 的内存读写。融合后的 kernel 可以一边算归一化，一边把结果喂给矩阵乘。

Listing 4.38 融合前后的图形态

```
before:
  y = rmsnorm(x, gamma)
  z = linear(y, packed_weight)

after:
  z = fused_rmsnorm_linear(x, gamma, packed_weight)
```

这个融合的前置条件至少包括：

- y 只有一个真实消费者，或者额外消费者只是可关闭的 profiler 统计。
- oracle 当前配置不要求比较 y 的逐元素输出。
- x 、 γ 和 packed_weight 的 dtype/layout 被后端 fused kernel 支持。
- quantized weight 的 scale metadata 能在 fused kernel 中按相同逻辑顺序读取。
- RMSNorm 的 ϵ 、累加类型和舍入规则与 reference profile 一致。
- 融合后 z 的 shape、dtype、device 和 quant contract 不变。

后置条件也要写清：原 `rmsnorm` 和 `linear` 被一个 fused node 替代； z 的逻辑 value id 可以保持或映射到新 id；use-def、liveness、cost analysis 和 memory plan 失效；shape/dtype 推导可以复用，但 pass verifier 必须重新检查输出合同。

核心思想

融合不是把两个函数塞进一个大函数。融合是一个受 use-def、oracle、dtype、layout、quant metadata 和后端能力共同约束的 IR 改写。

一个错误 fusion 的反例

再看一个不能随便融合的例子。假设图里有 RoPE、KV append 和 attention：

Listing 4.39 RoPE、KV append 与 attention 的状态边

```
q_rot, k_rot = rope(q, k, position)
kv_cache.append(layer, position, k_rot, v)
out = attention(q_rot, kv_cache.read(layer, 0..position))
```

有人可能想把它们全部融合成一个 `rope_attention` kernel，减少中间张量写回：

Listing 4.40 错误融合：丢掉了状态写入

```
out = fused_rope_attention(q, k, v, kv_cache, position)
```

如果这个 fused kernel 只在内部使用 k_rot 参与当前 attention，却没有把 k_rot/v 写入 KV cache，那么当前 token 可能看起来没错，但下一个 token 会读不到这一位置的 K/V。这个 bug 不一定立刻崩溃，因为 cache slot 里可能有旧值或零值；它会表现为后续 logits 漂移。

正确的融合必须把状态写入保留下来：

Listing 4.41 正确融合必须保留 state write

```
out = fused_rope_kv_append_attention(
    q, k, v,
    kv_cache_write_binding,
    kv_cache_read_binding,
    position)
```

pass verifier 要检查两件事。第一，融合前后的状态读写集合等价：KVCache[layer, position] 仍然被写入，attention 仍然读取 0..position。第二，执行顺序等价：当前位置写入必须在未来 token 读取前可见；若 GPU 上异步执行，还要有 stream/event 或 segment 顺序保证。

常见误区

AI 编译器里的状态边不能被当成普通 activation 边优化掉。KV cache、sampler state、随机数状态、debug/oracle materialization 都属于会影响后续行为的边界。

分析 pass：先收集事实，再做决定

很多优化应该先跑分析 pass，再跑变换 pass。分析 pass 不改 IR，只产出事实。变换 pass 使用这些事实做决定。把两者混在一个大函数里，后面很难调试和测试。

分析	产出事实	后续用途
Shape analysis	每条边 shape/dtype/layout	verifier、kernel 选择
Use-def analysis	每个 value 的消费者	fusion、liveness
Liveness analysis	defined/last use	activation arena
Alias analysis	是否共享存储或视图	in-place、内存复用
Quant analysis	scale/group/format 支持	quant lowering、oracle
Cost analysis	估算内存流和计算量	pass 决策、plan 选择

例如 fusion pass 不应该自己顺手扫描所有消费者并猜内存生命周期。它应该消费 use-def 和 liveness 结果。这样测试可以分别验证 use-def 是否正确、liveness 是否正确、fusion 是否尊重这些事实。

lowering：从语义逐步靠近后端

lowering 是把高层语义变成低层执行计划的过程。它不是一步到位，也不是直接把 graph node 换成函数指针。更稳妥的做法是分阶段：

Listing 4.42 AI 编译器 lowering 阶段

```
Graph IR:
    decoder block, attention, linear, mlp

Tensor IR:
    concrete shapes, layouts, quant descriptors

Backend-independent plan:
    fusion boundaries, liveness, memory plan

Backend-specific plan:
    CPU packed panels, SIMD kernel desc, thread partition
    GPU device buffers, kernel adapter, stream dependencies

Executable plan:
    segments, kernel calls, arena offsets, state bindings
```

分阶段的好处是：高层 verifier 可以检查语义，低层 verifier 可以检查机器合同。比如在 backend-independent plan 中，attention 仍然是语义操作；到了 CPU plan，decode attention 可能降低为按 head 和 position 分块读取 KV cache 的 kernel；到了 executable plan，runtime 只看到 segment、buffer offset、kernel handle 和 state binding。

CPU lowering: 从 tensor 到 SIMD plan

以 decode 阶段的 quantized linear 为例，CPU lowering 要做的事情包括：

- 判断当前 shape 更像 GEMV 还是 small-batch GEMM。
- 查询 CPU feature，选择 scalar、AVX2、AVX-512 或其他 ISA plan。
- 确认权重是否已经按目标 ISA packed，必要时触发 prepare。
- 根据 output channel、group size、tail 和 cache 选择 tile。
- 选择单线程或多线程分区策略。
- 生成 kernel 调用参数和 profiler 标签。

Listing 4.43 CPU lowering 输出的计划片段

```
CpuKernelCall:
  kernel_id: q4_gemv_avx2_g64
  input_binding: arena.hidden_norm
  packed_weight: packed.layers.12.mlp.wup
  output_binding: arena.mlp_up
  tile_n: 16
  tile_k: 64
  threads: partition_by_output_rows
  tolerance_profile: q4_weight_f16_output
```

这里和第一册、第二册的连接很直接。第一册帮助我们理解 ABI、函数调用、指针和反汇编；第二册帮助我们理解 AVX、cache、TLB、线程和 NUMA；第三册把它们组织成 lowering 计划。

GPU lowering: 把同步和 staging 写进计划

GPU lowering 不应该只输出一个 `launch(kernel)`。它还要表达 device buffer、stream、workspace、host-device staging 和同步点。特别是 logits 通常需要回到 CPU 采样，KV cache 可能常驻 GPU，某些 fallback 可能回到 CPU，这些边界都要在计划里显式出现。

Listing 4.44 GPU lowering 输出的计划片段

```
GpuSegment:
  stream: decode_stream
  device_inputs:
    hidden_buffer
    packed_weight_buffer
    kv_cache_buffer
  workspace: attention_workspace
  kernels:
    rmsnorm_linear_fused
    rope_kv_append
    attention_decode
    mlp_fused
  staging:
    copy logits top-k to host
  sync:
```


wait before sampler

如果计划不记录 staging 和 sync, profiler 就会只看到 kernel 时间, 看不到端到端 decode latency。第二册讲过异步 I/O、背压和运行时调度; GPU 后端里的 stream 和 host-device 同步就是同类问题在 AI infra 中的具体形态。

Executable Plan: runtime 执行的是计划, 不是重新解释图

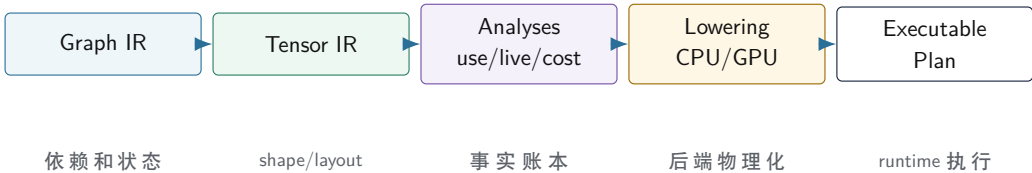
executable plan 是 AI 编译器交给 runtime 的产物。它应该把已经决定的事情固化下来: segment 顺序、kernel 选择、arena offset、packed weight、KV cache 绑定点、workspace 需求、profiler 标签和 oracle 边界。

Listing 4.45 ExecutablePlan 的核心结构

```
ExecutablePlan:
  plan_id
  stage: prefill | decode
  backend
  assumptions:
    max_tokens
    context_bucket
    dtype_profile
    quant_profile
  memory_plan:
    arena_size
    value_offsets
    workspace_requirements
  segments:
    Segment[]
  oracle_points:
    layer_output
    logits
  profiler_schema
```

runtime 拿到 plan 后, 不再做 shape inference, 不再做 fusion, 不再选择 tile, 不再 packing。它只绑定 session 状态, 例如当前 token、KV cache position、arena 基址、backend context, 然后执行 segment。这样热路径才不会退回解释器形态。

IR 到 executable plan 的编译流水线



图示: 本图要证明的命题是: AI 编译器的最终产物不是一个抽象图, 而是 runtime 可以直接绑定和执行的计划。

Verifier 要贯穿每一层

前端有 verifier, IR pass 之后也要有 verifier, lowering 后还要有 verifier。不同层检查不同不变量:

- Graph verifier: 节点输入输出存在, 状态边合法, 没有非法循环依赖。

- Tensor verifier: shape、stride、dtype、layout、quant descriptor 一致。
- Pass verifier: 改写前后输出语义和边界没有被破坏。
- Memory verifier: arena offset 不错误重叠, alignment 满足后端要求。
- Backend verifier: kernel 支持 dtype/layout/tail/quant, packed descriptor 完整。
- Plan verifier: segment 顺序、state binding、workspace 和 sync 边界完整。

这些 verifier 不一定都很复杂, 但必须存在。它们让错误在编译期或 prepare 阶段暴露, 而不是进入运行时热路径后才以崩溃或质量漂移形式出现。

C++20 实现边界

工程上, AI 编译器模块可以按下面方式组织:

Listing 4.46 AI 编译器模块组织建议

```
include/ai/graph/
graph_ir.hpp
tensor_desc.hpp
verifier.hpp

include/ai/compile/
pass.hpp
pass_pipeline.hpp
analyses.hpp
memory_plan.hpp
lowering.hpp
executable_plan.hpp

src/compile/analysis/
use_def_analysis.cpp
liveness_analysis.cpp
cost_analysis.cpp

src/compile/passes/
fuse_rmsnorm_linear.cpp
eliminate_redundant_reshape.cpp
quant_lowering.cpp

src/compile/lowering/
cpu_lowering.cpp
gpu_lowering.cpp
```

接口上要保持几条纪律。pass 接收已验证 IR, 返回改写结果和诊断; analysis 产出不可变事实, 不偷偷改图; lowering 消费 descriptor 和 analysis, 不重新解析模型文件; executable plan 是 runtime 的稳定输入, 不暴露临时编译器内部指针。C++20 中可以用 `std::span` 表示节点序列和 value 序列, 用 RAIi 管理 arena 和 backend 资源, 用显式 enum 和小值对象表达 opcode、dtype、layout 和 backend capability。

本节验收线

读完本节, 应该能真正理解 AI 编译器的内部结构:

- IR 是可检查、可改写的中间账本, 不是为了抽象而抽象。
- Graph IR 表达算子依赖和 KV cache 等状态边。
- Tensor IR 表达 shape、stride、layout、dtype、device 和 quant descriptor。
- Loop IR 或 LoopPlanDesc 表达 tile、vector、tail、thread 和 packed weight。

- pass 必须声明前置条件、后置条件、依赖分析和失效分析。
- lowering 要分阶段从语义计划走到 CPU/GPU 后端物理计划。
- executable plan 固化 segment、kernel、arena、workspace、state binding、oracle 和 profiler。
- verifier 要贯穿 Graph、Tensor、Pass、Memory、Backend 和 Plan 每一层。

这就是把编译原理落到 AI infra 的方式：不是背名词，而是用 IR 和 pass 把模型语义、量化合同、内存生命周期和后端机器事实串成一条可验证的工程流水线。

第 5 章

C++20 CPU/GPU 后端优化

5.1 C++20 后端优化总路线：从 reference 到可验收的 CPU/GPU plan

后端优化不能从“写快代码”开始

很多人第一次写推理后端，会直接跳到 SIMD intrinsic、线程池或者 GPU kernel。这样很容易得到一段看起来很专业、却无法证明正确和无法稳定复用的代码。真正的后端优化要先回答四个问题：

1. 正确答案由谁定义。
2. 当前 baseline 慢在哪里。
3. 本次优化改变了哪条地址流、生命周期、指令流或同步边界。
4. oracle 和 profiler 是否同时证明“没算错”和“真的快了”。

这和编译器后端的纪律一样。后端不是随便生成几条目标指令，而是把已经验证的 IR 降到目标机器，并保留语义。AI 推理后端也不是随便调用 kernel，而是把 tensor descriptor、packed weight、memory plan、KV cache 和 runtime segment 落到 CPU 或 GPU 的物理执行。

核心思想

后端优化的起点是 reference 和证据，不是 SIMD 或 GPU。没有 reference，就不知道优化后是否正确；没有 profiler，就不知道优化是否击中了真正瓶颈；没有清晰 descriptor，就无法把优化接回编译器和运行时。

C++20 后端的基本模块

一个可维护的 C++20 后端不应该只有 `kernel.cpp`。至少要把下面几类职责分开：

Listing 5.1 C++20 后端模块草图

```
backend/cpu/
cpu_backend.hpp           // backend facade, capability query
tensor_view.hpp           // non-owning typed/byte views
aligned_buffer.hpp        // RAII aligned allocation
packed_weight.hpp         // packed descriptors and cache
matmul/
  reference_kernel.hpp
  scalar_kernel.hpp
  simd_kernel.hpp
  packing.hpp
schedule/
  thread_pool.hpp
  work_partition.hpp
profile/
  cpu_events.hpp

backend/gpu/
gpu_backend.hpp
device_buffer.hpp
stream.hpp
kernel_adapter.hpp
```

```
workspace.hpp
staging.hpp
```

这些文件名只是示意，真正重要的是边界。C++20 的 RAII 对象负责释放资源，`std::span` 或类似轻量视图负责观察连续内存，descriptor 负责描述 shape、stride、dtype、layout 和 quantization，backend facade 负责能力查询和 plan 创建，kernel 只接收已经验证过的输入。

不要让 kernel 解析模型名。 kernel 应该知道输入指针、shape、stride、scale 和 packed descriptor，不应该知道 `layers.12.attn.wq.weight` 这种模型方言。模型名解析属于前端，后端只消费 canonical descriptor。

不要让 runtime 猜 SIMD 宽度。 runtime 选择 executable plan，但不应该在每个 token 上重新判断 AVX2、AVX-512 或 NEON 的细节。CPU backend 初始化时查询能力，编译/prepare 阶段生成候选计划，runtime 只做便宜选择。

不要让 packed weight 失去来源。 packed buffer 是物理布局，但诊断仍要能回到原始 tensor name、文件 offset、shape 和 checksum。否则一旦输出异常，只能在一坨重排后的字节中调试。

CPU 优化阶梯：每一级改变一个机器事实

CPU 后端可以按阶梯推进，每一级都应该能单独 benchmark 和回滚。

阶段	改变的机器事实	验收证据
reference	定义语义，不追求速度	oracle 作为比较基准
scalar baseline	连续循环和清楚地址公式	结果正确，profiler 有基线
layout 修正	stride、对齐、尾块、连续性	cache miss 和边界路径可解释
packing	把权重重排成 kernel 友好流	packing 成本与热路径收益分开统计
SIMD	向量 load、FMA/dot、累加器展开	指令形态和误差边界可验证
线程划分	work partition、barrier、false sharing	单线程到多线程 scaling 曲线
NUMA/TLB	内存放置、页、亲和、预取	长 prompt 和多请求下尾延迟稳定

这个阶梯的关键是一次只改变一个主要变量。若同一版同时改 packing、SIMD、线程池和 arena，很难判断速度来自哪里，也很难定位错误。教材后续写具体 kernel 时，每个优化都要说明“输入 descriptor 没变，输出语义没变，只改变了某条物理地址流或执行划分”。

SIMD micro-kernel 要写清楚的合作

SIMD 不是把标量循环换成 intrinsic 就结束。一个合格的 micro-kernel 至少要写清楚：

- 支持的 dtype: fp32、fp16/bf16、int8、int4 解包后计算，还是整数 dot。
- 输入 layout: 行主序、列主序、packed panel、group scale 相对位置。
- 向量宽度: AVX2、AVX-512、NEON 或运行时选择的抽象宽度。
- 累加器数量: 同时维护几个输出元素，是否避免 dependency chain。
- 尾部路径: `K`、`N`、`head_dim` 不能整除向量宽度时怎样处理。
- 数值合同: 累加类型、舍入、requantize、允许误差和 reference 比较方式。
- 内存合同: 输入/输出是否要求对齐，是否允许非连续 stride，是否完整覆盖输出。

Listing 5.2 micro-kernel descriptor 需要表达的信息

```
MatmulKernelDesc:
  isa: avx2 | avx512 | neon | scalar
  input_dtype
  weight_format
  accumulator_dtype
```

```

output_dtype
tile_m, tile_n, tile_k
alignment_bytes
tail_policy
quant_group_size
requires_packed_weight

```

descriptor 的价值是把 kernel 能做什么说清楚。编译器 lowering 可以根据它选择 kernel; runtime profiler 可以把耗时归到具体 kernel; oracle 可以知道该用哪个误差阈值; 测试可以枚举 tail、对齐和 dtype 组合。

线程调度：小 kernel 不一定适合抢满所有核心

CPU 推理的线程调度不能只写一句“并行 for”。prefill 和 decode 的形态不同，线程划分也不同。prefill 中 token 维度较大，GEMM/attention 更容易让多个核心保持忙碌；decode 中一次通常只有一个新 token，小 kernel 很多，过度并行会让 barrier、任务投递和 cache 竞争吞掉收益。

调度器至少要考虑：

- 按输出行、输出列、token、head、layer 还是 batch 切分。
- 每个 task 的最小工作量，太小的 task 不值得进入线程池。
- workspace 和 output buffer 是否会 false sharing。
- 多层连续 segment 是否能减少 barrier 次数。
- 线程亲和和 NUMA 放置是否与权重内存一致。
- profiler 是否能分别记录 compute time、wait time 和 barrier time。

一个实用规则是：先让单线程 kernel 正确且可解释，再做粗粒度并行，最后才优化小任务调度。否则多线程版本慢了，很难判断是 kernel 慢、切分错、同步重，还是内存带宽已经满了。

GPU 后端：共享语义合同，不共享 CPU 的物理假设

GPU 后端和 CPU 后端应该共享 graph/tensor/quant/KV 的语义合同，但不共享所有物理布局。CPU 关心 cache line、SIMD 寄存器和线程池；GPU 关心显存连续性、coalesced load、shared memory、kernel launch、stream、同步和 host-device staging。

GPU 后端至少要有这些 C++20 RAII 边界：

- **DeviceBuffer**：管理显存分配和释放，禁止裸指针泄漏生命周期。
- **Stream**：管理异步执行队列和事件同步。
- **Workspace**：为某些 kernel 或 vendor library 调用提供临时区。
- **KernelAdapter**：把 executable plan 的 descriptor 绑定到具体 kernel launch。
- **StagingBuffer**：处理 logits、token id 或小 metadata 的 host-device 传输。

GPU 优化要特别小心 decode 阶段。大 prefill 可能容易吃满 GPU，但 decode 单 token 可能被 launch overhead、同步、KV cache 读流或 logits 回传限制。若 profiler 只报告 GPU kernel 时间，不报告端到端 step latency，就会误判。

后端优化的验收报告

每次后端优化都应该产出一份小报告。它不一定长，但必须包含能复现和能判断的信息：

Listing 5.3 后端优化报告字段

```

optimization:
  name
  changed_contract
  affected_stage: prefill | decode | both
  backend: cpu | gpu

correctness:

```



```

reference
oracle_metric
tolerance
tested_shapes
tested_dtypes

performance:
  baseline_commit_or_build
  hardware
  threads_or_device
  prompt_lengths
  context_lengths
  latency
  tokens_per_second
  memory_peak
  profiler_breakdown

```

这份报告就是工程质量的底线。没有 tested shapes, 尾块可能没测; 没有 tested dtypes, 量化路径可能没测; 没有 context length, KV cache 长上下文可能没测; 没有 profiler breakdown, tokens/s 变快也不知道是哪部分变快。

一次真实优化应该怎样设计实验

以“把 decode 线性层从 fp16 权重改成 int4 weight-only”为例, 不要直接改完跑一个 prompt。正确实验要先拆变量:

1. 固定模型切片、prompt set、sampler seed、线程数或 GPU。
2. 先验证 converter: 原始权重坐标反量化与 reference 一致。
3. 再验证单算子: 同一输入下 fp16 reference 与 int4 kernel 输出误差。
4. 再验证层输出: 一个 decoder block 的 hidden drift。
5. 再验证 logits: top-k 是否变化, 最大误差和分位数是多少。
6. 最后 benchmark: packing time、decode step latency、tokens/s、内存峰值。

Listing 5.4 单次优化实验记录

```

Experiment:
  change: q4_weight_only_decode_linear
  fixed:
    model_fixture
    prompt_set
    sampler_seed
    backend
    thread_count
  correctness:
    byte_oracle_passed
    operator_error
    layer_error
    logits_topk_change
  performance:
    packing_time_ms
    decode_step_p50_p95
    tokens_per_second
    memory_peak_bytes
    bandwidth_estimate

```

这样实验失败时能定位。byte oracle 失败，查 nibble、scale、tail；operator 失败，查 kernel 地址流和累加；layer 失败，查 fusion、arena 或 residual；logits 失败，查误差是否被 softmax 放大；performance 没提升，查瓶颈是否根本不在权重带宽。

A/B 路径和回滚条件

后端优化必须可回滚。项目里应同时保留 reference、baseline 和 optimized plan，并让 runtime 能按配置选择。不是为了用户长期使用慢路径，而是为了调试和线上回滚。

Listing 5.5 后端 plan 选择策略

```
BackendPlanSet:
  reference_plan
  baseline_plan
  optimized_plan
  debug_oracle_plan

Selection:
  if oracle_enabled:
    use debug_oracle_plan
  else if optimized_supported and gate_passed:
    use optimized_plan
  else:
    use baseline_plan
```

回滚条件要提前定义。例如：

- logits top-k 变化率超过阈值；
- 固定 sampler 的 token mismatch 超过阈值；
- decode p95 latency 回归超过阈值；
- memory peak 超过容量预算；
- fallback 次数超过阈值；
- 某类 shape 或 context bucket 失败。

这不是保守，而是让优化可维护。没有 A/B 和回滚条件，项目会变成“某次优化之后好像快了，但后来没人敢动”。

CPU perf counters 应该服务假设

第二册讲过 cache、TLB、分支和指令吞吐。第三册用它们时要围绕假设，而不是堆计数器。比如你认为 int4 优化更快是因为少读权重，那么应观察：

- LLC miss 或内存带宽是否下降；
- 指令数是否因为解包显著上升；
- cycles per token 是否下降；
- SIMD 指令比例是否符合预期；
- TLB miss 是否因 packed layout 改善或恶化。

Listing 5.6 CPU 后端性能假设记录

```
Hypothesis:
  q4 reduces weight bandwidth in decode GEMV

Counters:
  cycles
```

```

instructions
cache_misses
dTLB_misses
memory_bandwidth_estimate
vector_instruction_count

```

Decision:

```

if bytes_down but instructions_up too much:
    inspect unpack and scale path
if cache_misses unchanged:
    bottleneck may not be weight stream

```

GPU 也同理。认为融合减少 launch，就要报告 launch count 和 total step latency；认为 KV layout 改善 coalescing，就要报告 memory throughput、load efficiency 或同类指标；认为 top-k on device 减少同步，就要报告 staging bytes 和 sampler wait。

优化完成的定义

一个后端优化完成，不是代码合并，也不是单个 benchmark 变快。最小完成定义是：

验收与评分标准

- descriptor 说明了新路径支持的 dtype、layout、shape、tail 和 quant profile。
- verifier 能拒绝不支持输入。
- reference/oracle 覆盖字节、单算子、层或 logits 中的相关层级。
- profiler 能拆分本次优化影响的时间和字节。
- benchmark 覆盖至少一个整齐形状和一个边界形状。
- 文档记录适用场景、收益、风险和回滚条件。

这套定义直接回答“学完能做成什么”。照这个标准推进，读者能做出一个有限但严肃的推理后端：支持明确模型范围、明确量化 profile、明确 CPU/GPU 能力、明确正确性阈值和性能报告。它未必马上追上成熟框架，但每个差距都能被定位。

本节验收线

读完本节，应该建立后端优化的基本纪律：

- C++20 后端要分清 descriptor、RAII buffer、packed weight、kernel、scheduler、profiler 和 oracle。
- CPU 优化按 reference、scalar baseline、layout、packing、SIMD、线程、NUMA/TLB 的阶梯推进。
- SIMD micro-kernel 必须声明 dtype、layout、tile、tail、alignment 和数值误差合同。
- 线程调度要根据 prefill/decode 形态选择粒度，不能盲目填满线程池。
- GPU 后端共享语义合同，但用自己的 device buffer、stream、workspace 和 staging 边界。
- 每个优化都必须同时有 correctness report 和 performance report。

后续扩写具体 kernel 时，本节就是验收模板：没有合同、没有 oracle、没有 profiler、没有可复现 benchmark 的优化，都不能算完成。

5.2 CPU 后端优化细讲：地址流、SIMD、线程、NUMA 与 C++20 工程边界

后端优化的第一原则：先解释慢在哪里

CPU 后端优化很容易被写成一串术语：AVX、packing、thread pool、NUMA、prefetch。对初学者来说，这些词如果没有机器账本支撑，只会变成新的黑箱。真正的优化要先回答一个朴素问题：这段推理为什么慢？

对 7B 级 decoder-only 模型，慢可能来自不同位置。prefill 阶段可能慢在大 GEMM、attention

和 activation 峰值；decode 阶段可能慢在 GEMV、KV cache 读流、小 kernel 调度和线程 barrier；量化路径可能慢在 int4 解包、scale 读取和 requantize；多线程可能慢在 false sharing、内存带宽和 NUMA 远端访问。优化不能一上来就写 intrinsic，而要先用 profiler 拆开这些来源。

核心理想

CPU 后端优化不是“把代码写得像汇编”，而是建立地址流、指令流、内存层级、线程调度和数值合同之间的证据链。每个优化都要能说明它减少了哪种机器成本。

本节把 CPU 后端细化到实现层。它会反复连接第一册和第二册：第一册提供 ABI、对象布局、对齐、汇编和 benchmark 身份；第二册提供 cache、TLB、SIMD、线程、NUMA 和运行时调度；第三册把这些基础应用到张量、量化、KV cache 和 executable plan。

C++20 后端对象：所有权和视图必须分开

后端代码最先要避免的是裸指针混乱。权重、packed weight、activation、KV cache 和 workspace 生命周期不同。C++20 中应该把所有权对象和非拥有视图区分开。

Listing 5.7 后端对象的所有权边界

```
Owned objects:
AlignedBuffer      owns CPU aligned memory
MappedFile         owns file mapping
PackedWeight       owns backend-specific packed bytes
ThreadPool         owns worker threads
ProfilerTrace      owns event storage

Non-owning views:
TensorView         span + shape + stride + dtype
ByteSpan           std::span<std::byte>
KernelArgs         views bound for one call
```

这条边界可以避免两个常见错误。第一，kernel 把某个临时指针缓存到对象里，arena 下一次复用地址后读到错误数据。第二，某个 descriptor 以为自己拥有 scale buffer，实际 scale buffer 已经随加载临时对象释放。所有权对象用 RAII 管理生命周期；kernel 只拿当次调用有效的 view。

常见误区

高性能代码也必须有清楚所有权。越是 SIMD、线程和异步后端，越不能依赖“这个指针应该还活着”的隐含约定。

TensorView：地址公式要显式

一个 tensor view 至少要包含元素类型、shape、stride 和数据指针。连续内存只是特殊情况。后端 kernel 可以要求某些输入连续，但这个要求必须写在 kernel descriptor 中，由 verifier 检查。

Listing 5.8 TensorView 的语义字段

```
TensorView:
data
dtype
shape
stride
layout
device
alignment
```

地址公式要能从 view 推导出来：

Listing 5.9 二维 view 的地址公式

```
address(i, j) = base + (i * stride0 + j * stride1) * sizeof(element)
```

为什么这很重要？因为很多优化只对连续内存成立。若 `stride1=1`，内层循环可以顺序 load；若 `stride1` 很大，SIMD load 可能变成 gather 或标量读；若输出 stride 不连续，store 也会变慢。第一册讲过指针和对象布局；在张量后端中，这些知识直接决定 kernel 合同。

GEMV 和 GEMM: prefill 与 decode 的不同账本

prefill 通常处理多个 prompt token，线性层形态接近 GEMM：

Listing 5.10 prefill linear 的抽象形态

```
X[T, H] * W[0, H]^T -> Y[T, 0]
```

decode 通常一次处理一个新 token，形态更接近 GEMV：

Listing 5.11 decode linear 的抽象形态

```
x[H] * W[0, H]^T -> y[0]
```

这两个形态的优化重点不同。GEMM 有更多数据复用机会，tile 可以同时复用 X 和 W；GEMV 中输入向量 `x` 较小，可以驻留 cache，但权重每步都要读很多，常常受权重带宽和 packed layout 影响。一个只为 GEMM 优化的 kernel，不一定适合 decode；一个只为 GEMV 优化的 kernel，也不一定吃满 prefill。

阶段	主要形态	优化重点
prefill	GEMM / batched GEMM	tile 复用、cache blocking、线程吞吐
decode	GEMV / small GEMM	权重读流、packing、低调度开销、KV cache
MLP	大线性层组合	gate/up/down 的融合边界和 activation 流
attention	softmax + KV 读取	数值稳定、历史读流、context 分档

所以 executable plan 应该区分 prefill plan 和 decode plan。后端优化报告也应该分别给出 prefill tokens/s、time to first token、decode tokens/s 和单 token latency。

packing: 让热循环读到它想读的顺序

模型文件布局通常服务于通用存储，不服务于某个 SIMD micro-kernel。packing 的目标是把权重重排成热循环顺序读取的布局。对 decode GEMV，常见目标是让一个 output tile 的若干行权重按 `K` 方向连续读取，便于向量 load 和累加多个输出。

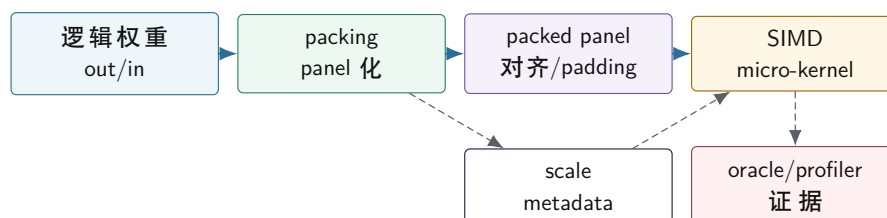
Listing 5.12 packing 前后的思维差异

```
logical:
  weight[out, in]

packed for kernel:
  panel[out_tile, in_tile, lane]
  scales[out_tile, group]
  tail padding for vector width
```

packing 的收益来自减少热路径地址计算、增加顺序读、避免跨 cache line 的随机访问，并让 scale metadata 与权重 code 靠近。但 packing 也有成本：准备时间、额外内存、可能的 TLB 压力、诊断复杂度。因此报告要区分 `pack_time_ms`、`packed_bytes` 和热路径收益。

CPU packed weight 与 micro-kernel 的关系



图示：本图要证明的命题是：packing 是 kernel 合同的一部分，权重 panel、scale metadata 和 profiler/oracle 必须一起设计。

SIMD micro-kernel：先设计寄存器账本

SIMD 优化不是把循环改成 intrinsic。要先设计寄存器账本：哪些寄存器放输入向量，哪些放权重，哪些放 accumulator，scale 在哪里乘，尾部怎样处理。以 fp32 GEMV 为例，一个 micro-kernel 可能同时累加 4 个输出通道：

Listing 5.13 GEMV micro-kernel 的寄存器账本

```

acc0, acc1, acc2, acc3 = 0
for k in 0..H step vector_width:
    x = load input[k]
    w0 = load weight[out0, k]
    w1 = load weight[out1, k]
    w2 = load weight[out2, k]
    w3 = load weight[out3, k]
    acc0 += x * w0
    acc1 += x * w1
    acc2 += x * w2
    acc3 += x * w3
horizontal_reduce acc0..acc3
  
```

对 int8 或 int4，账本更复杂。int8 可能使用 dot-product 指令或扩展到更宽类型累加；int4 要先解包 nibble，做符号扩展或 zero point 处理，再乘 scale 或进入整数累加。若 scale 每 64 个输入共享一次，micro-kernel 要在 group 边界加载 scale，并保证 packed layout 与 group 边界一致。

路径	热点	主要风险
fp32/fp16 风格	load、FMA、reduce	accumulator 依赖链、tail requantize、饱和、scale
int8 dot	dot 指令、int32 累加	解包开销、metadata 读流
int4 dequant	nibble 解包、scale 乘法	context 增长、数值稳定
KV attention	历史 K/V 读取、softmax	

micro-kernel descriptor 必须声明这些合同：ISA、tile、alignment、tail、accumulator dtype、输出 dtype、quant group size 和误差容忍。否则 lowering、verifier、oracle 和 benchmark 都无法正确选择和解释它。

int8 linear 的算术合同

int8 权重量化的 CPU 内核不能只写“把 int8 乘回 float”。至少有两条路线。第一条是教学和 reference 友好的路线：输入仍是 fp32/fp16，权重 int8 反量化成浮点参与 FMA。第二条是性能路

线：输入也量化或分块量化，使用整数 dot 指令累加到 int32，再按 scale 还原到目标 dtype。

Listing 5.14 float input + int8 weight 的合同

```
real_w[out, k] = int8_w[out, k] * scale[out or group]
acc_float += input_float[k] * real_w[out, k]
```

这条路线容易验证，但每个权重都要转换或乘 scale。若 scale 是 per-channel，scale 读取较轻；若 scale 是 per-group，内层每到 group 边界要换 scale。若 scale 与 packed weight 不同布局，cache locality 会变差。

整数 dot 路线可以写成：

Listing 5.15 int8 dot + int32 accumulator 的合同

```
q_x = quantize(input_float, input_scale)
q_w = int8_w
acc_i32 += int32(q_x) * int32(q_w)
real_acc = acc_i32 * input_scale * weight_scale
output = cast_or_requantize(real_acc)
```

这条路线的难点更多：输入 scale 从哪里来，activation 是否逐 token 量化，zero point 是否为 0，accumulator 是否溢出，输出是 fp16/fp32 还是再量化。它可能更快，但必须有更强的 verifier 和 oracle。第三册的工程路线应该先让 float-input/int8-weight 路径稳定，再引入 activation quantization 和整数 dot。

常见误区

不要把 int8 weight-only 和 int8 activation 混成一个词。前者主要节省权重容量和读带宽；后者改变输入表示、accumulator 合同和 requantize 路径，正确性风险更高。

用硬件计数器判断问题类型

CPU 优化不能只看 wall time。至少要把慢分成几类：前端指令供应不足、后端执行端口饱和、cache miss、TLB miss、分支或同步。不同机器可用事件不同，但思路一致。

现象	可能原因	下一步
IPC 低且 cache miss 高	权重/KV 读流不连续	查 layout、packing、page 大小
IPC 低但 miss 不高	依赖链、指令混乱、解包开销	看反汇编和 uops
多线程不扩展	带宽饱和、barrier、false sharing	改分区和线程数
长上下文尾延迟高	KV page 分散、TLB miss	查 paged KV 和大页策略
int4 不如 int8 快	解包和 scale 读取压过带宽收益	重排 scale 或改 kernel

benchmark 报告里最好同时写“性能数字”和“解释数字”。例如：[decodetokens/s](#) 提升 18%，同时 LLC miss 降低、pack time 增加多少、logits top-k 是否变化。只有这样，优化才不是偶然跑快一次。

tail path: 不要把边界条件当小事

真实模型 shape 不总是整齐满足 vector width 或 tile size。即使 hidden size 常常是 4096，intermediate size、vocab size、head 数、group size、batch 和 prompt 长度也可能产生尾部。tail path 写错会导致越界读、漏算或错误 padding。

Listing 5.16 tail path 需要覆盖的情况

```
K dimension not divisible by vector width
```

```
output channels not divisible by tile_n
group size does not divide logical tail
prompt token count not divisible by tile_m
vocab size not divisible by sampler block
```

优化代码常见坏味道是：主路径很快，tail path 用一堆临时判断散在热循环里。更好的做法是把 tail policy 写进 descriptor，并在 lowering 阶段选择专门主路径和尾部路径。测试必须覆盖 tail shape，不能只测漂亮的 4096x4096。

线程划分：decode 不一定适合细粒度并行

CPU 线程优化要先看任务粒度。prefill 的 GEMM 和 attention 块通常有足够工作量，可以按 token、output channel、head 或 tile 划分。decode 的单 token 路径包含许多小 kernel，若每个 kernel 都投递一批小任务，barrier 和线程唤醒可能抵消计算收益。

Listing 5.17 线程调度要记录的字段

```
ThreadPlan:
  stage: prefill | decode
  partition_axis: token | output_channel | head | layer | batch
  min_work_per_task
  barrier_count
  scratch_per_thread
  output_write_policy
  affinity_policy
```

false sharing 是一个具体风险。若多个线程写相邻很小的输出或 profiler counter，它们可能反复抢同一 cache line。第二册讲过 cache coherence；在推理后端里，它会表现为多线程不升反降。输出分区、per-thread scratch、counter padding 和归约策略都要考虑 cache line。

常见误区

不要用“CPU 利用率接近 100%”证明推理快。线程都忙可能是在抢 cache line、等内存带宽或反复 barrier。真正指标是端到端 latency、tokens/s、硬件计数器和可解释 scaling 曲线。

NUMA、TLB 与大页：长上下文会放大内存事实

当模型权重、packed weight 和 KV cache 变大，NUMA 和 TLB 开始影响尾延迟。若线程跑在一个 NUMA node，而权重页主要分配在另一个 node，远端访问会拖慢。若 KV cache 分散在很多小页，长上下文 attention 会放大 TLB miss。若多请求同时增长 KV cache，内存分配策略还会影响碎片和局部性。

后端报告至少要记录：

- CPU 型号、核心数、NUMA node、线程亲和策略。
- 权重和 KV cache 的分配策略，是否 first-touch。
- 是否使用大页或特定页大小。
- 长 prompt 和长 decode 下的 tail latency。
- cache miss、TLB miss、远端访问等可用硬件计数器。

这部分不能凭空优化。先测，再做策略。比如固定线程亲和可能改善稳定性，也可能降低操作系统调度弹性；大页可能减少 TLB miss，也可能增加分配失败或内存浪费。教材强调的是把这些选择纳入报告，而不是给出一个永远正确的开关。

KV cache CPU 路径：读流比写入更关键

KV cache 每生成一个 token 写一次当前 K/V，但 decode attention 每步要读取历史 K/V。上下文越长，读流越大。CPU 后端要围绕读取模式设计 layout。

Listing 5.18 decode attention 的读流

```

for head in heads:
    for pos in 0..context_length:
        k = load K[layer, head, pos, dim]
        score[pos] = dot(q[head], k)
    probs = softmax(score)
    for pos in 0..context_length:
        v = load V[layer, head, pos, dim]
        out[head] += probs[pos] * v

```

这里至少有三类优化。第一，layout 让 **pos, dim** 读取尽量连续，减少跨页和跨 cache line。第二，按 context length 分档，小上下文用简单路径，长上下文用分块路径。第三，若 KV cache 量化，需要把 dequant 和 attention 结合，但不能让 scale metadata 破坏读流。

数值上，softmax 需要稳定实现，通常要先减最大值。优化 attention 时不能只看速度，还要比较 logits 和生成质量。一个更快但数值不稳定的 attention kernel，会在长上下文下放大误差。

benchmark harness: 测量要能复现

后端优化最终要落到 benchmark。一个合格 harness 至少固定这些变量：

Listing 5.19 CPU 后端 benchmark 输入

```

model_or_fixture_id
quantization_profile
backend_plan_id
cpu_feature_path
thread_count
affinity_policy
prompt_set_id
prompt_lengths
decode_lengths
context_length_buckets
warmup_iterations
measurement_iterations

```

输出要同时包含性能和正确性：

Listing 5.20 CPU 后端 benchmark 输出

```

load_time_ms
pack_time_ms
prefill_tokens_per_s
time_to_first_token_ms
decode_tokens_per_s
p50_decode_latency_ms
p95_decode_latency_ms
arena_peak_bytes
kv_cache_bytes
max_logit_abs_error
topk_changed_count
hardware_counters_optional

```

测量本身也要避免常见误区：没有 warmup、把模型加载混入 decode、把 packing 成本藏进首

次 token、只测一个短 prompt、没有固定线程数、没有记录 CPU 频率状态、只报最好一次。第一册已经讲过可信测量；这里要把同样纪律应用到 AI 后端。

自研后端怎样对标成熟框架

用户要求“每一步每一个优化都做好”，这不等于一开始就超过成熟框架。正确路线是分阶段对标：

阶段	本项目目标	对标重点
reference	正确、可解释	logits 与参考一致
baseline	能端到端跑固定模型片段	找到最大瓶颈差距
packing/SIMD	单算子接近合理吞吐	microbenchmark 和反汇编
线程化	多核扩展稳定	scaling 曲线和尾延迟
量化	容量和速度同时改善	质量报告和 tokens/s
runtime plan	端到端稳定	prefill/decode 分项差距

对标报告应该同时写“我们还差在哪里”。例如：prefill 接近成熟框架 70%，decode 只有 40%，原因可能是 KV cache attention 读流没有优化；或者 int4 权重容量下降明显，但 scale metadata 布局导致解包路径慢。这样的报告比一个孤立 tokens/s 数字更有工程价值。

优化判定表：看到现象后查什么

后端优化最需要的是排查顺序。下面这张表把常见现象、优先假设和验证动作连起来。

现象	优先假设	验证动作
decode batch 1 慢	权重读流或小 kernel 调度	算 weight bytes、看 pack layout、拆 kernel count
长上下文变慢明显	KV cache 读流/TLB/page table	按 context bucket 画 latency，查 TLB/cache
int4 不如 int8	解包和 scale metadata 太重	分开测 unpack、scale load、FMA 时间
多线程不扩展	内存带宽、barrier、false sharing	画线程数曲线，记录 barrier wait
prefill 快但 TTFT 慢	tokenizer、prepare、packing 或 queue wait	分拆 load、prepare、prefill、sampler
GPU kernel 快但端到端慢	launch、copy、sync 或 sampler	看 StepTrace，不只看 kernel profiler

这张表的用法很直接：不要先改代码，先证明假设。若 decode 慢但 weight bytes 下限已经接近实测，继续优化 FMA 指令可能收益有限；若 int4 慢在 scale 读取，换更激进的位宽没有意义；若 p95 主要是 queue wait，kernel micro-optimization 不会解决业务 SLO。

本节验收线

读完本节，应该能把 CPU 后端优化拆成可实现、可验证的步骤：

- 后端对象要分清 RAII 所有权和 `std::span`/TensorView 非拥有视图。
- TensorView 的 shape、stride、layout 和 alignment 决定地址公式和 kernel 合同。
- prefill 更像 GEMM，decode 更像 GEMV，两者应使用不同 plan 和指标。
- packing 要同时组织权重 panel、scale metadata、padding、对齐和诊断映射。
- SIMD micro-kernel 要先设计寄存器账本，再声明 dtype、tile、tail、accumulator 和误差合同。
- tail path 是必须测试的正确性边界，不是附属小分支。
- 线程、NUMA、TLB 和 KV cache 读流必须用 profiler 和硬件事实解释。
- benchmark harness 要固定模型、量化、线程、prompt、context、warmup 和正确性指标。

这章的核心结论很简单：高质量 AI infra 后端不是某段神秘快代码，而是一套从 C++20 对象生命周期到机器地址流、从 SIMD micro-kernel 到 runtime plan、从 oracle 到 profiler 的完整证据系统。

5.3 GPU 硬件基础：SM、warp、显存层级、tensor core 与推理瓶颈

为什么第三册必须讲 GPU 硬件

只讲 GPU API 和 stream 还不够。推理引擎的 GPU 后端最终跑在硬件上：线程如何成组执行，访存如何合并，shared memory 为什么快，tensor core 适合什么形态，显存带宽和 PCIe 传输怎样限制 decode，这些都会决定 AI 编译器 lowering 的选择。如果不了解这些硬件事实，GPU 后端很容易停在“能 launch kernel”的层面。

可以先把 GPU 和第二册里的 CPU 对比。CPU 少量强核心，靠大 cache、乱序执行、分支预测和 SIMD 处理复杂控制流；GPU 大量较简单执行单元，靠海量线程隐藏内存延迟，适合规则、密集、数据并行的工作。两者都要关心内存层级和并行，但并行粒度和瓶颈不同。

核心思想

GPU 快不快，取决于工作是否能匹配它的执行模型：大量线程、规则访存、高算术强度、少同步、少 host-device 往返。prefill 往往更匹配，decode 单 token 往往更难匹配。

SM/CU：GPU 的基本执行岛

不同厂商术语不同，NVIDIA 常说 SM，AMD 常说 CU。本册用 SM/CU 表示 GPU 上一组局部执行资源：它包含调度器、寄存器文件、若干向量/标量执行单元、shared memory 或 local data share、load/store 单元，以及可能的 tensor core 或 matrix core。

一个 kernel launch 后，工作会被切成许多 thread block 或 workgroup。每个 block 被调度到某个 SM/CU 上执行。一个 block 内的线程共享一块 shared memory，并能做 block 内同步；不同 block 通常不能直接同步，只能通过全局内存和 kernel 边界间接协作。

Listing 5.21 GPU 执行层级的简化账本

```
kernel launch
  grid: many blocks
    block/workgroup
      warps/wavefronts
        lanes/threads
          registers
            shared memory
              global memory
```

这解释了为什么 GPU lowering 要选择 block 形状。一个 attention kernel 可以让一个 block 处理一个 head 的一段 position，也可以让一个 block 处理一组 token 和 head。选择不同，shared memory 用量、寄存器压力、访存合并和 occupancy 都会变。

warp/wavefront：线程不是完全独立执行

GPU 上的线程通常以 warp 或 wavefront 为单位执行。可以把它理解成“一组 lane 同时执行同一条指令，但每个 lane 处理不同数据”。这和 CPU SIMD 有相似性：CPU SIMD 是一条指令处理多个 lane；GPU warp 是多个线程 lockstep 执行同一指令。区别是 GPU 编程模型把 lane 暴露成线程。

如果 warp 内线程走不同分支，会发生 divergence：一部分 lane 先执行分支 A，另一部分 lane 再执行分支 B，吞吐下降。若相邻 lane 访问连续地址，硬件可以合并内存请求；若访问分散，global memory 事务变多。

现象	类比	推理后端影响
warp lockstep divergence	SIMD lane 同步执行 lane mask 分裂	分支越规则越好 sampler/稀疏路径不适合热 kernel
coalescing occupancy	连续向量 load 同时驻留的 warp 数	KV cache 和权重 layout 要顺序 寄存器/shared memory 过多会降并发

第二册讲 SIMD 时强调“相邻 lane 和规则控制流”。GPU 这里是同一原则的放大版。prefill 的大矩阵乘通常规则；decode 中某些小 batch、变长 context、条件路径和 host 同步更容易破坏规则性。

coalescing：连续地址决定一次访存能带回多少有效数据

GPU global memory 带宽很高，但前提是 warp/wavefront 里的 lane 访问足够连续。若 32 个 lane 分别访问相邻的 fp16 元素，硬件可以用较少内存事务带回一整段数据；若 32 个 lane 访问相隔很远的位置，就会产生更多事务，实际带宽下降。

Listing 5.22 coalesced 与 scattered 访问的差异

```
coalesced:
  lane 0 reads base + 0
  lane 1 reads base + 1
  lane 2 reads base + 2
  ...

scattered:
  lane 0 reads page_table[a]
  lane 1 reads page_table[b]
  lane 2 reads page_table[c]
  ...
```

KV cache layout 直接影响 coalescing。若一个 warp 负责同一个 head 的连续 `head_dim`，那么 `head_dim` 连续布局有利；若一个 warp 负责多个 position 的同一维度，那么 position 连续更有利。paged KV 会引入 page table，长上下文和多 session 调度更灵活，但 kernel 必须小心把 page 内连续读和跨页边界分开处理。

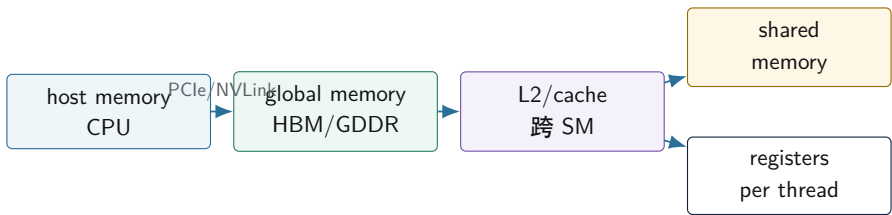
常见误区

GPU 上“逻辑上连续”不等于“lane 访问连续”。descriptor 只写 shape 不够，lowering 必须知道 warp mapping 和物理 stride。

显存层级：global、shared、register 和 cache

GPU 的 global memory 容量大、带宽高，但延迟也高。shared memory 在 SM/CU 内部，容量小但速度快，适合 tile 复用。register 是每个线程私有，最快但数量有限。L2 cache 连接多个 SM/CU，L1 或 texture cache 细节随架构不同。

GPU 推理中的内存层级



图示：本图要证明的命题是：GPU kernel 性能不只取决于算力，还取决于数据在哪一层内存里移动。

对推理来说，权重和 KV cache 通常在 global memory；tile 化 GEMM 会把输入/权重片段搬到 shared memory 或寄存器；accumulator 放在寄存器；logits 可能需要 staging 到 host。若每个 decode step 都把完整 logits 从 global memory 拷回 host，PCIe 或同步开销可能比某个小 kernel 更显眼。

shared memory：它解决的是复用，不是容量

shared memory 适合保存一个 block 内多个线程都会反复使用的小 tile。矩阵乘中，一个 input tile 和 weight tile 被多个 lane 复用，搬到 shared memory 后可以减少 global memory 读流。attention 中，也可以把 K/V 的一段 block 或 softmax 中间量放到 shared memory，减少重复读写。

但 shared memory 容量小，而且占用越多，每个 SM/CU 能同时驻留的 block 可能越少。用它之前要问三个问题：

- 这个 tile 会被多少次复用，是否值得从 global memory 搬进来。
- tile 大小是否会让 occupancy 明显下降。
- shared memory 的访问是否会发生 bank conflict。

例如 decode attention 读取历史 K/V 时，如果每个 K/V 元素只被一个 query head 用一次，把整段历史搬进 shared memory 可能不划算；若同一 K/V block 会被多个 query head 或多个 batch row 复用，就有价值。GQA 的共享 K/V head 也会改变这个判断：多个 query head 读同一个 K/V head，理论上有复用机会，但具体能否利用取决于 kernel mapping。

tensor core：不是所有矩阵都天然吃满

现代 GPU 常有 tensor core 或 matrix core，专门加速小矩阵块乘加。它们对 dtype、矩阵形状、对齐和 layout 有要求，常见于 fp16/bf16/tf32/int8/fp8 等路径。prefill 的大 GEMM 更容易使用 tensor core；decode 单 token 的 GEMV 不一定能有效使用。

场景	tensor core 适配性	关键条件
prefill QKV/MLP GEMM	通常较好	batch/token 足够大，layout 对齐
decode GEMV	通常较难	M 维太小，launch 和访存占比高
int4 权重 attention	取决于格式支持 取决于 kernel 设计	packed layout 和 scale 处理 softmax、mask、KV 读流复杂

这解释了一个常见现象：GPU prefill 很快，但 decode tokens/s 未必按峰值算力提升。decode 阶段每次只有一个 token，矩阵形态小，KV cache 读历史，logits 还要进 sampler。算力不是唯一瓶颈。

用 roofline 判断 GPU 推理瓶颈

roofline 的核心是算术强度：

$$AI = \frac{FLOPs}{Bytes}$$

若 **AI** 很低，即使 GPU 峰值 FLOPS 很高，也会被显存带宽限制；若 **AI** 很高，才更可能接近计算单元上限。prefill GEMM 的权重 tile 被多个 token 行复用，算术强度高；decode GEMV 的 batch 通常很小，权重复用弱，算术强度低。KV attention 还要随上下文读取历史 K/V，常常更像带宽问题。

路径	算术强度直觉	常见瓶颈
prefill GEMM	高，tile 复用好	tensor core、workspace、activation
decode GEMV	低到中，权重复用弱	显存带宽、launch、小矩阵效率
decode attention	随 context 读 KV	KV 布局、带宽、softmax 归约
logits staging	计算少，传输和同步明显	host-device copy、sampler boundary

一个简单反证方法是先算字节下限。若某个 decode step 至少要读 4GB 权重和 KV，而有效显存带宽是 800GB/s，那么单步光读数据就至少 5ms，还没算 kernel launch、softmax、MLP、staging 和 sampler。若 benchmark 报告低于这个数量级，就要检查是否统计了完整 step、是否用了 batch、是否跳过了某些层、是否缓存命中口径不同。

核心思想

GPU 性能分析不能只问“有没有 tensor core”。先算 FLOPs、bytes、launch、copy、sync，再判断瓶颈更像计算、带宽、调度还是 host-device 边界。

occupancy：并发多不等于一定快

occupancy 指一个 SM/CU 上同时驻留的 warp/block 相对硬件上限的比例。高 occupancy 可以帮助隐藏 global memory 延迟，但不是越高越好。寄存器用太多、shared memory 用太多、block 太大，都会降低 occupancy；反过来，为了提高 occupancy 把 tile 做得太小，也可能降低数据复用。推理 kernel 要平衡：

- 寄存器：accumulator 多，复用好，但 occupancy 可能降。
- shared memory：tile 大，global memory 读少，但可驻留 block 少。
- block 大小：并行度高，但同步和资源压力上升。
- 算术强度：每读一个字节做多少计算，决定更像算力瓶颈还是带宽瓶颈。

因此 GPU 后端的 profiler 不应只报告 occupancy。一个 kernel occupancy 很高但访存不合并，仍然慢；一个 tensor core kernel occupancy 不高但计算吞吐高，也可能很快。指标要和具体瓶颈一起看。

GPU profiler 指标怎样读

不同工具名字不同，但读法相近。不要孤立看一个百分比，而要把指标放回 kernel 目标。

指标	可能说明什么	需要结合什么看
achieved occupancy	驻留 warp 是否足够	寄存器、shared memory、block size
memory throughput	是否接近带宽上限	coalescing、cache hit、字节估算
tensor core utilization	矩阵单元是否忙	dtype、layout、tile、GEMM 形态
warp stall	等内存、同步或依赖	global load、barrier、寄存器依赖
launch count	小 kernel 数量	decode step latency、fusion 机会
copy time	host-device 传输	staging 大小、sampler 位置

例如 memory throughput 很高而 tensor core utilization 低，不一定是坏事；decode attention 可能本来就是读 KV 的带宽路径。反过来，occupancy 很高但 throughput 很低，可能是访问不合并、分支发散或 page table 间接访问导致。报告必须把这些指标和 executable plan 中的 kernel desc 对上。

prefill 和 decode 的 GPU 瓶颈不同

把前面硬件事实放回推理链路，可以得到一个清晰判断：

阶段	GPU 优势	主要风险
prefill	大 GEMM、并行 token、tensor core	activation 峰值、attention workspace、显存占用
decode	权重常驻、KV cache 常驻、可融合小算子	launch、同步、GEMV 小形态、KV 读流、logits staging
采样	可做 top-k 或局部 sampler	完整 logits 回传和 CPU 同步
量化	降低显存和带宽	格式不支持、scale metadata、de-quant 开销

这张表是 GPU lowering 的基础。prefill plan 应优先选择高吞吐 GEMM/attention；decode plan 应优先减少 launch 和 host-device 同步，优化 KV cache layout，并按 context length 分档。若把 prefill 的大矩阵思路直接套到 decode，通常会失望。

GPU 硬件事实怎样进入 AI 编译器

GPU 硬件不应该散落成后端里的魔法数字。它应该进入 capability 和 kernel descriptor：

Listing 5.23 GPU capability 和 kernel descriptor 字段

```
GpuCapability:
  backend_name
  max_threads_per_block
  warp_or_wavefront_size
  shared_memory_per_block
  register_file_limits
  supported_matrix_dtypes
  supported_quant_formats
  host_device_link

GpuKernelDesc:
  operator
  stage: prefill | decode
  block_shape
  warp_mapping
  shared_memory_bytes
  registers_estimate
  supported_layouts
  sync_points
```

编译器 lowering 使用这些字段选择 plan；verifier 用它们拒绝不支持的 shape 或 dtype；profiler 用它们解释瓶颈；benchmark 报告用它们保证可复现。这样 GPU 硬件知识就进入了工程合同，而不是停留在“这个 kernel 在我机器上更快”的经验。

本节验收线

读完本节，应该能把 GPU 硬件和推理后端连接起来：

- SM/CU 是 GPU 的局部执行资源，block/workgroup 被调度到其上。

- warp/wavefront 类似放大的 SIMD，喜欢规则控制流和连续访存。
- global memory、shared memory、register、cache 和 host-device 链路共同决定数据移动成本。
- tensor core 适合满足 dtype/layout/shape 条件的大矩阵块，不自动解决 decode GEMV。
- occupancy 要和寄存器、shared memory、访存合并、算术强度一起看。
- prefill 和 decode 的 GPU 瓶颈不同，lowering 和 profiler 必须分开。
- GPU capability 应进入 descriptor、verifier、lowering 和报告。

5.4 GPU 后端优化细讲：device buffer、stream、kernel launch 与端到端延迟

GPU 后端不是把 CPU 指针换成 device pointer

GPU 对 AI 推理很重要，但它也很容易被误解。初学者常以为只要把权重放到显存、把算子换成 GPU kernel，就一定更快。真实情况更细：大 prefill 可能非常适合 GPU，因为矩阵乘规模大、并行度高；decode 单 token 却可能被 kernel launch、同步、KV cache 读流、logits 回传和 sampler 边界限制。GPU 峰值 FLOPS 很高，不等于一次本地对话的端到端 latency 一定低。

从编译器角度看，GPU 后端是另一个 lowering 目标。它和 CPU 后端共享模型语义、shape、dtype、quant descriptor、KV cache 合同和 oracle，但物理执行完全不同。CPU 后端关心 cache line、SIMD 寄存器和线程池；GPU 后端关心 device memory、coalesced load、shared memory、warp/block、stream、kernel launch 和 host-device staging。

核心思想

GPU 后端优化的核心是端到端计划，不是单个 kernel 时间。device buffer、stream、workspace、staging、同步点和 fallback 都必须进入 executable plan 和 profiler。

本节不会依赖某个具体 GPU API。无论使用 CUDA、HIP、Metal、Vulkan compute 还是厂商库，工程边界都类似：资源要 RAII 管理，host/device 所有权要明确，异步执行要有 event 证据，kernel launch 要进入 profiler，fallback 不能悄悄打断语义合同。

GPU 后端的 C++20 RAII 边界

GPU 后端首先要把资源生命周期写清楚。下面这些对象不应该以裸句柄和裸指针散落在 runtime 中：

Listing 5.24 GPU 后端 RAII 对象

```
GpuContext:
  owns device selection and backend capability query

DeviceBuffer:
  owns device memory allocation and free

Stream:
  owns asynchronous execution queue

Event:
  owns timestamp/synchronization marker

Workspace:
  owns temporary device memory for algorithms

StagingBuffer:
  owns pinned or mapped host memory for transfers

KernelAdapter:
  binds executable plan descriptors to launch calls
```

这些对象的价值不是包装 API，而是建立工程不变量。DeviceBuffer 的析构释放显存；Stream 的生命周期覆盖异步 kernel；Event 让 profiler 能测真实 device 时间；StagingBuffer 显式表示 host-device 边界；KernelAdapter 把高层 descriptor 转换成具体 launch 参数。没有这些边界，GPU 后端很快会变成一堆到处传递的 `void*` 和句柄。

常见误区

异步后端最怕生命周期伪装。CPU 代码返回不代表 GPU kernel 已经读完输入；arena 或 staging buffer 不能在相关 stream/event 完成前复用。

host/device 驻留：数据在哪里比函数在哪里更重要

GPU 推理的第一条账本是数据驻留。权重、packed weight、activation、KV cache、logits、token id、sampler state 分别在 host 还是 device？什么时候传输？传输是否同步？这些问题往往比单个 kernel 代码更影响端到端延迟。

数据	理想驻留	风险
packed weight	device 常驻	首次上传/显存峰值/多 GPU 分片
activation	device arena	跨 stream 生命周期和 workspace 冲突
KV cache	device 常驻或分层	长上下文显存占用、page table、量化
logits	device 计算，少量回传	完整 vocab 回传太贵，sampler 边界同步
token id	host/device 双方可见	每步小传输和同步放大 latency

一个常见坏路径是：每个 decode step 在 GPU 上算 logits，然后把完整 vocab logits 拷回 CPU，再在 CPU 上做 sampler，再把下一个 token 拷回 GPU。对于大词表，这个回传可能成为固定开销。更好的计划可能是在 GPU 上做 top-k 或 sampler 的一部分，只回传 next token 或少量调试数据。但这会改变 sampler 的执行位置，必须进入 executable plan 和 oracle。

stream 和 event：异步不是自动并行

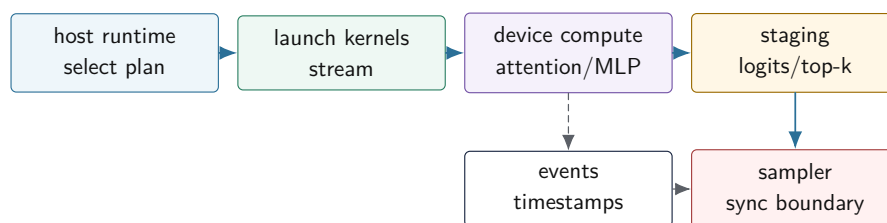
GPU API 往往是异步的。向 stream 提交 kernel 后，CPU 线程可以继续走；但这不等于工作已经完成，也不等于不同操作可以随便重排。stream 和 event 是计划中的同步语义。

Listing 5.25 GPU plan 中需要记录的同步边界

```
GpuPlan:
  stream decode_stream
  events:
    after_kv_append
    after_attention
    before_logits_staging
  dependencies:
    attention waits for kv_append
    sampler waits for logits staging
    arena slot reuse waits for producing kernels
```

profiler 也必须区分 CPU 提交时间和 GPU 执行时间。只测 kernel 内部时间，会漏掉 launch overhead、host-device copy、stream wait、CPU sampler 和计划选择。只测 CPU 墙钟时间，又难以定位慢在 device kernel 还是同步。两者都要记录。

GPU executable plan 中的 stream、event 与 staging



图示：本图要证明的命题是：GPU 后端的端到端延迟由 launch、device compute、event、staging 和 sampler 同时决定。

kernel launch：小算子会被固定开销吞掉

GPU kernel launch 有固定成本。prefill 的大 GEMM 能摊薄 launch 成本；decode 的许多小算子不一定能。如果每层都单独 launch RMSNorm、QKV、RoPE、KV append、attention、Wo、MLP gate、activation、down projection，decode 每个 token 会产生大量 launch 和同步。

解决方向有几类：

- 算子融合：把 RMSNorm 与 linear、RoPE 与 KV append 等合并。
- segment 调度：减少 host 侧逐算子解释和 profiler 细粒度开销。
- persistent 或批处理策略：对服务端多请求场景减少重复 launch。
- GPU sampler：减少 logits 完整回传和 host 同步。

融合仍然要受语义约束。比如 oracle 需要某层中间输出时，debug plan 不能完全隐藏物化点；量化权重需要 scale metadata，融合 kernel 必须正确读取；KV append 写入状态，后续 attention 必须看到正确 position。GPU fusion 不是“把越多函数塞进一个 kernel 越好”，而是减少内存流和 launch 开销，同时控制寄存器、shared memory 和调试边界。

prefill GPU plan：吞吐和 workspace

prefill 的输入 token 多，矩阵乘和 attention 更大，GPU 通常更有优势。编译器 lowering 要为 prefill 选择合适的 batched GEMM、attention kernel 和 workspace。workspace 是后端算法的临时内存，不能和 activation arena 随意混用。

Listing 5.26 prefill GPU plan 的主要字段

```

PrefillGpuPlan:
  prompt_length_bucket
  device_weight_layout
  activation_arena_size
  attention_workspace_size
  gemm_algorithm_id
  attention_algorithm_id
  stream_dependencies
  kv_cache_write_layout
  
```

prefill 指标也要拆开。time to first token 包含 tokenizer、权重准备、KV cache 初始化、prefill graph 和 logits/sampler；prompt tokens/s 只描述 prompt 处理吞吐。若报告只写一个“GPU prefill 很快”，但没有说明 prompt 长度、batch、workspace、是否包含权重上传，就无法对标。

decode GPU plan：小 batch、KV cache 与同步

decode 的输入通常是一个 token。即使 GPU 算力强，单 token 路径也可能被固定开销限制。decode GPU plan 要重点处理三件事：

- 减少每 token launch 和同步次数。

- 让 KV cache 读取尽量连续或分块，避免长上下文下显存读流退化。
- 减少 logits 回传，只同步 sampler 真正需要的数据。

Listing 5.27 decode GPU profiler 应拆分的事件

```
decode_step:
    host_plan_select_ns
    launch_overhead_ns
    rmsnorm_linear_kernel_ns
    kv_append_kernel_ns
    attention_kernel_ns
    mlp_kernel_ns
    logits_staging_ns
    sampler_ns
    total_step_latency_ns
```

如果只看 `attention_kernel_ns`，可能会误以为 GPU 很快；如果 `logits_staging_ns+sampler_ns` 很大，用户感受到的 tokens/s 仍然不高。端到端报告必须包含 `total_step_latency_ns`。

一个 decode step 的端到端时序

GPU decode 不能只看 kernel profiler。单 token 生成是一条 host 和 device 共同参与的流水线。即使每个 device kernel 都很快，host 侧计划选择、kernel launch、event wait、logits staging 和 sampler 也可能支配端到端延迟。

Listing 5.28 GPU decode step 的最小时序

```
host:
    bind current token and position
    submit rmsnorm/qkv/rope/attention/mlp kernels
    submit logits or top-k staging
    wait for sampler boundary
    sample next token

device stream:
    rmsnorm_qkv_kernel
    rope_kv_append_kernel
    attention_decode_kernel
    mlp_kernel
    logits_topk_kernel
    copy top-k or logits slice to host
```

这条时序里至少有三个同步边界。第一，KV append 与 attention 之间要有顺序，attention 必须看到当前 token 写入后的 K/V。第二，logits 或 top-k 结果回到 host 前，sampler 不能读。第三，arena 或 workspace 复用前，相关 stream 上的 kernel 必须完成。若这些边界只靠“代码看起来按顺序写”维持，就很容易在异步后端出错。

Listing 5.29 StepTrace 应记录的字段

```
StepTrace:
    request_id
    step_index
    model_position
```

```

context_length
selected_plan_id
host_submit_begin_ns
host_submit_end_ns
device_kernel_begin_ns
device_kernel_end_ns
staging_begin_ns
staging_end_ns
sampler_begin_ns
sampler_end_ns
sync_wait_ns
total_step_latency_ns

```

有了这个结构，报告才能回答具体问题：慢在 host 提交、device kernel、staging、wait，还是 sampler。没有这个拆分，只拿 GPU 工具里某个 kernel 的耗时去推断用户感受到的 tokens/s，是不完整的。

GPU decode 的最小可用优化顺序

GPU 后端优化也要分阶段，不要一开始就写一个巨大的 fused kernel。一个更稳的顺序是：

1. 先让所有权重和 KV cache 常驻 device，消除每步大块 host-device 传输。
2. 把 logits staging 从完整 vocab 缩小到 sampler 需要的最小集合，或把 top-k 放到 device。
3. 合并小算子，优先融合 RMSNorm+Linear、RoPE+KV append 这类语义边界清晰的路径。
4. 对 attention decode 按 context length 分档，短上下文、长上下文、paged KV 使用不同 plan。
5. 最后再做更激进的 persistent kernel、跨层 segment 或多请求 batch decode。

每一级都要同时看 oracle 和 StepTrace。比如把 top-k 放到 GPU 后，logits 全量 oracle 可能不再自然可得，这时 debug plan 要保留可选的 full logits materialization；生产 plan 可以只回传 top-k，但必须声明 sampler 语义没有改变。

GPU 量化：格式支持比位宽口号更重要

GPU 上的低比特量化并不天然快。关键是目标硬件和库是否支持该格式，以及支持发生在哪一层。如果 int4 权重格式能直接进入高效矩阵乘路径，收益可能明显；如果每次都要先用自定义 kernel 解包成 fp16，再调用 GEMM，收益要重新计算。scale metadata 的布局也会影响 coalesced load。

路线	优点	风险
vendor quant kernel 自定义 dequant + GEMM	利用硬件和库优化 实现直接，易验证	格式受限，descriptor 适配复杂 解包和额外写回可能很贵
fused dequant matmul	减少中间写回	kernel 复杂，寄存器/shared memory 压力
KV cache 量化	降低长上下文显存和带宽	scale metadata 和 attention 精度风险

量化 verifier 要知道 GPU 后端实际支持什么。不能因为 manifest 写了 q4，就假设 GPU plan 可用。若 GPU 不支持该格式，应明确 fallback 到 CPU、fallback 到 dequant path，或拒绝该 plan，并在报告里写出原因。

fallback：不能悄悄改变执行位置

真实工程中，某些算子或格式 GPU 后端暂时不支持，需要 fallback。fallback 可以是 CPU reference、CPU optimized path、GPU 上的慢路径，或者 vendor 库之外的自定义 kernel。关键是不能悄悄发生。

Listing 5.30 fallback report 字段

```

FallbackEvent:
  operator_id
  reason: unsupported_dtype | unsupported_shape | workspace_limit | ↔
    ↪ debug_oracle
  from_backend
  to_backend
  data_transfer_bytes
  added_sync_points
  correctness_profile

```

如果某层 attention fallback 到 CPU，会发生 device-to-host 拷贝、CPU 计算、host-to-device 拷贝和同步，端到端 latency 可能大幅上升。profiler 必须把它显示出来。oracle 也要知道 fallback 路径的误差合同可能不同。

多 GPU 和分片：先建立单设备合同

多 GPU 推理涉及权重分片、KV cache 分布、通信、pipeline/tensor parallel 和跨设备同步。本册后续可以扩展，但工程顺序应该先把单设备合同建立好。原因很简单：如果单 GPU 的 descriptor、stream、workspace、oracle 和 profiler 都不清楚，多 GPU 只会把错误放大。

单设备稳定后，多 GPU 至少要新增这些事实：

- 张量分片轴和每个 shard 的 shape。
- 跨设备通信算子和同步点。
- KV cache 是复制、分片还是按层分布。
- logits 聚合和 sampler 所在设备。
- profiler 能拆分 compute、communication 和 wait。

这些内容和第二册分布式边界、背压、运行时调度有关。它们不是本节主要目标，但要从一开始为 descriptor 和 plan 留出扩展空间。

GPU 后端验收报告

GPU 后端优化的报告至少要覆盖：

Listing 5.31 GPU 后端验收报告字段

```

hardware:
  gpu_model
  driver_runtime_version
  device_memory
  enabled_backend_features

plan:
  backend_plan_id
  device_layouts
  stream_count
  workspace_bytes
  fallback_events

performance:
  weight_upload_time_ms
  pack_or_convert_time_ms
  prefill_tokens_per_s
  time_to_first_token_ms
  decode_tokens_per_s

```

```

total_step_latency_ms
launch_time_ms
staging_time_ms
device_kernel_breakdown

correctness:
oracle_profile
logits_error
topk_changed
generated_token_mismatch

```

注意 `launch_time_ms` 和 `staging_time_ms` 必须出现。它们正是很多 GPU decode 路径慢的原因。如果报告只有 device kernel breakdown，就还不是端到端推理报告。

本节验收线

读完本节，应该能说明 GPU 后端的真实工程边界：

- GPU 后端共享 AI 编译器的语义合同，但拥有自己的 device buffer、stream、event、workspace、staging 和 kernel adapter。
- 数据驻留和 host-device 传输常常比单个 kernel 更影响端到端 latency。
- stream/event 必须进入 executable plan，arena 和 staging buffer 复用要等待异步工作完成。
- kernel launch 是 decode 小算子的固定成本，fusion 和 segment 调度要围绕它设计。
- prefill 关注吞吐和 workspace，decode 关注 launch、KV cache 读流、logits staging 和 sampler 同步。
- GPU 量化要看硬件和库是否真正支持目标格式，fallback 必须显式报告。
- GPU benchmark 必须同时报告 device kernel 时间、launch、staging、同步、端到端 tokens/s 和 oracle。

到这里，CPU 和 GPU 后端有了共同的工程语言：descriptor 表达语义，lowering 选择物理计划，runtime 绑定状态，oracle 证明正确，profiler 证明性能。两者的差异不是谁更“高级”，而是机器账本不同。

5.5 CPU decode 实战：从标量 GEMV 到 packed 低比特微内核

decode 优化先看形态

7B 推理的 decode 阶段每次只处理一个新 token。对线性层来说，输入是一个 hidden 向量，权重是一个大矩阵，输出是一个新向量。这更像 GEMV，而不是大 batch GEMM。CPU 优化要先承认这个形态：每 token 反复流过大块权重，权重字节和 KV 字节很容易支配延迟。

核心理想

CPU decode 微内核的目标不是写一个“通用最快矩阵乘”，而是为单 token 或小 batch 的固定权重读流建立可验证的 packed layout、累加策略、tail path、线程切分和性能账本。

本节沿同一个 linear 语义推进：标量 fp32 reference，普通 int8 baseline，packed int8，int4 packed，SIMD 微内核，线程切分，NUMA 和 profiler。每一步都要能回到 oracle。

本节固定一个 decode 形状作为样板，不再泛泛谈“某个矩阵”：

Listing 5.32 CPU decode 样板形状

```

input hidden K = 4096
output channels O = 4096
batch/token T = 1
semantic: y[out] = sum_{k=0}^{4095} W[out,k] * x[k]

```

```
weight layout before packing: row-major [0,K]
packed output block: BN = 8 for SIMD kernel, benchmark block field uses 16
```

这个 shape 对应 7B decoder 里的典型 hidden-to-hidden projection。它还不是完整一层，因为没有 RMSNorm、attention、MLP 和 KV cache，但它是 decode 阶段最容易被权重读流支配的核心切片。

标量 reference 不是性能 baseline

reference 的职责是正确，不是代表可接受性能。

Listing 5.33 fp32 reference GEMV

```
for out in 0..0:
    acc = 0
    for k in 0..K:
        acc += weight[out, k] * input[k]
    output[out] = acc
```

性能 baseline 可以仍然是标量，但要固定测量条件：连续内存、关闭 debug 检查、固定线程数、固定输入、warmup、重复次数和输出校验。否则 benchmark 比较的是调试开销或缓存偶然性。

int8 权重 baseline

最简单的 int8 权重实现是边读边反量化：

Listing 5.34 int8 权重 + fp32 输入 baseline

```
for out in 0..0:
    acc = 0
    scale = scales[out]
    for k in 0..K:
        w = float(weight_i8[out, k]) * scale
        acc += w * input[k]
    output[out] = acc
```

这比 fp32 权重少读字节，但每个权重多了转换和 scale。若 `scale` 是 per-channel，它可以在外层读取一次；若是 per-group，内层每隔 group size 切换 scale。这里必须先有 QuantDesc：

Listing 5.35 int8 QuantDesc 最小字段

```
QuantDesc:
    bit_width = 8
    signed = true
    scale_policy = per_channel | per_group
    group_size
    zero_point_policy
    scale_dtype
    axis
```

没有 descriptor，kernel 无法知道 scale 怎样对应权重，也无法写通用 oracle。

仓库中的 baseline 落在 [books/cpu-volume-3/labs/linux_cpu_inference/src/int8_kernels.cpp](https://github.com/llvm/cpuid/blob/main/books/cpu-volume-3/labs/linux_cpu_inference/src/int8_kernels.cpp)：`linear_i8` 使用 `int8weight+fp32input+fp32accumulator`。它不是完整整数推理，但它有清晰合同：

Listing 5.36 lqci 当前 int8 scalar 语义

```

row_base = out * input_size
acc = bias[out]
for k in 0..input_size:
    acc += float(weights[row_base + k]) * scale * input[k]
output[out] = acc

```

对 **4096x4096**，每次调用至少读取约 **16MiB** int8 权重、**16KiB** fp32 input、写 **16KiB** fp32 output。scale 当前是 per-layer 标量，所以它不是主要读流。这个账本解释了为什么 decode GEMV 首先要盯权重读流和 SIMD 转换，而不是盲目增加复杂图优化。

packed int8: 先改变地址流

原始权重按 **[out, k]** 行主序存储，对单输出行扫描友好。但 SIMD 微内核通常想一次计算多个 **out**，让同一个输入 **input[k]** 广播到多个输出 accumulator。为了让多个输出通道的权重连续，需要把小面板打包。

Listing 5.37 packed int8 panel 示意

```

for out_block in 0..0 step BN:
    for k in 0..K:
        for oi in 0..BN:
            packed.append(weight_i8[out_block + oi, k])

```

打包后，内核的热循环读取 **packed[k, oi]** 是连续的：

Listing 5.38 packed GEMV 热循环

```

for out_block in 0..0 step BN:
    acc[BN] = 0
    for k in 0..K:
        x = input[k]
        wv = load packed weights for BN outputs at this k
        acc += x * dequant(wv)
    store acc

```

packing 的成本应发生在模型 prepare 阶段，而不是每次请求 decode 阶段。plan report 要记录 packed weight 字节数、packing 时间和 layout id。否则上线后可能发现首个请求很慢，却不知道慢在 prepare。

当前 packed layout 使用 **out_block->k->lane** 顺序。原始 row-major 在固定 **k=0** 时，连续 8 个输出通道的地址是：

Listing 5.39 原始 row-major 多输出地址

```

W[0,0]    addr = 0
W[1,0]    addr = 4096
W[2,0]    addr = 8192
...
W[7,0]    addr = 28672

```

packed 后，固定 **k=0** 的 8 个输出通道变成连续地址：

Listing 5.40 packed panel 多输出地址

```

packed block out0..7, k=0:

```

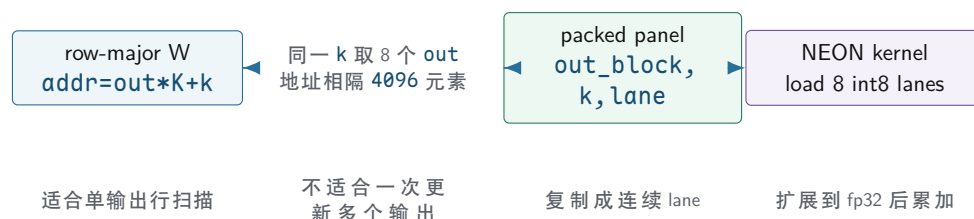


```
paddr 0..7 = W[0,0], W[1,0], ..., W[7,0]
```

```
packed block out0..7, k=1:
```

```
paddr 8..15 = W[0,1], W[1,1], ..., W[7,1]
```

4096x4096 decode GEMV 的地址流变化



图示：本图要证明的命题是：当前 packed int8 的直接收益是把跨步多输出读取变成连续 lane 读取。

int4 packed：字节减少，解包增加

int4 把两个 4-bit 权重放进一个 byte。常见 signed int4 范围是 $-8..7$ 或通过 zero point 表示。解包需要取低四位和高四位，再做符号扩展或减 zero point。

Listing 5.41 int4 解包语义

```
byte b contains two values:
  lo = b & 0x0f
  hi = (b >> 4) & 0x0f

signed value:
  if nibble >= 8:
    value = nibble - 16
  else:
    value = nibble
```

int4 的优势是权重字节减半，风险是解包、scale metadata 和误差。group quant 下每 G 个权重共享一个 scale。若 K 不能整除 G ，最后一组是 tail group。byte oracle 必须覆盖 nibble 顺序、符号扩展、group tail 和 scale 索引。

累加器：低比特不等于低精度累加

低比特权重不代表可以用低精度累加。常见策略有两类：

- 权重低比特，输入 fp16/fp32，反量化后用 fp32 accumulator。
- 权重和激活都量化，用 int32 accumulator，再 requantize 或转回 fp32。

第一类实现简单，适合学习和 CPU baseline；第二类更接近完整整数推理，但需要处理激活 scale、zero point 修正、bias scale 和输出 requantize。第三册前面量化章节已经讲过 zero point 修正，CPU 微内核进入整数路径时必须复用同一合同。

常见误区

不要把 int4 权重直接乘成 int8 accumulator。归约维度 $K=4096$ 时，误差和溢出风险都不可接受。accumulator dtype 是 kernel descriptor 的核心字段。

SIMD 微内核的寄存器账本

假设一次计算 $BN=16$ 个输出通道。内核维护 16 个 accumulator lane。每个 k ：

1. 读取一个 `input[k]`。
2. 广播到 SIMD 寄存器。
3. 读取 packed 权重的 16 个值。
4. 转换或解包到可乘格式。
5. 乘加到 accumulator。

Listing 5.42 抽象 SIMD GEMV 微内核

```
acc = zero_vector(BN)

for k in 0..K:
    x = broadcast(input[k])
    w = load_packed_weight_vector(k)
    wf = convert_or_dequant(w, scale_for(k))
    acc = fma(x, wf, acc)

store output block
```

这个账本要和 ISA 绑定。AVX2、AVX-512、NEON、SVE 的向量宽度、转换指令、整数 dot 能力和 mask tail 都不同。因此代码结构应分成 capability、descriptor、kernel 实现和 fallback，不要在业务调用点写一堆 ISA 分支。

当前 aarch64 NEON 实现位于 [books/cpu-volume-3/labs/linux_cpu_inference/src/int8_kernels_neon.cpp](#)。它一次处理 8 个输出通道，用两个 `float32x4_t` accumulator 保存低 4 lane 和高 4 lane：

Listing 5.43 当前 NEON kernel 的核心语义

```
weights_i8 = vld1_s8(packed_weights + packed_index)
weights_i16 = vmovl_s8(weights_i8)
weights_i32_low = vmovl_s16(low(weights_i16))
weights_i32_high = vmovl_s16(high(weights_i16))
weights_f32_low = vcvtq_f32_s32(weights_i32_low)
weights_f32_high = vcvtq_f32_s32(weights_i32_high)
input_scaled = vdupq_n_f32(input[k] * scale)
acc_low = vmlaq_f32(acc_low, weights_f32_low, input_scaled)
acc_high = vmlaq_f32(acc_high, weights_f32_high, input_scaled)
```

这段代码的价值是把 8 个输出通道的标量乘加合并成两个 4-lane NEON 累加账本；它的限制也很明确：它先把 int8 扩展成 fp32，再做 fp32 乘加，并没有使用 `sdot` 这类整数 dot 指令，也没有做 int32 accumulator + requantize。

tail path: 最后一块经常出错

若输出通道 0 不是 BN 的倍数，最后一个 block 需要 tail。常见策略：

- 单独标量处理 tail，简单但慢一点。
- SIMD mask store，要求 ISA 支持。
- padding packed weight 到 BN 对齐，输出只写有效部分。

若归约维度 K 不是 group size、解包块或向量宽度的倍数，也需要 tail。对 int4，K 为奇数还会出现半个 byte 的语义问题。虽然真实 7B 的很多维度规整，测试必须包含不规整维度，因为模型变体、lm head、adapter 或小 fixture 都可能触发。

线程切分：按输出通道还是按层

decode 的单个 linear 可以按输出通道切分：不同线程计算不同 **out** block，共享同一 input，读取不同权重。优点是没有输出写冲突；缺点是每个线程都读一遍 input，且小层线程开销可能过大。

Listing 5.44 按输出通道切分

```
parallel_for out_block_range:
    run_packed_gemv(input, packed_weight_range, output_range)
```

也可以在线路更高层调度：不同层或不同算子顺序依赖强，不适合随意并行；不同 request 的 decode step 可以并行，但会增加权重竞争和 cache 压力。最稳的起点是单算子内按输出 block 切分，并记录线程数和分区。

切分方式	优点	风险
按输出通道	简单，无输出冲突	权重带宽争用、线程开销
按 batch/session	可提高吞吐	状态复杂，p95 可能变差
按层流水	理论并行	decoder 层依赖强，收益有限

NUMA 和权重放置

多路 CPU 或 NUMA 机器上，权重放在哪个 NUMA node 会影响带宽。若线程在 node 0，权重页却在 node 1，远端内存访问会增加延迟。最小工程策略：

- prepare 阶段按计划触碰 packed weight，让页分配到目标 node。
- 线程池固定亲和，避免跨 node 随机迁移。
- plan report 记录线程数、亲和策略和 NUMA 策略。
- benchmark 报告硬件拓扑，不把不同机器数字混比。

NUMA 优化不要太早做，但 trace 要保留足够字段。否则后期发现 p95 抖动，只能猜测是调度还是内存放置。

perf counters：用证据判断瓶颈

CPU 优化不能只看耗时。至少要观察：

- cycles 和 instructions，判断 IPC。
- cache miss 和 LLC miss，判断权重/KV 读流。
- branch misses，判断 tail 或分支复杂度。
- vector instruction 比例，确认 SIMD path 真的执行。
- memory bandwidth，判断是否接近带宽上限。

如果引入 SIMD 后 instructions 降了但 LLC miss 上升，可能 packing layout 破坏了局部性。如果线程数增加后 tokens/s 不升反降，可能已经带宽饱和或 NUMA 远端访问增加。如果 int4 比 int8 不快，可能解包和 scale 成本抵消了字节收益。

本次环境已经安装 **perf7.0.12**，但容器没有暴露可用 PMU events，**perfstat** 对 **cycles, instructions, cache-misses, branches, branch-misses** 返回：

Listing 5.45 本机 perf 结果，不伪造计数器

```
Error:
No supported events found.
Access to performance monitoring and observability operations is limited.
```

这个结果记录在下面的 perf artifact。它不是通过项，而是缺口证据：当前样板章已经有源码、CSV 和反汇编，但还没有 cycles、IPC、cache miss、branch miss。下一次必须在有 PMU 权限的裸机或授权容器上补齐。

Listing 5.46 perf artifact

```
books/cpu-volume-3/results/
lcqi-aarch64-neon-decode4096-2026-06-23.perf.txt
```

当前 lab 的真实验收口径

本仓库的 `linux_cpu_inference` 已经把本节的一部分落成代码：同一个 int8 linear 语义下有四个 benchmark backend：

- **scalar**：普通 row-major 标量循环，是 correctness 和耗时基线。
- **packed_scalar**：使用 output block packing，但仍用标量累加，用来证明 packed layout、padding 和 tail 语义。
- **avx2**：x86-64 目标下编译的 8 输出通道 packed kernel。
- **neon**：aarch64 目标下编译的 8 输出通道 packed kernel。

benchmark 不再把跨平台源码混成一个数字，而是按 backend 逐行输出：

Listing 5.47 LCQI kernel benchmark CSV 字段

```
target_arch,input_size,output_size,output_block,repeat,
backend,available,average_us,max_abs_diff,checksum
```

这种格式有两个好处。第一，`available=0` 可以明确说明某个 ISA 后端没有在当前机器上实测；例如 aarch64 机器不能把 AVX2 源码当作本机性能证据。第二，所有 backend 都和同一份 scalar 输出比较 `max_abs_diff`，避免只看速度。

当前 aarch64 机器上的 Release 样本如下：

shape	scalar us	packed scalar us	NEON us
128x128	15.312	33.731	5.308
256x256	63.666	47.709	14.515
512x512	280.195	173.169	36.685
513x257	136.218	93.257	20.192
1024x4096	4757.070	3014.180	702.123

新增固定 decode 样板 `4096x4096` 后，CSV artifact 是 `books/cpu-volume-3/results/lcqi-aarch64-neon-decode4096-2026-06-23.csv`：

backend	average us	max abs diff	解释
scalar	20188.1	0	row-major 标量基线
packed scalar	25052.9	0	证明 layout，但本机更慢
NEON	4322.2	0	本机 SIMD 路径，约 4.67x 快于 scalar
AVX2	0	0	aarch64 上不可用，不能当实测

这些数字的解释要克制。packed scalar 变慢，说明 packing 本身不是魔法；如果没有 SIMD 消费这个布局，它只增加了循环和地址转换成本。NEON 变快，说明连续 lane + 向量扩展/乘加击中了当前瓶颈的一部分。它还没有证明达到了硬件上限，因为缺少 PMU 计数器、内存带宽、CPU 频率和 NUMA 信息。

反汇编 artifact 如下。热循环关键片段随后列出：

Listing 5.48 objdump artifact

```
books/cpu-volume-3/results/
lcqi-neon-linear-i8-objdump-2026-06-23.txt
```

Listing 5.49 NEON objdump 摘要

```

ldr    s26, [x20, x0, lsl #2]    // load input[k]
ldr    d28, [x2, x1]             // load 8 packed int8 weights
fmul   s26, s26, s29             // input * scale
sxtl   v28.8h, v28.8b           // int8 -> int16
sxtl   v27.4s, v28.4h           // low lanes int16 -> int32
sxtl2  v28.4s, v28.8h           // high lanes int16 -> int32
scvtf  v27.4s, v27.4s           // int32 -> fp32
scvtf  v28.4s, v28.4s
fmul   v27.4s, v27.4s, v26.s[0]
fmul   v28.4s, v28.4s, v26.s[0]
fadd   v31.4s, v31.4s, v27.4s
fadd   v30.4s, v30.4s, v28.4s

```

这段反汇编证明当前路径确实进入 NEON 向量计算；也证明它还不是成熟 int8 dot-product kernel。一个更强实现应减少 `scvtf/fmul/fadd` 链路，尝试整数 dot、更多 unroll、prefetch、线程分区和 per-channel/per-group scale 的更低开销组织。

oneDNN 对照：只能比较坐标，不能混淆 dtype

本次还加入了可选 oneDNN 对标目标 `lcqi_onednn_bench`。它在安装 `onednn-devel` 的机器上编译，执行 `1x4096` 乘 `4096x4096` 的 f32 matmul。CSV artifact 是 `books/cpu-volume-3/results/lcqi-onednn-f32-4096-2026-06-23.csv`：

library	dtype	shape	average us
oneDNN matmul	f32	<code>1x4096*4096x4096</code>	17187.8
LCQI scalar	int8 weight + fp32 input	<code>4096x4096</code>	20188.1
LCQI NEON	int8 weight + fp32 input	<code>4096x4096</code>	4322.2

这张表不能读成“LCQI 已经比 oneDNN 强”，因为 dtype、权重表示、primitive 选择和线程策略不同。它只能说明本仓库已经有了外部 primitive 坐标，并暴露下一步对标要求：要么实现同 dtype f32 GEMV/GEMM 与 oneDNN 比，要么找 oneDNN 支持的同量化路径；同时固定线程数、ISA、warmup、输出误差和 perf counters。

微内核进入主线的验收

一个 CPU decode 微内核进入主线前，至少要满足：

验收与评分标准

- descriptor 明确 dtype、layout、group size、accumulator、tail policy 和 ISA。
- packed weight oracle 能把 packed bytes 映射回原始权重。
- operator oracle 覆盖整齐尺寸、输出 tail、归约 tail、非对齐输入和 group tail。
- layer oracle 覆盖至少一个 tiny decoder block。
- performance report 分开 pack time、kernel time、decode step time 和内存峰值。
- unsupported ISA 有明确 fallback 或 plan 阶段拒绝。
- 反汇编或 profiler 证明确实走到目标 SIMD path。

这条验收线比“benchmark 快了”严格，但它能保护项目长期演进。否则下一次改量化、改 layout、接 GPU fallback 时，很容易破坏 CPU 路径而不自知。

本节验收线

读完本节，应该能把 CPU decode 优化拆成可实现任务：

- 从 fp32 reference 建立正确性基准，再写 int8/int4 baseline。

- 理解 packed layout 改变的是多个输出通道的权重地址流。
- 知道 int4 的 nibble 顺序、符号扩展、group scale 和 tail 为什么必须测试。
- 能说明 accumulator dtype 为什么是 kernel descriptor 的核心字段。
- 能把 SIMD 微内核写成 input broadcast、packed load、dequant、FMA、store 的账本。
- 能设计线程切分、NUMA 策略、perf counters 和主线验收。

5.6 GPU decode 端到端：常驻权重、KV 布局、sampler 与同步成本

GPU 快，不代表端到端一定快

GPU 在大 GEMM 上很强，但 decode 阶段常常是小 batch、单 token、长上下文、频繁同步。若只看单个 kernel 时间，很容易误判。端到端 GPU decode 要把权重常驻、KV cache、logits、sampler、copy、stream、event 和 fallback 一起算。

核心思想

GPU decode 的核心问题不是“能不能 launch kernel”，而是让权重、KV、activation 和 sampler 尽量留在设备侧，同时把每步同步和 host-device copy 控制到可解释范围。

一个最小 GPU decode step 包括：

Listing 5.50 GPU decode step 账本

```
input token on host
-> copy token or embedding input to device
-> run layer kernels
-> read/write device KV cache
-> compute logits on device
-> sampler on device or copy logits to host
-> emit token event on host
```

每个箭头都有成本。若每步把所有中间 activation 拷回 CPU 做调试，GPU kernel 再快也没意义。debug oracle 可以按 trace level 打开，但默认路径应尽量减少同步。

权重常驻设备

GPU 后端的基本要求是权重 prepare 后常驻设备。每次请求或每个 token 重新上传权重，端到端必然崩溃。

Listing 5.51 GPU prepare 阶段

```
load model files on host
verify manifest and quant descriptors
pack or convert weights for GPU backend
allocate DeviceBuffer for each weight group
copy weights to device
record device_weight_table in plan report
```

plan report 要记录 device weight 字节、上传时间、quant format 和 fallback。若 GPU 显存不够，应该在 compile/prepare 阶段拒绝或选择 CPU fallback，而不是在请求运行中途失败。

activation 和 workspace

decode 每层会产生临时 activation。GPU 后端应使用 workspace 或 arena，避免每个 step 调用设备分配器。

Listing 5.52 GPU workspace 字段


```
GpuWorkspace:
  activation_buffers
  attention_scratch
  logits_buffer
  sampler_temp
  size_bytes
  alignment
  stream_owner
```

workspace 的生命周期通常是 plan 或 session。若 workspace 被多个 session 共享，调度器必须保证不会并发写同一 buffer。若每个 session 独占 workspace，并发容量要把它算进 admission control。

Device KV cache

若 layer kernel 在 GPU 上运行，KV cache 最好也在 GPU 上。否则每层 attention 都要在主机和设备之间传 K/V，成本不可接受。

Listing 5.53 GPU KV cache 设计点

```
DeviceKvCache:
  layout: contiguous | paged
  dtype: fp16 | int8 | int4
  block_table_optional
  scale_table_optional
  max_context
  resident_device
```

连续 KV 在 GPU 上地址简单，但并发浪费显存；分页 KV 更灵活，但 kernel 要通过 block table 找 K/V。长上下文下，attention kernel 的读取模式决定了 coalescing。若每个 thread 读取不连续地址，带宽会下降。

GPU executable plan

GPU 后端也需要 executable plan，而不是在 runtime 中临时拼 kernel。

Listing 5.54 GPU segment descriptor

```
GpuSegmentDesc:
  segment_id
  op_or_fused_region
  kernel_name
  grid_shape
  block_shape
  input_buffers
  output_buffers
  state_reads
  state_writes
  workspace_range
  stream_policy
  fallback_segment_optional
```

state_reads 和 **state_writes** 对 attention 很重要：decode attention 读历史 KV，同时写当前 K/V。若 plan 不表达状态边，调度器可能错误重排 segment；若 workspace range 不明确，多个 segment 可能覆盖同一临时区域；若 fallback segment 不记录，性能报告会失真。

device residency report

GPU prepare 完成后，要输出哪些对象在设备上。

Listing 5.55 DeviceResidencyReport

```
DeviceResidencyReport:
  model_id
  device_id
  weights:
    total_bytes
    quant_profile
    packed_layout
  kv_cache:
    layout
    max_reserved_bytes
    dtype
  workspace:
    bytes
  logits:
    device_buffer_bytes
  fallback:
    unsupported_segments
```

这份报告能提前发现“显存看似够，实际 packed weight 加 workspace 不够”的问题。它也能解释为什么同一模型在不同 GPU 上 plan 不同：某些设备支持某种低比特 kernel，另一些设备只能 fallback。

prefill 和 decode 的 GPU 策略不同

prefill 有较大的 token 维度，适合 GEMM 和 batched attention。decode 单 token 时，kernel launch、KV 读流和小矩阵效率更突出。

阶段	GPU 机会	GPU 风险
prefill	大 GEMM、并行 attention、吞吐高	显存峰值、长 prompt attention
decode	多 session 合批、KV 常驻、低同步	小 batch 低占用、launch/copy 成本

因此 GPU 报告必须拆 prefill 和 decode。只报总 tokens/s 会掩盖问题：一个后端可能 prefill 非常快，但单用户 decode 受 launch 成本限制；也可能多 session 合批后吞吐高，但 p95 变差。

sampler 放设备侧的收益和代价

若 logits 在 GPU 上，CPU sampler 需要把 logits 或 top-k 候选拷回 host。完整词表 logits 可能是几十 KB 到几百 KB，每步还会同步 stream。设备侧 sampler 可以减少同步，但要实现 top-k/top-p、随机数和 stop 检查。

折中做法是 GPU 先选 top-k 候选，只拷回候选 id 和分数给 CPU 最终采样。这样减少拷贝，但仍保留业务逻辑和可复现随机数在 CPU。

策略	拷贝量	实现复杂度
拷完整 logits 到 CPU	高	低
GPU top-k, CPU 采样	中低	中
全 GPU sampler	低	高

贯穿项目可以先用 CPU sampler, 但必须在 trace 中单独记录 `logits_copy_ns` 和 `sampler_ns`。这样后续把 sampler 下沉到 GPU 时, 收益能被证明。

stream 和 event

GPU 后端不能只用一个全局同步。最小资源模型:

Listing 5.56 GPU 执行资源

```
GpuExecutionContext:
  device_id
  stream
  events:
    prefill_start
    prefill_end
    decode_step_start
    decode_step_end
  workspace
  error_state
```

event 用来测量 GPU 时间, host 计时用来测端到端时间。二者都要记录。若 GPU event 显示 kernel 很快, 但 host 端 decode step 慢, 问题可能在 copy、sync、sampler、队列或事件等待。

同步点要显式列出

GPU 性能问题经常藏在同步点。常见同步来源:

- 拷回 logits 给 CPU sampler。
- debug 模式 materialize 中间张量。
- host 等待 GPU event 才能发 token。
- fallback segment 在 CPU 和 GPU 之间来回搬数据。
- 多 stream 之间的依赖 event。
- device allocator 或临时 buffer 初始化。

每个同步点都要进入 trace。若只是看 kernel profile, 可能看不到 host 在等待。端到端优化时, 减少一个同步点往往比把某个小 kernel 优化 10% 更有效。

合批 decode

GPU decode 要提高吞吐, 通常需要合批多个 session 的当前 token。合批后, 一个 step 的输入形状从 `[1,H]` 变成 `[B,H]`, 权重 GEMV 变成小 GEMM, GPU 利用率提高。但合批有代价: 请求要等待同批其他 session, KV cache 来自不同 session, stop 和 sampler 也各自独立。

Listing 5.57 GPU decode 合批元数据

```
DecodeBatch:
  batch_size
  session_ids
  model_positions
  kv_block_tables
  input_token_ids
  sampler_states
```

合批不是把 token 拼起来就完事。每个 session 的 `model_position`、KV block table、stop 条件和 sampler state 都不同。kernel 可以共享权重, 但不能混淆状态。调度器需要设置最大等待时间, 否则为了凑 batch 会拉高 p95。

GPU oracle 策略

GPU 路径也要有 oracle，但不能每步都拷完整中间张量。可用策略：

- smoke 模式：只比较最终 logits 和 top-k。
- checkpoint 模式：抽样比较某几层输出。
- debug 模式：materialize 指定 segment 的输入输出。
- byte 模式：只比较 packed weight 或量化解包。

trace level 控制 oracle 成本。默认生产路径不应频繁同步中间张量；调试时可以打开指定 segment 的 checkpoint。这样既保留定位能力，又不让 oracle 扭曲性能数据。

fallback 不是静默降级

GPU 可能不支持某个 dtype、某个 group size、某个上下文长度或某个算子融合。fallback 可以是好策略，但必须显式记录：

Listing 5.58 GpuFallbackEvent 字段

```
GpuFallbackEvent:
  segment_id
  requested_backend
  actual_backend
  reason:
    dtype_unsupported
    group_size_unsupported
    workspace_exceeded
    kernel_missing
    device_memory_exceeded
  performance_impact_estimate
```

静默 fallback 会制造严重误导：用户以为在测 GPU，实际跑 CPU；或者某个 segment 来回拷贝，端到端比纯 CPU 更慢。

GPU StepTrace

GPU decode 每步至少要记录：

Listing 5.59 GPU StepTrace 字段

```
GpuStepTrace:
  request_id
  step
  context_length
  batch_size
  stream_id
  kernel_time_ns
  copy_h2d_bytes
  copy_d2h_bytes
  copy_time_ns
  sync_time_ns
  sampler_time_ns
  fallback_count
  device_memory_peak
```

这些字段能定位典型问题：kernel 时间低但同步高，说明 host-device 边界有问题；copy 字节突然上升，说明某个 debug 或 fallback materialize 了中间张量；batch size 太小，说明调度器没有把

decode 合并起来。

发布前 GPU 检查表

验收与评分标准

- prepare 阶段完成权重上传，decode step 不重复上传权重。
- device residency report 记录权重、KV、workspace、logits 和 fallback。
- GPU segment descriptor 记录 kernel、buffer、state read/write、workspace 和 stream。
- StepTrace 同时有 GPU event 时间和 host 端 elapsed 时间。
- logits copy、sampler、sync 和 fallback 单独计时。
- 合批 decode 记录 batch size、等待时间和每个 session 的 position。
- debug oracle 不污染默认性能报告。

本节验收线

读完本节，应该能设计 GPU decode 的端到端合同：

- 权重在 prepare 后常驻 device，并写入 plan report。
- activation 和 workspace 不在每 step 动态分配。
- KV cache 在 device 侧维护，连续或分页布局都有明确地址合同。
- prefill 和 decode 分开报告，不用一个 tokens/s 混过。
- sampler、logits copy、stream sync 和 fallback 都进入 StepTrace。
- fallback 必须显式记录原因和实际 backend。
- 用 GPU event 和 host trace 同时解释 kernel 时间与端到端时间。

工程验收、路线图与作品集证据

6.1 工程验收与回归体系：reference、测试、覆盖率、性能门禁和框架对标

为什么“能跑”不是完成

一台本地大模型推理引擎最容易给人错觉：只要 prompt 能生成文字，项目就算跑通。对学习项目来说，这只是第一步；对高质量 AI infra 来说，这远远不够。生成文本看起来正常，不代表 tokenizer 合同正确，不代表量化 scale 正确，不代表 KV cache 没有越界，不代表 SIMD tail path 被覆盖，不代表 GPU fallback 没有偷偷发生，也不代表性能优化真的击中了瓶颈。

工程验收要回答四个问题：

- 1. 正确性由哪个 reference 定义。
- 2. 哪些层次的 oracle 已经比较过。
- 3. 哪些性能指标和硬件环境可复现。
- 4. 哪些回归测试会在后续改动时保护这些事实。

核心思想

AI infra 的完成标准不是“输出了 token”，而是 reference、oracle、profiler、覆盖率、性能回归和外部框架对标形成闭环。没有闭环，优化无法被信任，也无法长期维护。

reference 分层：先有正确答案

reference 是不追求速度的正确性来源。它应该清晰、可读、可调试。后端优化越激进，reference 越重要。项目中至少需要下面几类 reference：

reference 层次	作用	不追求的东西
字节/量化 converter	定义 int4/int8 bytes 如何还原	SIMD 和吞吐
单算子 reference	定义 RMSNorm/linear/attention 输出	cache 和线程效率
层级 reference	定义 decoder block 输出	融合和内存复用
端到端 reference	定义 logits/生成序列合同	生产级速度

reference 代码要尽量少依赖后端复杂路径。比如 int4 reference converter 不应该复用 AVX2 解包函数；否则 SIMD 解包错了，reference 也跟着错。单算子 reference 可以用标量循环和 fp32 累加；端到端 reference 可以只跑小模型、小层数或模型切片，保证测试可承受。

oracle 分层：从字节到生成序列

oracle 消费 reference 和被测实现，判断误差是否在合同内。不同层 oracle 关注不同问题：

Listing 6.1 oracle 层次与定位能力

```
byte oracle:
    packed bytes -> decoded values
    catches nibble order, zero point, scale, tail

operator oracle:
    optimized kernel output vs scalar reference
    catches SIMD, accumulation, layout, quant

layer oracle:
```



```
decoder block output vs reference block
catches fusion, arena overwrite, state edge
```

```
logits oracle:
  final logits distribution
  catches visible model quality drift
```

```
sequence oracle:
  fixed sampler and seed token sequence
  catches end-to-end behavior changes
```

误差容忍度必须和 dtype、量化、后端和 reduction 顺序匹配。fp32 reference 对 fp32 scalar 可以要求很严；fp16、bf16、int8、int4、KV cache 量化都需要不同 tolerance profile。oracle 报告里要写清使用的是哪个 profile，不要把阈值埋在测试代码里。

测试矩阵：不要只测漂亮形状

AI 后端最容易漏测的是边界形状。测试矩阵要覆盖整齐形状，也要覆盖 tail、非对齐、不同 group size、不同 context length 和不同阶段。

Listing 6.2 后端测试矩阵示例

```
shapes:
  H = 4096
  H = 4097 for tail test
  0 not divisible by tile_n
  prompt length: 1, 7, 32, 512
  context length: 1, 128, 1024, 4096

dtypes:
  f32 reference
  f16/bf16 style
  int8 per-channel
  int4 group-size 32/64/128

layouts:
  contiguous
  padded
  packed CPU panel
  device layout

backends:
  scalar
  CPU SIMD
  CPU threaded
  GPU kernel
  fallback path
```

这些测试不一定全部在每次提交跑。可以分层：快速单元测试常跑，较大的端到端和性能测试在夜间或发布前跑。但测试矩阵必须被设计出来，否则项目只会在“常见 happy path”上看起来稳定。

coverage: 覆盖率要服务风险，不服务数字

用户要求每一步优化都做好，C++20 项目也应该有覆盖率意识。但覆盖率不是盲目追数字。高风险路径要优先覆盖：verifier 错误诊断、descriptor 解析、quant converter、tail path、arena liveness、KV cache position、fallback、sampler 和 profiler report。

模块	高风险分支	覆盖重点
model/verifier	坏 shape、坏 dtype、坏 quant	拒绝输入和诊断质量
quant	tail、scale shape、packed order	reference converter 和 oracle
memory planner	last use、alignment、debug materialization	offset 不重叠和生命周期
backend CPU	tail、非对齐、多线程分区	正确性和性能回归
backend GPU	fallback、staging、sync	端到端 latency 和资源释放
runtime	prefill/decode 切换、stop、context limit	session 状态不串扰

如果项目工具链支持覆盖率，核心模块应该接近或超过 90% 的有意义覆盖。若某些 GPU 分支或硬件计数器难以在 CI 中覆盖，要用 fake backend、mock event、small fixture 或本地验收脚本补充。重要的是把不可覆盖原因写清，而不是假装没有风险。

CI 分层：不是所有测试都每次全跑

AI 推理项目的测试成本差异很大。字节级 quant converter 可以毫秒级完成；7B 模型长上下文 benchmark 可能需要专门机器。工程上要分层，而不是把所有检查都塞进每次提交，也不能因为全量测试太慢就放弃测试。

层级	触发时机	内容
L0	本地开发频繁运行	格式化、编译、descriptor/verifier 单测、字节级 quant 测试
L1	每次提交或 PR	小模型 reference、单算子 oracle、memory planner、CPU scalar/SIMD 小矩阵
L2	夜间或合并前	模型切片端到端、长 context、paged KV、fallback、覆盖率报告
L3	发布前或专用机器	真实模型 benchmark、GPU、框架对标、p50/p95、峰值内存、质量回归

失败报告也要结构化。不要只贴一段日志。最小字段可以是：

Listing 6.3 CI failure 结构

```

CiFailure:
  layer: L0 | L1 | L2 | L3
  component: verifier | quant | backend | runtime | benchmark
  fixture_id
  backend
  expected
  actual
  tolerance_or_gate
  first_bad_artifact
  reproduction_command

```

这样开发者能直接知道问题属于哪层合同。若 byte oracle 失败，先查 nibble order、scale 和 tail；若 layer oracle 失败，查 fusion、arena 和状态边；若 L3 benchmark 回归，先看硬件、热身、频率、prompt set 和 profiler breakdown。CI 的作用不是制造红叉，而是把问题定位到正确工程层。

性能门禁：基线、阈值和报告要稳定

性能回归比功能回归更难测，因为机器负载、频率、温度和系统噪声都会影响数字。工程上不能用单次最好成绩做门禁，而要固定环境、固定输入、固定 warmup 和迭代，并报告分位数。

Listing 6.4 性能门禁报告字段

```
PerformanceGate:
  hardware_id
  backend_plan_id
  model_fixture_id
  quant_profile_id
  prompt_set_id
  thread_count_or_device
  warmup_iterations
  measurement_iterations
  baseline_version
  current_version
  allowed_regression_percent
  metrics:
    time_to_first_token
    prefill_tokens_per_s
    decode_tokens_per_s
    p50_step_latency
    p95_step_latency
    peak_memory
```

门禁阈值要分指标。比如 peak memory 回归 20% 可能不能接受；单次 p50 latency 回归 3% 可能在噪声内；p95 latency 回归 15% 需要调查；pack time 变慢但 decode 变快是否可接受，要看目标场景是短请求还是长期服务。性能门禁不是机械判分，而是让退化可见。

服务化 SLO：把 kernel 数字换成线上账本

底层 kernel 快，不等于服务能扛业务。推理服务还要处理排队、取消、流式返回、上下文容量、KV cache 内存和 OOM。最小 serving 验收要把单请求 trace 聚合成下面的账本：

Listing 6.5 ServingSloEnvelope

```
ServingSloEnvelope:
  max_waiting_requests
  max_running_requests
  admission_policy: reject | queue | degrade
  ttft_p50_ms
  ttft_p95_ms
  decode_step_p95_ms
  end_to_end_p95_ms
  queue_wait_p95_ms
  kv_cache_reserved_bytes
  kv_cache_used_bytes
  oom_reject_count
  cancel_reclaim_p95_ms
```

用 7B 典型 shape 粗算一次 KV 容量：layers=32, kv_heads=32, head_dim=128, fp16 时，每 token KV 字节为：

$$2 \times 32 \times 32 \times 128 \times 2 = 524288 \text{ bytes} = 512 \text{ KiB}$$

一个 4K context session 约 **2GiB** KV cache。若机器只给 serving 预留 **16GiB** KV 空间，理论上最多同时容纳 8 个满长 session；实际还要扣除 allocator 碎片、paged metadata、workspace 和安全水位。admission control 因此不能只看请求数，必须看 **prompt_tokens+max_new_tokens** 对 KV budget 的占用。

调度也要定量。若单请求 decode step p95 是 **40ms**，同一 worker 串行跑 4 个活跃 session，最坏情况下某个 session 下一个 token 要等另外 3 个 step，队列等待可接近 **120ms**。若业务要求流式 token 间隔 p95 小于 **80ms**，就必须降低同 worker 并发、改 continuous batching、或把长请求降级/拒绝。这里的重点不是精确排队论，而是 SLO 要能反推出调度策略。

线上现象	应记录的字段	常见根因
TTFT 高 token 间隔抖动	queue wait、load、prefill、pack time decode p95、active sessions、batch size	冷启动、长 prompt、排队 调度过载、同步点、抢占
OOM 或频繁拒绝	KV reserved/used、workspace、arena peak	admission 不看 token 预算
取消后内存不降	cancel reclaim、session state、KV refcount	资源生命周期错
p50 正常但 p95 差	queue wait、page fault、fallback count	尾部排队、缺页、后端降级

因此服务化验收至少要有三条故障演练：超长上下文被 admission 拒绝并返回 token budget；取消请求后 KV、arena、workspace 在限定时间内释放；强制注入慢请求后，报告能拆出 queue wait 与 kernel time，而不是只看到端到端超时。

profiler schema：优化证据要可比较

profiler 输出不能每章一个格式。统一 schema 让不同优化可以比较。最小 schema 应该覆盖 stage、segment、backend、context、耗时、内存和可选硬件计数器。

Listing 6.6 统一 profiler event schema

```
ProfilerEvent:
  run_id
  stage: load | prepare | prefill | decode | sample
  segment_id
  layer_id
  backend
  kernel_or_operation
  start_ns
  end_ns
  input_tokens
  context_length
  bytes_read_estimate
  bytes_written_estimate
  arena_peak_bytes
  workspace_bytes
  kv_cache_bytes
  counters_optional
```

同一 schema 可以服务 CPU 和 GPU。CPU 可能填硬件计数器；GPU 可能填 stream event 时间和 staging bytes；reference 路径也可以填 stage 和耗时。这样后续报告不需要人工拼日志。

诊断质量：错误要能定位到合同

高质量工程不是永远不出错，而是出错时能定位。诊断应该回到合同，而不是只报底层异常。

Listing 6.7 诊断应该包含的信息

```
Diagnostic:
  severity
  component: manifest | verifier | compiler | backend | runtime
  object: tensor/operator/segment/session
  expected
  actual
  derived_from
  suggested_next_check
```

例如 backend 不支持某个 q4 格式时，应该写明 tensor、format、group size、backend capability 和 fallback 选择；KV cache 越界时，应该写明 session position、capacity、max context 和触发的 token；arena 复用冲突时，应该写明两个 value 的生命周期和 offset。这样的诊断能让读者把错误和书中讲过的合同对应起来。

框架对标：尊重成熟实现，也要知道差距在哪里

自研引擎需要对标成熟框架，但对标不能只贴一个 tokens/s。成熟框架可能使用不同量化格式、不同 tokenizer、不同线程数、不同 GPU kernel、不同 batch 策略。对标前要固定变量：

- 同一模型或同一模型切片。
- 同一 tokenizer 和 prompt set。
- 同一量化格式或明确说明格式差异。
- 同一线程数、亲和策略或同一 GPU。
- 同样区分加载、prepare、prefill、decode 和 sampler。
- 同样报告正确性或至少 logits/生成差异。

对标表应该能说明差距：

指标	本项目	成熟框架	解释
load/prepare	A	B	mmap、packing、上传差异
prefill	A	B	GEMM/attention kernel 差异
decode	A	B	GEMV、KV cache、launch 差异
memory peak	A	B	arena、packed weight、KV 策略
quality drift	A	B	量化和 sampler 差异

如果差距很大，也不是失败。高质量书稿应该教读者定位差距：是算法、layout、kernel、线程、设备同步、量化、还是 runtime 调度。只要能定位，就能进入下一轮优化。

发布前验收清单

第三册对应的 C++20 项目在发布一个阶段性版本前，至少应该完成下面清单：

验收与评分标准

- 模型 manifest、tokenizer 合同、shape/dtype/quant verifier 有负例测试。
- reference converter、单算子 reference、层级 reference 和小端到端 reference 可运

行。

- CPU scalar、CPU optimized、GPU 或 fallback 后端都通过 oracle。
- activation arena、workspace、packed weight、KV cache 有生命周期和容量测试。
- prefill/decode 分别有 benchmark，报告 time to first token、tokens/s、latency 分位数和内存峰值。
- 量化报告同时包含容量、scale metadata、logits/top-k、生成序列和性能。
- profiler event schema 稳定，报告能追溯到 segment、kernel、backend 和 context length。
- 性能回归门禁记录硬件、线程、prompt、baseline 和允许退化阈值。
- 与成熟框架的对标至少能解释最大差距来源。

这份清单看起来严格，但它正是 AI infra 工程和普通 demo 的区别。demo 证明“能跑”；验收体系证明“能长期改、能定位错、能解释快慢、能对用户负责”。

阶段发布报告模板

每个阶段发布时都应该留一份短报告。模板可以固定，内容不需要冗长，但必须能复现。

Listing 6.8 阶段发布报告模板

```
ReleaseReport:
  version
  model_scope
  supported_backends
  supported_quant_profiles
  correctness:
    reference_commit
    oracle_profiles
    failed_or_skipped_cases
  performance:
    hardware
    prompt_set
    prefill_tokens_per_s
    ttft_ms
    decode_tokens_per_s
    p95_step_latency_ms
    memory_peak_bytes
  known_limits:
    max_context
    unsupported_dtypes
    unsupported_shapes
    fallback_paths
  comparison:
    baseline_version
    mature_framework_reference
    largest_gap_explanation
```

这个报告能直接回答三个问题：能跑哪些模型和 profile，正确性证据是什么，性能瓶颈还在哪里。若报告写不出来，说明项目某些合同还没有建立好。

验收失败时的处理顺序

失败后不要直接调阈值。处理顺序应该是：

1. 先确认 fixture、模型、prompt、backend plan 和量化 profile 是否一致。

2. 再确认 reference 是否可信，尤其 byte converter 和 tokenizer。
3. 定位最早失败层级：byte、operator、layer、logits、sequence、performance。
4. 若是 correctness，先修实现或合同；若是性能，先确认测量口径和硬件状态。
5. 只有在误差来源被解释后，才调整 tolerance profile。

阈值是合同，不是让测试通过的旋钮。尤其是量化和 GPU fallback，不能因为输出文本看起来还行就放宽 logits/top-k 检查。

一次 CPU kernel 优化的发布门禁示例

假设本周把 **linear** 的 CPU 路径从标量 baseline 换成 packed int8 SIMD kernel。一个合格发布门禁不应该只跑一个 benchmark，而要按照证据链通过。

Listing 6.9 CPU int8 kernel 发布门禁

1. byte oracle:
int8 weights and scale metadata decode to expected values
2. packed weight oracle:
packed panel maps back to original W[out, in]
3. operator oracle:
scalar reference vs packed SIMD output
4. layer oracle:
decoder block output under fixed fixture
5. logits oracle:
final logits and top-k overlap
6. performance gate:
decode latency, p95, memory peak and pack time
7. fallback check:
unsupported ISA falls back to scalar or refuses at plan time

这条门禁能定位失败。例如 byte oracle 失败，说明量化字节解释有错；packed weight oracle 失败，说明 layout 转换有错；operator oracle 失败，说明 SIMD lane、accumulator、tail 或 scale 有错；layer oracle 失败，可能是 arena 复用、fusion 或状态边；performance gate 失败，才进入 cache、线程、NUMA、带宽和反汇编分析。

一次 KV cache bug 的定位示例

KV cache bug 很常见，因为它跨 token 存活，且 shape 往往看起来正确。假设长上下文续写时 logits 从第 200 个 token 开始漂移，可以按下面顺序定位：

Listing 6.10 KV cache 漂移定位流程

- ```
check tokenizer:
 token ids match reference

check model position:
 each token position is monotonic and not reset by cache slot

check RoPE checkpoint:
 q/k after rope match reference at positions 0, 1, 127, 200
```

```

check KV append:
 key/value written at expected logical position

check KV read:
 query head maps to correct kv head
 attention reads positions 0..current_position

check attention probability:
 score and softmax drift starts at which position

```

如果只比较最终生成文本，很难知道问题在哪。把 checkpoint 放在 RoPE、KV append、KV read 和 attention probability，能把“长文本质量差”拆成具体合同错误。

### 一次性能回归的排查示例

性能回归也要按阶段拆。假设版本 B 比版本 A 的 decode tokens/s 下降 18%，不要直接改 kernel。先确认报告坐标一致：

- 模型、prompt set、quant profile、backend plan 是否相同。
- 线程数、亲和、频率、温度和系统负载是否可比。
- pack time 是否被算进 decode。
- fallback count 是否增加。
- context length 分布是否变化。
- p50 和 p95 是否一起下降，还是只有尾延迟变差。

随后看 profiler breakdown：

| 现象                      | 可能原因                          | 下一步证据                       |
|-------------------------|-------------------------------|-----------------------------|
| 权重 segment 变慢           | packing layout、SIMD path、NUMA | packed oracle、perf counters |
| KV segment 随 context 变差 | KV layout、cache miss、TLB      | context bucket 报告           |
| sampler 时间上升            | logits copy、top-k 实现          | sampler trace               |
| p95 上升但 p50 稳定          | 调度、队列、页错误                     | queue wait、page fault       |
| GPU kernel 快但端到端慢       | copy、sync、launch、fallback     | stream event trace          |

这类排查流程应该写进 release report 的失败处理说明。否则团队会在每次回归时重新发明定位方法。

### 验收报告中的最小可读示例

报告不需要长，但要能回答“这次改动可信吗”。下面是一个压缩示例：

Listing 6.11 release gate 摘要示例

```

ReleaseGateSummary:
 change: cpu int8 packed linear kernel
 model_fixture: tiny_decoder_v3
 backend_plan: cpu_avx2_int8_pack_v1
 correctness:
 byte_oracle: pass
 packed_weight_oracle: pass
 operator_oracle_max_abs: 0.0061
 logits_topk_overlap: 9/10

```

```

performance:
 decode_tokens_per_s: 18.4 -> 24.7
 p95_step_latency_ms: 68.2 -> 49.5
 pack_time_ms: 143
 memory_peak_mb: 812 -> 856
risk:
 only group size 64 covered
 avx512 path not implemented

```

这样的摘要比“快了 34%”更有价值。它同时暴露了收益、误差、内存代价和限制。

## 本节验收线

读完本节，应该能建立工程闭环：

- reference 定义正确答案，oracle 分层比较误差，profiler 证明性能变化。
- 测试矩阵必须覆盖 tail、非对齐、量化 group、context length、fallback 和不同后端。
- 覆盖率要优先服务 verifier、quant、memory planner、backend、runtime 等高风险路径。
- 性能门禁要固定硬件、输入、warmup、迭代、baseline 和允许退化阈值。
- profiler schema 要统一，让 CPU/GPU/reference/fallback 报告可以比较。
- 诊断要回到 manifest、IR、descriptor、backend plan 和 session 状态这些合同。
- 框架对标要拆开 load、prepare、prefill、decode、memory 和 quality，而不是只报一个总分。

至此，第六章形成闭环：CPU 和 GPU 后端不是孤立优化技巧，而是以 C++20 资源边界为基础，以编译器 lowering 为入口，以 oracle/profiler/回归门禁为出口的工程系统。

## 6.2 实施路线图：从最小闭环到可对标后端

### 不要从最终形态开始

第一册和第二册体量大，是因为它们把基础、实验和系统边界都展开了。第三册也需要完整链路，但不能用“同时实现所有后端、所有量化、所有 GPU kernel”的方式推进。正确路线是先建立最小闭环，再逐步替换瓶颈路径。

Listing 6.12 第三册项目的实施顺序

```

M0: reference scalar slice
M1: manifest + verifier + typed graph
M2: prefill/decode runtime with KV cache
M3: CPU packed weight + SIMD kernel
M4: quantization descriptor + oracle
M5: GPU hardware-aware backend plan
M6: profiler + regression gate + framework comparison

```

每一阶段都必须能构建、能运行、能报告。否则项目会变成一堆互相等待的半成品。

### 阶段任务拆到可执行粒度

路线图只有阶段名还不够。下面把每个阶段拆成能写代码、能验收、能形成章节实验的任务。

**M0: reference scalar slice。** 目标是写出最小可复现推理数学，而不是追求速度。任务包括：固定 tokenizer fixture；构造一层或两层 decoder slice；实现 RMSNorm、linear、RoPE、causal attention、SwiGLU、lm head 的标量 reference；固定 prompt 和 sampler seed；输出 logits golden。验收标准是同一输入在不同机器上得到同一组 logits 或在严格阈值内一致。

**M1: manifest + verifier + typed graph。** 目标是让模型资产变成可检查数据。任务包括：定义 TensorDesc、QuantDesc、OperatorDesc、KVCacheDesc；读取模型 manifest；检查 shape、dtype、axis、group size、scale count、RoPE 参数、GQA head 映射；为坏资产写负例测试。验收标准不是“好模型能加载”，而是坏模型能被准确拒绝。

**M2: prefill/decode runtime with KV cache。** 目标是建立真正推理循环。任务包括：实现 SessionState；区分 prefill 与 decode；分配 KV cache；记录 model position；实现 append/read 合同；支持 context limit；把 logits 接给 sampler。验收标准是多轮生成不会串状态，prefix、左 padding 或截断场景能定位 position。

**M3: CPU packed weight + SIMD kernel。** 目标是把 decode 热路径从 reference 推到可测优化。任务包括：实现 packed weight cache；写 scalar baseline；写一个受限 dtype 的 SIMD micro-kernel；处理 tail；记录 packing time 与 kernel time；用 oracle 对比 reference。验收标准是小矩阵、非对齐、tail、不同 context length 都有测试。

**M4: quantization descriptor + oracle。** 目标是让低比特不是黑箱。任务包括：实现 int8/int4 converter；支持 per-channel 或 per-group scale；记录 calibration id；实现 byte oracle、operator oracle、logits oracle；报告容量、metadata、误差和 latency。验收标准是能解释量化带来的收益和误差，不只报告模型文件变小。

**M5: GPU hardware-aware backend plan。** 目标不是一开始写最快 GPU kernel，而是建立 GPU 后端合同。任务包括：DeviceBuffer、Stream、Event、Workspace、StagingBuffer；capability query；device 常驻权重；KV cache device layout；StepTrace；fallback report。验收标准是 prefill/decode 分开报告 launch、copy、kernel、sync、sampler 和端到端 latency。

**M6: profiler + regression gate + framework comparison。** 目标是让项目能长期演进。任务包括：统一 profiler event schema；固定 prompt set；固定 benchmark warmup 和迭代；记录硬件和 backend plan；建立 L0-L3 CI；与成熟框架拆项对标。验收标准是一次优化能说明改了什么、快在哪里、误差多少、是否影响 p95 和内存峰值。

这些任务连起来以后，读者能做的不是一个“会打印几句话”的玩具，而是一个可解释的本地推理原型：能加载受限模型资产，能跑 prefill/decode，能维护 KV cache，能做 CPU 低比特优化，能接 GPU plan，能用 oracle/profiler 判断改动是否成立。它仍然不是完整商业框架，但已经具备业务推理服务最核心的工程骨架。

### 每个阶段交付什么

| 阶段 | 交付物                        | 验收                                     |
|----|----------------------------|----------------------------------------|
| M0 | 小模型或模型切片 reference         | 固定 prompt 的 logits 可复现                 |
| M1 | manifest、schema、verifier   | 坏模型资产能给明确诊断                            |
| M2 | session、KV cache、arena     | prefill/decode 状态不串扰                   |
| M3 | CPU packing、SIMD、线程初版      | 单算子和端到端都有 profiler                     |
| M4 | int8/int4/KV quant 合同      | oracle 报告误差和容量收益                       |
| M5 | GPU capability、device plan | prefill/decode 分别报告 launch/copy/kernel |
| M6 | benchmark 与对标              | 与外部框架差距可解释                             |

这张表补齐了“链路不完善”的问题：每一节讲的对象都必须落到某个阶段。manifest 不是文档，属于 M1；KV cache 不是注意力小技巧，属于 M2；SIMD 和 packing 属于 M3；量化不是脚本，属于 M4；GPU 硬件不是背景知识，属于 M5；工程验收属于 M6。

### 每阶段只允许欠一种债

路线图不能变成口号。每个阶段可以保守，但不能同时欠正确性、性能和诊断三种债。

| 阶段 | 可以暂缓      | 不能暂缓                                |
|----|-----------|-------------------------------------|
| M0 | 速度、完整模型   | reference 可复现                       |
| M1 | 激进优化      | manifest/verifier 诊断                |
| M2 | 多后端       | prefill/decode 状态边界                 |
| M3 | GPU       | CPU oracle 与 profiler               |
| M4 | 所有格式      | 量化合同和误差报告                           |
| M5 | 最优 kernel | capability、fallback、copy/sync<br>账本 |
| M6 | 新功能       | 回归门禁和对标解释                           |

### C++20 目录怎样随阶段增长

目录应随阶段增长，而不是一开始创建一堆空模块。

Listing 6.13 目录增长路线

```

M0:
include/ai/reference/
tests/reference/

M1:
include/ai/model/
include/ai/graph/
src/model/
src/graph/

M2:
include/ai/runtime/
src/runtime/

M3:
include/ai/backend/cpu/
src/backend/cpu/

M4:
include/ai/quant/
src/quant/

M5:
include/ai/backend/gpu/
src/backend/gpu/

M6:
tools/bench/
tools/oracle/
tools/report/

```

这个路线遵守 C++20 工程纪律：所有权对象和 view 分开，descriptor 是小值对象，pass 和 analysis 分开，backend 不解析模型文件，runtime 不重新做编译。每个阶段新增目录时，都要同步新增测试和报告字段。

### 每个里程碑的最小代码形态

路线图还要能落到最小代码。下面不是完整目录，而是每个阶段至少要出现的核心类型。

Listing 6.14 M0-M2 的核心类型

```
M0:
 TensorView
 ReferenceLinear
 ReferenceRmsNorm
 ReferenceAttention
 GoldenLogitsFixture

M1:
 ModelManifest
 TensorDesc
 QuantDesc
 GraphNode
 ShapeVerifier
 Diagnostic

M2:
 SessionState
 KVCache
 ActivationArena
 SamplerState
 RunTrace
```

Listing 6.15 M3-M6 的核心类型

```
M3:
 CpuCapability
 PackedWeight
 MatmulKernelDesc
 WorkPartition
 CpuProfilerEvent

M4:
 QuantConverter
 PackedWeightOracle
 OperatorOracle
 QuantErrorReport

M5:
 GpuCapability
 DeviceBuffer
 Stream
 Event
 Workspace
 StepTrace

M6:
 BenchmarkFixture
 PerformanceGate
 ComparisonReport
 CiFailure
```



这些类型的存在比目录名更重要。若项目已经有 `SessionState` 和 `KVCache`，但没有 `RunTrace`，说明能跑但不可解释；若有 `PackedWeight`，但没有 `PackedWeightOracle`，说明优化不可验证；若有 GPU kernel，但没有 `StepTrace`，说明只能看 kernel 时间，不能看端到端。

### 最小可演示业务闭环

第三册项目的第一个业务闭环不应该追求全功能聊天系统，而应该是一个可解释的本地生成服务：

Listing 6.16 最小可演示业务闭环

```
load_model(model_dir)
compile_plan(model_id, backend_policy, quant_profile)

request:
 prompt
 max_new_tokens
 temperature
 trace_level

response:
 streamed tokens
 finish reason
 usage
 trace summary
```

验收不是“能回答问题”，而是：

- 错模型能被 manifest/verifier 拒绝。
- 正模型能输出固定 prompt 的 deterministic logits。
- 开启 trace 后能看到 prefill、decode、sampler 耗时。
- 超过 context limit 会被 admission control 拒绝。
- 取消请求会释放 KV cache。
- 改量化 profile 后能看到容量、误差和速度变化。

这个闭环已经能支撑真实业务原型：本地助手、RAG 回答、批量摘要、代码补全都可以接在它上面。后续优化只是在同一合同下替换 backend plan，而不是重写整条服务链。

### 何时算第三册链路完成

第三册不是要在一章里实现完整工业推理框架，但书稿链路应该完整。链路完成的标准是：读者能从任意一个对象追到上游语义和下游执行。

- 看到 `QuantDescriptor`，能追到 manifest、verifier、IR、packing、oracle。
- 看到 `KVCacheDesc`，能追到 prefill 写入、decode 读取、layout、量化、profiler。
- 看到 `GpuKernelDesc`，能追到 GPU capability、block/warp、stream、staging、fallback。
- 看到 benchmark 数字，能追到模型、prompt、backend plan、硬件、oracle 和 baseline。

这比单纯增加页数更重要。内容要补，但每一页都要服务链路。

### 八周实现节奏：每周都要有可运行产物

如果把第三册内容落实成一个学习项目，可以按八周推进。这里的“周”不是日历硬要求，而是任务切片：每一段都必须能构建、能运行、能验收。

| 阶段 | 本周交付物                                | 必须证明的事                           |
|----|--------------------------------------|----------------------------------|
| W1 | tiny tensor、RMSNorm、Linear reference | shape、stride、dtype 错误能被拒绝        |
| W2 | RoPE、attention、KV cache reference    | position、head 映射、causal mask 正确  |
| W3 | tiny decoder end-to-end logits       | 固定 prompt 的 logits 和 token 序列可复现 |
| W4 | manifest、verifier、typed graph        | 坏模型资产有明确诊断                       |
| W5 | executable plan、arena、trace          | prefill/decode 生命周期可追踪           |
| W6 | CPU packed weight 和一个 SIMD path      | operator oracle 通过，tail path 被覆盖 |
| W7 | int8/int4 descriptor 和量化 oracle      | 容量、误差、速度能同时报告                    |
| W8 | bench、release report、框架对标            | 性能差距能拆到阶段和后端原因                   |

每周都要保留一个命令行入口。比如 W1 不是“写好了类”，而是能运行：

Listing 6.17 W1-W3 的命令行验收

```
ai-oracle --fixture fixtures/tiny_rmsnorm.json
ai-oracle --fixture fixtures/tiny_linear.json
ai-oracle --fixture fixtures/tiny_rope_attention.json
ai-generate --fixture fixtures/tiny_decoder.json --max-new-tokens 4
```

W4-W8 则开始进入真实工程链路：

Listing 6.18 W4-W8 的命令行验收

```
ai-inspect-model --model fixtures/tiny_model
ai-compile-plan --model fixtures/tiny_model --backend cpu
ai-generate --model fixtures/tiny_model --trace summary
ai-bench --model fixtures/tiny_model --prompt-set prompts/smoke.jsonl
ai-report --compare reports/current.json reports/baseline.json
```

命令能跑并不等于通过。每个命令都要输出结构化报告，报告里要有 fixture id、model id、backend plan id、quant profile、硬件、误差阈值、失败定位和复现命令。否则下一次回归时仍然要靠猜。

### 用户故事：从业务需求反推底层对象

第三册项目不是只给算法同学看的。一个业务开发者接本地推理服务时，会提出具体需求。每个需求都能反推出底层对象和测试。

| 业务需求         | 底层对象                            | 验收方式                     |
|--------------|---------------------------------|--------------------------|
| 限制最长上下文      | tokenizer、admission、KV cache    | 超限请求被拒绝并报告 token 数       |
| 流式返回 token   | session、decode loop、event queue | token/final/error 事件顺序稳定 |
| 用户取消生成       | scheduler、KV cache、arena        | 取消后资源计数归零                |
| 切换 CPU/GPU   | backend policy、plan report      | fallback 可见且不改变语义        |
| 切换量化 profile | QuantDesc、packed weight、oracle  | 容量、误差、速度差异可见             |
| 定位响应慢        | RunTrace、ProfilerEvent          | TTFT 和 decode p95 可拆分    |

这张表说明“能不能做实例业务开发”的答案不是靠宣传，而是靠对象边界。没有 admission control，就不能可靠限制上下文；没有 session 和资源释放，就不能安全取消；没有 trace，就不能解释慢请求；没有 oracle，就不能安全切换量化和后端。

### 每个里程碑的测试夹具

测试夹具要跟随路线图增长。不要等到真实 7B 模型才开始测，那样一次失败会同时涉及 tokenizer、权重、RoPE、KV cache、量化和后端。

Listing 6.19 夹具增长路线

```

M0:
 tiny_tensor_contiguous
 tiny_tensor_strided
 tiny_rmsnorm_zero_and_large
 tiny_linear_known_answer

M1:
 manifest_good
 manifest_bad_vocab_embedding
 manifest_bad_qkv_shape
 manifest_bad_quant_group

M2:
 rope_position_0
 rope_prefix_position
 attention_causal_mask
 kv_append_read_gqa
 context_limit_reject

M3:
 packed_weight_roundtrip
 cpu_tile_tail_n
 cpu_tile_tail_k
 cpu_non_aligned_input
 cpu_thread_partition

M4:
 int8_per_channel
 int4_group32
 int4_group64_tail
 kv_quant_score_error

```

M5:

```
gpu_capability_missing
gpu_fallback_cpu
gpu_staging_copy
gpu_stream_event_trace
```

M6:

```
smoke_prompt_set
long_context_prompt_set
release_benchmark_set
```

每个 fixture 都要有一句解释：它保护哪条合同。比如 `manifest_bad_qkv_shape` 保护模型导入和 head 维度, `kv_append_read_gqa` 保护 query head 到 KV head 的映射, `int4_group64_tail` 保护最后一组不足 group size 时的 scale 和 nibble 顺序。

### 负例诊断要从第一天写

只测成功路径会让项目看起来进展快，但后期会付出更大代价。下面是必须尽早写的负例。

| 负例                                                  | 正确诊断                                         | 不合格表现                              |
|-----------------------------------------------------|----------------------------------------------|------------------------------------|
| tokenizer vocab 与 embedding 行数不一致                   | 指出两者实际值和来源文件                                 | decode 时访问越界                       |
| <code>query_heads</code> 不能整除 <code>kv_heads</code> | 拒绝 GQA head 映射                               | attention 中取整凑合                    |
| RoPE base 或 scaling 缺失                              | 指出缺失字段和默认策略                                  | 静默使用错误默认值                          |
| int4 scale 数量不匹配<br>context 超过容量                    | 指出 group 数、实际 scale 数<br>返回 context limit 错误 | 解包时读越界或错 scale<br>KV cache 自动扩容到失控 |
| GPU 不支持某 dtype                                      | plan 阶段 fallback 或拒绝                         | kernel launch 失败才报错                |

诊断不要求长，但必须具体。读者应该从错误里知道下一步检查什么，而不是只看到 `invalidargument`。

### 从 M0 到 M6 的代码增长原则

路线图容易出现两种坏倾向：一开始设计过度，或者一路写临时函数不回收。比较稳的原则是：

**M0-M2 先保守。** 类型少、逻辑直、输出多。reference 可以慢，但不能模糊。所有 state 都显式传参，尤其是 `model_position`、`layer_id` 和 `kv_head`。

**M3-M4 开始分离描述符。** 一旦出现 packed weight、SIMD、int4、scale，就必须有 descriptor。不要让 kernel 通过约定猜 layout。descriptor 要能打印到 plan report, oracle 要能使用同一 descriptor 解释字节。

**M5 接 GPU 时先建资源边界。** 先实现 `DeviceBuffer`、`Stream`、`Event`、`Workspace` 和 fallback report，再追求 kernel 性能。否则 GPU 路径会把 copy、sync、launch 和 kernel 时间混成一团。

**M6 固化报告。** 性能报告、release report 和 comparison report 的字段要稳定。优化可以变，报告坐标不能变；否则两次 benchmark 无法比较。

### 不要用这些方式推进

有几种推进方式看似快，其实会破坏全书主线。

## 常见误区

- 不要先下载一个真实大模型，靠改到能跑来倒推架构。真实模型只会同时暴露太多问题。
- 不要直接调用外部框架得到结果后，把项目包装成服务。那样学不到 manifest、IR、KV cache、量化和后端合同。
- 不要在没有 reference 的情况下写 SIMD 或 GPU。错误会被隐藏到 logits 漂移里。
- 不要把量化当文件压缩。量化必须有 scale、zero point、accumulator、requantize、oracle 和质量报告。
- 不要只报平均 tokens/s。必须拆 TTFT、prefill、decode、p95、内存峰值和 fallback。

## 大师级 AI Infra 的硬证据

读完第三册不应该只证明“系统学习过”。大师级 AI Infra 岗位看的是工程证据：能不能把原理落实成可运行系统、可复现实验、可解释性能差距和可维护测试。当前第三册的书稿能打基础，但仅靠书稿本身不能证明生产级能力。必须把 [labs/linux\\_cpu\\_inference/](#) 从教学 demo 推进到证据项目。

| 证据层        | 必须交付                                                   | 不合格表现           |
|------------|--------------------------------------------------------|-----------------|
| 手写 kernel  | scalar、packed、AVX2/AVX-512、tail、反汇编                    | 只讲 GEMM 原理      |
| 推理 runtime | tensor、operator registry、executor、planner、thread pool  | 只有单函数 demo      |
| 系统对标       | oneDNN/OpenBLAS/ONNX Runtime/llama.cpp/ggml 至少两个       | 只报自家数字          |
| 性能报告       | latency、吞吐、IPC、cache miss、branch miss、NUMA、shape sweep | 只报平均 tokens/s   |
| 测试体系       | golden、随机 shape、边界 shape、fuzz、sanitizer、CI             | 只测一个 happy path |
| 开源证据       | 文档、benchmark、测试或小 bug 修复被合并                            | 闭门写报告           |

因此，[linux\\_cpu\\_inference](#) 的验收线要升级。第一阶段已经有 int8 MLP、tiny reference decoder、int8 scalar baseline、packed layout 和 shape sweep benchmark；这只能说明项目开始接触真实工程边界。下一阶段必须补：

- Release benchmark 固定 CSV 输出，记录 compiler、flags、CPU、thread count 和 commit。
- AVX2 或 AVX-512 int8 GEMV/GEMM kernel，至少覆盖一个 decode 热路径 shape。
- Linux [perfstat](#) wrapper，收集 cycles、instructions、IPC、cache misses、branches、branch misses。
- oneDNN 或 OpenBLAS 对照，再选 llama.cpp 或 ggml 做推理侧对照。
- random shape、tail shape、非对齐 shape、bad descriptor 和 sanitizer。

这条线不容易，但它是“刀胚开刃”的标准。没有这些证据，第三册最多是一套学习材料；补齐之后，它才会变成能拿给 AI Infra 团队看的工程作品。

## 本节验收线

读完本节，应该能把第三册项目拆成可推进的阶段：

- 先 reference，再 manifest/verifier，再 runtime，再 CPU/GPU/quant 优化。
- 每阶段都有交付物和验收，不把 correctness、performance 和 diagnostics 分开欠账。
- C++20 目录随阶段增长，避免大而空或单文件堆叠。

- 链路完成标准是对象能追溯，不是页数本身。



## 术语表

**张量** 带元素类型、形状、步长和生命周期的数据视图。

**算子** 有输入合同、输出合同和错误边界的计算单元。

**量化** 用较低位宽的整数近似表示实数值，并用 scale 和 zero point 记录映射关系。

**校准** 为量化选择数值范围和 scale 的过程。权重可以离线扫描，activation 和 KV cache 通常需要代表性输入或运行时策略。

**Requantize** 把较宽累加结果重新映射到目标低位宽输出的步骤，通常包含固定点乘法、舍入、加 zero point 和 clamp。

**Group Quantization** 按一组连续元素共享 scale 的量化方式，常用于 int4/int8 大模型权重；它降低误差但要求 kernel 正确读取 group scale。

**QuantDescriptor** 描述量化格式的结构化元数据，包括 bit width、scale dtype、zero point policy、group size、axis、packed order 和 tail policy。

**CalibrationRecord** 记录量化校准来源和结果的工程对象，包括校准集、算法、范围、饱和率、scale 摘要和工具版本。

**Reference Converter** 用清晰标量实现把低比特字节、scale 和 zero point 还原为参考浮点值的组件，用来验证 packed order、tail policy 和 metadata 解释。

**Reference** 用来定义正确结果的清晰实现，不以速度为目标。

**Oracle** 判定实现输出是否满足 reference 和误差合同的比较器。

**Kernel** 某个算子在特定 dtype、layout 和目标机器上的具体实现。

**推理图** 算子节点和张量边组成的有向计算结构。

**AI 编译器** 把模型图、张量合同、量化元数据和后端能力转换成可执行计划的编译流水线。

**编译前端** 把外部输入转换成已检查内部表示的阶段。在 AI 推理引擎里，它读取模型文件、tokenizer、权重索引和量化 metadata，输出 manifest 与 typed graph。

**IR** 中间表示。AI 编译器中常分为表达算子依赖的 graph IR、表达 shape/layout/dtype 的 tensor IR，以及表达 tile/vector/thread 的 loop IR。

**Pass** 读取一种 IR 并产出改写后 IR 或分析事实的编译步骤，例如融合 pass、layout pass、liveness pass 和 lowering pass。

**Analysis Pass** 不改写 IR、只收集事实的 pass，例如 use-def、liveness、alias、shape、quant 和 cost analysis。

**Transform Pass** 根据已有分析事实改写 IR 的 pass，例如融合、消除冗余 reshape、量化 lowering 和 layout 转换。

**Lowering** 把较高层表示逐步降到更接近目标后端的表示，例如从算子图降到 CPU SIMD kernel 或 GPU device kernel 调用。

**后端** 把已验证的 IR 或 executable plan 落到具体硬件和运行环境的实现层，例如 CPU SIMD/多线程后端或 GPU stream/kernel 后端。

**Executable Plan** 编译器输出给 runtime 的可执行计划，包含 segment、kernel 选择、arena offset、外部状态绑定点和后端同步要求。

**内存规划** 根据张量生命周期、大小、对齐和后端位置安排 arena、workspace、packed weight 和 KV cache 的过程。

**Manifest** 模型导入后形成的机器可读清单，记录 config、tokenizer、张量位置、shape、dtype、layout 和量化元数据。

**Verifier** 检查 IR、shape、dtype、量化和状态边是否满足合同的组件；失败时应给出可定位诊断，而不是让后端继续执行。

**Schema Resolution** 把不同模型格式或命名方言映射到引擎内部 canonical role 的过程。

**Activation Arena** 根据中间张量生命周期预先规划的一块或多块临时内存，用 offset 复用短生命

周期 activation, 避免热路径反复分配。

**Packed Weight** 后端准备阶段从逻辑权重转换出的物理布局, 通常包含 tile 重排、对齐、padding、量化 scale 布局和 kernel 读取合同。

**KV Cache Quantization** 对推理过程中动态增长的 K/V 状态做低比特存储的策略。它不同于静态权重量化, 必须同时考虑写入成本、metadata、长上下文误差和 decode attention 热路径。

**Segment** executable plan 中比单个算子更粗的调度单元, 绑定 arena offset、外部状态和一段 kernel 序列, 用来降低运行时解释开销。

**Time to First Token** 从请求进入到第一个生成 token 可用之间的延迟, 主要受 tokenizer、prefill、KV cache 初始化和首轮 logits 影响。

**Micro-kernel** 面向特定 dtype、layout、tile 和指令集的小型核心计算内核, 例如一个 AVX2 int8 GEMV tile。它通常由外层 packing、线程调度和尾部路径包围。

**RAII** C++ 中用对象生命周期管理资源的惯用法。推理后端中, 文件映射、aligned buffer、device buffer、stream 和 profiler event 都应由 RAII 对象管理。

**Baseline** 优化前可重复测量的基线实现。它可以不快, 但必须正确、可构建、可 benchmark, 并能作为后续优化的比较对象。

**Tail Path** 处理维度不能整除 SIMD 宽度、tile 大小或 group size 时的边界路径。它必须被 descriptor、verifier、kernel 和测试共同覆盖。

**ThreadPlan** CPU 后端对任务划分、最小任务粒度、barrier、scratch、输出写策略和亲和策略的描述。

**DeviceBuffer** GPU 后端中拥有显存生命周期的 RAII 对象, 避免裸 device pointer 在计划和运行时之间泄漏所有权。

**SM/CU** GPU 上的基本执行资源组。NVIDIA 常称 SM, AMD 常称 CU; 它们包含调度、寄存器、执行单元、shared memory 和矩阵计算单元等资源。

**Warp/Wavefront** GPU 中一组 lockstep 执行的线程。它类似放大的 SIMD lane 组, 喜欢规则分支和连续访存。

**Shared Memory** GPU SM/CU 内部的快速小容量内存, 常用于 tile 复用; 容量和使用量会影响 occupancy。

**Tensor Core** GPU 中面向矩阵块乘加的专用计算单元, 通常对 dtype、shape、layout 和对齐有要求。

**Occupancy** 一个 SM/CU 上可同时驻留的 warp/block 相对上限的比例。它影响隐藏延迟的能力, 但不能单独代表性能。

**Stream** GPU 后端的异步执行队列。kernel launch、拷贝和 event 通常挂在 stream 上, runtime 必须用它表达同步边界。

**StagingBuffer** Host 与 device 之间传输 logits、token id 或小 metadata 的中转缓冲。它的大小、同步和复用会影响 decode latency。

**Performance Gate** 性能回归门禁, 记录硬件、输入、baseline、阈值和指标, 用来判断一次改动是否让关键路径退化。

**ProfilerEvent** 统一性能事件记录, 通常包含 stage、segment、backend、kernel、时间、context length、内存和可选硬件计数器。

# 索引

instruction  
  add, 29  
  max, 24, 30  
  mul, 29  
IR, 150  
  
manifest, 144  
  
operator, 25  
  
packing, 36

Pass, 152  
  
quantization, 25  
  
SIMD, 37  
  
tensor, 24  
TTFT, 9  
  
可执行计划, 7  
因果语言建模, 88